

2021年最新Android中高进阶面试集锦

前言

近几年大家对Android的热度依旧高涨，在各种招聘网站上也可以看到Android开发者的薪资标准也在逐渐提高，但随之而来的高薪资对应的一定是对从业人员更高标准的要求。有些程序员可能已经实战多年，但是但换工作的时候面临着对原理知识的遗漏，所以面试的时候显得有些力不从心；另外一些朋友可能接触Android不久，处于只能掌握基础理论，但缺乏实践的情况。。。

对大家不同的情况，码牛学院在这里为大家整合出了这本《Android进阶面试圣经》，希望可以给到各位Android开发者更多的帮助与收获。

第一章 Android 高频面试之必考Java基础

1，面向对象和面向过程的区别

面向过程：面向过程性能比面向对象高。因为对象调用需要实例化，开销比较大，较消耗资源，所以当性能是最重要的考量因素的时候，比如单片机、嵌入式开发、Linux/Unix 等，一般采用面向过程开发。但是，面向过程没有面向对象易维护、易复用、易扩展。

面向对象：面向对象易维护、易复用、易扩展。因为面向对象有封装、继承、多态性的特性，所以可设计出低耦合的系统，使得系统更加灵活、更加易于维护。

那为什么，面向过程性能比面向对象高呢？

面向过程也需要分配内存，计算内存偏移量，Java 性能差的主要原因并不是因为它是面向对象语言，而是因为 Java 是半编译语言，最终的执行代码并不是可以直接被 CPU 执行的二进制机器码。而面向过程语言大多都是直接编译成机器码在电脑上执行，并且其它一些面向过程的脚本语言性能也并不一定比 Java 好。

2，面向对象的特征有哪些

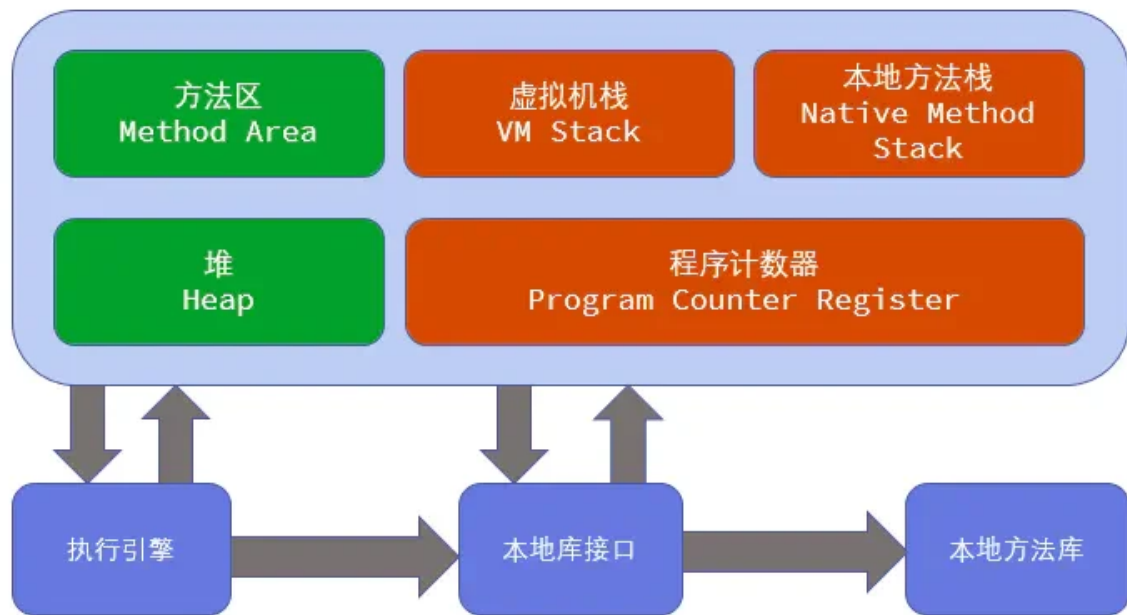
- **封装**：通常认为封装是把数据和操作数据的方法绑定起来，对数据的访问只能通过已定义的接口。
- **继承**：继承是从已有类得到继承信息创建新类的过程。提供继承信息的类被称为父类（超类、基类）；得到继承信息的类被称为子类（派生类）。
- **抽象**：抽象是将一类对象的共同特征总结出来构造类的过程，包括数据抽象和行为抽象两方面。抽象只关注对象有哪些属性和行为，并不关注这些行为的细节是什么。
- **多态性**：多态性是指允许不同子类型的对象对同一消息作出不同的响应。即同一消息可以根据发送对象的不同而采取不同的行为方式。

3，解释下Java的编译与解释并存的现象

当 .class 字节码文件通过 JVM 转为机器可以执行的二进制机器码时，JVM 类加载器首先加载字节码文件，然后通过解释器逐行进行解释执行，这种方式的执行速度相对比较慢。而且有些方法和代码块是反复被调用的(也就是所谓的热点代码)，所以后面引进了 JIT 编译器，而 JIT 属于运行时编译。当 JIT 编译器完成一次编译后，会将字节码对应的机器码保存下来，下次可以直接调用。这也解释了我们为什么经常会说 Java 是编译与解释共存的语言。

4，简单介绍下JVM的内存模型

Java虚拟机所管理的内存包含程序计数器、Java虚拟机栈、本地方法栈、Java堆和方法区5个部分，模型图如下图所示。



4.1 程序计数器

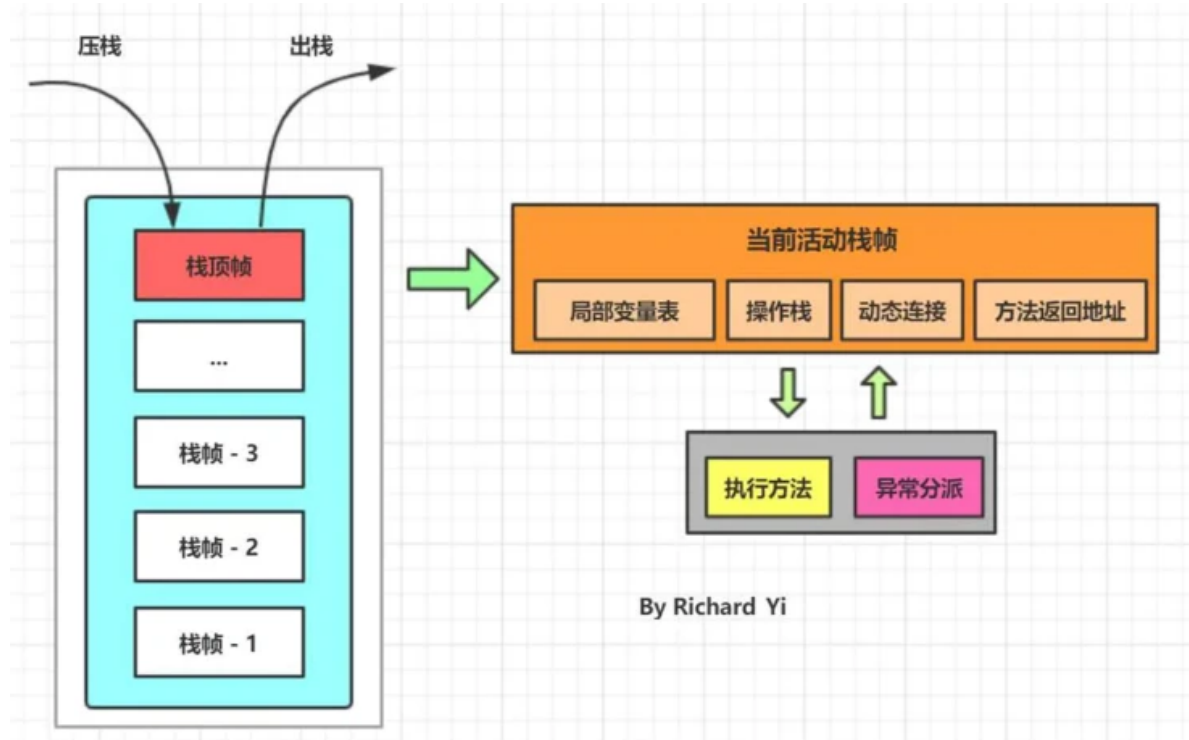
由于Java虚拟机的多线程是通过线程轮流切换、分配处理器执行时间的方式来实现的，在任何一个确定的时刻，一个处理器只会执行一条线程中的指令。为了线程切换后能恢复到正确的执行位置，每条线程都需要有一个独立的程序计数器，各个线程之间的计数器互不影响，独立存储，这类内存区域为【线程私有】的内存。

程序计数器具有如下的特点：

- 是一块较小的内存空间。
- 线程私有，每条线程都有自己的程序计数器。
- 生命周期方面，随着线程的创建而创建，随着线程的结束而销毁。
- 是唯一一个不会出现OutOfMemoryError的内存区域。

4.2 Java虚拟机栈

Java虚拟机栈也是线程私有的，它的生命周期与线程的生命周期同步，虚拟机栈描述的是Java方法执行的线程内存模型。每个方法被执行的时候，Java虚拟机都会同步创建一个内存块，用于存储在该方法运行过程中的信息，每个方法被调用的过程都对应着一个栈帧在虚拟机中从入栈到出栈的过程。



Java虚拟机栈有如下的特点：

- 局部变量表所需的内存空间在编译期间完成分配，进入一个方法时，这个方法需要在栈帧中分配的局部变量空间是完全确定的，在方法运行期间不会改变局部变量表的大小。
- Java虚拟机栈会出现两种异常：StackOverflowError 和 OutOfMemoryError。

4.3 本地方法栈

本地方法栈与虚拟机所发挥的作用很相似，区别在于虚拟机栈为虚拟机执行Java方法服务，而本地方法栈则是为虚拟机使用到的本地方法服务。

4.4 Java堆

Java堆是虚拟机所管理的内存中最大的一块，Java堆是被所有线程共享的一块内存区域，在虚拟机启动时创建。

此内存区域的唯一目的就是存放对象实例，Java中“几乎”所有的对象实例都在这里分配内存。这里使用“几乎”是因为Java语言的发展，及时编译的技术发展，逃逸分析技术的日渐强大，栈上分配、标量替换等优化手段，使Java对象实例都分配在堆上变得不那么绝对。

Java堆是垃圾收集器管理的主要区域，因此很多时候也被称做“GC堆”。从内存回收的角度来看，由于现在收集器基本都采用分代收集算法（G1之后开始变得不一样，引入了region，但是依旧采用了分代思想），Java堆中还可以细分为：新生代和老年代。再细致一点的有Eden空间、From Survivor空间、ToSurvivor空间等。从内存分配的角度来看，线程共享的Java堆中可能划分出多个线程私有的分配缓冲区（Thread Local Allocation Buffer，简写TLAB）。

OOM异常

Java堆的大小既可以固定也可以扩展，但是主流的虚拟机，堆的大小都是支持扩展的。如果需要线程请求分配内存，但堆已满且内存已无法再扩展时，就抛出 OutOfMemoryError 异常。比如：

```
/**
 * VM Args: -Xms10m -Xmx10m -XX:+HeapDumpOnOutOfMemoryError
 */
public class HeapOOMTest {
```

```

public static final int _1MB = 1024 * 1024;

public static void main(String[] args) {
    List<Integer[]> list = new ArrayList<>();
    for (int i = 0; i < 10; i++) {
        Integer[] ints = new Integer[2 * _1MB];
        list.add(ints);
    }
}
}

```

4.5 方法区

方法区和Java堆一样，是各个线程共享的内存区域，他用于存储已被虚拟机加载的类信息、常量、静态变量、即时编译器编译后的代码缓存等数据。

在 HotSpot JVM 中，永久代（永久代实现方法区）中用于存放类和方法的元数据以及常量池，比如 Class 和 Method。每当一个类初次被加载的时候，它的元数据都会放到永久代中。永久代是有大小限制的，因此如果加载的类太多，很有可能导致永久代内存溢出，为此我们不得不对虚拟机做调优。

后来 HotSpot 放弃永久代（PermGen），jdk1.7 版本中，HotSpot 已经把原本放在永久代的字符串常量池、静态变量等移出，到了 jdk1.8，完全废弃了永久代，方法区移至元空间（Metaspace）。比如类元信息、字段、静态属性、方法、常量等都移动到元空间区。元空间的本质和永久代类似，都是对 JVM 规范中方法区的实现。不过元空间与永久代之间最大的区别在于：元空间并不在虚拟机中，而是使用本地内存。因此，默认情况下，元空间的大小仅受本地内存限制。

常用的 JVM 调参如下表：

参数	作用描述
-XX:MetaspaceSize	分配给 Metaspace（以字节计）的初始大小。如果不设置的话，默认是 20.79M，这个初始大小是触发首次 Metaspace Full GC 的阈值，例如 -XX:MetaspaceSize=256M
-XX:MaxMetaspaceSize	分配给 Metaspace 的最大值，超过此值就会触发 Full GC，此值默认没有限制，但应取决于系统内存的大小。JVM 会动态地改变此值。但是线上环境建议设置，例如 -XX:MaxMetaspaceSize=256M
-XX:MinMetaspaceFreeRatio	最小空闲比，当 Metaspace 发生 GC 后，会计算 Metaspace 的空闲比，如果空闲比(空闲空间/当前 Metaspace 大小)小于此值，就会触发 Metaspace 扩容。默认值是 40，也就是 40%，例如 -XX:MinMetaspaceFreeRatio=40
-XX:MaxMetaspaceFreeRatio	最大空闲比，当 Metaspace 发生 GC 后，会计算 Metaspace 的空闲比，如果空闲比(空闲空间/当前 Metaspace 大小)大于此值，就会触发 Metaspace 释放空间。默认值是 70，也就是 70%，例如 -XX:MaxMetaspaceFreeRatio=70

运行时常量池

运行时常量池是方法区的一部分，Class 文件中除了有类的版本、字段、方法、接口等描述信息外，还有一项信息是常量池表，用于存放编译期间生成的各种字面量与符号引用，这部分内容将在类加载后存放在方法区的运行时常量池中。

方法区中存放：类信息、常量、静态变量、即时编译器编译后的代码。常量就存放在运行时常量池中。

当类被 Java 虚拟机加载后，.class 文件中的常量就存放在方法区的运行时常量池中。而且在运行期间，可以向常量池中添加新的常量。如 String 类的 intern() 方法就能在运行期间向常量池中添加字符串常量。

4.6 直接内存

直接内存并不是虚拟机运行时数据区的组成部分，在 NIO 中引入了一种基于通道和缓冲的 IO 方式。它可以通过调用本地方法直接分配 Java 虚拟机之外的内存，然后通过一个存储在堆中的 **DirectByteBuffer** 对象直接操作该内存，而无须先将外部内存中的数据复制到堆中再进行操作，从而提高了数据操作的效率。

由于直接内存并非 Java 虚拟机的组成部分，因此直接内存的大小不受 Java 虚拟机控制，但既然是内存，如果内存不足时还是会抛出 OutOfMemoryError 异常。

下面是直接内存与堆内存的一些异同点：

- 直接内存申请空间耗费更高的性能；
- 直接内存读取 IO 的性能要优于普通的堆内存。
- 直接内存作用链：本地 IO -> 直接内存 -> 本地 IO
- 堆内存作用链：本地 IO -> 直接内存 -> 非直接内存 -> 直接内存 -> 本地 IO

服务器管理员在配置虚拟机参数时，会根据实际内存设置 -Xmx 等参数信息，但经常忽略直接内存，使得各个内存区域总和大于物理内存限制，从而导致动态扩展时出现 OutOfMemoryError 异常。

5, 简单介绍下Java的类加载器

Java 的类加载器可以分为 BootstrapClassLoader、ExtClassLoader 和 AppClassLoader，它们的作用如下。

- **BootstrapClassLoader**：Bootstrap 类加载器负责加载 rt.jar 中的 JDK 类文件，它是所有类加载器的父加载器。Bootstrap 类加载器没有任何父类加载器，如果调用 String.class.getClassLoader()，会返回 null，任何基于此的代码会抛出 NullPointerException 异常，因此 Bootstrap 加载器又被称为初始类加载器。
- **ExtClassLoader**：Extension 将加载类的请求先委托给它的父加载器，也就是 Bootstrap，如果没有成功加载的话，再从 jre/lib/ext 目录下或者 java.ext.dirs 系统属性定义的目录下加载类。Extension 加载器由 sun.misc.Launcher\$ExtClassLoader 实现。
- **AppClassLoader**：Java 默认的加载器就是 System 类加载器，又叫作 Application 类加载器。它负责从 classpath 环境变量中加载某些应用相关的类，classpath 环境变量通常由 -classpath 或 -cp 命令行选项来定义，或者是 JAR 中的 Manifest 的 classpath 属性，Application 类加载器是 Extension 类加载器的子加载器。

类加载会涉及一些加载机制。

- **委托机制**：加载任务委托交给父类加载器，如果不行就向下传递委托任务，由其子类加载器加载，保证 Java 核心库的安全性。
- **可见性机制**：子类加载器可以看到父类加载器加载的类，而反之则不行。
- **单一性原则**：父加载器加载过的类不能被子加载器加载第二次。

6, 谈一下Java的垃圾回收，以及常用的垃圾回收算法。

Java 的内存管理主要涉及三个部分：堆（Java 代码可及的 Java 堆和 JVM 自身使用的方法区）、栈（服务 Java 方法的虚拟机栈和服务 Native 方法的本地方法栈）和保证程序在多线程环境下能够连续执行的程序计数器。

Java 堆是进行垃圾回收的主要区域，故其也被称为 GC 堆；而方法区的垃圾回收主要针对的是新生代和中年代。总的来说，堆（包括 Java 堆和方法区）是垃圾回收的主要对象，特别是 Java 堆。

6.1 垃圾回收算法

6.1.1 对象存活判断

引用计数

每个对象有一个引用计数属性，新增一个引用时计数加1，引用释放时计数减1，计数为0时可以回收。此方法虽然简单，但无法解决对象相互循环引用的问题。

可达性分析

从 GC Roots 开始向下搜索，搜索所走过的路径称为引用链，当一个对象到 GC Roots 没有任何引用链相连时，则证明此对象是不可用的。在Java中，GC Roots包括：

- 虚拟机栈中引用的对象。
- 方法区中类静态属性实体引用的对象。
- 方法区中常量引用的对象。
- 本地方法栈中 JNI 引用的对象。

6.2 垃圾收集算法

标记清除法

如它的名字一样，算法分为“标记”和“清除”两个阶段：首先标记出所有需要回收的对象，在标记完成后统一回收掉所有被标记的对象。之所以说它是最基础的收集算法，是因为后续的收集算法都是基于这种思路并对其缺点进行改进而得到的。

标记复杂算法有两个主要的缺点：一个是效率问题，标记和清除过程的效率都不高；另外一个空间问题，标记清除之后会产生大量不连续的内存碎片，空间碎片太多可能会导致，当程序在以后的运行过程中需要分配较大对象时无法找到足够的连续内存而不得不提前触发另一次垃圾收集动作。

复制算法

复制的收集算法，它将可用内存按容量划分为大小相等的两块，每次只使用其中的一块。当这一块的内存用完了，就将还存活着的对象复制到另外一块上面，然后再把已使用过的内存空间一次清理掉。

它的优点是每次只需要对其中的一块进行内存回收，内存分配时就不用考虑内存碎片等复杂情况，只要移动堆顶指针，按顺序分配内存即可，实现简单，运行高效。而缺点也是显而易见的，内存缩小为原来的一半，持续复制长生存期的对象则导致效率降低。

标记整理法

复制收集算法在对象存活率较高时就要执行较多的复制操作，效率将会变低。更关键的是，如果不想浪费50%的空间，就需要有额外的空间进行分配担保，以应对被使用的内存中所有对象都100%存活的极端情况，所以在老年代一般不能直接选用这种算法。

根据老年代的特点，有人提出了另外一种“标记-整理”（Mark-Compact）算法，标记过程仍然与“标记-清除”算法一样，但后续步骤不是直接对可回收对象进行清理，而是让所有存活的对象都向一端移动，然后直接清理掉端边界以外的内存。

分代收集算法

分代收集算法，就是把Java堆分为新生代和老年代，这样就可以根据各个年代的特点采用最适当的收集算法。在新生代中，每次垃圾收集时都发现有大批对象死去，只有少量存活，那就选用复制算法，只需要付出少量存活对象的复制成本就可以完成收集。而老年代中因为对象存活率高、没有额外空间对它进行分配担保，就必须使用“标记-清理”或“标记-整理”算法来进行回收。

7，成员变量和局部变量的区别

- 从语法形式上看：成员变量是属于类的，而局部变量是在方法中定义的变量或是方法的参数；成员变量可以被 public、private、static 等修饰符所修饰，而局部变量不能被这些修饰符所修饰；但是它们都可以被 final 所修饰。

- 从变量在内存中的存储方式来看：如果成员变量被 `static` 所修饰，那么这个成员变量属于类，如果没有被 `static` 修饰，则该成员变量属于对象实例。对象存在于堆内存，局部变量存在于栈内存（具体是Java虚拟机栈）。
- 从变量在内存中的生存时间来看：成员变量是对象的一部分，它随着对象的创建而存在，而局部变量随着方法的调用结束而自动消失。
- 成员变量如果没有赋初始值，则会自动以类型的默认值而赋值（例外：被 `final` 修饰的成员变量必须在初始化时赋值），局部变量则不会自动赋值。

8, Java 中的方法重写(Overriding)和方法重载(Overload)的含义

方法重写

在Java程序中，类的继承关系可以产生一个子类，子类继承父类，它具备了父类所有的特征，继承了父类所有的方法和变量。子类可以定义新的特征，当子类需要修改父类的一些方法进行扩展，增大功能，程序设计者常常把这样的一种操作方法称为重写，也叫称为覆写或覆盖。

方法重写有如下一些特点：

- 方法名，参数列表必须相同，返回类型可以相同也可以是原类型的子类型
- 重写方法不能比原方法访问性差（即访问权限不允许缩小）。
- 重写方法不能比原方法抛出更多的异常。
- 重写发生在子类和父类之间。
- 重写实现运行时的多态性。

方法重载

方法重载是让类以统一的方式处理不同类型数据的一种手段。调用方法时通过传递给它们的不同个数和类型的参数来决定具体使用哪个方法，这就是多态性。所谓方法重载是指在一个类中，多个方法的方法名相同，但是参数列表不同。参数列表不同指的是参数个数、参数类型或者参数的顺序不同。

- 方法名必须相同，参数列表必须不同（个数不同、或类型不同、参数类型排列顺序不同等）。
- 方法的返回类型可以相同也可以不相同。
- 重载发生在同一类中。
- 重载实现编译时的多态性。

9, 简单介绍下传递和引用传递

按值传递：值传递是指在调用函数时将实际参数复制一份传递到函数中，这样在函数中如果对参数进行修改，将不会影响到实际参数。简单来说就是直接复制了一份数据过去，因为是直接复制，所以这种方式在传递时如果数据量非常大的话，运行效率自然就变低了，所以Java在传递数据量很小的数据是值传递，比如Java中的各种基本类型：`int`、`float`、`double`、`boolean`等类型。

引用传递：引用传递其实就弥补了上面说的不足，如果每次传参数的时候都复制一份的话，如果这个参数占用的内存空间太大的话，运行效率会很底下，所以引用传递就是直接把内存地址传过去，也就是说引用传递时，操作的其实都是源数据，这样的话修改有时候会冲突，记得用逻辑弥补下就好了，具体的数据类型就比较多了，比如Object，二维数组，List，Map等除了基本类型的参数都是引用传递。

10, 为什么重写 equals 时必须重写 hashCode 方法

下面是使用`hashCode()`与`equals()`的相关规定：

- 如果两个对象相等（即用 `equals` 比较返回 `true`），则 `hashCode` 一定也是相同的；
- 两个对象有相同的 `hashCode` 值，它们也不一定是相等的（不同的对象也可能产生相同的 `hashCode`，概率性问题）；
- `equals` 方法被覆盖过，则 `hashCode` 方法也必须被覆盖。

为什么必须要重写 hashCode 方法？其实就是为了保证同一个对象，保证在 equals 相同的情况下 hashCode 值必定相同，如果重写了 equals 而未重写 hashCode 方法，可能就会出现两个没有关系的对象 equals 相同的（因为 equals 都是根据对象的特征进行重写的），但 hashCode 确实不相同的。

11，接口和抽象类的区别和相同点是什么

相同点：

- 接口是绝对抽象的，不可以被实例化，抽象类也不可以被实例化。
- 类可以不实现抽象类和接口声明的所有方法，当然，在这种情况下，类也必须得声明成是抽象的。

异同点：

- 从设计层面来说，抽象是对类的抽象，是一种模板设计，接口是行为的抽象，是一种行为的规范。
- 定义接口的关键字是 interface，抽象类的关键字是 abstract class
- 接口中所有的方法隐含的都是抽象的。而抽象类则可以同时包含抽象和非抽象的方法。
- 类可以实现很多个接口，但是只能继承一个抽象类，接口可以继承多个接口
- Java 接口中声明的变量默认都是 public static final 的。抽象类可以包含非 final 的变量。
- 在JDK1.8之前，接口中不能有静态方法，抽象类中可以有普通方法和静态方法；在JDK1.8后，接口中可以有默认方法和静态方法，并且有方法体。
- 抽象类可以有构造方法，但是不能直接被 new 关键字实例化。
- 在JDK1.8 前，抽象类的抽象方法默认访问权限为 protected，1.8默认访问权限为 default，共有 default、protected、public 三种修饰符，非抽象方法可以使用四种修饰符；在JDK1.8 前，接口方法默认为 public，1.8时默认为 public，此时可以使用 public 和 default，1.9时接口方法还支持 private。

12，简述下HashMap

HashMap底层采用了数组+链表的数据结构，数组是HashMap的主体，链表则是主要为了解决哈希冲突而存在的。

如果定位到的数组位置不含链表，那么执行查找、添加等操作很快，仅需一次寻址即可；如果定位到的数组包含链表，对于添加操作，其时间复杂度为O(n)，首先遍历链表，存在即覆盖，否则新增；对于查找操作来讲，仍需遍历链表，然后通过key对象的equals方法逐一比对查找。所以，性能考虑，HashMap中的链表出现越少，性能才会越好。

HashMap有4个构造器，其他构造器如果用户没有传入initialCapacity 和loadFactor这两个参数，会使用默认值initialCapacity默认为16，loadFactory默认为0.75。

```
public HashMap(int initialCapacity, float loadFactor) {
    //此处对传入的初始容量进行校验，最大不能超过MAXIMUM_CAPACITY = 1<<30(230)
    if (initialCapacity < 0)
        throw new IllegalArgumentException("Illegal initial capacity: " +
                                           initialCapacity);

    if (initialCapacity > MAXIMUM_CAPACITY)
        initialCapacity = MAXIMUM_CAPACITY;
    if (loadFactor <= 0 || Float.isNaN(loadFactor))
        throw new IllegalArgumentException("Illegal load factor: " +
                                           loadFactor);

    this.loadFactor = loadFactor;
    threshold = initialCapacity;

    init(); //init方法在HashMap中没有实际实现，不过在其子类如 LinkedHashMap中就会有对
应实现
}
```

加载因子存在的原因，还是因为减缓哈希冲突，如果初始桶为16，等到满16个元素才扩容，某些桶里可能就有不止一个元素了。所以加载因子默认为0.75，也就是说大小为16的HashMap，到了第13个元素，就会扩容成32。

Put过程

- 判断当前数组是否要初始化。
- 如果key为空，则put一个空值进去。
- 根据key计算出hashcode。
- 根据hashcode定位出在桶内的位置。
- 如果桶是链表，则需要遍历判断hashcode，如果key和原来的key是否相等，相等则进行覆盖，返回原来的值。
- 如果桶是空的，说明当前位置没有数据存入，新增一个 Entry 对象写入当前位置。当调用 addEntry 写入 Entry 时需要判断是否需要扩容。如果需要就进行两倍扩充，并将当前的 key 重新 hash 并定位。而在 createEntry中会将当前位置的桶传入到新建的桶中，如果当前桶有值就会在位置形成链表。

Get过程

- 根据key计算出hashcode，并定位到桶内的位置。
- 判断是不是链表，如果是则需要根据遍历直到 key 及 hashcode 相等时候就返回值，如果不是就根据 key、key 的 hashcode 是否相等来返回值。
- 如果啥也没取到就返回null。

JDK 1.8的HashMap底层采用的是链表+红黑树，增加一个阈值进行判断是否将链表转红黑树，HashEntry 修改为 Node，目的是解决hash冲突造成的链表越来越长、查询慢的问题。

Get过程

- 判断当前桶是不是空,空就需要初始化;
- 根据key, 计算出hashcode, 根据hashcode, 定位到具体的桶中, 并判断当前桶是不是为空,为空表明没有hash冲突创建一个新桶即可;
- 如果有hash冲突,那么就要比较当前桶中的 key、key 的 hashcode 与写入的 key 是否相等, 相等就赋值给 e,在第 8 步的时候会统一进行赋值及返回;
- 如果当前位置是红黑树, 就按照红黑树的方式写入数据;
- 如果当前位置是链表,则需要把key,value封装一个新的节点,添加到当前的桶后面(尾插法),形成链表;
- 接着判断当前链表的大小是否大于预设的阈值, 大于时就要转换为红黑树;
- 如果在遍历过程中找到 key 相同时直接退出遍历;
- 如果 e != null 就相当于存在相同的 key,那就需要将值覆盖;
- 最后判断是否需要扩容;

Get过程

- 首先将 key hash 之后取得所定位的桶。
- 如果桶为空则直接返回 null 。
- 否则判断桶的第一个位置(有可能是链表、红黑树)的 key 是否为查询的 key，是直接返回 value。
- 如果第一个不匹配，则判断它的下一个是红黑树还是链表。
- 红黑树就按照树的查找方式返回值。
- 不然就按照链表的方式遍历匹配返回值。

13, ConcurrentHashMap

JDK8中ConcurrentHashMap参考了JDK8 HashMap的实现，采用了数组+链表+红黑树的实现方式来设计，内部大量采用CAS操作，那什么是CAS。

CAS是compare and swap的缩写，中文称为【比较交换】。CAS是一种基于锁的操作，而且是乐观锁。在Java中锁分为乐观锁和悲观锁。悲观锁是将资源锁住，等一个之前获得锁的线程释放锁之后，下一个线程才可以访问。而乐观锁采取了一种宽泛的态度，通过某种方式不加锁来处理资源，性能较悲观锁有很大的提高。

CAS 操作包含三个操作数 —— 内存位置 (V)、预期原值 (A) 和新值(B)。如果内存地址里面的值和A的值是一样的，那么就将内存里面的值更新成B。CAS是通过无限循环来获取数据的，如果在第一轮循环中，a线程获取地址里面的值被b线程修改了，那么a线程需要自旋，到下次循环才有可能机会执行。

14，介绍下什么是乐观锁、悲观锁

Java 按照锁的实现分为乐观锁和悲观锁，乐观锁和悲观锁并不是一种真实存在的锁，而是一种设计思想。

悲观锁

悲观锁是一种悲观思想，它总认为最坏的情况可能会出现，它认为数据很可能会被其他人所修改，所以悲观锁在持有数据的时候总会把资源 或者 数据 锁住，这样其他线程想要请求这个资源的时候就会阻塞，直到等到悲观锁把资源释放为止。传统的关系型数据库里边就用到了很多这种锁机制，比如行锁，表锁等，读锁，写锁等，都是在做操作之前先上锁。悲观锁的实现往往依靠数据库本身的锁功能实现。

Java 中的 Synchronized 和 ReentrantLock 等独占锁(排他锁)也是一种悲观锁思想的实现，因为 Synchronized 和 ReentrantLock 不管是否持有资源，它都会尝试去加锁，生怕自己心爱的宝贝被别人拿走。

乐观锁

乐观锁的思想与悲观锁的思想相反，它总认为资源和数据不会被别人所修改，所以读取不会上锁，但是乐观锁在进行写入操作的时候会判断当前数据是否被修改过(具体如何判断我们下面再说)。乐观锁的实现方案一般来说有两种：版本号机制 和 CAS实现。乐观锁多适用于多度的应用类型，这样可以提高吞吐量。

在Java中java.util.concurrent.atomic包下面的原子变量类就是使用了乐观锁的一种实现方式CAS实现的。

15，谈谈对Java线程的理解

线程是进程中可独立执行的最小单位，也是 CPU 资源（时间片）分配的基本单位，同一个进程中的线程可以共享进程中的资源，如内存空间和文件句柄。线程有一些基本的属性，如id、name、以及priority。

id：线程 id 用于标识不同的线程，编号可能被后续创建的线程使用，编号是只读属性，不能修改。

name：线程的名称，默认值是 Thread-(id)

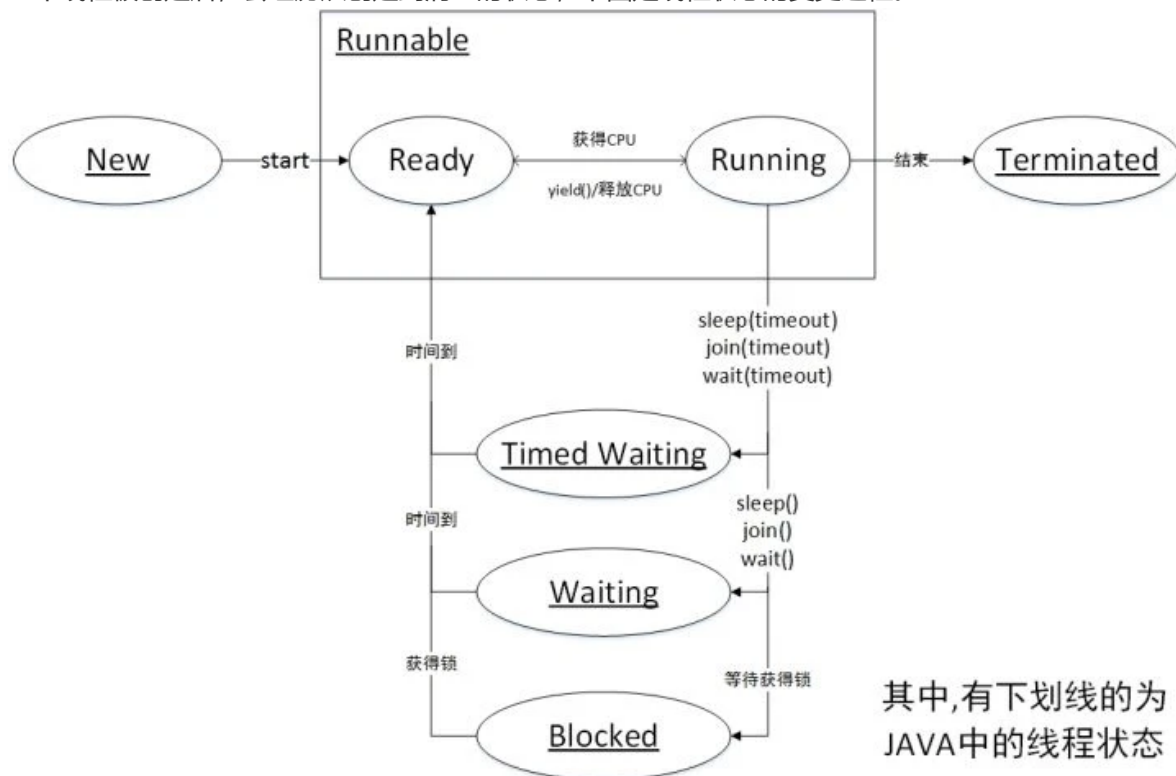
daemon：分为守护线程和用户线程，我们可以通过 setDaemon(true) 把线程设置为守护线程。守护线程通常用于执行不重要的任务，比如监控其他线程的运行情况，GC 线程就是一个守护线程。

setDaemon() 要在线程启动前设置，否则 JVM 会抛出非法线程状态异常，可被继承。

priority：线程调度器会根据这个值来决定优先运行哪个线程（不保证），优先级的取值范围为 1~10，默认值是 5，可被继承。Thread 中定义了下面三个优先级常量：

- 最低优先级：MIN_PRIORITY = 1
- 默认优先级：NORM_PRIORITY = 5
- 最高优先级：MAX_PRIORITY = 10

一个线程被创建后，会经历从创建到消亡的状态，下图是线程状态的变更过程。



下表是展示了线程的生命周期状态变化：

状态	说明
New	新创建了一个线程对象，但还没有调用start()方法。
Runnable	Ready 状态 线程对象创建后，其他线程(比如 main 线程) 调用了该对象的 start() 方法。该状态的线程位于可运行线程池中，等待被线程调度选中 获取 cpu 的使用权。Running 绪状态的线程在获得 CPU 时间片后变为运行中状态（running）。
Blocked	线程因为某种原因放弃了cpu 使用权（等待锁），暂时停止运行。
Waiting	线程进入等待状态因为以下几个方法：Object#wait()、Thread#join()、LockSupport#park()
Terminated	该线程已经执行完毕。

16, Synchronized、volatile、Lock并发

线程同步和并发通常会问到Synchronized、volatile、Lock的作用。其中，Lock是一个类，而其余两个则是Java关键字。

Synchronized

Synchronized是Java的关键字，也是Java的内置特性，在JVM层面实现了对临界资源的同步互斥访问，通过对对象的头文件来操作，从而达到加锁和释放锁的目的。使用Synchronized修饰的代码或方法，通常有如下特性：

- Synchronized在发生异常时，会自动释放线程占有的锁，因此不会导致死锁现象发生。
- 不能响应中断。
- 同一时刻不管是读还是写都只能有一个线程对共享资源操作，其他线程只能等待，性能不高。

正是因为上面的特性，所以Synchronized的缺点也是显而易见的：即如果一个代码块被synchronized修饰了，当一个线程获取了对应的锁，并执行该代码块时，其他线程便只能一直等待，因此效率很低。

volatile

保证了不同线程对这个变量进行操作时的可见性，即一个线程修改了某个变量的值，这新值对其他线程来说是立即可见的。并且volatile是禁止进行指令重排序。

所谓指令重排序，指的是处理器为了提高程序运行效率，可能会对输入代码进行优化，它不保证程序中各个语句的执行先后顺序同代码中的顺序一致，但是它会保证程序最终执行结果和代码顺序执行的结果是一致的。

volatile为了保证原子性，必须具备以下条件：

- 对变量的写操作不依赖于当前值
- 该变量没有包含在具有其他变量的不变式中

17, 锁

按照作用的不同，Java的锁可以分为如下：



悲观锁、乐观锁

悲观锁认为自己在在使用数据的时候一定有别的线程来修改数据，因此在获取数据的时候会先加锁，确保数据不会被别的线程修改。Java 中，synchronized 关键字和 Lock 的实现类都是悲观锁。悲观锁适合写操作多的场景，先加锁可以保证写操作时数据正确。

而乐观锁认为自己在在使用数据时不会有别的线程修改数据，所以不会添加锁，只是在更新数据的时候去判断之前有没有别的线程更新了这个数据。如果这个数据没有被更新，当前线程将自己修改的数据成功写入。如果数据已经被其他线程更新，则根据不同的实现方式执行不同的操作（例如报错或者自动重试）。乐观锁在 Java 中是通过使用无锁编程来实现，最常采用的是 CAS 算法，Java 原子类中的递增操作就通过 CAS 自旋实现。乐观锁适合读操作多的场景，不加锁的特点能够使其读操作的性能大幅提升。

这里说到了CAS算法，那么什么是CAS算法呢？

CAS算法

一个线程失败或挂起并不会导致其他线程也失败或挂起，那么这种算法就被称为非阻塞算法。而CAS就是一种非阻塞算法实现，也是一种乐观锁技术，它能在不使用锁的情况下实现多线程安全，因此是一种无锁算法。

CAS算法的定义：CAS的主要作用是不使用加锁就可以实现线程安全，CAS 算法又称为比较交换算法，是一种实现并发算法时常用到的技术，Java并发包中的很多类都使用了CAS技术。CAS具体包括三个参数：当前内存值V、旧的预期值A、即将更新的值B，当且仅当预期值A和内存值V相同时，将内存值修改为B并返回true，否则什么都不做，并返回false。

原子更新的基本操作包括：

- AtomicBoolean：原子更新布尔变量；
- AtomicInteger：原子更新整型变量；
- AtomicLong：原子更新长整型变量；

以AtomicInteger为例，代码如下：

```
public class AtomicInteger extends Number implements java.io.Serializable {
    //返回当前的值
    public final int get() {
        return value;
    }
    //原子更新为新值并返回旧值
    public final int getAndSet(int newValue) {
        return unsafe.getAndSetInt(this, valueOffset, newValue);
    }
    //最终会设置成新值
    public final void lazySet(int newValue) {
        unsafe.putOrderedInt(this, valueOffset, newValue);
    }
    //如果输入的值等于预期值，则以原子方式更新为新值
    public final boolean compareAndSet(int expect, int update) {
        return unsafe.compareAndSwapInt(this, valueOffset, expect, update);
    }
    //原子自增
    public final int getAndIncrement() {
        return unsafe.getAndAddInt(this, valueOffset, 1);
    }
    //原子方式将当前值与输入值相加并返回结果
    public final int getAndAdd(int delta) {
        return unsafe.getAndAddInt(this, valueOffset, delta);
    }
}
```

再如，下面是使用多线程对一个int值进行自增操作的代码，如下所示。

```
public class AtomicIntegerDemo {

    private static AtomicInteger atomicInteger = new AtomicInteger(0);

    public static void main(String[] args){
        for (int i = 0; i < 5; i++){
            new Thread(new Runnable() {
```

```

        public void run() {
            //调用AtomicInteger的getAndIncrement返回的是增加之前的值
            System.out.println(atomicInteger.getAndIncrement());
        }
    }).start();
}
System.out.println(atomicInteger.get());
}
}

```

自旋锁、适应性自旋锁

阻塞或唤醒一个 Java 线程需要操作系统切换 CPU 状态来完成，这种状态转换需要耗费处理器时间。在许多场景中，同步资源的锁定时间很短，为了这一小段时间去切换线程，线程挂起和恢复现场的花费可能会让系统得不偿失。如果物理机器有多个处理器，能够让两个或以上的线程同时并行执行，我们就可以让后面那个请求锁的线程不放弃CPU的执行时间，看看持有锁的线程是否很快就会释放锁。

而为了让当前线程【稍等一下】，我们需让当前线程进行自旋，如果在自旋完成后前面锁定同步资源的线程已经释放了锁，那么当前线程就可以不必阻塞而是直接获取同步资源，从而避免切换线程的开销，这就是自旋锁。

死锁

当前线程拥有其他线程需要的资源，当前线程等待其他线程已拥有的资源，都不放弃自己拥有的资源。

18，谈谈你对Java 反射的理解

所谓反射，指的是在运行状态中，对于任意一个类，都能够获取这个类的所有属性和方法；对于任意一个对象，都能够调用它的任意一个方法和属性，而这种动态获取的信息以及动态调用对象的方法的功能就被称为Java语言的反射机制。

使用反射前需要事先获取到的字节码，在Java中，获取字节码的方式有三种：

1. Class.forName(className)
2. 类名.class
3. this.getClass()

19，注解

Java 语言中的类、方法、变量、参数和包等都可以被标注。和 Javadoc 不同，Java 标注可以通过反射获取标注内容。根据作用时机的不同，Java的注解可以分为三种：

- **SOURCE**：注解将被编译器丢弃（该类型的注解信息只会保留在源码里，源码经过编译后，注解信息会被丢弃，不会保留在编译好的class文件里），如 @Override。
- **CLASS**：注解在class文件中可用，但会被 VM 丢弃（该类型的注解信息会保留在源码里和 class 文件里，在执行的时候，不会加载到虚拟机中），请注意，当注解未定义 Retention 值时，默认值是 CLASS。
- **RUNTIME**：注解信息将在运行期 (JVM) 也保留，因此可以通过反射机制读取注解的信息（源码、class 文件和执行的时候都有注解的信息），如 @Deprecated。

20，单例

为了保证只有一个对象存在，可以使用单例模式，网上有，单例模式的七种写法。我们介绍一下常见的几种：

懒汉式

懒汉式使用的是static关键字，因此是线程不安全的。

```

public class Singleton {
    private static Singleton instance;
    private Singleton (){}
    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}

```

如果要线程安全，那么需要使用synchronized关键字。

```

public class Singleton {
    private static Singleton instance;
    private Singleton (){}
    public static synchronized Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}

```

不过，使用synchronized锁住之后，运行效率明显降低。

静态内部类

静态内部类利用了classloader的机制来保证初始化instance时只有一个线程。

```

public class Singleton {
    private static class SingletonHolder {
        private static final Singleton INSTANCE = new Singleton();
    }
    private Singleton (){}
    public static final Singleton getInstance() {
        return SingletonHolder.INSTANCE;
    }
}

```

双重校验锁

```

public class Singleton {
    private volatile static Singleton singleton;
    private Singleton (){}
    public static Singleton getSingleton() {
        if (singleton == null) {
            synchronized (Singleton.class) {
                if (singleton == null) {
                    singleton = new Singleton();
                }
            }
        }
        return singleton;
    }
}

```

双重检查锁定是synchronized的升级的写法，那为什么要使用volatile关键字呢，是为了禁止初始化实例时的重排序。我们知道，初始化一个实例在java字节码中会有4个步骤：

1. 申请内存空间
2. 初始化默认值（区别于构造器方法的初始化）
3. 执行构造器方法
4. 连接引用和实例

而后两步是有可能重排序，而使用volatile可以禁止指令重排序。

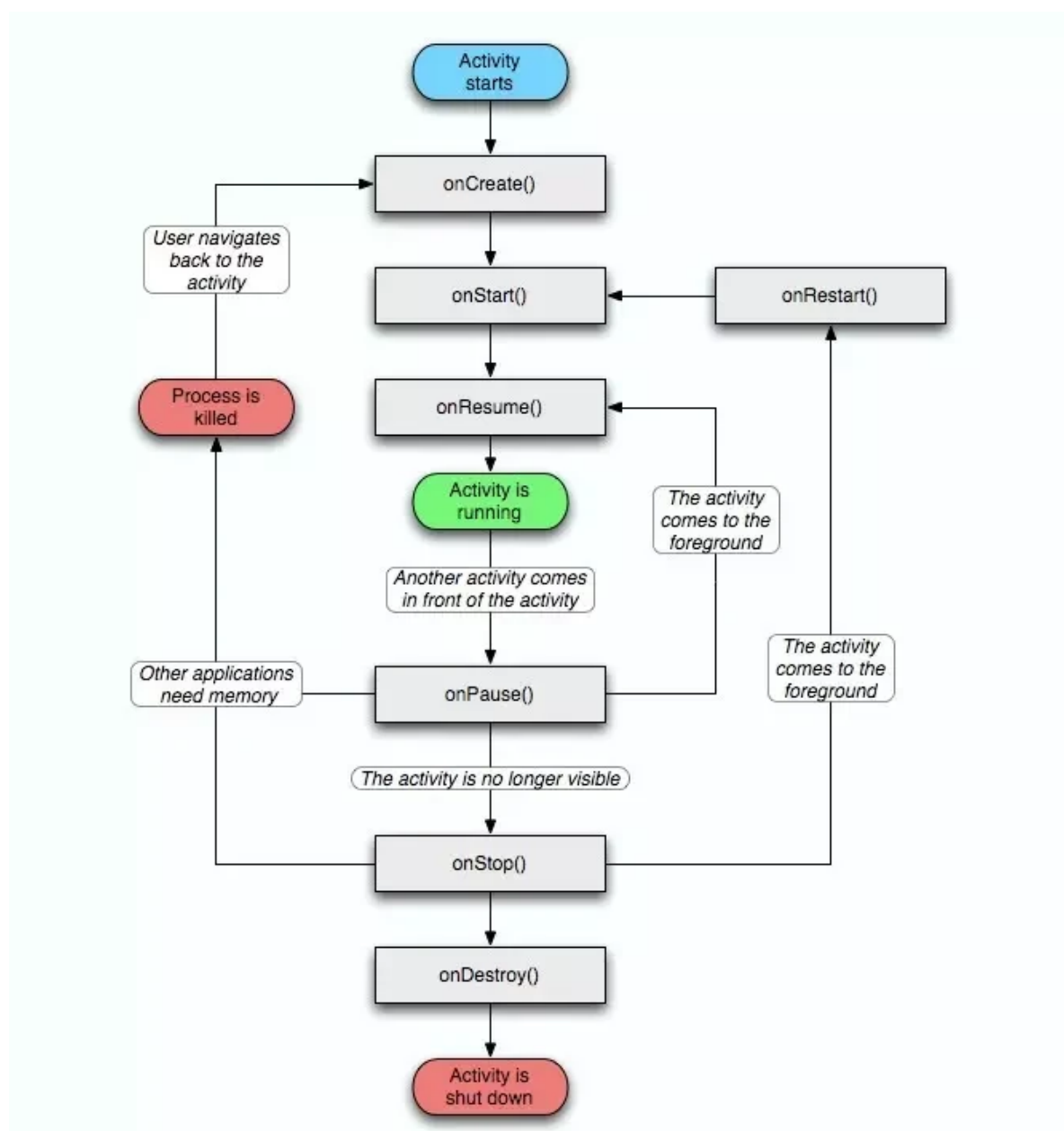
第二章 Android 面试之必问Android基础

1、Activity

1.1 生命周期

正常情况系，Activity会经历如下几个阶段：

- onCreate：表示Activity正在被创建。
- onRestart：表示Activity正在被重新启动。
- onStart：表示Activity正在被启动，这时已经可见，但没有出现在前台无法进行交互。
- onResume：表示Activity已经可见，并且处于前台。
- onPause：表示Activity正在停止（可做一次保存状态停止动画等非耗时操作）。
- onStop：表示Activity即将停止（可进行重量级回收工作）。
- onDestroy：表示Activity即将被销毁。



对于生命周期，通常还会问如下的一些问题：

- 第一次启动： `onCreate`->`onStart`->`onResume`；
- 打开新的Activity或者返回桌面： `onPause`->`onStop`。如果打开新的Activity为透明主题，则不会调用`onStop`；
- 当回到原来Activity时： `onRestart`->`onStart`->`onResume`；
- 当按下返回键： `onPause`->`onStop`->`onDestroy`

1.2 启动模式

Activity的启动模式有四种：Standard、SingleTop、SingleTask和SingleInstance。

- **Standard**：标准模式，也是默认模式。每次启动都会创建一个全新的实例。
- **SingleTop**：栈顶复用模式。这种模式下如果Activity位于栈顶，不会新建实例。`onNewIntent`会被调用，接收新的请求信息，不会再低啊用`onCreate`和`onStart`。
- **SingleTask**：栈内复用模式。升级版singleTop，如果栈内有实例，则复用，并会将该实例之上的Activity全部清除。
- **SingleInstance**：系统会为它创建一个单独的任务栈，并且这个实例独立运行在一个task中，这个task只有这个实例，不允许有别的Activity存在（可以理解为手机内只有一个）。

1.3 启动流程

在理解Activity的启动流程之前，先让我们来看一下Android系统启动流程。总的来说，Android系统启动流程的主要经历init进程 -> Zygote进程 -> SystemServer进程 -> 各种系统服务 -> 应用进程等阶段。

1. **启动电源以及系统启动**：当电源按下时引导芯片从预定义的地方（固化在ROM）开始执行，加载引导程序BootLoader到RAM，然后执行。
2. **引导程序BootLoader**：BootLoader是在Android系统开始运行前的一个小程序，主要用于把系统OS拉起来并运行。
3. **Linux内核启动**：当内核启动时，设置缓存、被保护存储器、计划列表、加载驱动。当其完成系统设置时，会先在系统文件中寻找init.rc文件，并启动init进程。
4. **init进程启动**：初始化和启动属性服务，并且启动Zygote进程。
5. **Zygote进程启动**：创建JVM并为其注册JNI方法，创建服务器端Socket，启动SystemServer进程。
6. **SystemServer进程启动**：启动Binder线程池和SystemServiceManager，并且启动各种系统服务。
7. **Launcher启动**：被SystemServer进程启动的AMS会启动Launcher，Launcher启动后会将已安装应用的快捷图标显示到系统桌面上。

参考：

[Android系统启动流程之init进程启动](#)

[Android系统启动流程之Zygote进程启动](#)

[Android系统启动流程之SystemServer进程启动](#)

[Android系统启动流程之Launcher进程启动](#)

Launcher进程启动后，就会调用Activity的启动了。首先，Launcher会调用ActivityTaskManagerService，然后ActivityTaskManagerService会调用ApplicationThread，然后ApplicationThread再通过ActivityThread启动Activity，完整的分析可以参考[Android之Activity启动流程](#)

2, Fragment

2.1 简介

Fragment，是Android 3.0 (API 11) 提出的，为了兼容低版本，support-v4库中也开发了一套Fragment API，最低兼容Android 1.6，如果要在最新的版本中使用Fragment，需要引入AndroidX的包。

相比Activity，Fragment具有如下一些特点：

- **模块化 (Modularity)**：我们不必把所有代码全部写在Activity中，而是把代码写在各自的Fragment中。
- **可重用 (Reusability)**：多个Activity可以重用一個Fragment。
- **可适配 (Adaptability)**：根据硬件的屏幕尺寸、屏幕方向，能够方便地实现不同的布局，这样用户体验更好。

Fragment有如下几个核心的类：

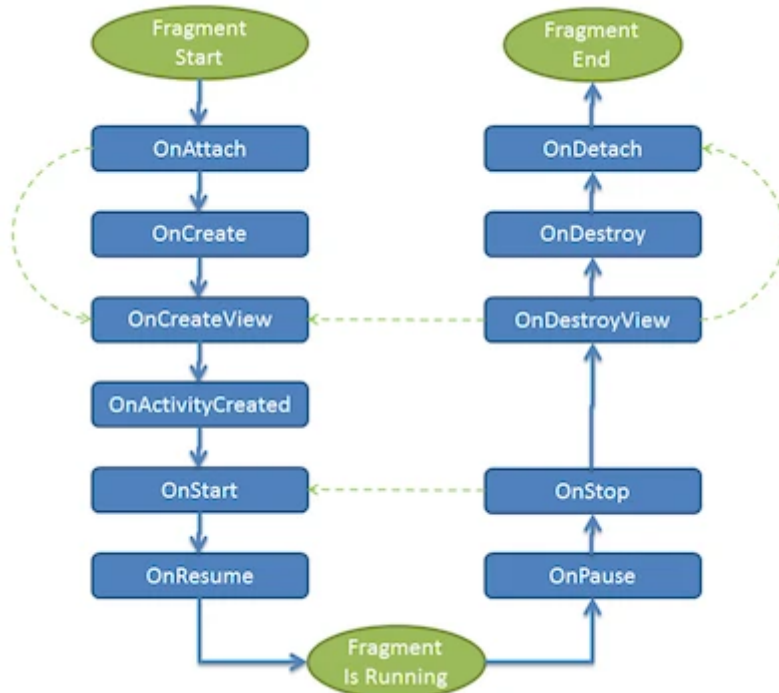
- **Fragment**：Fragment的基类，任何创建的Fragment都需要继承该类。
- **FragmentManager**：管理和维护Fragment。他是抽象类，具体的实现类是FragmentManagerImpl。
- **FragmentTransaction**：对Fragment的添加、删除等操作都需要通过事务方式进行。他是抽象类，具体的实现类是BackStackRecord。

2.2 生命周期

Fragment必须是依存于Activity而存在的，因此Activity的生命周期会直接影响到Fragment的生命周期。相比Activity的生命周期，Fragment的生命周期如下所示。

- **onAttach()**: Fragment和Activity相关联时调用。如果不是一定要使用具体的宿主 Activity 对象的话, 可以使用这个方法或者getContext()获取 Context 对象, 用于解决Context上下文引用的问题。同时还可以在此方法中可以通过getArguments()获取到需要在Fragment创建时需要的参数。
- **onCreate()**: Fragment被创建时调用。
- **onCreateView()**: 创建Fragment的布局。
- **onActivityCreated()**: 当Activity完成onCreate()时调用。
- **onStart()**: 当Fragment可见时调用。
- **onResume()**: 当Fragment可见且可交互时调用。
- **onPause()**: 当Fragment不可交互但可见时调用。
- **onStop()**: 当Fragment不可见时调用。
- **onDestroyView()**: 当Fragment的UI从视图结构中移除时调用。
- **onDestroy()**: 销毁Fragment时调用。
- **onDetach()**: 当Fragment和Activity解除关联时调用。

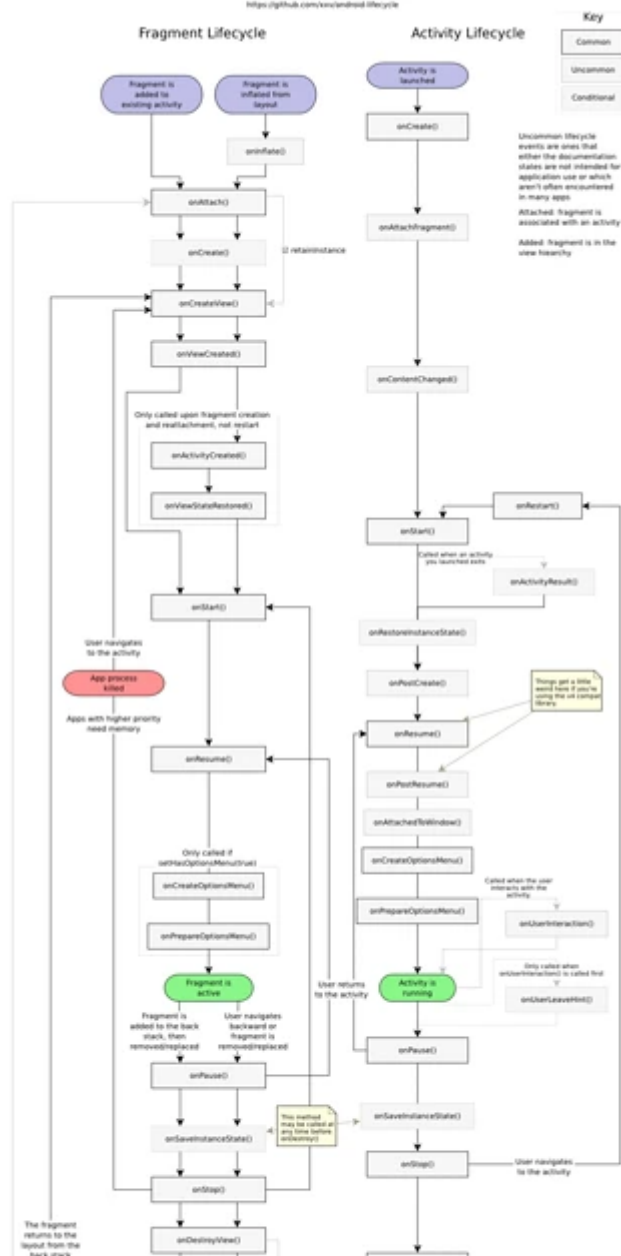
如下图所示。

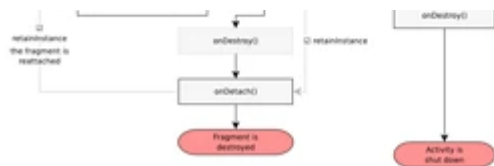


下面是Activity的生命周期和Fragment的各个生命周期方法的对应关系。

The Complete Android Activity/Fragment Lifecycle

© 2014-2015 Steve Pomeroy <stev@pomeroy.com>
CC BY-SA 4.0
<https://github.com/stevepomeroy/android-lifecycle>





2.3 与Activity传递数据

2.3.1 Fragment向Activity传递数据

首先，在Fragment中定义接口，并让Activity实现该接口，如下所示。

```
public interface OnFragmentInteractionListener {
    void onItemClick(String str);
}
```

然后，在Fragment的onAttach()中，将参数Context强转为OnFragmentInteractionListener对象传递过去。

```
public void onAttach(Context context) {
    super.onAttach(context);
    if (context instanceof OnFragmentInteractionListener) {
        mListener = (OnFragmentInteractionListener) context;
    } else {
        throw new RuntimeException(context.toString()
            + " must implement OnFragmentInteractionListener");
    }
}
```

2.3.2 Activity向Fragment传递数据

在创建Fragment的时候，可以通过setArguments(Bundle bundle)方式将值传递给Activity，如下所示。

```
public static Fragment newInstance(String str) {
    FragmentTest fragment = new FragmentTest();
    Bundle bundle = new Bundle();
    bundle.putString(ARG_PARAM, str);
    fragment.setArguments(bundle); // 设置参数
    return fragment;
}
```

3, Service

3.1 启动方式

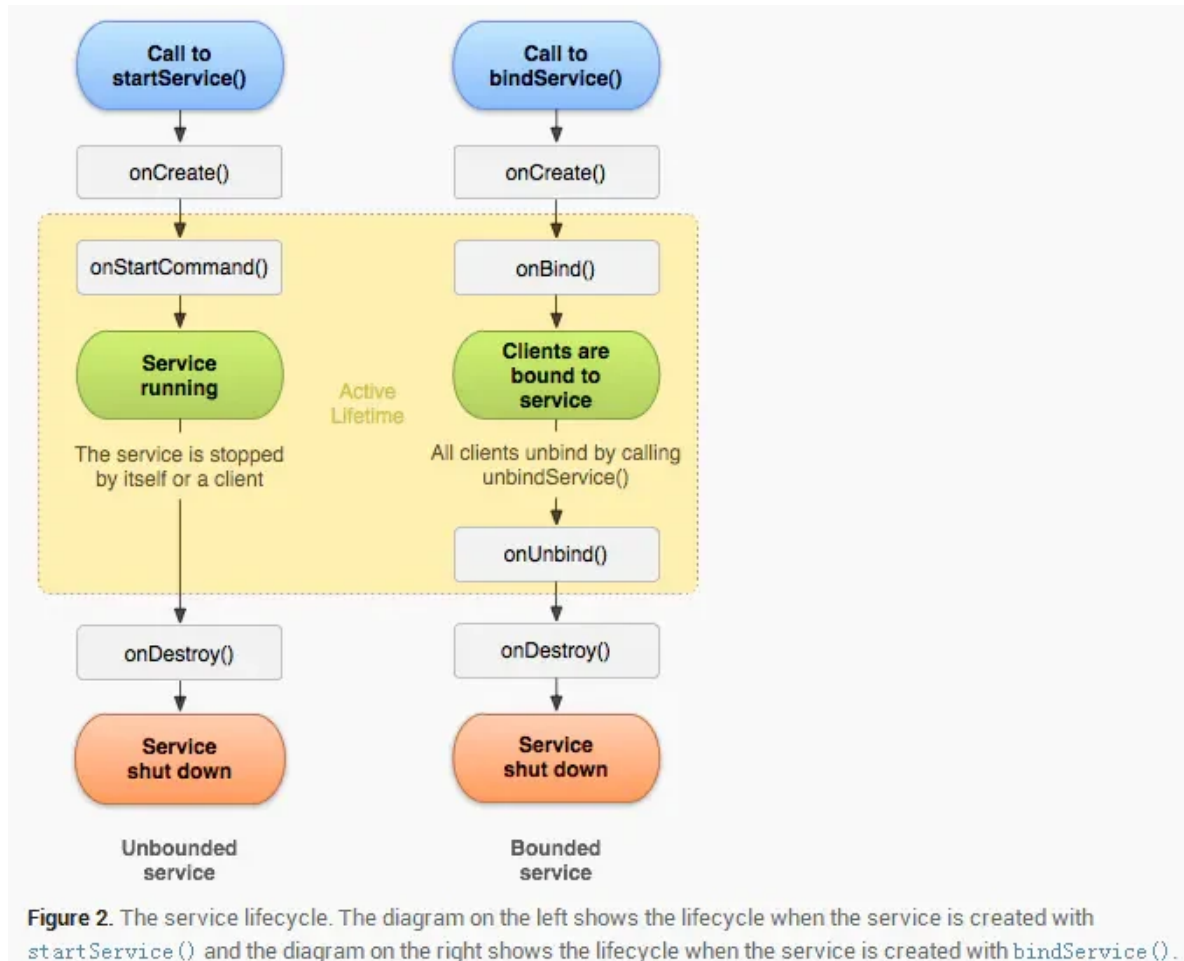
Service的启动方式主要有两种，分别是startService和bindService。

其中，startService使用的是同一个Service，因此onStart()会执行多次，onCreate()只执行一次，onStartCommand()也会执行多次。使用bindService启动时，onCreate()与onBind()都只会调用一次。

使用startService启动时是单独开一个服务，与Activity没有任何关系，而bindService方式启动时，Service会和Activity进行绑定，当对应的activity销毁时，对应的Service也会销毁。

3.2 生命周期

下图是startService和bindService两种方式启动Service的示意图。



3.2.1 startService

- **onCreate():** 如果service没被创建过，调用startService()后会执行onCreate()回调；如果service已处于运行中，调用startService()不会执行onCreate()方法。
- **onStartCommand():** 多次执行了Context的startService()方法，那么Service的onStartCommand()方法也会相应的多次调用。
- **onBind():** Service中的onBind()方法是抽象方法，Service类本身就是抽象类，所以onBind()方法是必须重写的，即使我们用不到。
- **onDestroy():** 在销毁Service的时候该方法。

```
public class TestOneService extends Service{

    @Override
    public void onCreate() {
        Log.i("Kathy","onCreate - Thread ID = " +
        Thread.currentThread().getId());
        super.onCreate();
    }

    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        Log.i("Kathy", "onStartCommand - startId = " + startId + ", Thread ID =
        " + Thread.currentThread().getId());
        return super.onStartCommand(intent, flags, startId);
    }

    @Nullable
    @Override
```

```

    public IBinder onBind(Intent intent) {
        Log.i("Kathy", "onBind - Thread ID = " +
            Thread.currentThread().getId());
        return null;
    }

    @Override
    public void onDestroy() {
        Log.i("Kathy", "onDestroy - Thread ID = " +
            Thread.currentThread().getId());
        super.onDestroy();
    }
}

```

3.2.2 bindService

bindService启动的服务和调用者之间是典型的Client-Server模式。调用者是client，Service则是Server端。Service只有一个，但绑定到Service上面的Client可以有一个或很多个。bindService启动服务的生命周期与其绑定的client息息相关。

- 1，首先，在Service的onBind()方法中返回IBinder类型的实例。
- 2，onBind()方法返回的IBinder的实例需要能够返回Service实例本身。

3.3 Service不被杀死

现在，由于系统API的限制，一些常见的不被杀死Service方式已经过时，比如下面是之前的一些方式。

3.3.1， onStartCommand方式中，返回START_STICKY。

调用Context.startService方式启动Service时，如果Android面临内存匮乏，可能会销毁当前运行的Service，待内存充足时可以重建Service。而Service被Android系统强制销毁并再次重建的行为依赖于Service的onStartCommand()方法的返回值，常见的返回值有如下一些。

START_NOT_STICKY：如果返回START_NOT_STICKY，表示当Service运行的进程被Android系统强制杀掉之后，不会重新创建该Service。

START_STICKY：如果返回START_STICKY，表示Service运行的进程被Android系统强制杀掉之后，Android系统会将该Service依然设置为started状态（即运行状态），但是不再保存onStartCommand方法传入的intent对象，即获取不到intent的相关信息。

START_REDELIVER_INTENT：如果返回START_REDELIVER_INTENT，表示Service运行的进程被Android系统强制杀掉之后，与返回START_STICKY的情况类似，Android系统会将再次重新创建该Service，并执行onStartCommand回调方法，但是不同的是，Android系统会再次将Service在被杀掉之前最后一次传入onStartCommand方法中的Intent再次保留下来并再次传入到重新创建后的Service的onStartCommand方法中，这样我们就能读取到intent参数。

4， BroadcastReceiver

4.1 BroadcastReceiver是什么

BroadcastReceiver，广播接收者，它是一个系统全局的监听器，用于监听系统全局的Broadcast消息，所以它可以很方便的进行系统组件之间的通信。BroadcastReceiver属于系统级的监听器，它拥有自己的进程，只要存在与之匹配的Broadcast被以Intent的形式发送出来，BroadcastReceiver就会被激活。

和其他的四大组件一样，BroadcastReceiver也有自己独立的声明周期，但是它又和Activity、Service不同。当在系统注册一个BroadcastReceiver之后，每次系统以一个Intent的形式发布Broadcast的时候，系统都会创建与之对应的BroadcastReceiver广播接收者实例，并自动触发它的onReceive()方法，当onReceive()方法被执行完成之后，BroadcastReceiver的实例就会被销毁。

从不同的纬度区分，BroadcastReceiver可以分为不同的类别。

- 系统广播/非系统广播
- 全局广播/本地广播
- 无序广播/有序广播/粘性广播

4.2 基本使用

4.2.1 注册广播

广播的注册分为静态注册和动态注册。静态注册是在Manifest清单文件中进行注册，比如。

```
<receiver android:name=".MyBroadcastReceiver" android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.BOOT_COMPLETED"/>
        <action android:name="android.intent.action.INPUT_METHOD_CHANGED" />
    </intent-filter>
</receiver>
```

动态注册是在代码中，使用registerReceiver方法代码进行注册，比如。

```
val br: BroadcastReceiver = MyBroadcastReceiver()
val filter = IntentFilter(ConnectivityManager.CONNECTIVITY_ACTION).apply {
    addAction(Intent.ACTION_AIRPLANE_MODE_CHANGED)
}
registerReceiver(br, filter)
```

4.2.2 发送广播

然后，我们使用sendBroadcast方法发送广播。

```
Intent().also { intent ->
    intent.setAction("com.example.broadcast.MY_NOTIFICATION")
    intent.putExtra("data", "Notice me senpai!")
    sendBroadcast(intent)
}
```

4.2.3 接收广播

发送广播的时候，我们会添加一个发送的标识，那么接收的时候使用这个标识接收即可。接收广播需要继承BroadcastReceiver，并重写onReceive回调方法接收广播数据。

```
private const val TAG = "MyBroadcastReceiver"

class MyBroadcastReceiver : BroadcastReceiver() {

    override fun onReceive(context: Context, intent: Intent) {
        StringBuilder().apply {
            append("Action: ${intent.action}\n")
            append("URI: ${intent.toUri(Intent.URI_INTENT_SCHEME)}\n")
            toString().also { log ->
                Log.d(TAG, log)
                Toast.makeText(context, log, Toast.LENGTH_LONG).show()
            }
        }
    }
}
```

```
}
```

5, ContentProvider

ContentProvider是Android四大组件之一，不过平时使用的机会比较少。如果你看过它的底层源码，那么就应该知道ContentProvider是通过Binder进行数据共享。因此，如果我们需要对第三方应用提供数据，可以考虑使用ContentProvider实现。

6, Android View知识点

Android本身的View体系非常庞大的，如果要完全看懂View的原理是很困难的，我们这里捡一些比较重要的概念来给大家讲解。

6.1 测量流程

Android View本身的绘制流程需要经过measure测量、layout布局、draw绘制三个过程，最终才能够将其绘制出来并展示在用户面前。

首先，我们看一下Android的MeasureSpec，Android的MeasureSpec分为3中模式，分别是EXACTLY、AT_MOST 和 UNSPECIFIED，含义如下。

- **MeasureSpec.EXACTLY**：精确模式，在这种模式下，尺寸的值是多少组件的长或宽就是多少。
- **MeasureSpec.AT_MOST**：最大模式，由父组件能够给出的最大的空间决定。
- **MeasureSpec.UNSPECIFIED**：未指定模式，当前组件可以随便使用空间，不受限制。

相关文章：[谈谈对 MeasureSpec 的理解](#)

6.2 事件分发

Android的事件分发由dispatchTouchEvent、onInterceptTouchEvent和onTouchEvent三个方法构成。

- **dispatchTouchEvent**：方法返回值为true，表示事件被当前视图消费掉；返回为super.dispatchTouchEvent表示继续分发该事件，返回为false表示交给父类的onTouchEvent处理。
- **onInterceptTouchEvent**：方法返回值为true，表示拦截这个事件并交由自身的onTouchEvent方法进行消费；返回false表示不拦截，需要继续传递给子视图。如果return super.onInterceptTouchEvent(ev)，事件拦截分两种情况：即一种是有子View的情况，另一种是没有子View的情况。
如果该View存在子View且点击到了该子View，则不拦截,继续分发
给子View 处理，此时相当于return false。如果该View没有子View或者有子View但是没有点击中子View，则交由该View的onTouchEvent响应，此时相当于return true。
- **onTouchEvent**：方法返回值为true表示当前视图可以处理对应的事件；返回值为false表示当前视图不处理这个事件，它会被传递给父视图的onTouchEvent方法进行处理。如果return super.onTouchEvent(ev)，事件处理分为两种情况，即自己消费还是还是向上传递。

在Android系统中，拥有事件传递处理能力的类有以下三种：

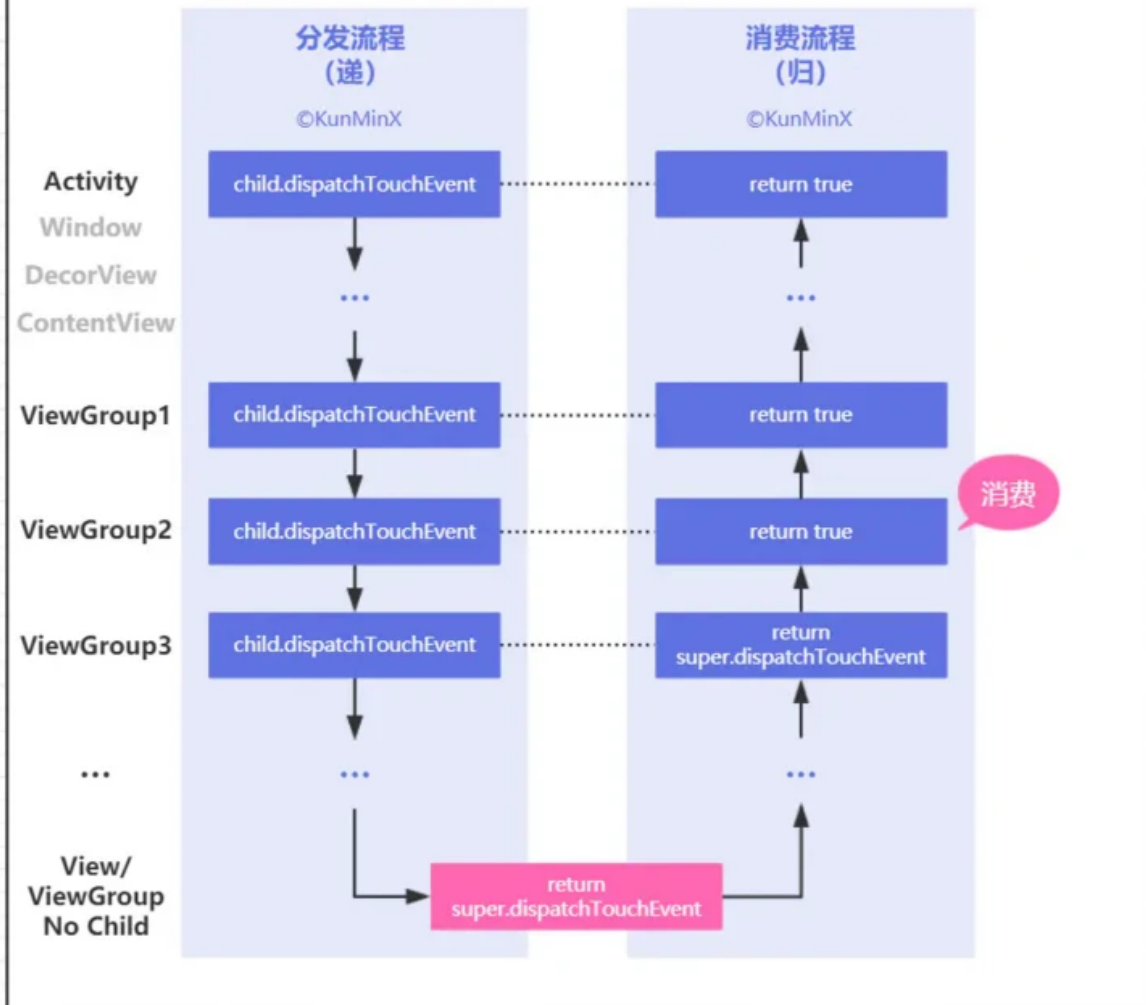
- Activity：拥有分发和消费两个方法。
- ViewGroup：拥有分发、拦截和消费三个方法。
- View：拥有分发、消费两个方法。

在事件分发中，有时候会问：**ACTION_CANCEL**什么时候触发，触摸button然后滑动到外部抬起会触发点击事件吗，再滑动回去抬起会么？

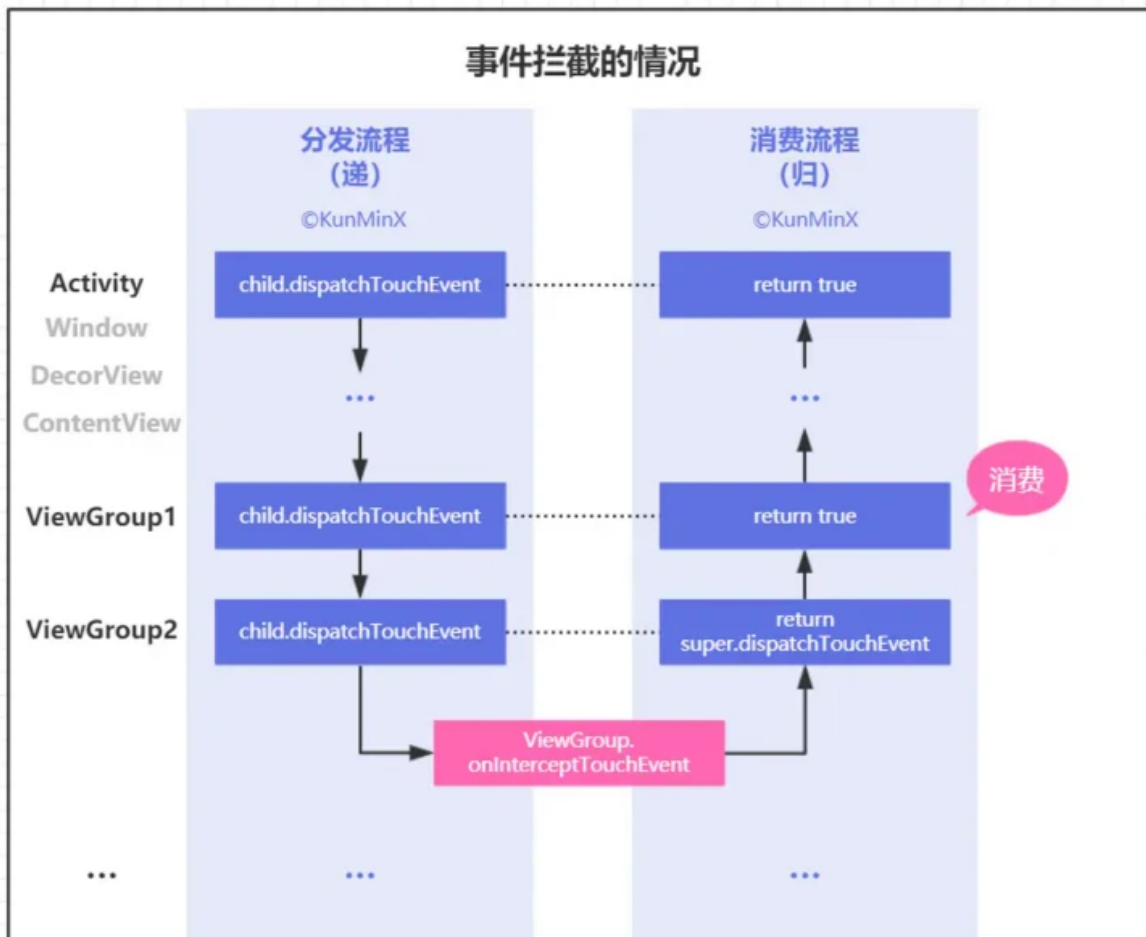
对于这个问题，我们需要明白以下内容：

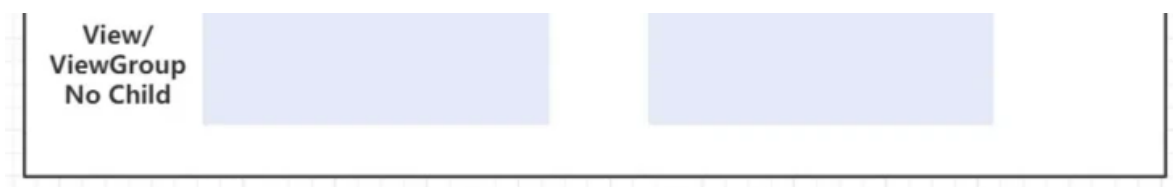
- 一般ACTION_CANCEL和ACTION_UP都作为View一段事件处理的结束。如果在父View中拦截ACTION_UP或ACTION_MOVE，在第一次父视图拦截消息的瞬间，父视图指定子视图不接受后续消息了，同时子视图会收到ACTION_CANCEL事件。
- 如果触摸某个控件，但是又不是在这个控件的区域上抬起，也会出现ACTION_CANCEL。

一次事件分发的全流程



事件拦截的情况





- ViewGroup 默认不拦截任何事件。ViewGroup 的 `onInterceptTouchEvent` 方法默认返回 `false`。
- View 没有 `onInterceptTouchEvent` 方法，一旦有点击事件传递给它，`onTouchEvent` 方法就会被调用。
- View 在可点击状态下，`onTouchEvent` 默认会消耗事件。
- `ACTION_DOWN` 被拦截了，`onInterceptTouchEvent` 方法执行一次后，就会留下记号 (`mFirstTouchTarget == null`) 那么往后的 `ACTION_MOVE` 和 `ACTION_UP` 都会拦截。

6.3 MotionEvent

Android的MotionEvent事件主要有以下几个：

- `ACTION_DOWN` 手指刚接触到屏幕
- `ACTION_MOVE` 手指在屏幕上移动
- `ACTION_UP` 手机从屏幕上松开的一瞬间
- `ACTION_CANCEL` 触摸事件取消

下面是事件的举例：点击屏幕后松开，事件序列为 `DOWN -> UP`，点击屏幕滑动松开，事件序列为 `DOWN -> MOVE -> ... -> MOVE -> UP`。同时，`getX/getY` 返回相对于当前View左上角的坐标，`getRawX/getRawY` 返回相对于屏幕左上角的坐标。`TouchSlop`是系统所能识别出的被认为滑动的最小距离，不同设备值可能不相同，可通过`ViewConfiguration.get(getContext()).getScaledTouchSlop()` 获取。

6.4 Activity、Window、DecorView之间关系

首先，来看一下Activity中`setContentView`的源代码。

```
public void setContentView(@LayoutRes int layoutResID) {
    //将xml布局传递到window当中
    getWindow().setContentView(layoutResID);
    initWindowDecorActionBar();
}
```

可以看到，Activity的 `setContentView` 实质是将 View 传递到 Window 的 `setContentView()` 方法中，Window 的 `setContentView` 会在内部调用 `installDecor()` 方法创建 `DecorView`，代码如下。

```
public void setContentView(int layoutResID) {
    if (mContentParent == null) {
        //初始化DecorView以及其内部的content
        installDecor();
    } else if (!hasFeature(FEATURE_CONTENT_TRANSITIONS)) {
        mContentParent.removeAllViews();
    }
    if (hasFeature(FEATURE_CONTENT_TRANSITIONS)) {
        .....
    } else {
        //将contentView加载到DecorView当中
        mLayoutInflater.inflate(layoutResID, mContentParent);
    }
    .....
}
```

```

private void installDecor() {
    .....
    if (mDecor == null) {
        //实例化DecorView
        mDecor = generateDecor(-1);
        .....
    }
    else {
        mDecor.setWindow(this);
    }
    if (mContentParent == null) {
        //获取Content
        mContentParent = generateLayout(mDecor);
    }
    .....
}
protected DecorView generateDecor(int featureId) {
    .....
    return new DecorView(context, featureId, this, getAttributes());
}

```

通过 generateDecor()的new一个 DecorView，然后调用 generateLayout()获取 DecorView中 content，最终通过 inflate将 Activity视图添加到 DecorView中的 content中，但此时 DecorView还未被添加到 Window中。添加操作需要借助 ViewRootImpl。

ViewRootImpl的作用是用来衔接 WindowManager和 DecorView，在 Activity被创建后会通过 WindowManager将 DecorView添加到 PhoneWindow中并且创建 ViewRootImpl实例，随后将 DecorView与 ViewRootImpl进行关联，最终通过执行 ViewRootImpl的 performTraversals()开启整个 View树的绘制。

6.5 Draw 绘制流程

Android的Draw过程可以分为六个步骤：

1. 首先，绘制View的背景；
2. 如果需要的话，保持canvas的图层，为fading做准备；
3. 然后，绘制View的内容；
4. 接着，绘制View的子View；
5. 如果需要的话，绘制View的fading边缘并恢复图层；
6. 最后，绘制View的装饰(例如滚动条等等)。

涉及到的代码如下：

```

public void draw(Canvas canvas) {
    ...
    // 步骤一：绘制View的背景
    drawBackground(canvas);
    ...
    // 步骤二：如果需要的话，保持canvas的图层，为fading做准备
    saveCount = canvas.getSaveCount();
    ...
    canvas.saveLayer(left, top, right, top + length, null, flags);
    ...
    // 步骤三：绘制View的内容
    onDraw(canvas);
    ...
    // 步骤四：绘制View的子View

```

```

dispatchDraw(canvas);
...
// 步骤五: 如果需要的话, 绘制View的fading边缘并恢复图层
canvas.drawRect(left, top, right, top + length, p);
...
canvas.restoreToCount(saveCount);
...
// 步骤六: 绘制View的装饰(例如滚动条等等)
onDrawForeground(canvas)
}

```

6.6 Requestlayout, onlayout, onDraw, DrawChild区别与联系

- **requestLayout()**: 会导致调用 measure()过程 和 layout()过程, 将会根据标志位判断是否需要 ondraw。
- **onLayout()**: 如果该View是ViewGroup对象, 需要实现该方法, 对每个子视图进行布局。
- **onDraw()**: 绘制视图本身 (每个View都需要重载该方法, ViewGroup不需要实现该方法)。
- **drawChild()**: 去重新回调每个子视图的draw()方法。

6.7 invalidate() 和 postInvalidate()的区别

invalidate()与postInvalidate()都用于刷新View, 主要区别是invalidate()在主线程中调用, 若在子线程中使用需要配合handler; 而postInvalidate()可在子线程中直接调用。

7, Android进程

7.1 概念

进程 (Process) 是计算机中的程序关于某数据集上的一次运行活动, 是系统进行资源分配和调度的基本单位, 是操作系统结构的基础。

当一个程序第一次启动的时候, Android会启动一个LINUX进程和一个主线程。默认的情况下, 所有该程序的组件都将在该进程和线程中运行。同时, Android会为每个应用程序分配一个单独的LINUX用户。Android会尽量保留一个正在运行进程, 只在内存资源出现不足时, Android会尝试停止一些进程从而释放足够的资源给其他新的进程使用, 也能保证用户正在访问的当前进程有足够的资源去及时地响应用户的事件。

我们可以将一些组件运行在其他进程中, 并且可以为任意的进程添加线程。组件运行在哪个进程中是在manifest文件里设置的, 其中, 和都有一个process属性来指定该组件运行在哪个进程之中。我们可以设置这个属性, 使得每个组件运行在它们自己的进程中, 或是几个组件共享一个进程, 或是不共享。元素也有一个process属性, 用来指定所有的组件的默认属性。

Android中的所有组件都在指定的进程中的主线程中实例化的, 对组件的系统调用也是由主线程发出的。每个实例不会建立新的线程。对系统调用进行响应的方法——例如负责执行用户动作的View.onKeyDown()和组件的生命周期函数——都是运行在这个主线程中的。这意味着当系统调用这个组件时, 这个组件不能长时间的阻塞主线程。例如进行网络操作时或是更新UI时, 如果运行时间较长, 就不能直接在主线程中运行, 因为这样会阻塞这个进程中其他的组件, 我们可以将这样的组件分配到新建的线程中或是其他的线程中运行。

7.2 进程生命周期

按照生命周期的不同, Android的进程可以分为前台进程、后台进程、可见进程、服务进程和空进程等。

前台进程

前台进程是用户当前正在使用的进程，一些前台进程可以在任何时候都存在，当内存低的时候前台进程也可能被销毁。对于这种情况下，设备会进行内存调度，中止一些前台进程来保持对用户交互的响应。

如果有以下的情形，那么它就是前台进程：

1. 托管用户正在交互的 Activity（已调用 Activity 的 onResume() 方法）
2. 托管某个 Service，后者绑定到用户正在交互的 Activity
3. 托管正在“前台”运行的 Service（服务已调用 startForeground()）
4. 托管正执行一个生命周期回调的 Service（onCreate()、onStart() 或 onDestroy()）
5. 托管正执行其 onReceive() 方法的 BroadcastReceiver

可见进程

可见进程指的是不包含前台组件，但是会在屏幕上显示一个可见的进程。

如果有如下的一种情形，那就是可见进程：

1. 托管不在前台、但仍对用户可见的 Activity（已调用其 onPause() 方法）。例如，如果 re前台 Activity 启动了一个对话框，允许在其后显示上一 Activity，则有可能发生这种情况。
2. 托管绑定到可见（或前台）Activity 的 Service。

服务进程

通过 startService() 方法启动的 Service，这个 Service 没有上面的两种进程重要，一般会随着应用的生命周期。

一般来说，使用 startService() 方法启动的服务且不属于上述两个更高类别进程的就是服务进程。

后台进程

包含目前对用户不可见的 Activity 的进程（已调用 Activity 的 onStop() 方法）。通常会有很多后台进程在运行，因此它们会保存在 LRU（最近最少使用）列表中，以确保包含用户最近查看的 Activity 的进程最后一个被终止。

空进程

不含任何活动应用组件的进程。保留这种进程的的唯一目的是用作缓存，以缩短下次在其中运行组件所需的启动时间。为使总体系统资源在进程缓存和底层内核缓存之间保持平衡，系统往往会终止这些进程。

7.3 多进程

首先，进程一般指一个执行单元，在移动设备上就是一个程序或应用，我们在 Android 中所说的多进程（IPC）一般指一个应用包含多个进程。之所以要使用多进程有两方面原因：某些模块由于特殊的需求要运行在单独的进程；增加应用可用的内存空间。

在 Android 中开启多进程只有一种方法，就是在 AndroidManifest.xml 中注册 Service、Activity、Receiver、ContentProvider 时指定 android:process 属性，如下所示。

```
<service
    android:name=".MyService"
    android:process=":remote">
</service>

<activity
    android:name=".MyActivity"
    android:process="com.shh.ipctest.remote2">
</activity>
```

可以看到，MyService和MyActivity指定的android:process属性值有所不同，它们的区别如下：

- :remote：以:开头是一种简写，系统会在当前进程名前附加当前包名，完整的进程名为：com.shh.ipctest:remote，同时以:开头的进程属于当前应用的私有进程，其它应用的组件不能和它跑在同一进程。
- com.shh.ipctest.remote2：这是完整的命名方式，不会附加包名，其它应用如果和该进程的ShareUID、签名相同，则可以和它跑在同一个进程，实现数据共享。

不过，开启多进程会引发如下问题，必须引起注意：

- 静态成员和单例模式失效
- 线程同步机制失效
- SharedPreferences可靠性降低
- Application被多次创建

对于前两个问题，可以这么理解，在Android中，系统会为每个应用或进程分配独立的虚拟机，不同的虚拟机自然占有不同的内存地址空间，所以同一个类的对象会产生不同的副本，导致共享数据失败，必然也不能实现线程的同步。

由于SharedPreferences底层采用读写XML的文件的方式实现，多进程并发的读写很可能导致数据异常。

Application被多次创建和前两个问题类似，系统在分配多个虚拟机时相当于把同一个应用重新启动多次，必然会导致Application多次被创建，为了防止在Application中出现无用的重复初始化，可使用进程名来做过滤，只让指定进程的才进行全局初，如下所示。

```
public class MyApplication extends Application{
    @Override
    public void onCreate() {
        super.onCreate();
        String processName = "com.xzh.ipctest";
        if (getPackageName().equals(processName)){
            // do some init
        }
    }
}
```

7.4 多进程通信方式

目前，Android中支持的多进程通信方式主要有以下几种：

- AIDL：功能强大，支持进程间一对多的实时并发通信，并可实现RPC（远程过程调用）。
- Messenger：支持一对多的串行实时通信，AIDL的简化版本。
- Bundle：四大组件的进程通信方式，只能传输Bundle支持的数据类型。
- ContentProvider：强大的数据源访问支持，主要支持CRUD操作，一对多的进程间数据共享，例如我们的应用访问系统的通讯录数据。
- BroadcastReceiver：即广播，但只能单向通信，接收者只能被动的接收消息。
文件共享：在非高并发情况下共享简单的数据。
- Socket：通过网络传输数据。

8，序列化

8.1 Parcelable 与 Serializable

- Serializable 使用I/O读写存储在硬盘上，而Parcelable是直接在内存中读写。
- Serializable 会使用反射，序列化和反序列化过程需要大量I/O操作，Parcelable自己实现封送和解封（marshalled & unmarshalled）操作不需要用反射，数据也存放在Native内存中，效率要

快很多。

8.2 示例

Serializable实例:

```
import java.io.Serializable;
class serializableObject implements Serializable {
    String name;
    public serializableObject(String name) {
        this.name = name;
    }
    public String getName() {
        return name;
    }
}
```

Parcelable实例:

```
import android.os.Parcel;
import android.os.Parcelable;
class parcleObject implements Parcelable {
    private String name;
    protected parcleObject(Parcel in) {
        this.name = in.readString();
    }
    public parcleObject(String name) {
        this.name = name;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public static final Creator<parcleObject> CREATOR = new Creator<parcleObject>() {
        @Override
        public parcleObject createFromParcel(Parcel in) {
            return new parcleObject(in);
        }
        @Override
        public parcleObject[] newArray(int size) {
            return new parcleObject[size];
        }
    };
    @Override
    public int describeContents() {
        return 0;
    }
    @Override
    public void writeToParcel(Parcel dest, int flags) {
        dest.writeString(this.name);
    }
}
```

使用Parcelable时，一般需要用以下几个方法：

- `createFromParcel(Parcel in)`：从序列化后的对象中创建原始对象。
- `newArray(int size)`：创建指定长度的原始对象数组。
- `User(Parcel in)` 从序列化后的对象中创建原始对象。
- `writeToParcel(Parcel dest, int flags)`：将当前对象写入序列化结构中，其中 `flags` 标识有两种值：0 或者 1。为 1 时标识当前对象需要作为返回值返回，不能立即释放资源，几乎所有情况都为 0。
- `describeContents`：返回当前对象的内容描述。如果含有文件描述符，返回 1，否则返回 0，几乎所有情况都返回 0。

9, Window

9.1 基本概念

Window 是一个抽象类，它的具体实现是 `PhoneWindow`。`WindowManager` 是外界访问 Window 的入口，Window 的具体实现位于 `WindowManagerService` 中，`WindowManager` 和 `WindowManagerService` 的交互是一个 IPC 过程。Android 中所有的视图都是通过 Window 来呈现，因此 Window 实际是 View 的直接管理者。

依据作用的不同，Window可以分为如下几种：

- **Application Window**：对应着一个 Activity；
- **Sub Window**：不能单独存在，只能附属在父 Window 中，如 Dialog 等；
- **System Window**：需要权限声明，如 Toast 和 系统状态栏等；

9.2 内部机制

Window 是一个抽象的概念，每一个 Window 对应着一个 View 和一个 `ViewRootImpl`。Window 实际是不存在的，它是以 View 的形式存在。对 Window 的访问必须通过 `WindowManager`，`WindowManager` 的实现类是 `WindowManagerImpl`，源码如下：

```
@Override
public void addView(@NonNull View view, @NonNull ViewGroup.LayoutParams params)
{
    applyDefaultToken(params);
    mGlobal.addView(view, params, mContext.getDisplay(), mParentWindow);
}

@Override
public void updateViewLayout(@NonNull View view, @NonNull ViewGroup.LayoutParams params) {
    applyDefaultToken(params);
    mGlobal.updateViewLayout(view, params);
}

@Override
public void removeView(View view) {
    mGlobal.removeView(view, false);
}
```

`WindowManagerImpl` 没有直接实现 Window 的三大操作，而是全部交给 `WindowManagerGlobal` 处理，`WindowManagerGlobal` 以工厂的形式向外提供自己的实例，涉及的代码如下：

```
// 添加
public void addView(View view, ViewGroup.LayoutParams params,
    Display display, Window parentWindow) {
```

```

...
// 子 window 的话需要调整一些布局参数
final WindowManager.LayoutParams wparams = (WindowManager.LayoutParams)
params;
if (parentwindow != null) {
    parentwindow.adjustLayoutParamsForSubWindow(wparams);
} else {
    ...
}
ViewRootImpl root;
View panelParentView = null;
synchronized (mLock) {
    // 新建一个 ViewRootImpl, 并通过其 setView 来更新界面完成 window 的添加过程
    ...
    root = new ViewRootImpl(view.getContext(), display);
    view.setLayoutParams(wparams);
    mViews.add(view);
    mRoots.add(root);
    mParams.add(wparams);
    // do this last because it fires off messages to start doing things
    try {
        root.setView(view, wparams, panelParentView);
    } catch (RuntimeException e) {
        // BadTokenException or InvalidDisplayException, clean up.
        if (index >= 0) {
            removeViewLocked(index, true);
        }
        throw e;
    }
}
}

// 删除
@UnsupportedAppUsage
public void removeView(View view, boolean immediate) {
    ...
    synchronized (mLock) {
        int index = findViewLocked(view, true);
        View curView = mRoots.get(index).getView();
        removeViewLocked(index, immediate);
        ...
    }
}

private void removeViewLocked(int index, boolean immediate) {
    ViewRootImpl root = mRoots.get(index);
    View view = root.getView();
    if (view != null) {
        InputMethodManager imm = InputMethodManager.getInstance();
        if (imm != null) {
            imm.windowDismissed(mViews.get(index).getWindowToken());
        }
    }
    boolean deferred = root.die(immediate);
    if (view != null) {
        view.assignParent(null);
        if (deferred) {
            mDyingViews.add(view);

```

```

    }
}

// 更新
public void updateViewLayout(View view, ViewGroup.LayoutParams params) {
    ...
    final WindowManager.LayoutParams wparams =
        (WindowManager.LayoutParams)params;
    view.setLayoutParams(wparams);
    synchronized (mLock) {
        int index = findViewLocked(view, true);
        ViewRootImpl root = mRoots.get(index);
        mParams.remove(index);
        mParams.add(index, wparams);
        root.setLayoutParams(wparams, false);
    }
}

```

10, 消息机制

谈Android的消息机制主要就是Handler机制。

10.1 Handler 机制

Handler 有两个主要用途：

- 安排 Message 和 runnables 在将来的某个时刻执行;
- 将要在不同于自己的线程上执行的操作排入队列。(在多个线程并发更新UI的同时保证线程安全。)

Android 规定访问 UI 只能在主线程中进行，因为 Android 的 UI 控件不是线程安全的，多线程并发访问会导致 UI 控件处于不可预期的状态。为什么系统不对 UI 控件的访问加上锁机制？缺点有两个：加锁会让 UI 访问的逻辑变得复杂；其次锁机制会降低 UI 访问的效率。如果子线程访问 UI，那么程序就会抛出异常。为了保证线程安全，ViewRootImpl 对UI操作做了验证，这个验证工作是由 ViewRootImpl 的 checkThread 方法完成。

```

void checkThread() {
    if (mThread != Thread.currentThread()) {
        throw new CalledFromWrongThreadException(
            "Only the original thread that created a view hierarchy can
            touch its views.");
    }
}

```

谈Handler机制时，通常会包含以下三个对象：

- Message：Handler 接收和处理的消息对象。
- MessageQueue：Message 的队列，先进先出，每一个线程最多可以拥有一个。
- Looper：消息泵，是 MessageQueue 的管理者，会不断从 MessageQueue 中取出消息，并将消息分给对应的 Handler 处理，每个线程只有一个 Looper。

Handler 创建的时候会采用当前线程的 Looper 来构造消息循环系统，需要注意的是，线程默认是没有 Looper 的，直接使用 Handler 会报错，如果需要使用 Handler 就必须为线程创建 Looper，因为默认的 UI 主线程，也就是 ActivityThread，ActivityThread 被创建的时候就会初始化 Looper，这也是在主线程中默认可以使用 Handler 的原因。

10.2 工作原理

ThreadLocal

ThreadLocal 是一个线程内部的数据存储类，通过它可以在指定的线程中存储数据，其他线程则无法获取。Looper、ActivityThread 以及 AMS 中都用到 ThreadLocal。当不同线程访问同一个 ThreadLocal 的 get 方法，ThreadLocal 内部会从各自的线程中取出一个数组，然后再从数组中根据当前 ThreadLocal 的索引去查找对应的 value 值，源码如下：

```
public void set(T value) {
    Thread t = Thread.currentThread();
    ThreadLocalMap map = getMap(t);
    if (map != null)
        map.set(this, value);
    else
        createMap(t, value);
}

...
public T get() {
    Thread t = Thread.currentThread();
    ThreadLocalMap map = getMap(t);
    if (map != null) {
        ThreadLocalMap.Entry e = map.getEntry(this);
        if (e != null) {
            @SuppressWarnings("unchecked")
            T result = (T)e.value;
            return result;
        }
    }
    return setInitialValue();
}
```

MessageQueue

MessageQueue 主要包含两个操作：插入和读取。读取操作本身会伴随着删除操作，插入和读取对应的方法分别是 enqueueMessage 和 next。MessageQueue 内部实现并不是用的队列，实际上通过一个单链表的数据结构来维护消息列表。next 方法是一个无限循环的方法，如果消息队列中没有消息，那么 next 方法会一直阻塞。当有新消息到来时，next 方法会放回这条消息并将其从单链表中移除，源码如下。

```
boolean enqueueMessage(Message msg, long when) {
    ...
    synchronized (this) {
        ...
        msg.markInUse();
        msg.when = when;
        Message p = mMessages;
        boolean needWake;
        if (p == null || when == 0 || when < p.when) {
            // New head, wake up the event queue if blocked.
            msg.next = p;
            mMessages = msg;
            needWake = mBlocked;
        } else {
            // Inserted within the middle of the queue. Usually we don't have
            to wake
            // up the event queue unless there is a barrier at the head of the
            queue
        }
    }
}
```

```

        // and the message is the earliest asynchronous message in the
queue.

        needwake = mBlocked && p.target == null && msg.isAsynchronous();
        Message prev;
        for (;;) {
            prev = p;
            p = p.next;
            if (p == null || when < p.when) {
                break;
            }
            if (needwake && p.isAsynchronous()) {
                needwake = false;
            }
        }
        msg.next = p; // invariant: p == prev.next
        prev.next = msg;
    }

    // We can assume mPtr != 0 because mQuitting is false.
    if (needwake) {
        nativeWake(mPtr);
    }
}
return true;
}
...
Message next() {
    // Return here if the message loop has already quit and been disposed.
    // This can happen if the application tries to restart a looper after quit
    // which is not supported.
    ...
    for (;;) {
        ...
        synchronized (this) {
            // Try to retrieve the next message. Return if found.
            final long now = SystemClock.uptimeMillis();
            Message prevMsg = null;
            Message msg = mMessages;
            if (msg != null && msg.target == null) {
                // Stalled by a barrier. Find the next asynchronous message in
the queue.

                do {
                    prevMsg = msg;
                    msg = msg.next;
                } while (msg != null && !msg.isAsynchronous());
            }
            if (msg != null) {
                if (now < msg.when) {
                    // Next message is not ready. Set a timeout to wake up when
it is ready.

                    nextPollTimeoutMillis = (int) Math.min(msg.when - now,
Integer.MAX_VALUE);
                } else {
                    // Got a message.
                    mBlocked = false;
                    if (prevMsg != null) {
                        prevMsg.next = msg.next;
                    } else {

```

```

        mMessages = msg.next;
    }
    msg.next = null;
    if (DEBUG) Log.v(TAG, "Returning message: " + msg);
    msg.markInUse();
    return msg;
}
} else {
    // No more messages.
    nextPollTimeoutMillis = -1;
}
...
}

// Run the idle handlers.
// We only ever reach this code block during the first iteration.
for (int i = 0; i < pendingIdleHandlerCount; i++) {
    final IdleHandler idler = mPendingIdleHandlers[i];
    mPendingIdleHandlers[i] = null; // release the reference to the
handler

    boolean keep = false;
    try {
        keep = idler.queueIdle();
    } catch (Throwable t) {
        Log.wtf(TAG, "IdleHandler threw exception", t);
    }

    if (!keep) {
        synchronized (this) {
            mIdleHandlers.remove(idler);
        }
    }
}

// Reset the idle handler count to 0 so we do not run them again.
pendingIdleHandlerCount = 0;

// while calling an idle handler, a new message could have been
delivered
// so go back and look again for a pending message without waiting.
nextPollTimeoutMillis = 0;
}
}

```

Looper

Looper 会不停地从 MessageQueue 中 查看是否有新消息，如果有新消息就会立刻处理，否则会一直阻塞，Looper源码。

```

private Looper(boolean quitAllowed) {
    mQueue = new MessageQueue(quitAllowed);
    mThread = Thread.currentThread();
}

```

可通过 Looper.prepare() 为当前线程创建一个 Looper，默认情况下，Activity会创建一个Looper对象。

除了 prepare 方法外，Looper 还提供了 prepareMainLooper 方法，主要是给 ActivityThread 创建 Looper 使用，本质也是通过 prepare 方法实现的。由于主线程的 Looper 比较特殊，所以 Looper 提供了一个 getMainLooper 方法来获取主线程的 Looper。

同时，Looper 提供了 quit 和 quitSafely 来退出一个 Looper，二者的区别是：quit 会直接退出 Looper，而 quitSafely 只是设定一个退出标记，然后把消息队列中的已有消息处理完毕后才安全地退出。Looper 退出后，通过 Handler 发送的消息会失败，这个时候 Handler 的 send 方法会返回 false，因此在不需要的时候需要及时终止 Looper。

Handler

Handler 的工作主要包含消息的发送和接收的过程。消息的发送可以通过 post/send 的一系列方法实现，post 最终也是通过 send 来实现的。

11, RecyclerView优化

在Android开发中，经常会遇到长列表的问题，因此很多时候，就会涉及到RecyclerView的优化问题。对于列表卡顿的原因，通常有如下一些：

11.1 卡顿场景

notifyDataSetChanged

如果数据需要全局刷新时，可以使用 notifyDataSetChanged；对于增加或减少数据，可以使用局部刷新，如下所示。

```
void onNewDataArrived(List<News> news) {
    List<News> oldNews = myAdapter.getItems();
    DiffResult result = DiffUtil.calculateDiff(new MyCallback(oldNews, news));
    myAdapter.setNews(news);
    result.dispatchUpdatesTo(myAdapter);
}
```

RecyclerView嵌套

在实际开发中，经常会看到竖直滚动的RecyclerView嵌套一个横向滚动的RecyclerView的场景。由于单个RecyclerView都拥有独立的itemView对象池，对于嵌套的情况，可以设置共享对象池，如下。

```
class OuterAdapter extends RecyclerView.Adapter<OuterAdapter.ViewHolder> {
    RecyclerView.RecycledViewPool mSharedPool = new
    RecyclerView.RecycledViewPool();

    ...

    @Override
    public void onCreateViewHolder(ViewGroup parent, int viewType) {
        // inflate inner item, find innerRecyclerView by ID...
        LinearLayoutManager innerLLM = new
        LinearLayoutManager(parent.getContext(),
            LinearLayoutManager.HORIZONTAL);
        innerRv.setLayoutManager(innerLLM);
        innerRv.setRecycledViewPool(mSharedPool);
        return new OuterAdapter.ViewHolder(innerRv);
    }
}
```

嵌套层级过深

通过Systemtrace工具可以检测Layout的性能，如果耗时太长或者调用次数过多，需要考察一下是否过度使用RelativeLayout或者嵌套多层LinearLayout，每层Layout都会导致其child多次的measure/layout。

对象分配和垃圾回收

虽然Android 5.0上使用ART来减少GC停顿时间，但仍然会造成卡顿。尽量避免在循环内创建对象导致GC。要知道，创建对象需要分配内存，而这个时机检查内存是否足够来决定需不需要进行GC。

11.2 其他优化策略

除了上面的典型场景外，RecyclerView的优化还需要注意以下几点：

- 升级 RecyclerView 版本到 25.1.0 及以上使用 Prefetch 功能；
- 通过重写 RecyclerView.onViewRecycled(holder) 来回收资源；
- 如果 Item 高度是固定的话，可以使用 RecyclerView.setHasFixedSize(true); 来避免 requestLayout 浪费资源；
- 对 ItemView 设置监听器，不要对每个 Item 都调用 addXxxListener，应该大家公用一个 XxxListener，根据 ID 来进行不同的操作，优化了对象的频繁创建带来的资源消耗；
- 尽量不要对ViewHolder使用有透明度改变的绘制；
- 增加预加载策略。

Android 面试之必问高级知识点

1，编译模式

1.1 概念

在Android早期的版本中，应用程序的运行环境是需要依赖Dalvik虚拟机的。不过，在后来的版本（大概是4.x版本），Android的运行环境却换到了 Android Runtime，其处理应用程序执行的方式完全不同于 Dalvik，Dalvik 是依靠一个 Just-In-Time (JIT) 编译器去解释字节码。

不过，Dalvik模式下，开发者编译后的应用代码需要通过一个解释器在用户的设备上运行，这一机制并不高效，但让应用能更容易在不同硬件和架构上运行。ART 则完全改变了这套做法，在应用安装时就预编译字节码到机器语言，这一机制叫 Ahead-Of-Time (AOT) 编译。在移除解释代码这一过程后，应用程序执行效率更高、启动也更快。

1.2 AOT优点

下面是AOT编译方式的一些优点：

1.2.1 预先编译

ART 引入了预先编译机制，可提高应用的性能。ART 还具有比 Dalvik 更严格的安装时验证。在安装时，ART 使用设备自带的 dex2oat 工具来编译应用。该实用工具接受 DEX 文件作为输入，并为目标设备生成经过编译的应用可执行文件，该工具能够顺利编译所有有效的 DEX 文件。

1.2.2 垃圾回收优化

垃圾回收 (GC) 可能有损于应用性能，从而导致显示不稳定、界面响应速度缓慢以及其他问题。ART模式从以下几个方面优化了垃圾回收的策略：

- 只有一次（而非两次）GC 暂停
- 在 GC 保持暂停状态期间并行处理
- 在清理最近分配的短时对象这种特殊情况中，回收器的总 GC 时间更短
- 优化了垃圾回收的工效，能够更加及时地进行并行垃圾回收，这使得 GC_FOR_ALLOC 事件在典型用例中极为罕见
- 压缩 GC 以减少后台内存使用和碎片

1.2.3 开发和调试方面的优化

支持采样分析器

一直以来，开发者都使用 Traceview 工具（用于跟踪应用执行情况）作为分析器。虽然 Traceview 可提供有用的信息，但每次方法调用产生的开销会导致 Dalvik 分析结果出现偏差，而且使用该工具明显会影响运行时性能。ART 添加了对没有这些限制的专用采样分析器的支持，因而可更准确地了解应用执行情况，而不会明显减慢速度。支持的版本从 KitKat（4.4）版本开始，为 Dalvik 的 Traceview 添加了采样支持。

支持更多调试功能

ART 支持许多新的调试选项，特别是与监控和垃圾回收相关的功能。例如，查看堆栈跟踪中保留了哪些锁，然后跳转到持有锁的线程；询问指定类的当前活动的实例数、请求查看实例，以及查看使对象保持有效状态的参考；过滤特定实例的事件（如断点）等。

优化了异常和崩溃报告中的诊断详细信息

当发生运行时异常时，ART 会为您提供尽可能多的上下文和详细信息。ART 会提供 `java.lang.ClassCastException`、`java.lang.ClassNotFoundException` 和 `java.lang.NullPointerException` 的更多异常详细信息（较高版本的 Dalvik 会提供 `java.lang.ArrayIndexOutOfBoundsException` 和 `java.lang.ArrayStoreException` 的更多异常详细信息，这些信息现在包括数组大小和越界偏移量；ART 也提供这类信息）。

1.3 垃圾回收

ART 提供了多个不同的 GC 方案，这些方案运行着不同垃圾回收器，默认的 GC 方案是 CMS（并发标记清除），主要使用粘性 CMS 和部分 CMS。粘性 CMS 是 ART 的不移动分代垃圾回收器。它仅扫描堆中自上次 GC 后修改的部分，并且只能回收自上次 GC 后分配的对象。除 CMS 方案外，当应用将进程状态更改为察觉不到卡顿的进程状态（例如，后台或缓存）时，ART 将执行堆压缩。

除了新的垃圾回收器之外，ART 还引入了一种基于位图的新内存分配程序，称为 RosAlloc（插槽运行分配器）。此新分配器具有分片锁，当分配规模较小时可添加线程的本地缓冲区，因而性能优于 D1MAlloc（内存分配器）。

内存分配器的相关知识可以参考：[内存分配器](#)

同时，与 Dalvik 相比，ART 的 CMS 垃圾回收也带来了其他方面的改善，如下：

- 与 Dalvik 相比，暂停次数从 2 次减少到 1 次。Dalvik 的第一次暂停主要是为了进行根标记，即在 ART 中进行并发标记，让线程标记自己的根，然后马上恢复运行。
- 与 Dalvik 类似，ART GC 在清除过程开始之前也会暂停 1 次。两者在这方面的主要差异在于：在此暂停期间，某些 Dalvik 环节在 ART 中并发进行。这些环节包括 `java.lang.ref.Reference` 处理、系统弱清除（例如，jni 弱全局等）、重新标记非线程根和卡片预清理。在 ART 暂停期间仍进行的阶段包括扫描脏卡片以及重新标记线程根，这些操作有助于缩短暂停时间。
- 相对于 Dalvik，ART GC 改进的最后一个方面是粘性 CMS 回收器增加了 GC 吞吐量。不同于普通的分代 GC，粘性 CMS 不移动。系统会将年轻对象保存在一个分配堆栈（基本上是 `java.lang.Object` 数组）中，而非为其设置一个专属区域。这样可以避免移动所需的对象以维持低暂停次数，但缺点是容易在堆栈中加入大量复杂对象图像而使堆栈变长。

ART GC 与 Dalvik 的另一个主要区别在于 ART GC 引入了移动垃圾回收器。使用移动 GC 的目的在于通过堆压缩来减少后台应用使用的内存。目前，触发堆压缩的事件是 ActivityManager 进程状态的改变。当应用转到后台运行时，它会通知 ART 已进入不再“感知”卡顿的进程状态。此时 ART 会进行一些操作（例如，压缩和监视器压缩），从而导致应用线程长时间暂停。

目前，Android的ART正在使用的两个移动 GC 是同构空间压缩和半空间压缩，它们的区别如下：

- **半空间压缩**：将对象在两个紧密排列的碰撞指针空间之间进行移动。这种移动 GC 适用于小内存设备，因为它可以比同构空间压缩稍微多节省一点内存，额外节省出的空间主要来自紧密排列的对象，这样可以避免 RosAlloc/DIMalloc 分配器占用开销。
- 同构空间压缩通过将对象从一个 RosAlloc 空间复制到另一个 RosAlloc 空间来实现。这有助于通过减少堆碎片来减少内存使用量。这是目前非低内存设备的默认压缩模式。相比半空间压缩，同构空间压缩的主要优势在于应用从后台切换到前台时无需进行堆转换。

2，类加载器

2.1 类加载器分类

目前，Android的类加载器从下到上主要分为BootstrapClassLoader（根类加载器）、ExtensionClassLoader（扩展类加载器）和 AppClassLoader（应用类加载器）三种。

- **根类加载器**：该加载器没有父加载器。它负责加载虚拟机的核心类库，如java.lang.*等。例如java.lang.Object就是由根类加载器加载的。根类加载器从系统属性sun.boot.class.path所指定的目录中加载类库。根类加载器的实现依赖于底层操作系统，属于虚拟机的实现的一部分，它并没有继承java.lang.ClassLoader类。
- **扩展类加载器**：它的父加载器为根类加载器。它从java.ext.dirs系统属性所指定的目录中加载类库，或者从JDK的安装目录的jre/lib/ext子目录（扩展目录）下加载类库，如果把用户创建的JAR文件放在这个目录下，也会自动由扩展类加载器加载。扩展类加载器是纯Java类，是java.lang.ClassLoader类的子类。
- **系统类加载器**：也称为应用类加载器，它的父加载器为扩展类加载器。它从环境变量classpath或者系统属性java.class.path所指定的目录中加载类，它是用户自定义的类加载器的默认父加载器。系统类加载器是纯Java类，是java.lang.ClassLoader类的子类。父子加载器并非继承关系，也就是说子加载器不一定是继承了父加载器。

2.2 双亲委托模式

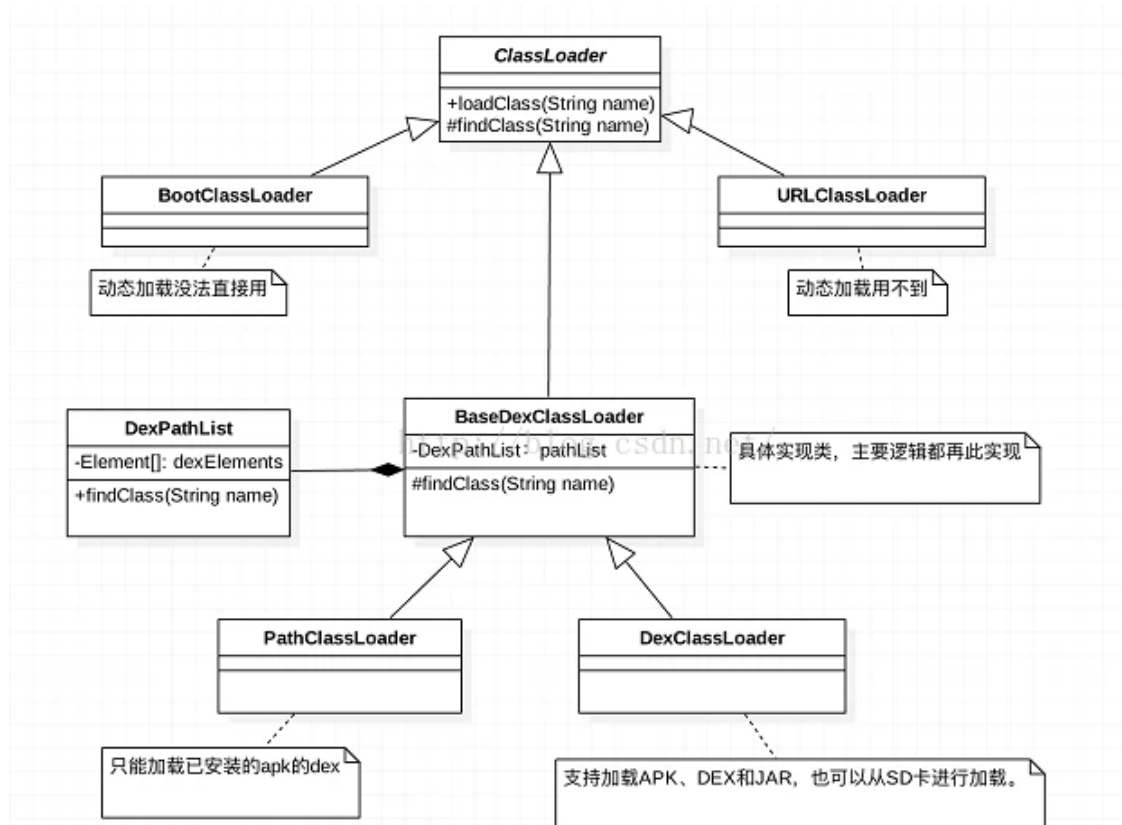
所谓双亲委托模式，指的是某个特定的类加载器在接到加载类的请求时，首先将加载任务委托给父类加载器，依次递归，如果父类加载器可以完成类加载任务，就成功返回；只有父类加载器无法完成此加载任务时，才自己去加载。

因为这样可以避免重复加载，当父亲已经加载了该类的时候，就没有必要子ClassLoader再加载一次。如果不使用这种委托模式，那我们就可以随时使用自定义的类来动态替代一些核心的类，存在非常大的安全隐患。

举个例子，事实上，java.lang.String这个类并不会被我们自定义的classloader加载，而是由bootstrap classloader进行加载，为什么会这样？实际上这就是双亲委托模式的原因，因为在任何一个自定义ClassLoader加载一个类之前，它都会先委托它的父亲ClassLoader进行加载，只有当父亲ClassLoader无法加载成功后，才会由自己加载。

2.3 Android的类加载器

下面是Android类加载器的模型图：



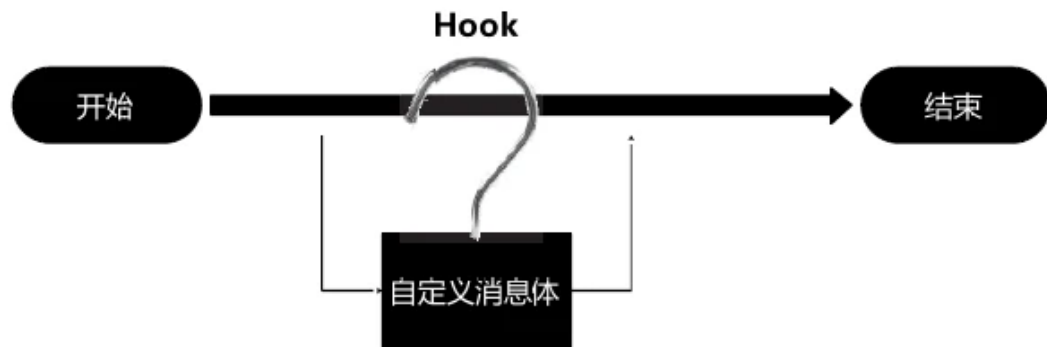
下面看一下DexClassLoader，DexClassLoader 重载了 findClass 方法，在加载类时会调用其内部的 DexPathList 去加载。DexPathList 是在构造 DexClassLoader 时生成的，其内部包含了 DexFile，涉及的源码如下。

```
...
public Class findClass(String name) {
    for (Element element : dexElements) {
        DexFile dex = element.dexFile;
        if (dex != null) {
            Class clazz = dex.loadClassBinaryName(name, definingContext);
            if (clazz != null) {
                return clazz;
            }
        }
    }
    return null;
}
...
```

类加载器更多的内容，可以参考：[android 类加载器双亲委托模式](#)

3, Android Hook

所谓Hook，就是在程序执行的过程中去截取其中的某段信息，示意图如下。



Android的Hook大体的流程可以分为如下几步：

- 1、根据需求确定需要 hook 的对象
- 2、寻找要hook的对象的持有者，拿到需要 hook 的对象
- 3、定义“要 hook 的对象”的代理类，并且创建该类的对象
- 4、使用上一步创建出来的对象，替换掉要 hook 的对象

下面是一段简单的Hook的示例代码，用到了Java的反射机制。

```
@SuppressWarnings({"DiscouragedPrivateApi", "PrivateApi"})
public static void hook(Context context, final View view) {
    try {
        // 反射执行View类的getListenerInfo()方法，拿到v的mListenerInfo对象，这个对象就是点击事件的持有者
        Method method = View.class.getDeclaredMethod("getListenerInfo");
        method.setAccessible(true); // 由于getListenerInfo()方法并不是public的，所以要加这个代码来保证访问权限
        Object mListenerInfo = method.invoke(view); // 这里拿到的就是mListenerInfo对象，也就是点击事件的持有者

        // 要从这里面拿到当前的点击事件对象
        Class<?> listenerInfoClz =
            Class.forName("android.view.View$ListenerInfo"); // 这是内部类的表示方法
        Field field = listenerInfoClz.getDeclaredField("mOnClickListener");
        final View.OnClickListener onClickListenerInstance =
            (View.OnClickListener) field.get(mListenerInfo); // 取得真实的mOnClickListener对象

        // 2. 创建我们自己的点击事件代理类
        // 方式1: 自己创建代理类
        // ProxyOnClickListener proxyOnClickListener = new
        ProxyOnClickListener(onClickListenerInstance);
        // 方式2: 由于View.OnClickListener是一个接口，所以可以直接用动态代理模式
        // Proxy.newProxyInstance的3个参数依次分别是:
        // 本地的类加载器;
        // 代理类的对象所继承的接口（用Class数组表示，支持多个接口）
        // 代理类的实际逻辑，封装在new出来的InvocationHandler内
        Object proxyOnClickListener =
            Proxy.newProxyInstance(context.getClass().getClassLoader(), new Class[]
            {View.OnClickListener.class}, new InvocationHandler() {
                @Override
                public Object invoke(Object proxy, Method method, Object[] args)
                throws Throwable {
                    Log.d("HookSetOnClickListener", "点击事件被hook到了"); // 加入自己的逻辑
                }
            });
    }
}
```

```

        return method.invoke(onClickListenerInstance, args); // 执行被
代理的对象的逻辑
    }
    });
    // 3. 用我们自己的点击事件代理类，设置到"持有者"中
    field.set(mListenerInfo, proxyOnClickListener);
} catch (Exception e) {
    e.printStackTrace();
}
}

// 自定义代理类
static class ProxyOnClickListener implements View.OnClickListener {
    View.OnClickListener oriLis;

    public ProxyOnClickListener(View.OnClickListener oriLis) {
        this.oriLis = oriLis;
    }

    @Override
    public void onClick(View v) {
        Log.d("HookSetOnClickListener", "点击事件被hook到了");
        if (oriLis != null) {
            oriLis.onClick(v);
        }
    }
}
}

```

而在Android开发中，想要实现Hook，肯定是没有这么简单的，我们需要借助一些Hook框架，比如Xposed、Cydia Substrate、Legend等。

参考资料：[Android Hook机制](#)

4，代码混淆

4.1 Proguard

众所周知，Java代码是很容易反编译的，为了更好的保护Java源代码，我们往往会对编译好的Class类文件进行混淆处理。而ProGuard就是一个混淆代码的开源项目。它的主要作用就是混淆，当然它还能对字节码进行缩减体积、优化等，但那些对于我们来说都算是次要的功能。

具体来说，ProGuard具有如下功能：

- 压缩 (Shrink)：检测和删除没有使用的类，字段，方法和特性。
- 优化 (Optimize)：分析和优化Java字节码。
- 混淆 (Obfuscate)：使用简短的无意义的名称，对类，字段和方法进行重命名。

在Android开发中，开启混淆需要将app/build.gradle文件下的minifyEnabled属性设置为true，如下所示。

```

minifyEnabled true
proguardFiles getDefaultProguardFile('proguard-android.txt'), 'proguard-
rules.pro'

```

proguard-android.txt是Android提供的默认混淆配置文件，我们需要的混淆的规则都放在这个文件中。

4.2 混淆规则

混淆命令

- keep: 保留类和类中的成员，防止被混淆或移除
- keepnames: 保留类和类中的成员，防止被混淆，成员没有被引用会被移除
- keepclassmembers: 只保留类中的成员，防止被混淆或移除
- keepclassmembersnames: 只保留类中的成员，防止被混淆，成员没有引用会被移除
- keepclasseswithmembers: 保留类和类中的成员，防止被混淆或移除，保留指明的成员
- keepclasseswithmembersnames: 保留类和类中的成员，防止被混淆，保留指明的成员，成员没有引用会被移除

混淆通配符

- `<field>`: 匹配类中的所有字段
- `<method>`: 匹配类中所有的方法
- `<init>`: 匹配类中所有的构造函数
- `*`: 匹配任意长度字符，不包含包名分隔符(.)
- `**`: 匹配任意长度字符，包含包名分隔符(.)
- `***`: 匹配任意参数类型

keep的规则格式如下:

```
[keep命令] [类] {  
    [成员]  
}
```

4.3 混淆模版

ProGuard中有些公共的模版是可以复用的，比如压缩比、大小写混合和一些系统提供的Activity、Service不能混淆等。

```
# 代码混淆压缩比，在 0~7 之间，默认为 5，一般不做修改  
-optimizationpasses 5  
  
# 混合时不使用大小写混合，混合后的类名为小写  
-dontusemixedcaseclassnames  
  
# 指定不去忽略非公共库的类  
-dontskipnonpubliclibraryclasses  
  
# 这句话能够使我们的项目混淆后产生映射文件  
# 包含有类名->混淆后类名的映射关系  
-verbose  
  
# 指定不去忽略非公共库的类成员  
-dontskipnonpubliclibraryclassmembers  
  
# 不做预校验，preverify 是 proguard 的四个步骤之一，Android 不需要 preverify，去掉  
这一步能够加快混淆速度。  
-dontpreverify  
  
# 保留 Annotation 不混淆  
-keepattributes *Annotation*,InnerClasses  
  
# 避免混淆泛型
```


`-keepattributes Signature`

抛出异常时保留代码行号

`-keepattributes SourceFile,LineNumberTable`

指定混淆是采用的算法，后面的参数是一个过滤器

这个过滤器是谷歌推荐的算法，一般不做更改

`-optimizations !code/simplification/cast,!field/*,!class/merging/*`

#####

#

Android开发中一些需要保留的公共部分

#

#####

保留我们使用的四大组件，自定义的 `Application` 等等这些类不被混淆

因为这些子类都有可能被外部调用

`-keep public class * extends android.app.Activity`

`-keep public class * extends android.app.Application`

`-keep public class * extends android.app.Service`

`-keep public class * extends android.content.BroadcastReceiver`

`-keep public class * extends android.content.ContentProvider`

`-keep public class * extends android.app.backup.BackupAgentHelper`

`-keep public class * extends android.preference.Preference`

`-keep public class * extends android.view.View`

`-keep public class com.android.vending.licensing.ILicensingService`

保留 `support` 下的所有类及其内部类

`-keep class android.support.** { *; }`

保留继承的

`-keep public class * extends android.support.v4.**`

`-keep public class * extends android.support.v7.**`

`-keep public class * extends android.support.annotation.**`

保留 `R` 下面的资源

`-keep class **.R$* { *; }`

保留本地 `native` 方法不被混淆

`-keepclasseswithmembernames class * {`

`native <methods>;`

`}`

保留在 `Activity` 中的方法参数是 `view` 的方法，

这样以来我们在 `layout` 中写的 `onClick` 就不会被影响

`-keepclassmembers class * extends android.app.Activity {`

`public void *(android.view.View);`

`}`

保留枚举类不被混淆

`-keepclassmembers enum * {`

`public static **[] values();`

`public static ** valueOf(java.lang.String);`

`}`

保留我们自定义控件（继承自 `view`）不被混淆

```

-keep public class * extends android.view.View {
    *** get*();
    void set*(***);
    public <init>(android.content.Context);
    public <init>(android.content.Context, android.util.AttributeSet);
    public <init>(android.content.Context, android.util.AttributeSet, int);
}

# 保留 Parcelable 序列化类不被混淆
-keep class * implements android.os.Parcelable {
    public static final android.os.Parcelable$Creator *;
}

# 保留 Serializable 序列化的类不被混淆
-keepnames class * implements java.io.Serializable
-keepclassmembers class * implements java.io.Serializable {
    static final long serialVersionUID;
    private static final java.io.ObjectStreamField[] serialPersistentFields;
    !static !transient <fields>;
    !private <fields>;
    !private <methods>;
    private void writeObject(java.io.ObjectOutputStream);
    private void readObject(java.io.ObjectInputStream);
    java.lang.Object writeReplace();
    java.lang.Object readResolve();
}

# 对于带有回调函数的 onXXEvent、**On*Listener 的，不能被混淆
-keepclassmembers class * {
    void *(**On*Event);
    void *(**On*Listener);
}

# webView 处理，项目中没有使用到 webView 忽略即可
-keepclassmembers class fqcn.of.javascript.interface.for.webview {
    public *;
}

-keepclassmembers class * extends android.webkit.WebViewClient {
    public void *(android.webkit.Webview, java.lang.String,
    android.graphics.Bitmap);
    public boolean *(android.webkit.Webview, java.lang.String);
}

-keepclassmembers class * extends android.webkit.WebViewClient {
    public void *(android.webkit.webview, java.lang.String);
}

# js
-keepattributes JavascriptInterface
-keep class android.webkit.JavascriptInterface { *; }
-keepclassmembers class * {
    @android.webkit.JavascriptInterface <methods>;
}

# @Keep
-keep,allowobfuscation @interface android.support.annotation.Keep
-keep @android.support.annotation.Keep class *
-keepclassmembers class * {
    @android.support.annotation.Keep *;
}

```

```
}
```

如果是aar这种插件，可以在aar的build.gradle中添加如下混淆配置。

```
android {  
    ...  
    defaultConfig {  
        ...  
        consumerProguardFile 'proguard-rules.pro'  
    }  
    ...  
}
```

5, NDK

如果要问Android的高级开发知识，那么NDK肯定是必问的。那么什么的NDK，NDK 全称是 Native Development Kit，是一组可以让开发者在 Android 应用中使用C/C++ 的工具。通常，NDK可以用在如下的场景中：

- 从设备获取更好的性能以用于计算密集型应用，例如游戏或物理模拟。
- 重复使用自己或其他开发者的 C/C++ 库，便于跨平台。
- NDK 集成了譬如 OpenGL、Vulkan 等 API 规范的特定实现，以实现在 Java 层无法做到的功能，如音视频开发、渲染。
- 增加反编译难度。

5.1, JNI基础

JNI即java native interface，是Java和Native代码进行交互的接口。

5.1.1 JNI 访问 Java 对象方法

假如，有如下一个Java类，代码如下。

```
package com.xzh.jni;  
  
public class MyJob {  
    public static String JOB_STRING = "my_job";  
    private int jobId;  
  
    public MyJob(int jobId) {  
        this.jobId = jobId;  
    }  
  
    public int getJobId() {  
        return jobId;  
    }  
}
```

然后，在cpp目录下，新建native_lib.cpp，添加对应的native实现。

```
#include <jni.h>  
  
extern "C"  
JNIEXPORT jint JNICALL
```

```

Java_com_xzh_jni_MainActivity_getJobId(JNIEnv *env, jobject thiz, jobject
job) {

    // 根据实例获取 class 对象
    jclass jobClz = env->GetObjectClass(job);
    // 根据类名获取 class 对象
    jclass jobClz = env->FindClass("com/xzh/jni/MyJob");

    // 获取属性 id
    jfieldID fieldId = env->GetFieldID(jobClz, "jobId", "I");
    // 获取静态属性 id
    jfieldID sFieldId = env->GetStaticFieldID(jobClz, "JOB_STRING",
"Ljava/lang/String;");

    // 获取方法 id
    jmethodID methodId = env->GetMethodID(jobClz, "getJobId", "()I");
    // 获取构造方法 id
    jmethodID initMethodId = env->GetMethodID(jobClz, "<init>", "(I)V");

    // 根据对象属性 id 获取该属性值
    jint id = env->GetIntField(job, fieldId);
    // 根据对象方法 id 调用该方法
    jint id = env->CallIntMethod(job, methodId);

    // 创建新的对象
    jobject newJob = env->NewObject(jobClz, initMethodId, 10);
    return id;
}

```

5.2 NDK开发

5.2.1 基本流程

首先，在 Java 代码中声明 Native 方法，如下所示。

```

public class MainActivity extends AppCompatActivity {

    // Used to load the 'native-lib' library on application startup.
    static {
        System.loadLibrary("native-lib");
    }

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        Log.d("MainActivity", stringFromJNI());
    }
    private native String stringFromJNI();
}

```

然后，新建一个 cpp 目录，并且新建一个名为 native-lib.cpp 的 cpp 文件，实现相关方法。

```
#include <jni.h>

extern "C" JNIEXPORT jstring JNICALL
Java_com_xzh_jni_MainActivity_stringFromJNI(
    JNIEnv *env,
    jobject /* this */) {
    std::string hello = "Hello from C++";
    return env->NewStringUTF(hello.c_str());
}
```

cpp文件遵循如下的规则：

- 函数名的格式遵循如下规则：Java 包名类名_方法名。
- extern "C" 指定采用 C 语言的命名风格来编译，否则由于 C 与 C++ 风格不同，导致链接时无法找到具体的函数
- JNIEnv*：表示一个指向 JNI 环境的指针，可以通过他来访问 JNI 提供的接口方法
- jobject：表示 java 对象中的 this
- JNIEXPORT 和 JNICALL：JNI 所定义的宏，可以在 jni.h 头文件中查找到

System.loadLibrary()的代码位于java/lang/System.java文件中，源码如下：

```
@CallerSensitive
public static void load(String filename) {
    Runtime.getRuntime().load0(Reflection.getCallerClass(), filename);
}
```

5.3 CMake 构建 NDK

CMake 是一个开源的跨平台工具系列，旨在构建、测试和打包软件，从 Android Studio 2.2 开始，Android Studio 默认地使用 CMake 与 Gradle 搭配使用来构建原生库。具体来说，我们可以使用 Gradle 将 C/C++ 代码 编译到原生库中，然后将这些代码打包到我们的应用中，Java 代码随后可以通过 Java 原生接口 (JNI) 调用 我们原生库中的函数。

使用CMake开发NDK项目需要下载如下一些套件：

- Android 原生开发工具包 (NDK)：这套工具集允许我们 开发 Android 使用 C 和 C++ 代码，并提供众多平台库，让我们可以管理原生 Activity 和访问物理设备组件，例如传感器和触摸输入。
- CMake：一款外部构建工具，可与 Gradle 搭配使用来构建原生库。如果你只计划使用 ndk-build，则不需要此组件。
- LLDB：一种调试程序，Android Studio 使用它来调试原生代码。

我们可以打开Android Studio，依次选择【Tools】>【Android】>【SDK Manager】>【SDK Tools】选中LLDB、CMake 和 NDK即可。

启用CMake还需要在 app/build.gradle 中添加如下代码。

```
android {
    ...
    defaultConfig {
        ...
        externalNativeBuild {
            cmake {
                cppFlags ""
            }
        }
    }
}
```

```

        ndk {
            abiFilters 'arm64-v8a', 'armeabi-v7a'
        }
    }
    ...
    externalNativeBuild {
        cmake {
            path "CMakeLists.txt"
        }
    }
}

```

然后，在对应目录新建一个 CMakeLists.txt 文件，添加代码。

```

# 定义了所需 CMake 的最低版本
cmake_minimum_required(VERSION 3.4.1)

# add_library() 命令用来添加库
# native-lib 对应着生成的库的名字
# SHARED 代表为分享库
# src/main/cpp/native-lib.cpp 则是指明了源文件的路径。
add_library( # Sets the name of the library.
            native-lib

            # Sets the library as a shared library.
            SHARED

            # Provides a relative path to your source file(s).
            src/main/cpp/native-lib.cpp)

# find_library 命令添加到 CMake 构建脚本中以定位 NDK 库，并将其路径存储为一个变量。
# 可以使用此变量在构建脚本的其他部分引用 NDK 库
find_library( # Sets the name of the path variable.
             log-lib

             # Specifies the name of the NDK library that
             # you want CMake to locate.
             log)

# 预构建的 NDK 库已经存在于 Android 平台上，因此，无需再构建或将其打包到 APK 中。
# 由于 NDK 库已经是 CMake 搜索路径的一部分，只需要向 CMake 提供希望使用的库的名称，并将其关联到自己的原生库中

# 要将预构建库关联到自己的原生库
target_link_libraries( # Specifies the target library.
                      native-lib

                      # Links the target library to the log library
                      # included in the NDK.
                      ${log-lib})

...

```

参考: [Android NDK开发基础](#)

6，动态加载

6.1 基本概念

动态加载技术在Web中很常见，对于Android项目来说，动态加载的目的是让用户不用重新安装APK就能升级应用的功能，主要的应用场景是插件化和热修复。

首先需要明确的一点，插件化和热修复不是同一个概念，虽然站在技术实现的角度来说，他们都是从系统加载器的角度出发，无论是采用hook方式，亦或是代理方式或者是其他底层实现，都是通过“欺骗”Android系统的方式来让宿主正常的加载和运行插件（补丁）中的内容；但是二者的出发点是不同的。

插件化，本质上是把需要实现的模块或功能当做一个独立的功能提取出来，减少宿主的规模，当需要使用到相应的功能时再去加载相应的模块。而热修复则往往是从修复bug的角度出发，强调的是在不需要二次安装应用的前提下修复已知的bug。

为了方便说明，我们先理清几个概念：

- 宿主：当前运行的APP。
- 插件：相对于插件化技术来说，就是要加载运行的apk类文件。
- 补丁：相对于热修复技术来说，就是要加载运行的.patch,.dex,*.apk等一系列包含dex修复内容的文件。

下图展示了Android动态化开发框架的整体的架构。



6.2 插件化

关于插件化技术，最早可以追溯到2012年的 [AndroidDynamicLoader](#)，其原理是动态加载不同的Fragment实现UI替换，不过随着15，16年更好的方案，这个方案渐渐的被淘汰了。再后来有了任玉刚的[dynamic-load-apk](#)方案，开始有了插件化的标准方案。而后面的方案大多基于Hook和动态代理两个方向进行。

目前，插件化的开发并没有一个官方的插件化方案，它是国内提出的一种技术实现，利用虚拟机的类的加载机制实现的一种技术手段，往往需要hook一些系统api，而Google从Android9.0开始限制对系统私有api的使用，也就造成了插件化的兼容性问题，现在几个流行的插件化技术框架，都是大厂根据自己的需求，开源出来的，如滴滴的VirtualAPK，360的RePlugin等，大家可以根据需要自行了解技术的实现原理。

6.3 热修复

6.3.1 热修复原理

说到热修复的原理，就不得不提到类的加载机制，和常规的JVM类似，在Android中类的加载也是通过ClassLoader来完成，具体来说就是PathClassLoader 和 DexClassLoader 这两个Android专用的类加载器，这两个类的区别如下。

- PathClassLoader：只能加载已经安装到Android系统中的apk文件（/data/app目录），是Android默认使用的类加载器。
- DexClassLoader：可以加载任意目录下的dex/jar/apk/zip文件，也就是我们一开始提到的补丁。

这两个类都是继承自BaseDexClassLoader，BaseDexClassLoader的构造函数如下。

```
public BaseDexClassLoader(String dexPath, File optimizedDirectory,
    String libraryPath, ClassLoader parent) {
    super(parent);
    this.pathList = new DexPathList(this, dexPath, libraryPath,
    optimizedDirectory);
}
```

这个构造函数只做了一件事，就是通过传递进来的相关参数，初始化了一个DexPathList对象。DexPathList的构造函数，就是将参数中传递进来的程序文件（就是补丁文件）封装成Element对象，并将这些对象添加到一个Element的数组集合dexElements中去。

前面说过类加载器的作用，就是将一个具体的类（class）加载到内存中，而这些操作是由虚拟机完成的，对于开发者来说，只需要关注如何去找到这个需要加载的类即可，这也是热修复需要干的事情。

在Android中，查找一个名为name的class需要经历如下两步：

1. 在DexClassLoader的findClass 方法中通过一个DexPathList对象findClass()方法来获取class。
2. 在DexPathList的findClass 方法中，对之前构造好dexElements数组集合进行遍历，一旦找到类名与name相同的类时，就直接返回这个class，找不到则返回null。

因此，基于上面的理论，我们可以想到一个最简单的热修复方案。假设现在代码中的某一个类出现Bug，那么我们可以在修复Bug之后，将这些个类打包成一个补丁文件，然后通过这个补丁文件封装出一个Element对象，并且将这个Element对象插到原有dexElements数组的最前端。这样，当DexClassLoader去加载类时，由于双亲加载机制的特点，就会优先加载插入的这个Element，而有缺陷的Element则没有机会再被加载。事实上，QQ早期的热修复方案就是这样的。

6.3.2 QQ 空间超级补丁方案

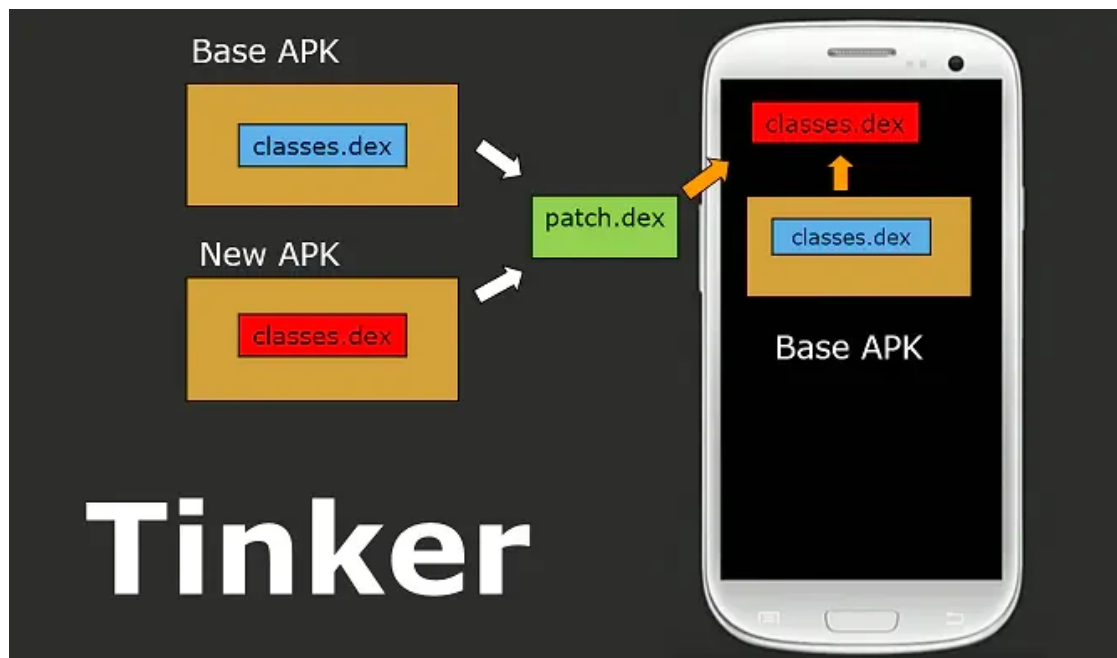
QQ 空间补丁方案就是使用javaassist 插桩的方式解决了CLASS_ISPREVERIFIED的难题。涉及的步骤如下：

- 在apk安装的时候系统会将dex文件优化成odex文件，在优化的过程中会涉及一个预校验的过程。
- 如果一个类的static方法，private方法，override方法以及构造函数中引用了其他类，而且这些类都属于同一个dex文件，此时该类就会被打上CLASS_ISPREVERIFIED。
 - 如果在运行时被打上CLASS_ISPREVERIFIED的类引用了其他dex的类，就会报错。
- 正常的分包方案会保证相关类被打入同一个dex文件。
- 想要使得patch可以被正常加载，就必须保证类不会被打上CLASS_ISPREVERIFIED标记。而要实现这个目的就必须要在分完包后的class中植入对其他dex文件中类的引用。

6.3.3 Tinker

QQ空间超级补丁方案在遇到补丁文件很大的时候耗时是非常严重的，因为一个大文件夹加载到内存中构建一个Element对象时，插入到数组最前端是需要耗费时间的，而这非常影响应用的启动速度。基于这些问题，微信提出了Tinker 方案。

Tinker的思路是，通过修复好的class.dex 和原有的class.dex比较差生差量包补丁文件 patch.dex，在手机上这个patch.dex又会和原有的class.dex 合并生成新的文件fix_class.dex，用这个新的fix_class.dex 整体替换原有的dexPathList中的内容，进而从根本上修复Bug，下图是演示图。



相比QQ空间超级补丁方案，Tinker 提供的思路可以说效率更高。对Tinker热修复方案感兴趣的同学可以去看看[Tinker 源码分析之DexDiff / DexPatch](#)

6.3.4 HotFix

以上提到的两种方式，虽然策略有所不同，但总的来说都是从上层ClassLoader的角度出发，由于ClassLoader的特点，如果想要新的补丁文件再次生效，无论你是插桩还是提前合并，都需要重新启动应用来加载新的DexPathList，从而实现Bug的修复。

AndFix 提供了一种运行时在Native修改Filed指针的方式，实现方法的替换，达到即时生效无需重启，对应用无性能消耗的目的。不过，由于Android在国内变成了安卓，各大手机厂商定制了自己的ROM，所以很多底层实现的差异，导致AndFix的兼容性并不是很好。

6.3.5 Sophix

Sophix采用的是类似类修复反射注入方式,把补丁so库的路径插入到nativeLibraryDirectories数组的最前面,这样加载so库的时候就是补丁so库而不是原来的so库。

在修复类代码的缺陷时，Sophix对旧包与补丁包中classes.dex的顺序进行了打破与重组，使得系统可以自然地识别到这个顺序,以实现类覆盖的目的。

在修复资源的缺陷时，Sophix构造了一个package id 为 0x66 的资源包，这个包里只包含改变了的资源项，然后直接在原有AssetManager中addAssetPath这个包即可，无需变更AssetManager对象的引用。

除了这些方案外，热修复方案还有美团的Robust、饿了么的Amigo等。不过，对于Android的热修复来说，很难有一种十分完美的解决方案。比如，在Android开发中，四大组件使用前需要在AndroidManifest中提前声明，而如果需要使用热修复的方式，无论是提前占坑亦或是动态修改，都会带来很强的侵入性。同时，Android碎片化的问题，对热修复方案的适配也是一大考验。

第三章 Android 面试之必问高级知识点

1, 编译模式

1.1 概念

在Android早期的版本中, 应用程序的运行环境是需要依赖Dalvik虚拟机的。不过, 在后来的版本(大概是4.x版本), Android的运行环境却换到了 Android Runtime, 其处理应用程序执行的方式完全不同于 Dalvik, Dalvik 是依靠一个 Just-In-Time (JIT) 编译器去解释字节码。

不过, Dalvik模式下, 开发者编译后的应用代码需要通过一个解释器在用户的设备上运行, 这一机制并不高效, 但让应用能更容易在不同硬件和架构上运行。ART 则完全改变了这套做法, 在应用安装时就预编译字节码到机器语言, 这一机制叫 Ahead-Of-Time (AOT) 编译。在移除解释代码这一过程后, 应用程序执行效率更高、启动也更快。

1.2 AOT优点

下面是AOT编译方式的一些优点:

1.2.1 预先编译

ART 引入了预先编译机制, 可提高应用的性能。ART 还具有比 Dalvik 更严格的安装时验证。在安装时, ART 使用设备自带的 dex2oat 工具来编译应用。该实用工具接受 DEX 文件作为输入, 并为目标设备生成经过编译的应用可执行文件, 该工具能够顺利编译所有有效的 DEX 文件。

1.2.2 垃圾回收优化

垃圾回收 (GC) 可能有损于应用性能, 从而导致显示不稳定、界面响应速度缓慢以及其他问题。ART模式从以下几个方面优化了垃圾回收的策略:

- 只有一次 (而非两次) GC 暂停
- 在 GC 保持暂停状态期间并行处理
- 在清理最近分配的短对象这种特殊情况中, 回收器的总 GC 时间更短
- 优化了垃圾回收的工效, 能够更加及时地进行并行垃圾回收, 这使得 GC_FOR_ALLOC 事件在典型用例中极为罕见
- 压缩 GC 以减少后台内存使用和碎片

1.2.3 开发和调试方面的优化

支持采样分析器

一直以来, 开发者都使用 Traceview 工具 (用于跟踪应用执行情况) 作为分析器。虽然 Traceview 可提供有用的信息, 但每次方法调用产生的开销会导致 Dalvik 分析结果出现偏差, 而且使用该工具明显会影响运行时性能。ART 添加了对没有这些限制的专用采样分析器的支持, 因而可更准确地了解应用执行情况, 而不会明显减慢速度。支持的版本从 KitKat (4.4) 版本开始, 为 Dalvik 的 Traceview 添加了采样支持。

支持更多调试功能

ART 支持许多新的调试选项, 特别是与监控和垃圾回收相关的功能。例如, 查看堆栈跟踪中保留了哪些锁, 然后跳转到持有锁的线程; 询问指定类的当前活动的实例数、请求查看实例, 以及查看使对象保持有效状态的参考; 过滤特定实例的事件 (如断点) 等。

优化了异常和崩溃报告中的诊断详细信息

当发生运行时异常时，ART 会为您提供尽可能多的上下文和详细信息。ART 会提供 `java.lang.ClassCastException`、`java.lang.ClassNotFoundException` 和 `java.lang.NullPointerException` 的更多异常详细信息（较高版本的 Dalvik 会提供 `java.lang.ArrayIndexOutOfBoundsException` 和 `java.lang.ArrayStoreException` 的更多异常详细信息，这些信息现在包括数组大小和越界偏移量；ART 也提供这类信息）。

1.3 垃圾回收

ART 提供了多个不同的 GC 方案，这些方案运行着不同垃圾回收器，默认的 GC 方案是 CMS（并发标记清除），主要使用粘性 CMS 和部分 CMS。粘性 CMS 是 ART 的不移动分代垃圾回收器。它仅扫描堆中自上次 GC 后修改的部分，并且只能回收自上次 GC 后分配的对象。除 CMS 方案外，当应用将进程状态更改为察觉不到卡顿的进程状态（例如，后台或缓存）时，ART 将执行堆压缩。

除了新的垃圾回收器之外，ART 还引入了一种基于位图的新内存分配程序，称为 RosAlloc（插槽运行分配器）。此新分配器具有分片锁，当分配规模较小时可添加线程的本地缓冲区，因而性能优于 DIMalloc（内存分配器）。

内存分配器的相关知识可以参考：[内存分配器](#)

同时，与 Dalvik 相比，ART 的 CMS 垃圾回收也带来了其他方面的改善，如下：

- 与 Dalvik 相比，暂停次数从 2 次减少到 1 次。Dalvik 的第一次暂停主要是为了进行根标记，即在 ART 中进行并发标记，让线程标记自己的根，然后马上恢复运行。
- 与 Dalvik 类似，ART GC 在清除过程开始之前也会暂停 1 次。两者在这方面的主要差异在于：在此暂停期间，某些 Dalvik 环节在 ART 中并发进行。这些环节包括 `java.lang.ref.Reference` 处理、系统弱清除（例如，jni 弱全局等）、重新标记非线性根和卡片预清理。在 ART 暂停期间仍进行的阶段包括扫描脏卡片以及重新标记线程根，这些操作有助于缩短暂停时间。
- 相对于 Dalvik，ART GC 改进的最后一个方面是粘性 CMS 回收器增加了 GC 吞吐量。不同于普通的分代 GC，粘性 CMS 不移动。系统会将年轻对象保存在一个分配堆栈（基本上是 `java.lang.Object` 数组）中，而非为其设置一个专属区域。这样可以避免移动所需的对象以维持低暂停次数，但缺点是容易在堆栈中加入大量复杂对象图像而使堆栈变长。

ART GC 与 Dalvik 的另一个主要区别在于 ART GC 引入了移动垃圾回收器。使用移动 GC 的目的在于通过堆压缩来减少后台应用使用的内存。目前，触发堆压缩的事件是 ActivityManager 进程状态的变化。当应用转到后台运行时，它会通知 ART 已进入不再“感知”卡顿的进程状态。此时 ART 会进行一些操作（例如，压缩和监视器压缩），从而导致应用线程长时间暂停。

目前，Android 的 ART 正在使用的两个移动 GC 是同构空间压缩和半空间压缩，它们的区别如下：

- **半空间压缩**：将对象在两个紧密排列的碰撞指针空间之间进行移动。这种移动 GC 适用于小内存设备，因为它可以比同构空间压缩稍微多节省一点内存，额外节省出的空间主要来自紧密排列的对象，这样可以避免 RosAlloc/DIMalloc 分配器占用开销。
- 同构空间压缩通过将对象从一个 RosAlloc 空间复制到另一个 RosAlloc 空间来实现。这有助于通过减少堆碎片来减少内存使用量。这是目前非低内存设备的默认压缩模式。相比半空间压缩，同构空间压缩的主要优势在于应用从后台切换到前台时无需进行堆转换。

2，类加载器

2.1 类加载器分类

目前，Android 的类加载器从下到上主要分为 BootstrapClassLoader（根类加载器）、ExtensionClassLoader（扩展类加载器）和 AppClassLoader（应用类加载器）三种。

- **根类加载器**：该加载器没有父加载器。它负责加载虚拟机的核心类库，如 `java.lang.*` 等。例如 `java.lang.Object` 就是由根类加载器加载的。根类加载器从系统属性 `sun.boot.class.path` 所指定的

目录中加载类库。根类加载器的实现依赖于底层操作系统，属于虚拟机的实现的一部分，它并没有继承java.lang.ClassLoader类。

- **扩展类加载器**：它的父加载器为根类加载器。它从java.ext.dirs系统属性所指定的目录中加载类库，或者从JDK的安装目录的jre/lib/ext子目录（扩展目录）下加载类库，如果把用户创建的JAR文件放在这个目录下，也会自动由扩展类加载器加载。扩展类加载器是纯Java类，是java.lang.ClassLoader类的子类。
 - **系统类加载器**：也称为应用类加载器，它的父加载器为扩展类加载器。它从环境变量classpath或者系统属性java.class.path所指定的目录中加载类，它是用户自定义的类加载器的默认父加载器。系统类加载器是纯Java类，是java.lang.ClassLoader类的子类。
- 父子加载器并非继承关系，也就是说子加载器不一定是继承了父加载器。

2.2 双亲委托模式

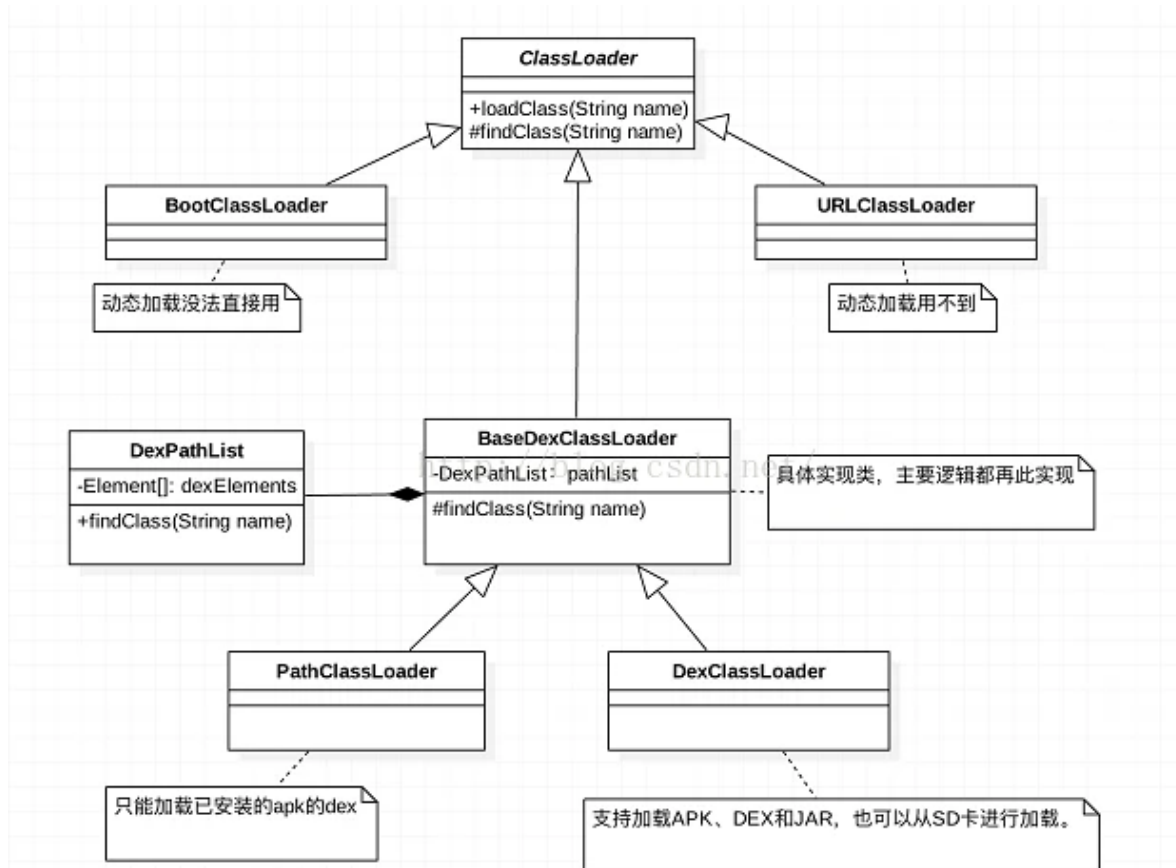
所谓双亲委托模式，指的是某个特定的类加载器在接到加载类的请求时，首先将加载任务委托给父类加载器，依次递归，如果父类加载器可以完成类加载任务，就成功返回；只有父类加载器无法完成此加载任务时，才自己去加载。

因为这样可以避免重复加载，当父亲已经加载了该类的时候，就没有必要子ClassLoader再加载一次。如果不使用这种委托模式，那我们就可以随时使用自定义的类来动态替代一些核心的类，存在非常大的安全隐患。

举个例子，事实上，java.lang.String这个类并不会被我们自定义的classloader加载，而是由bootstrap classloader进行加载，为什么会这样？实际上这就是双亲委托模式的原因，因为在任何一个自定义ClassLoader加载一个类之前，它都会先委托它的父亲ClassLoader进行加载，只有当父亲ClassLoader无法加载成功后，才会由自己加载。

2.3 Android的类加载器

下面是Android类加载器的模型图：



下面看一下DexClassLoader，DexClassLoader重载了findClass方法，在加载类时会调用其内部的DexPathList去加载。DexPathList是在构造DexClassLoader时生成的，其内部包含了DexFile，涉及的源码如下。

```

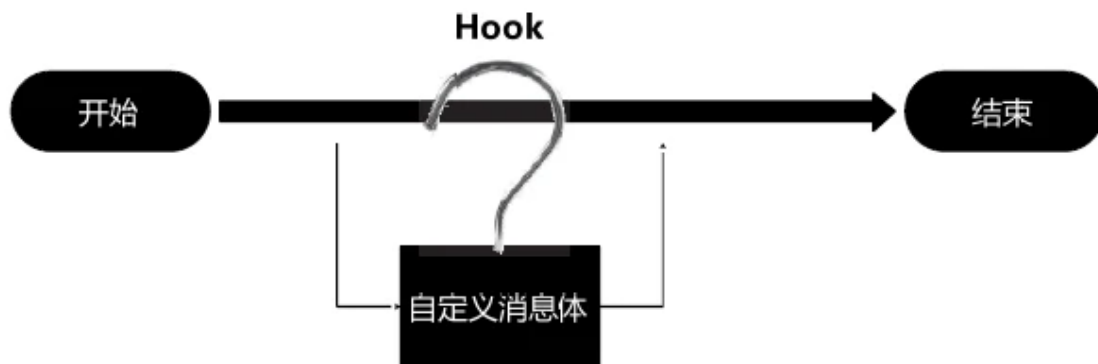
...
public Class findClass(String name) {
    for (Element element : dexElements) {
        DexFile dex = element.dexFile;
        if (dex != null) {
            Class clazz = dex.loadClassBinaryName(name, definingContext);
            if (clazz != null) {
                return clazz;
            }
        }
    }
    return null;
}
...

```

类加载器更多的内容，可以参考：[android 类加载器双亲委托模式](#)

3, Android Hook

所谓Hook，就是在程序执行的过程中去截取其中的某段信息，示意图如下。



Android的Hook大体的流程可以分为如下几步：

- 1、根据需求确定需要 hook 的对象
- 2、寻找要hook的对象的持有者，拿到需要 hook 的对象
- 3、定义“要 hook 的对象”的代理类，并且创建该类的对象
- 4、使用上一步创建出来的对象，替换掉要 hook 的对象

下面是一段简单的Hook的示例代码，用到了Java的反射机制。

```

@SuppressLint("DiscouragedPrivateApi", "PrivateApi")
public static void hook(Context context, final View view) {
    try {
        // 反射执行View类的getListenerInfo()方法，拿到v的mListenerInfo对象，这个对象就是
        // 点击事件的持有者
        Method method = view.getClass().getDeclaredMethod("getListenerInfo");
        method.setAccessible(true); // 由于getListenerInfo()方法并不是public的，所以要
        // 加这个代码来保证访问权限
        Object mListenerInfo = method.invoke(view); // 这里拿到的就是mListenerInfo对
        // 象，也就是点击事件的持有者

        // 要从这里面拿到当前的点击事件对象
        Class<?> listenerInfoClz =
        Class.forName("android.view.View$ListenerInfo"); // 这是内部类的表示方法
        Field field = listenerInfoClz.getDeclaredField("mOnClickListener");
        final View.OnClickListener onClickListenerInstance =
        (View.OnClickListener) field.get(mListenerInfo); // 取得真实的mOnClickListener对象
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```



```

// 2. 创建我们自己的点击事件代理类
// 方式1: 自己创建代理类
// ProxyOnClickListener proxyOnClickListener = new
ProxyOnClickListener(onClickListenerInstance);
// 方式2: 由于View.OnClickListener是一个接口, 所以可以直接用动态代理模式
// Proxy.newProxyInstance的3个参数依次分别是:
// 本地的类加载器;
// 代理类的对象所继承的接口 (用Class数组表示, 支持多个接口)
// 代理类的实际逻辑, 封装在new出来的InvocationHandler内
Object proxyOnClickListener =
Proxy.newProxyInstance(context.getClass().getClassLoader(), new Class[]
{View.OnClickListener.class}, new InvocationHandler() {
    @Override
    public Object invoke(Object proxy, Method method, Object[] args)
throws Throwable {
        Log.d("HookSetOnClickListener", "点击事件被hook到了");//加入自己的逻辑

        return method.invoke(onClickListenerInstance, args);//执行被代理的
对象的逻辑
    }
});
// 3. 用我们自己的点击事件代理类, 设置到"持有者"中
field.set(mListenerInfo, proxyOnClickListener);
} catch (Exception e) {
    e.printStackTrace();
}
}

// 自定义代理类
static class ProxyOnClickListener implements View.OnClickListener {
    View.OnClickListener oriLis;

    public ProxyOnClickListener(View.OnClickListener oriLis) {
        this.oriLis = oriLis;
    }

    @Override
    public void onClick(View v) {
        Log.d("HookSetOnClickListener", "点击事件被hook到了");
        if (oriLis != null) {
            oriLis.onClick(v);
        }
    }
}
}

```

而在Android开发中, 想要实现Hook, 肯定是没有这么简单的, 我们需要借助一些Hook框架, 比如Xposed、Cydia Substrate、Legend等。

参考资料: [Android Hook机制](#)

4, 代码混淆

4.1 Proguard

众所周知，Java代码是很容易反编译的，为了更好的保护Java源代码，我们往往会对编译好的Class类文件进行混淆处理。而ProGuard就是一个混淆代码的开源项目。它的主要作用就是混淆，当然它还能对字节码进行缩减体积、优化等，但那些对于我们来说都算是次要的功能。

具体来说，ProGuard具有如下功能：

- 压缩 (Shrink)：检测和删除没有使用的类，字段，方法和特性。
- 优化 (Optimize)：分析和优化Java字节码。
- 混淆 (Obfuscate)：使用简短的无意义的名称，对类，字段和方法进行重命名。

在Android开发中，开启混淆需要将app/build.gradle文件下的minifyEnabled属性设置为true，如下所示。

```
minifyEnabled true
proguardFiles getDefaultProguardFile('proguard-android.txt'), 'proguard-rules.pro'
```

proguard-android.txt是Android提供的默认混淆配置文件，我们需要的混淆的规则都放在这个文件中。

4.2 混淆规则

混淆命令

- keep：保留类和类中的成员，防止被混淆或移除
- keepnames：保留类和类中的成员，防止被混淆，成员没有被引用会被移除
- keepclassmembers：只保留类中的成员，防止被混淆或移除
- keepclassmembersnames：只保留类中的成员，防止被混淆，成员没有引用会被移除
- keepclasseswithmembers：保留类和类中的成员，防止被混淆或移除，保留指明的成员
- keepclasseswithmembersnames：保留类和类中的成员，防止被混淆，保留指明的成员，成员没有引用会被移除

混淆通配符

- `<field>`：匹配类中的所有字段
- `<method>`：匹配类中所有的方法
- `<init>`：匹配类中所有的构造函数
- `*`：匹配任意长度字符，不包含包名分隔符(.)
- `**`：匹配任意长度字符，包含包名分隔符(.)
- `***`：匹配任意参数类型

keep的规则格式如下：

```
[keep命令] [类] {
    [成员]
}
```

4.3 混淆模版

ProGuard中有些公共的模版是可以复用的，比如压缩比、大小写混合和一些系统提供的Activity、Service不能混淆等。

```
# 代码混淆压缩比，在 0~7 之间，默认为 5，一般不做修改
-optimizationpasses 5

# 混合时不使用大小写混合，混合后的类名为小写
```

-dontusemixedcaseclassnames

指定不去忽略非公共库的类

-dontskipnonpubliclibraryclasses

这句话能够使我们的项目混淆后产生映射文件

包含有类名->混淆后类名的映射关系

-verbose

指定不去忽略非公共库的类成员

-dontskipnonpubliclibraryclassmembers

不做预校验, preverify 是 proguard 的四个步骤之一, Android 不需要 preverify, 去掉这一步能够加快混淆速度。

-dontpreverify

保留 Annotation 不混淆

-keepattributes *Annotation*,InnerClasses

避免混淆泛型

-keepattributes Signature

抛出异常时保留代码行号

-keepattributes SourceFile,LineNumberTable

指定混淆是采用的算法, 后面的参数是一个过滤器

这个过滤器是谷歌推荐的算法, 一般不做更改

-optimizations !code/simplification/cast,!field/*,!class/merging/*

#####

#

Android开发中一些需要保留的公共部分

#

#####

保留我们使用的四大组件, 自定义的 Application 等等这些类不被混淆

因为这些子类都有可能被外部调用

-keep public class * extends android.app.Activity

-keep public class * extends android.app.Application

-keep public class * extends android.app.Service

-keep public class * extends android.content.BroadcastReceiver

-keep public class * extends android.content.ContentProvider

-keep public class * extends android.app.backup.BackupAgentHelper

-keep public class * extends android.preference.Preference

-keep public class * extends android.view.View

-keep public class com.android.vending.licensing.ILicensingService

保留 support 下的所有类及其内部类

-keep class android.support.** { *; }

保留继承的

-keep public class * extends android.support.v4.**

-keep public class * extends android.support.v7.**

-keep public class * extends android.support.annotation.**

保留 R 下面的资源


```

-keep class **.R$* { *; }

# 保留本地 native 方法不被混淆
-keepclasseswithmembernames class * {
    native <methods>;
}

# 保留在 Activity 中的方法参数是view的方法，
# 这样以来我们在 layout 中写的 onClick 就不会被影响
-keepclassmembers class * extends android.app.Activity {
    public void *(android.view.View);
}

# 保留枚举类不被混淆
-keepclassmembers enum * {
    public static **[] values();
    public static ** valueOf(java.lang.String);
}

# 保留我们自定义控件（继承自 View）不被混淆
-keep public class * extends android.view.View {
    *** get*();
    void set*(***);
    public <init>(android.content.Context);
    public <init>(android.content.Context, android.util.AttributeSet);
    public <init>(android.content.Context, android.util.AttributeSet, int);
}

# 保留 Parcelable 序列化类不被混淆
-keep class * implements android.os.Parcelable {
    public static final android.os.Parcelable$Creator *;
}

# 保留 Serializable 序列化的类不被混淆
-keepnames class * implements java.io.Serializable
-keepclassmembers class * implements java.io.Serializable {
    static final long serialVersionUID;
    private static final java.io.ObjectStreamField[] serialPersistentFields;
    !static !transient <fields>;
    !private <fields>;
    !private <methods>;
    private void writeObject(java.io.ObjectOutputStream);
    private void readObject(java.io.ObjectInputStream);
    java.lang.Object writeReplace();
    java.lang.Object readResolve();
}

# 对于带有回调函数的 onXXEvent、**On*Listener 的，不能被混淆
-keepclassmembers class * {
    void **(**On*Event);
    void **(**On*Listener);
}

# webView 处理，项目中没有使用到 webView 忽略即可
-keepclassmembers class fqcn.of.javascript.interface.for.webview {
    public *;
}
-keepclassmembers class * extends android.webkit.WebViewClient {

```

```

        public void *(android.webkit.Webview, java.lang.String,
        android.graphics.Bitmap);
        public boolean *(android.webkit.Webview, java.lang.String);
    }
    -keepclassmembers class * extends android.webkit.webviewClient {
        public void *(android.webkit.webview, java.lang.String);
    }

    # js
    -keepattributes JavascriptInterface
    -keep class android.webkit.JavascriptInterface { *; }
    -keepclassmembers class * {
        @android.webkit.JavascriptInterface <methods>;
    }

    # @Keep
    -keep,allowobfuscation @interface android.support.annotation.Keep
    -keep @android.support.annotation.Keep class *
    -keepclassmembers class * {
        @android.support.annotation.Keep *;
    }

```

如果是aar这种插件，可以在aar的build.gralde中添加如下混淆配置。

```

android {
    ...
    defaultConfig {
        ...
        consumerProguardFile 'proguard-rules.pro'
    }
    ...
}

```

5, NDK

如果要问Android的高级开发知识，那么NDK肯定是必问的。那么什么的NDK，NDK 全称是 Native Development Kit，是一组可以让开发者在 Android 应用中使用C/C++ 的工具。通常，NDK可以用在如下的场景中：

- 从设备获取更好的性能以用于计算密集型应用，例如游戏或物理模拟。
- 重复使用自己或其他开发者的 C/C++ 库，便于跨平台。
- NDK 集成了譬如 OpenGL、Vulkan 等 API 规范的特定实现，以实现在 Java 层无法做到的功能，如音视频开发、渲染。
- 增加反编译难度。

5.1, JNI基础

JNI即java native interface，是Java和Native代码进行交互的接口。

5.1.1 JNI 访问 Java 对象方法

假如，有如下一个Java类，代码如下。

```

package com.xzh.jni;

public class MyJob {

```

```

public static String JOB_STRING = "my_job";
private int jobId;

public MyJob(int jobId) {
    this.jobId = jobId;
}

public int getJobId() {
    return jobId;
}
}

```

然后，在cpp目录下，新建native_lib.cpp，添加对应的native实现。

```

#include <jni.h>

extern "C"
JNIEXPORT jint JNICALL
Java_com_xzh_jni_MainActivity_getJobId(JNIEnv *env, jobject thiz, jobject job) {

    // 根据实例获取 class 对象
    jclass jobClz = env->GetObjectClass(job);
    // 根据类名获取 class 对象
    jclass jobClz = env->FindClass("com/xzh/jni/MyJob");

    // 获取属性 id
    jfieldID fieldId = env->GetFieldID(jobClz, "jobId", "I");
    // 获取静态属性 id
    jfieldID sFieldId = env->GetStaticFieldID(jobClz, "JOB_STRING",
        "Ljava/lang/String;");

    // 获取方法 id
    jmethodID methodId = env->GetMethodID(jobClz, "getJobId", "()I");
    // 获取构造方法 id
    jmethodID initMethodId = env->GetMethodID(jobClz, "<init>", "(I)V");

    // 根据对象属性 id 获取该属性值
    jint id = env->GetIntField(job, fieldId);
    // 根据对象方法 id 调用该方法
    jint id = env->CallIntMethod(job, methodId);

    // 创建新的对象
    jobject newJob = env->NewObject(jobClz, initMethodId, 10);
    return id;
}

```

5.2 NDK开发

5.2.1 基本流程

首先，在Java代码中声明 Native 方法，如下所示。

```

public class MainActivity extends AppCompatActivity {

    // Used to load the 'native-lib' library on application startup.
    static {
        System.loadLibrary("native-lib");
    }
}

```

```

    }

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        Log.d("MainActivity", stringFromJNI());
    }
    private native String stringFromJNI();
}

```

然后，新建一个 cpp 目录，并且新建一个名为 native-lib.cpp 的 cpp 文件，实现相关方法。

```

#include <jni.h>

extern "C" JNIEXPORT jstring JNICALL
Java_com_xzh_jni_MainActivity_stringFromJNI(
    JNIEnv *env,
    jobject /* this */) {
    std::string hello = "Hello from C++";
    return env->NewStringUTF(hello.c_str());
}

```

cpp 文件遵循如下的规则：

- 函数名的格式遵循如下规则：Java *包名类名_方法名*。
- extern "C" 指定采用 C 语言的命名风格来编译，否则由于 C 与 C++ 风格不同，导致链接时无法找到具体的函数
- JNIEnv*: 表示一个指向 JNI 环境的指针，可以通过他来访问 JNI 提供的接口方法
- jobject: 表示 java 对象中的 this
- JNIEXPORT 和 JNICALL: JNI 所定义的宏，可以在 jni.h 头文件中查找到

System.loadLibrary() 的代码位于 java/lang/System.java 文件中，源码如下：

```

@CallerSensitive
public static void load(String filename) {
    Runtime.getRuntime().load0(Reflection.getCallerClass(), filename);
}

```

5.3 CMake 构建 NDK

CMake 是一个开源的跨平台工具系列，旨在构建、测试和打包软件，从 Android Studio 2.2 开始，Android Studio 默认地使用 CMake 与 Gradle 搭配使用来构建原生库。具体来说，我们可以使用 Gradle 将 C/C++ 代码编译到原生库中，然后将这些代码打包到我们的应用中，Java 代码随后可以通过 Java 原生接口 (JNI) 调用我们原生库中的函数。

使用 CMake 开发 NDK 项目需要下载如下一些套件：

- Android 原生开发工具包 (NDK): 这套工具集允许我们开发 Android 使用 C 和 C++ 代码，并提供众多平台库，让我们可以管理原生 Activity 和访问物理设备组件，例如传感器和触摸输入。
- CMake: 一款外部构建工具，可与 Gradle 搭配使用来构建原生库。如果你只计划使用 ndk-build，则不需要此组件。
- LLDB: 一种调试程序，Android Studio 使用它来调试原生代码。

我们可以打开Android Studio, 依次选择【Tools】>【Android】>【SDK Manager】>【SDK Tools】选中LLDB、CMake 和 NDK即可。

启用CMake还需要在 app/build.gradle 中添加如下代码。

```
android {  
    ...  
    defaultConfig {  
        ...  
        externalNativeBuild {  
            cmake {  
                cppFlags ""  
            }  
        }  
  
        ndk {  
            abiFilters 'arm64-v8a', 'armeabi-v7a'  
        }  
    }  
    ...  
    externalNativeBuild {  
        cmake {  
            path "CMakeLists.txt"  
        }  
    }  
}
```

然后, 在对应目录新建一个 CMakeLists.txt 文件, 添加代码。

```
# 定义了所需 CMake 的最低版本  
cmake_minimum_required(VERSION 3.4.1)  
  
# add_library() 命令用来添加库  
# native-lib 对应着生成的库的名字  
# SHARED 代表为分享库  
# src/main/cpp/native-lib.cpp 则是指明了源文件的路径。  
add_library( # Sets the name of the library.  
            native-lib  
  
            # Sets the library as a shared library.  
            SHARED  
  
            # Provides a relative path to your source file(s).  
            src/main/cpp/native-lib.cpp)  
  
# find_library 命令添加到 CMake 构建脚本中以定位 NDK 库, 并将其路径存储为一个变量。  
# 可以使用此变量在构建脚本的其他部分引用 NDK 库  
find_library( # Sets the name of the path variable.  
             log-lib  
  
             # Specifies the name of the NDK library that  
             # you want CMake to locate.  
             log)  
  
# 预构建的 NDK 库已经存在于 Android 平台上, 因此, 无需再构建或将其打包到 APK 中。  
# 由于 NDK 库已经是 CMake 搜索路径的一部分, 只需要向 CMake 提供希望使用的库的名称, 并将其关联到自己的原生库中
```

```
# 要将预构建库关联到自己的原生库
target_link_libraries( # Specifies the target library.
    native-lib

    # Links the target library to the log library
    # included in the NDK.
    ${log-lib})

...
```

参考: [Android NDK开发基础](#)

6, 动态加载

6.1 基本概念

动态加载技术在Web中很常见, 对于Android项目来说, 动态加载的目的是让用户不用重新安装APK就能升级应用的功能, 主要的应用场景是插件化和热修复。

首先需要明确的一点, 插件化和热修复不是同一个概念, 虽然站在技术实现的角度来说, 他们都是从系统加载器的角度出发, 无论是采用hook方式, 亦或是代理方式或者是其他底层实现, 都是通过“欺骗”Android 系统的方式来让宿主正常的加载和运行插件(补丁)中的内容; 但是二者的出发点是不同的。

插件化, 本质上是把需要实现的模块或功能当做一个独立的功能提取出来, 减少宿主的规模, 当需要使用时到相应的功能时再去加载相应的模块。而热修复则往往是从修复bug的角度出发, 强调的是在不需要二次安装应用的前提下修复已知的bug。

为了方便说明, 我们先理清几个概念:

- 宿主: 当前运行的APP。
- 插件: 相对于插件化技术来说, 就是要加载运行的apk类文件。
- 补丁: 相对于热修复技术来说, 就是要加载运行的.patch,.dex,* .apk等一系列包含dex修复内容的文件。

下图展示了Android动态化开发框架的整体的架构。



6.2 插件化

关于插件化技术，最早可以追溯到2012年的 [AndroidDynamicLoader](#)，其原理是动态加载不同的Fragment实现UI替换，不过随着15，16年更好的方案，这个方案渐渐的被淘汰了。再后来有了任玉刚的[dynamic-load-apk](#)方案，开始有了插件化的标准方案。而后面的方案大多基于Hook和动态代理两个方向进行。

目前，插件化的开发并没有一个官方的插件化方案，它是国内提出的一种技术实现，利用虚拟机的类的加载机制实现的一种技术手段，往往需要hook一些系统api，而Google从Android9.0开始限制对系统私有api的使用，也就造成了插件化的兼容性问题，现在几个流行的插件化技术框架，都是大厂根据自己的需求，开源出来的，如滴滴的VirtualAPK，360的RePlugin等，大家可以根据需要自行了解技术的实现原理。

6.3 热修复

6.3.1 热修复原理

说到热修复的原理，就不得不提到类的加载机制，和常规的JVM类似，在Android中类的加载也是通过ClassLoader来完成，具体来说就是PathClassLoader 和 DexClassLoader 这两个Android专用的类加载器，这两个类的区别如下。

- PathClassLoader：只能加载已经安装到Android系统中的apk文件（/data/app目录），是Android默认使用的类加载器。
- DexClassLoader：可以加载任意目录下的dex/jar/apk/zip文件，也就是我们一开始提到的补丁。

这两个类都是继承自BaseDexClassLoader，BaseDexClassLoader的构造函数如下。

```
public BaseDexClassLoader(String dexPath, File optimizedDirectory,
    String libraryPath, ClassLoader parent) {
    super(parent);
    this.pathList = new DexPathList(this, dexPath, libraryPath,
    optimizedDirectory);
}
```

这个构造函数只做了一件事，就是通过传递进来的相关参数，初始化了一个DexPathList对象。DexPathList的构造函数，就是将参数中传递进来的程序文件（就是补丁文件）封装成Element对象，并将这些对象添加到一个Element的数组集合dexElements中去。

前面说过类加载器的作用，就是将一个具体的类（class）加载到内存中，而这些操作是由虚拟机完成的，对于开发者来说，只需要关注如何去找到这个需要加载的类即可，这也是热修复需要干的事情。

在Android中，查找一个名为name的class需要经历如下两步：

1. 在DexClassLoader的findClass 方法中通过一个DexPathList对象findClass()方法来获取class。
2. 在DexPathList的findClass 方法中，对之前构造好dexElements数组集合进行遍历，一旦找到类名与name相同的类时，就直接返回这个class，找不到则返回null。

因此，基于上面的理论，我们可以想到一个最简单的热修复方案。假设现在代码中的某一个类出现Bug，那么我们可以在修复Bug之后，将这些个类打包成一个补丁文件，然后通过这个补丁文件封装出一个Element对象，并且将这个Element对象插到原有dexElements数组的最前端。这样，当DexClassLoader去加载类时，由于双亲加载机制的特点，就会优先加载插入的这个Element，而有缺陷的Element则没有机会再被加载。事实上，QQ早期的热修复方案就是这样的。

6.3.2 QQ 空间超级补丁方案

QQ 空间补丁方案就是使用javaassist 插桩的方式解决了CLASS_ISPREVERIFIED的难题。涉及的步骤如下：

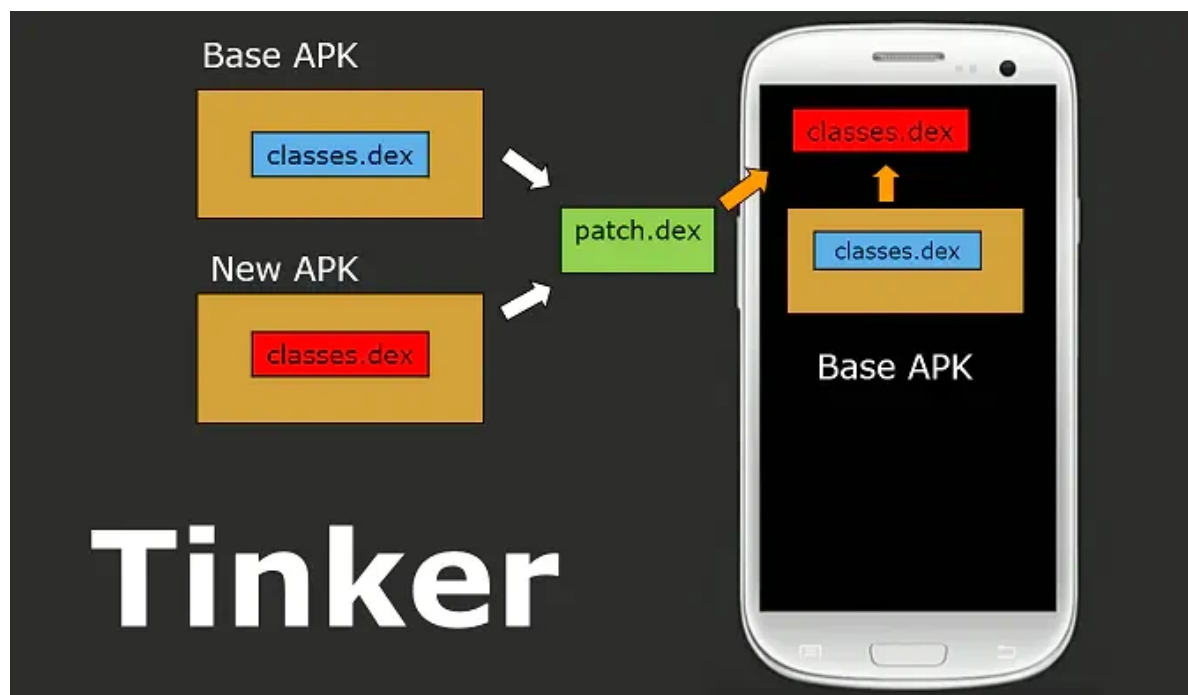
- 在apk安装的时候系统会将dex文件优化成odex文件，在优化的过程中会涉及一个预校验的过程。

- 如果一个类的static方法，private方法，override方法以及构造函数中引用了其他类，而且这些类都属于同一个dex文件，此时该类就会被打上CLASS_ISPREVERIFIED。
- 如果在运行时被打上CLASS_ISPREVERIFIED的类引用了其他dex的类，就会报错。
- 正常的分包方案会保证相关类被打入同一个dex文件。
- 想要使得patch可以被正常加载，就必须保证类不会被打上CLASS_ISPREVERIFIED标记。而要实现这个目的就必须要在分完包后的class中植入对其他dex文件中类的引用。

6.3.3 Tinker

QQ空间超级补丁方案在遇到补丁文件很大的时候耗时是非常严重的，因为一个大文件夹加载到内存中构建一个Element对象时，插入到数组最前端是需要耗费时间的，而这非常影响应用的启动速度。基于这些问题，微信提出了Tinker 方案。

Tinker的思路是，通过修复好的class.dex 和原有的class.dex比较差生差量包补丁文件patch.dex，在手机上这个patch.dex又会和原有的class.dex 合并生成新的文件fix_class.dex，用这个新的fix_class.dex 整体替换原有的dexPathList中的内容，进而从根本上修复Bug，下图是演示图。



相比QQ空间超级补丁方案，Tinker 提供的思路可以说效率更高。对Tinker热修复方案感兴趣的同学可以去看看[Tinker 源码分析之DexDiff / DexPatch](#)

6.3.4 HotFix

以上提到的两种方式，虽然策略有所不同，但总的来说都是从上层ClassLoader的角度出发，由于ClassLoader的特点，如果想要新的补丁文件再次生效，无论你是插桩还是提前合并，都需要重新启动应用来加载新的DexPathList，从而实现Bug的修复。

AndFix 提供了一种运行时在Native修改Filed指针的方式，实现方法的替换，达到即时生效无需重启，对应用无性能消耗的目的。不过，由于Android在国内变成了安卓，各大手机厂商定制了自己的ROM，所以很多底层实现的差异，导致AndFix的兼容性并不是很好。

6.3.5 Sophix

Sophix采用的是类似类修复反射注入方式,把补丁so库的路径插入到NativeLibraryDirectories数组的最前面,这样加载so库的时候就是补丁so库而不是原来的so库。

在修复类代码的缺陷时，Sophix对旧包与补丁包中classes.dex的顺序进行了打破与重组，使得系统可以自然地识别到这个顺序,以实现类覆盖的目的。

在修复资源的缺陷时，Sophix构造了一个package id 为 0x66 的资源包，这个包里只包含改变了的资源项，然后直接在原有AssetManager中addAssetPath这个包即可，无需变更AssetManager对象的引用。

除了这些方案外，热修复方案还有美团的Robust、饿了么的Amigo等。不过，对于Android的热修复来说，很难有一种十分完美的解决方案。比如，在Android开发中，四大组件使用前需要在AndroidManifest中提前声明，而如果需要使用热修复的方式，无论是提前占坑亦或是动态修改，都会带来很强的侵入性。同时，Android碎片化的问题，对热修复方案的适配也是一大考验。

参考：[Android 热修复的简析](#)
[深入探索Android热修复技术原理](#)

第四章 Android 面试之必问性能优化

1，启动优化

一个应用的启动快慢是能够直接影响用户的使用体验的，如果启动较慢可能会导致用户卸载放弃该应用程序。

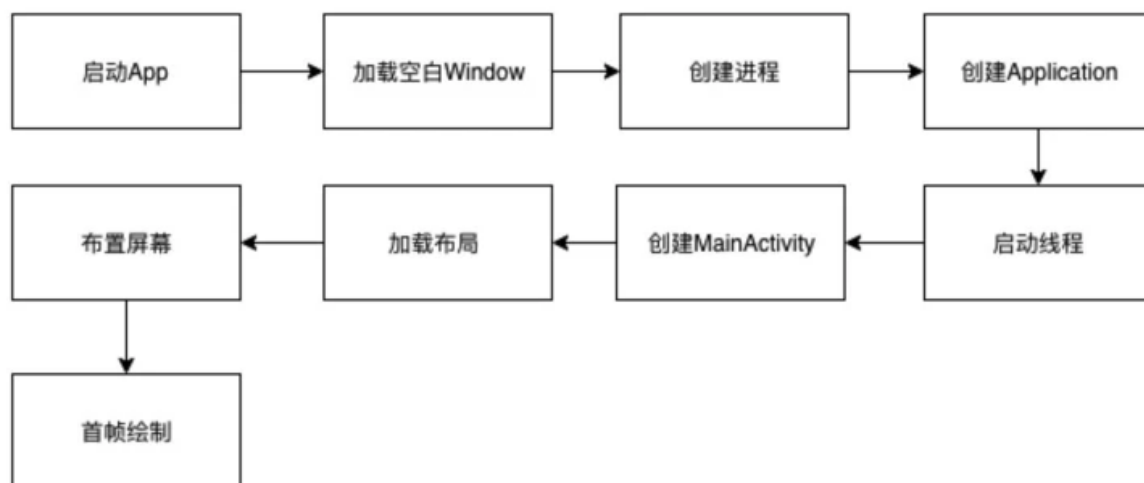
1.1 冷启动、热启动和温启动的优化

1.1.1 概念

对于Android应用程序来说，根据启动方式可以分为冷启动，热启动和温启动三种。

- 冷启动：系统不存在App进程（如APP首次启动或APP被完全杀死）时启动App称为冷启动。
- 热启动：按了Home键或其它情况app被切换到后台，再次启动App的过程。
- 温启动：温启动包含了冷启动的一些操作，不过App进程依然存在，这代表着它比热启动有更多的开销。

可以看到，热启动是启动最快的，温启动则是介于冷启动和热启动之间的一种启动方式。而冷启动则是最慢的，因为它会涉及很多进程的创建，下面是冷启动相关的任务流程：



1.1.2 视觉优化

在冷启动模式下，系统会启动三个任务：

- 加载并启动应用程序。
- 启动后立即显示应用程序空白的启动窗口。
- 创建应用程序进程。

一旦系统创建应用程序进程，应用程序进程就会进入下一阶段，并完成如下的一些事情。

- 创建app对象
- 启动主线程(main thread)
- 创建应用入口的Activity对象
- 填充加载布局View
- 在屏幕上执行View的绘制过程.measure -> layout -> draw

应用程序进程完成第一次绘制后，系统进程会交换当前显示的背景窗口，将其替换为主活动。此时，用户可以开始使用该应用程序了。因为App应用进程的创建过程是由手机的软硬件决定的，所以我们只能在这个创建过程中进行一些视觉优化。

1.1.3 启动主题优化

在冷启动的时候，当应用程序进程被创建后，就需要设置启动窗口的主题。目前，大部分的应用在启动时都会先进入一个闪屏页(LaunchActivity)来展示应用信息，如果在Application初始化了其它第三方的服务，就会出现启动的白屏问题。

为了更顺滑无缝衔接我们的闪屏页，可以在启动Activity的Theme中设置闪屏页图片，这样启动窗口的图片就会是闪屏页图片，而不是白屏。

```
<style name="AppTheme" parent="Theme.AppCompat.Light.NoActionBar">
    <item name="android:windowBackground">@drawable/lunch</item>    //闪屏页图片
    <item name="android:windowFullscreen">true</item>
    <item name="android:windowDrawsSystemBarBackgrounds">false</item>
</style>
```

1.2 代码方面的优化

设置主题的方式只能应用在要求不是很高的场景，并且这种优化治标不治本，关键还在于代码的优化。为了进行优化，我们需要掌握一些基本的数据。

1.2.1 冷启动耗时统计

ADB命令方式

在Android Studio的Terminal中输入以下命令可以查看页面的启动的时间，命令如下：

```
adb shell am start -w packagename/[packagename].首屏Activity
```

执行完成之后，会在控制台输出如下的信息：

```
Starting: Intent { act=android.intent.action.MAIN cat=
[android.intent.category.LAUNCHER] cmp=com.optimize.performance/.MainActivity }
Status: ok
Activity: com.optimize.performance/.MainActivity
ThisTime: 563
TotalTime: 563
WaitTime: 575
Complete
```

在上面的日志中有三个字段信息，即ThisTime、TotalTime和WaitTime。

- **ThisTime**：最后一个Activity启动耗时
- **TotalTime**：所有Activity启动耗时
- **WaitTime**：AMS启动Activity的总耗时

日志方式

埋点方式是另一种统计线上时间的方式，这种方式通过记录启动时的时间和结束的时间，然后取二者差值即可。首先，需要定义一个统计时间的工具类：

```
class LaunchRecord {  
  
    companion object {  
  
        private var sStart: Long = 0  
  
        fun startRecord() {  
            sStart = System.currentTimeMillis()  
        }  
  
        fun endRecord() {  
            endRecord("")  
        }  
  
        fun endRecord(postion: String) {  
            val cost = System.currentTimeMillis() - sStart  
            println("===$postion===$cost")  
        }  
    }  
}
```

启动时埋点我们直接在Application的attachBaseContext中进行打点。那么启动结束应该在哪里打点呢？结束埋点建议是在页面数据展示出来进行埋点。可以使用如下方法：

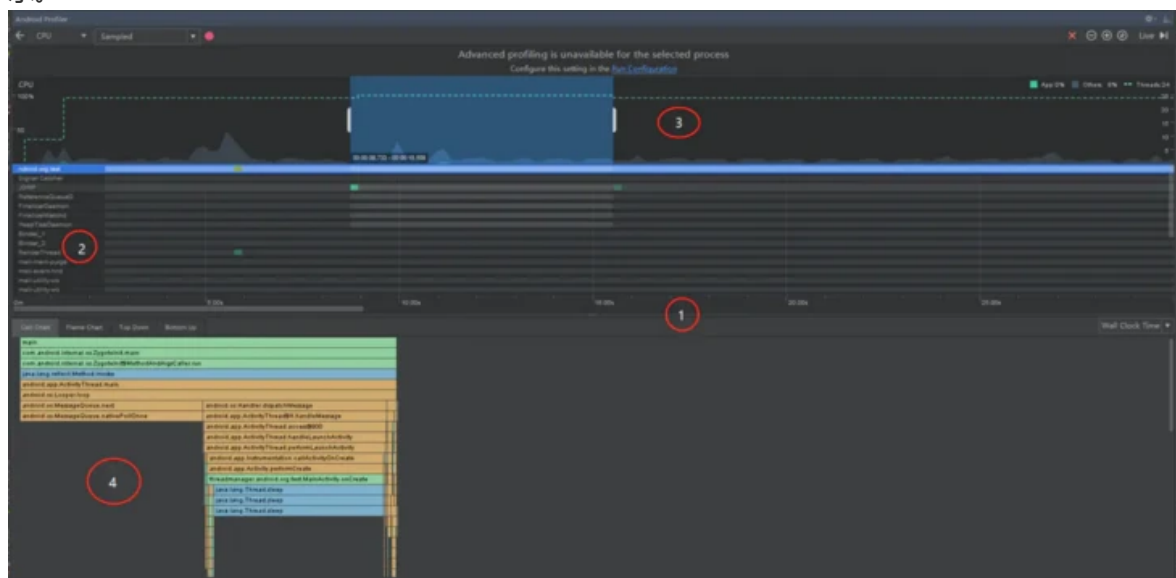
```
class MainActivity : AppCompatActivity() {  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        mTextView.viewTreeObserver.addOnDrawListener {  
            LaunchRecord.endRecord("onDraw")  
        }  
    }  
  
    override fun onWindowFocusChanged(hasFocus: Boolean) {  
        super.onWindowFocusChanged(hasFocus)  
        LaunchRecord.endRecord("onWindowFocusChanged")  
    }  
}
```

1.2.2 优化检测工具

在做启动优化的时候，可以借助三方工具来帮助我们理清各个阶段的方法或者线程、CPU的执行耗时等情况。这里主要介绍以下TraceView和SysTrace两款工具。

TraceView

TraceView是以图形的形式展示执行时间、调用栈等信息，信息比较全面，包含所有线程，如下图所示。



使用TraceView检测生成的结果会放在Andriid/data/packageName/files路径下。因为Traceview收集的信息比较全面，所以会导致运行开销严重，整体APP的运行会变慢，因此我们无法区分是不是Traceview影响了我们的启动时间。

SysTrace

SysTrace是结合Android内核数据，生成HTML报告，从报告中我们可以看到各个线程的执行时间以及方法耗时和CPU执行时间等。



再API 18以上版本，可以直接使用TraceCompat来抓取数据，因为这是兼容的API。

```
开始: TraceCompat.beginSection("tag ")
结束: TraceCompat.endSection()
```

然后，执行如下脚本。

```
python systrace.py -b 32768 -t 10 -a packagename -o outputfile.html sched gfx
view wm am app
```

这里可以大家普及下各个字端的含义：

- b：收集数据的大小
- t：时间
- a：监听的应用包名
- o：生成文件的名称

Systrace开销较小，属于轻量级的工具，并且可以直观反映CPU的利用率。

2，UI渲染优化

Android系统每隔16ms就会重新绘制一次Activity，因此，我们的应用必须在16ms内完成屏幕刷新的全部逻辑操作，每一帧只能停留16ms，否则就会出现掉帧现象。Android应用卡顿与否与UI渲染有直接的关系。

2.1 CPU、GPU

对于大多数手机的屏幕刷新频率是60hz，也就是如果在 $1000/60=16.67\text{ms}$ 内没有把这一帧的任务执行完毕，就会发生丢帧的现象，丢帧是造成界面卡顿的直接原因，渲染操作通常依赖于两个核心组件：CPU与GPU。CPU负责包括Measure，Layout等计算操作，GPU负责Rasterization（栅格化）操作。

所谓栅格化，就是将矢量图形转换为位图的过程，手机上显示是按照一个个像素来显示的，比如将一个Button、TextView等组件拆分成一个个像素显示到手机屏幕上。而UI渲染优化的目的就是减轻CPU、GPU的压力，除去不必要的操作，保证每帧16ms以内处理完所有的CPU与GPU的计算、绘制、渲染等操作，使UI顺滑、流畅的显示出来。

2.2 过度绘制

UI渲染优化的第一步就是找到Overdraw（过度绘制），即描述的是屏幕上的某个像素在同一帧的时间内被绘制了多次。在重叠的UI布局中，如果不可见的UI也在做绘制的操作或者后一个控件将前一个控件遮挡，会导致某些像素区域被绘制了多次，从而增加了CPU、GPU的压力。

那么如何找出布局中Overdraw的地方呢？很简单，就是打开手机里开发者选项，然后将调试GPU过度绘制的开关打开即可，然后就可以看到应用的布局是否被Overdraw，如下图所示。

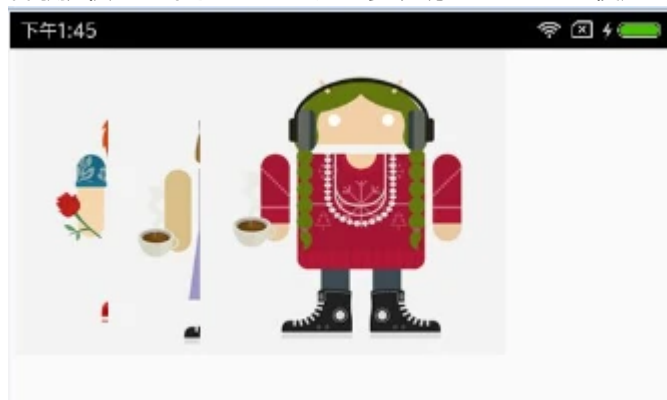


蓝色、淡绿、淡红、深红代表了4种不同程度的Overdraw情况，1x、2x、3x和4x分别表示同一像素上同一帧的时间内被绘制了多次，1x就表示一次(最理想情况)，4x表示4次(最差的情况)，而我们需要消除的就是3x和4x。

2.3 解决自定义View的OverDraw

我们知道，自定义View的时候有时会重写onDraw方法，但是Android系统是无法检测onDraw里面具体会执行什么操作，从而系统无法为我们做一些优化。这样对编程人员要求就高了，如果View有大量重叠的地方就会造成CPU、GPU资源的浪费，此时我们可以使用`canvas.clipRect()`来帮助系统识别那些可见的区域。

这个方法可以指定一块矩形区域，只有在这个区域内才会被绘制，其他的区域会被忽视。下面我们通过谷歌提供的一个小的Demo进一步说明OverDraw的使用。



在下面的代码中，DroidCard类封装的是卡片的信息，代码如下。

```
public class DroidCard {  
  
    public int x;//左侧绘制起点  
    public int width;  
    public int height;
```



```

public Bitmap bitmap;

public DroidCard(Resources res,int resId,int x){
    this.bitmap = BitmapFactory.decodeResource(res,resId);
    this.x = x;
    this.width = this.bitmap.getWidth();
    this.height = this.bitmap.getHeight();
}
}

```

自定义View的代码如下：

```

public class DroidCardsView extends View {
    //图片与图片之间的间距
    private int mCardSpacing = 150;
    //图片与左侧距离的记录
    private int mCardLeft = 10;

    private List<DroidCard> mDroidCards = new ArrayList<DroidCard>();

    private Paint paint = new Paint();

    public DroidCardsView(Context context) {
        super(context);
        initCards();
    }

    public DroidCardsView(Context context, AttributeSet attrs) {
        super(context, attrs);
        initCards();
    }
    /**
     * 初始化卡片集合
     */
    protected void initCards(){
        Resources res = getResources();
        mDroidCards.add(new DroidCard(res,R.drawable.alex,mCardLeft));

        mCardLeft+=mCardSpacing;
        mDroidCards.add(new DroidCard(res,R.drawable.claire,mCardLeft));

        mCardLeft+=mCardSpacing;
        mDroidCards.add(new DroidCard(res,R.drawable.kathryn,mCardLeft));
    }

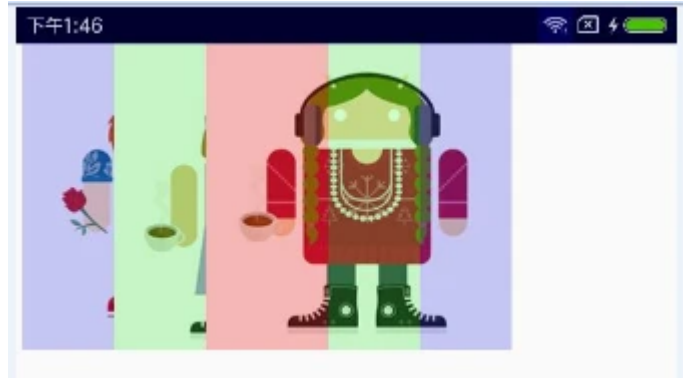
    @Override
    protected void onDraw(Canvas canvas) {
        super.onDraw(canvas);
        for (DroidCard c : mDroidCards){
            drawDroidCard(canvas, c);
        }
        invalidate();
    }

    /**
     * 绘制DroidCard
     */
}

```

```
private void drawDroidCard(Canvas canvas, DroidCard c) {
    canvas.drawBitmap(c.bitmap,c.x,0f,paint);
}
}
```

然后，我们运行代码，打开手机的overdraw开关，效果如下：



可以看到，淡红色区域明显被绘制了三次，是因为图片的重叠造成的。那怎么解决这种问题呢？其实，分析可以发现，最下面的图片只需要绘制三分之一即可，保证最下面两张图片只需要回执其三分之一最上面图片完全绘制出来就可。优化后的代码如下：

```
public class DroidCardsView extends View {

    //图片与图片之间的间距
    private int mCardSpacing = 150;
    //图片与左侧距离的记录
    private int mCardLeft = 10;

    private List<DroidCard> mDroidCards = new ArrayList<DroidCard>();

    private Paint paint = new Paint();

    public DroidCardsView(Context context) {
        super(context);
        initCards();
    }

    public DroidCardsView(Context context, AttributeSet attrs) {
        super(context, attrs);
        initCards();
    }
    /**
     * 初始化卡片集合
     */
    protected void initCards(){
        Resources res = getResources();
        mDroidCards.add(new DroidCard(res, R.drawable.alex,mCardLeft));

        mCardLeft+=mCardSpacing;
        mDroidCards.add(new DroidCard(res, R.drawable.claire,mCardLeft));

        mCardLeft+=mCardSpacing;
        mDroidCards.add(new DroidCard(res, R.drawable.kathryn,mCardLeft));
    }

    @Override
    protected void onDraw(Canvas canvas) {
```

```

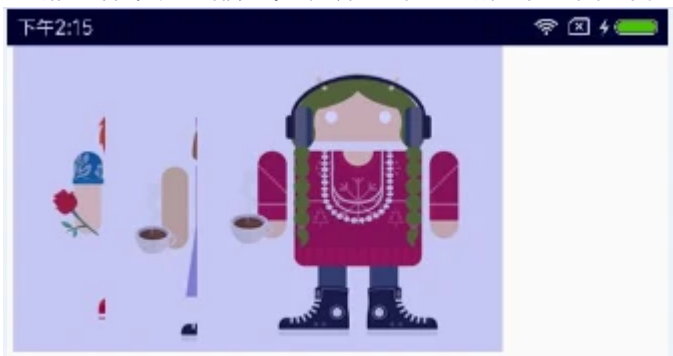
super.onDraw(canvas);
for (int i = 0; i < mDroidCards.size() - 1; i++){
    drawDroidCard(canvas, mDroidCards,i);
}
drawLastDroidCard(canvas,mDroidCards.get(mDroidCards.size()-1));
invalidate();
}

/**
 * 绘制最后一个DroidCard
 * @param canvas
 * @param c
 */
private void drawLastDroidCard(Canvas canvas,DroidCard c) {
    canvas.drawBitmap(c.bitmap,c.x,0f,paint);
}

/**
 * 绘制DroidCard
 * @param canvas
 * @param mDroidCards
 * @param i
 */
private void drawDroidCard(Canvas canvas,List<DroidCard> mDroidCards,int i) {
    DroidCard c = mDroidCards.get(i);
    canvas.save();
    canvas.clipRect((float)c.x,0f,(float)(mDroidCards.get(i+1).x),(float)c.height);
    canvas.drawBitmap(c.bitmap,c.x,0f,paint);
    canvas.restore();
}
}

```

在上面的代码中，我们使用Canvas的clipRect方法，绘制之前裁剪出一个区域，这样绘制的时候只在这区域内绘制，超出部分不会绘制出来。重新运行上面的代码，效果如下图所示。



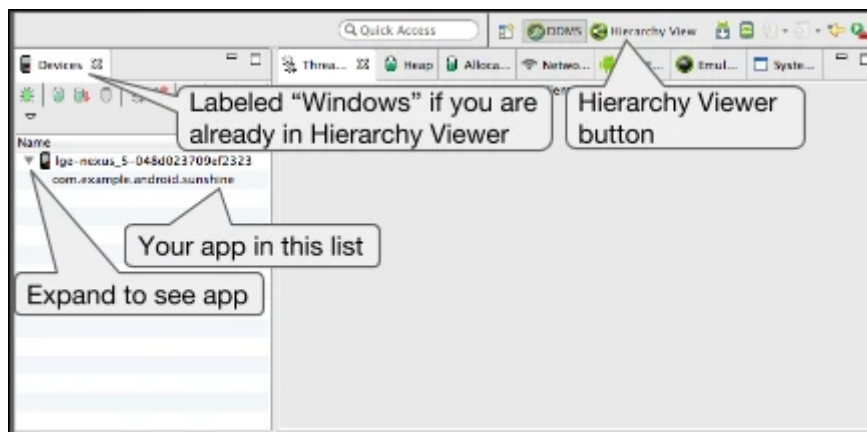
2.4 Hierarchy Viewer

Hierarchy Viewer 是 Android Device Monitor 中内置的一种工具，可让开发者测量布局层次结构中每个视图的布局速度，以及帮助开发者查找视图层次结构导致的性能瓶颈。Hierarchy Viewer可以通过红、黄、绿三种不同的颜色来区分布局的Measure、Layout、Executive的相对性能表现情况。

打开

1. 将设备连接到计算机。如果设备上显示对话框提示您允许 USB 调试吗？，请点击确定。
2. 在 Android Studio 中打开您的项目，在您的设备上构建并运行项目。
3. 启动 Android Device Monitor。Android Studio 可能会显示 Disable adb integration 对话框，因为一次只能有一个进程可以通过 adb 连接到设备，并且 Android Device Monitor 正在请求连接。因此，请点击 Yes。

4. 在菜单栏中，依次选择 Window > Open Perspective，然后点击 Hierarchy View。
5. 在左侧的 Windows 标签中双击应用的软件包名称。这会使用应用的视图层次结构填充相关窗格。

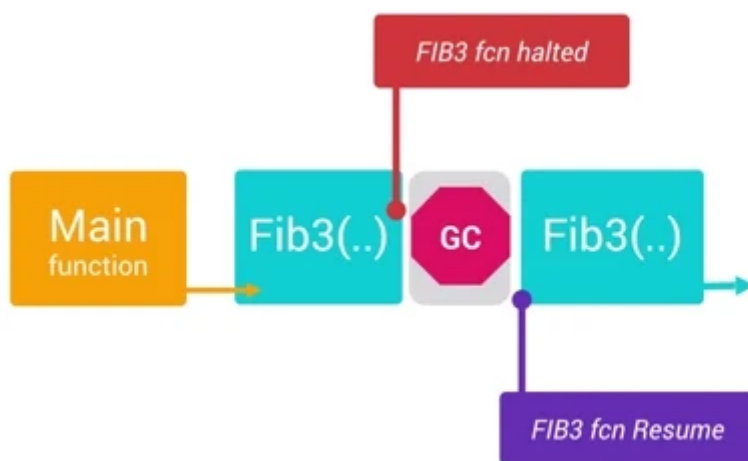


提升布局性能的关键点是尽量保持布局层级的扁平化，避免出现重复的嵌套布局。如果我们写的布局层级比较深会严重增加CPU的负担，造成性能的严重卡顿，关于Hierarchy Viewer的使用可以参考：[使用 Hierarchy Viewer 分析布局](#)。

2.5 内存抖动

在我们优化过view的树形结构和overdraw之后，可能还是感觉自己的app有卡顿和丢帧，或者滑动慢等问题，我们就要查看一下是否存在内存抖动情况了。所谓内存抖动，指的是内存频繁创建和GC造成的UI线程被频繁阻塞的现象。

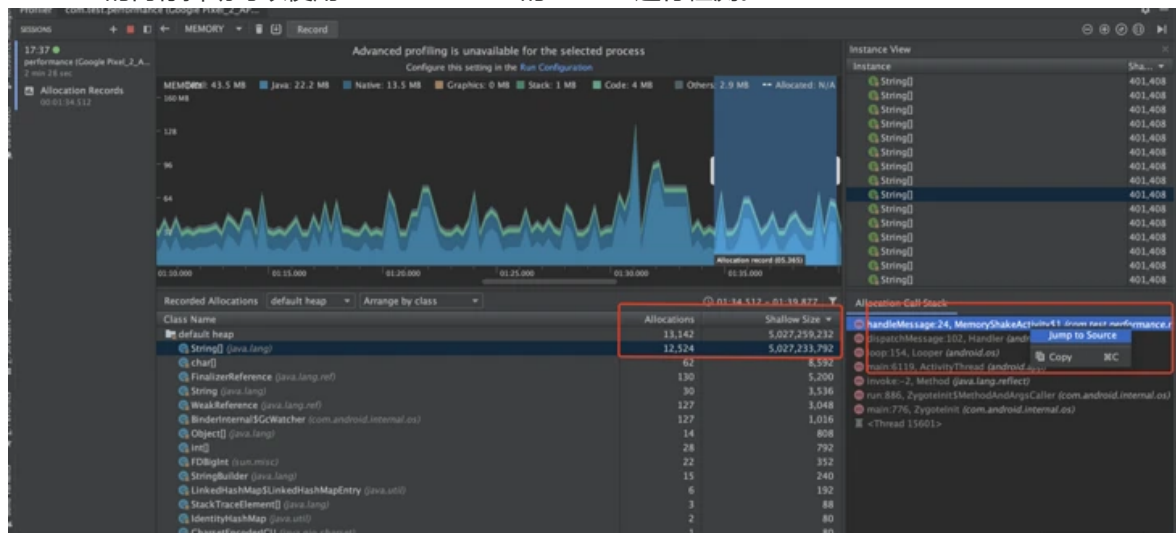
Android有自动管理内存的机制，但是对内存的不恰当使用仍然容易引起严重的性能问题。在同一帧里面创建过多的对象是件需要特别引起注意的事情，在同一帧里创建大量对象可能引起GC的不停操作，执行GC操作的时候，所有线程的任何操作都会需要暂停，直到GC操作完成。大量不停的GC操作则会显著占用帧间隔时间。如果在帧间隔时间里面做了过多的GC操作，那么就会造成页面卡顿。



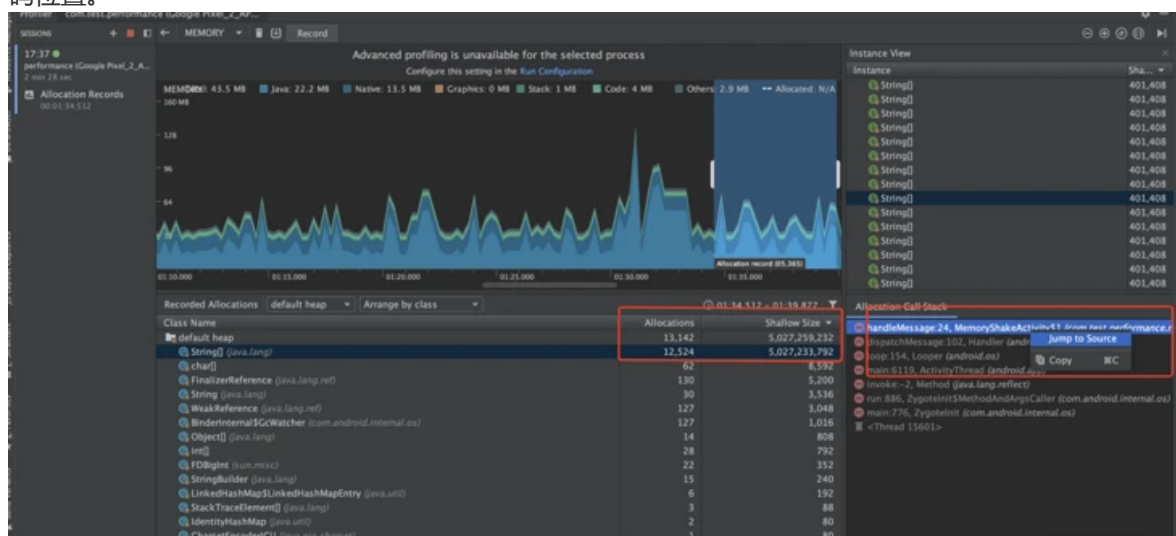
在Android开发中，导致GC频繁操作有两个主要原因：

- 内存抖动，所谓内存抖动就是短时间产生大量对象又在短时间内马上释放。
- 短时间产生大量对象超出阈值，内存不够，同样会触发GC操作。

Android的内存抖动可以使用Android Studio的Profiler进行检测。



然后，点击record记录内存信息，查找发生内存抖动位置，当然也可直接通过jump to Source定位到代码位置。



为了避免发生内存抖动，我们需要避免在for循环里面分配对象占用内存，需要尝试把对象的创建移到循环体之外，自定义View中的onDraw方法也需要引起注意，每次屏幕发生绘制以及动画执行过程中，onDraw方法都会被调用到，避免在onDraw方法里面执行复杂的操作，避免创建对象。对于那些无法避免需要创建对象的情况，我们可以考虑对象池模型，通过对象池来解决频繁创建与销毁的问题，但是这里需要注意结束使用之后，需要手动释放对象池中的对象。

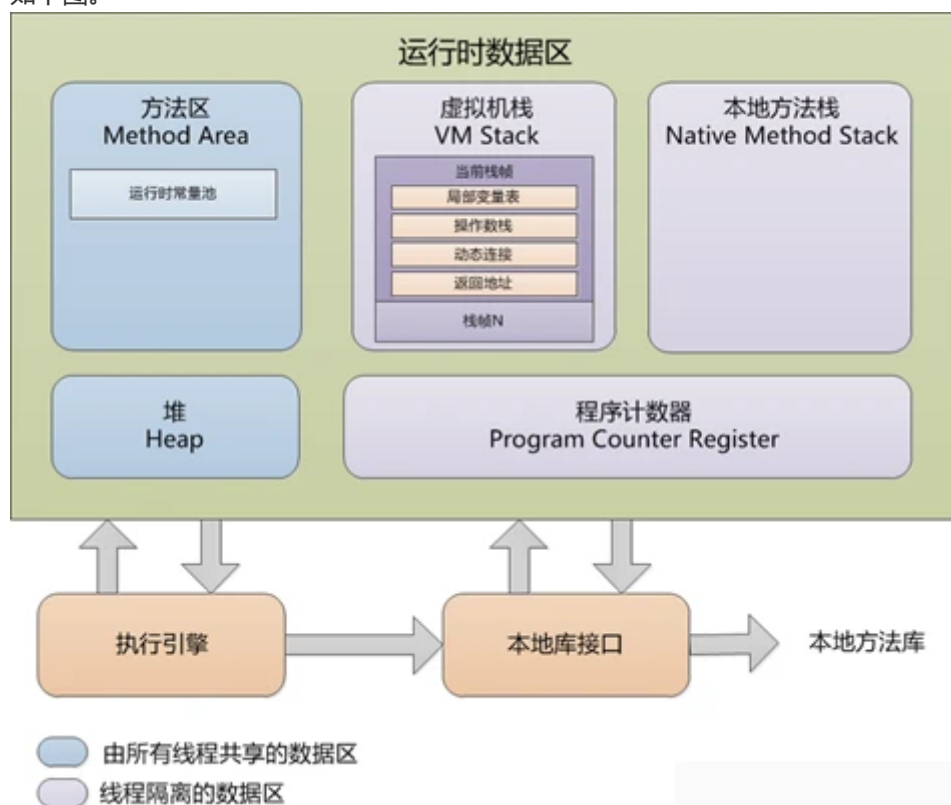
3，内存优化

3.1 内存管理

在前面Java基础环节，我们对Java的内存管理模型也做了基本的介绍，参考链接：[Android 面试之必问Java基础](#)

3.1.1 内存区域

在Java的内存模型中，将内存区域划分为方法区、堆、程序计数器、本地方法栈、虚拟机栈五个区域，如下图。



方法区

- 线程共享区域，用于存储类信息、静态变量、常量、即时编译器编译出来的代码数据。
- 无法满足内存分配需求时会发生OOM。

堆

- 线程共享区域，是JAVA虚拟机管理的内存中最大的一块，在虚拟机启动时创建。
- 存放对象实例，几乎所有的对象实例都在堆上分配，GC管理的主要区域。

虚拟机栈

- 线程私有区域，每个java方法在执行的时候会创建一个栈帧用于存储局部变量表、操作数栈、动态链接、方法出口等信息。方法从执行开始到结束过程就是栈帧在虚拟机栈中入栈出栈过程。
- 局部变量表存放编译期可知的基本数据类型、对象引用、returnAddress类型。所需的内存空间会在编译期间完成分配，进入一个方法时在帧中局部变量表的空间是完全确定的，不需要运行时改变。
- 若线程申请的栈深度大于虚拟机允许的最大深度，会抛出StackOverflowError错误。
- 虚拟机动态扩展时，若无法申请到足够内存，会抛出OutOfMemoryError错误。

本地方法栈

- 为虚拟机中Native方法服务，对本地方法栈中使用的语言、数据结构、使用方式没有强制规定，虚拟机可自有实现。
- 占用的内存区大小是不固定的，可根据需要动态扩展。

程序计数器

- 一块较小的内存空间，线程私有，存储当前线程执行的字节码行号指示器。
- 字节码解释器通过改变这个计数器的值来选取下一条需要执行的字节码指令：分支、循环、跳转等。
- 每个线程都有一个独立的程序计数器
- 唯一一个在java虚拟机中不会OOM的区域

3.1.2 垃圾回收

标记清除算法

标记清除算法主要分为有两个阶段，首先标记出需要回收的对象，然后咋标记完成后统一回收所有标记的对象；

缺点：

- 效率问题：标记和清除两个过程效率都不高。
- 空间问题：标记清除之后会导致很多不连续的内存碎片，会导致需要分配大对象时无法找到足够的连续空间而不得不触发GC的问题。

复制算法

将可用内存按空间分为大小相同的两小块，每次只使用其中的一块，等这块内存使用完了将还存活的对象复制到另一块内存上，然后将这块内存区域对象整体清除掉。每次对整个半区进行内存回收，不会导致碎片问题，实现简单且效率高效。

缺点：

需要将内存缩小为原来的一半，空间代价太高。

标记整理算法

标记整理算法标记过程和标记清除算法一样，但清除过程并不是对可回收对象直接清理，而是将所有存活对象像一端移动，然后集中清理到端边界以外的内存。

分代回收算法

当代虚拟机垃圾回收算法都采用分代收集算法来收集，根据对象存活周期不同将内存划分为新生代和老年代，再根据每个年代的特点采用最合适的回收算法。

- 新生代存活对象较少，每次垃圾回收都有大量对象死去，一般采用复制算法，只需要付出复制少量存活对象的成本就可以实现垃圾回收；
- 老年代存活对象较多，没有额外空间进行分配担保，就必须采用标记清除算法和标记整理算法进行回收；

3.2 内存泄漏

所谓内存泄露，指的是内存中存在的没有用的确无法回收的对象。表现的现象是会导致内存抖动，可用内存减少，进而导致GC频繁、卡顿、OOM。

下面是一段模拟内存泄漏的代码：

```
/**
 * 模拟内存泄露的Activity
 */
public class MemoryLeakActivity extends AppCompatActivity implements CallBack{
    @Override
    protected void onCreate(@Nullable Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_memoryleak);
        ImageView imageView = findViewById(R.id.iv_memoryleak);
        Bitmap bitmap = BitmapFactory.decodeResource(getResources(),
R.mipmap.splash);
        imageView.setImageBitmap(bitmap);

        // 添加静态类引用
        CallBackManager.addCallBack(this);
    }
    @Override
    protected void onDestroy() {
        super.onDestroy();
        // CallBackManager.removeCallBack(this);
    }
}
```



```
@Override
public void doperate() {
    // do sth
}
```

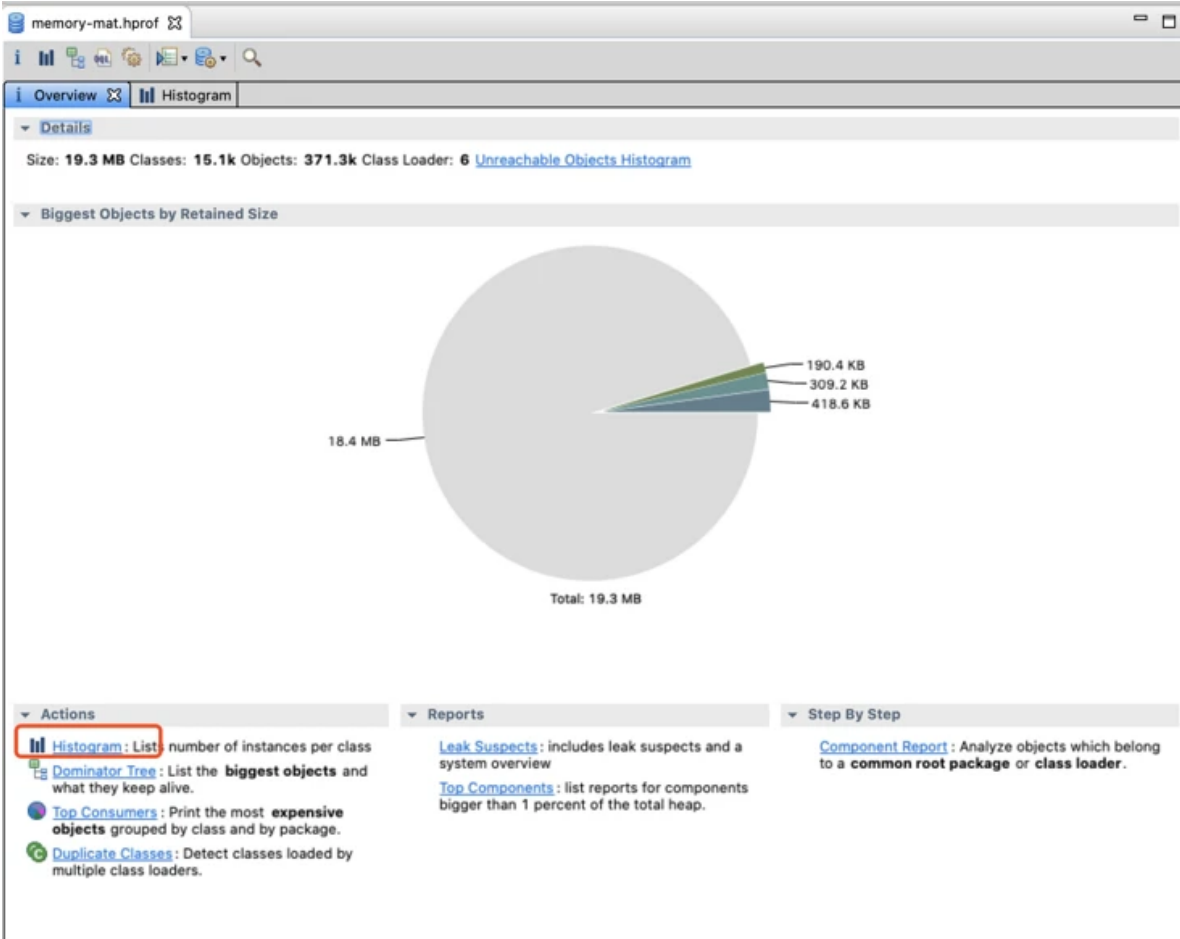
当我们使用Memory Profiler工具查看内存曲线，发现内存存在不断的上升，如下图所示。



如果想分析定位具体发生内存泄露位置，我们可以借助MAT工具。首先，使用MAT工具生成hprof文件，点击dump将当前内存信息转成hprof文件，需要对生成的文件转换成MAT可读取文件。执行一下转换命令即可完成转换，生成的文件位于Android/sdk/platform-tools路径下。

```
hprof-conv 刚刚生成的hprof文件 memory-mat.hprof
```

使用mat打开刚刚转换的hprof文件，然后使用Android Studio打开hprof文件，如下图所示。



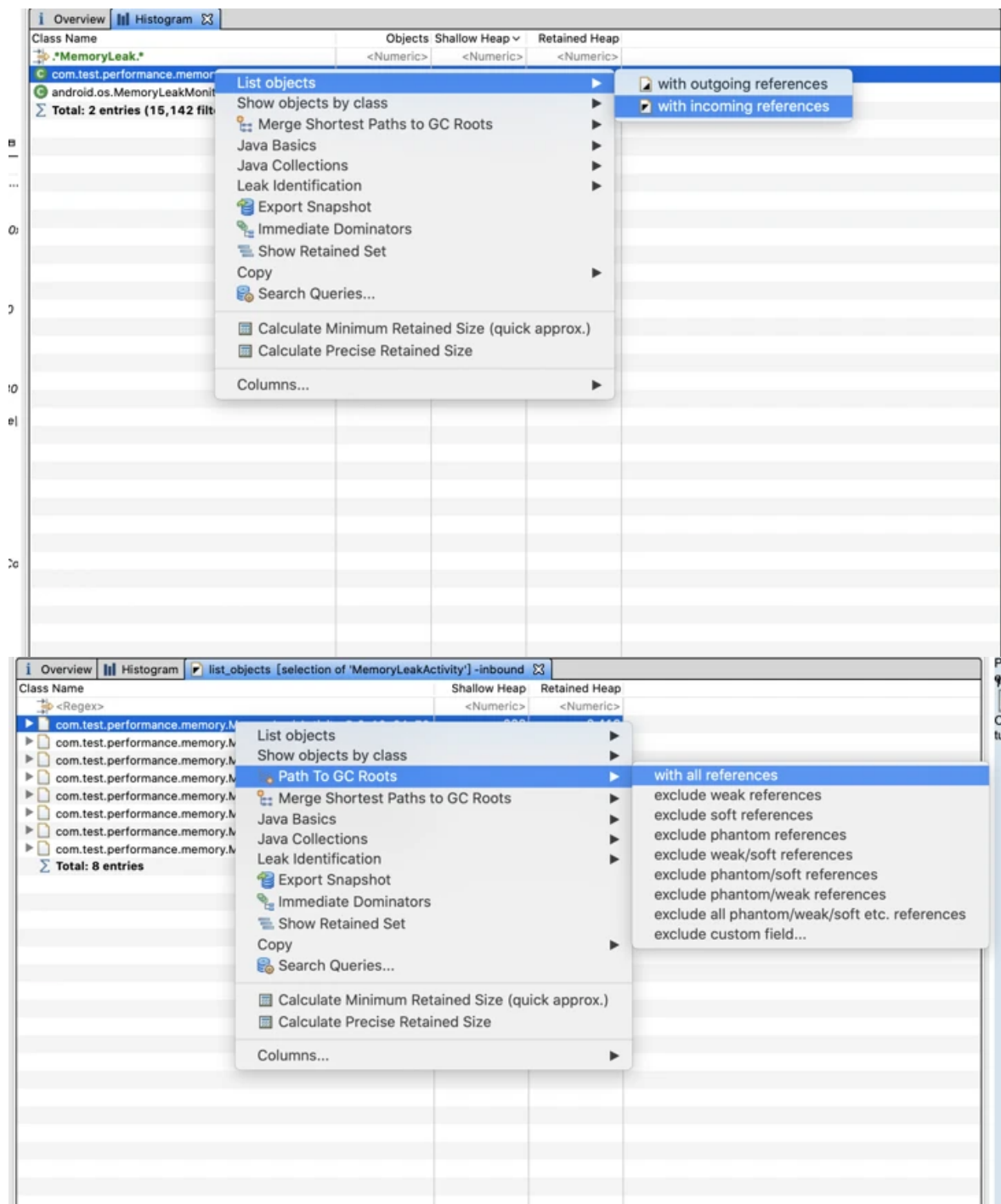
然后点击面板的【Histogram】，搜索MemoryLeakActivity，即可查看对应的泄漏文件的相关信息。

memory-mat.hprof

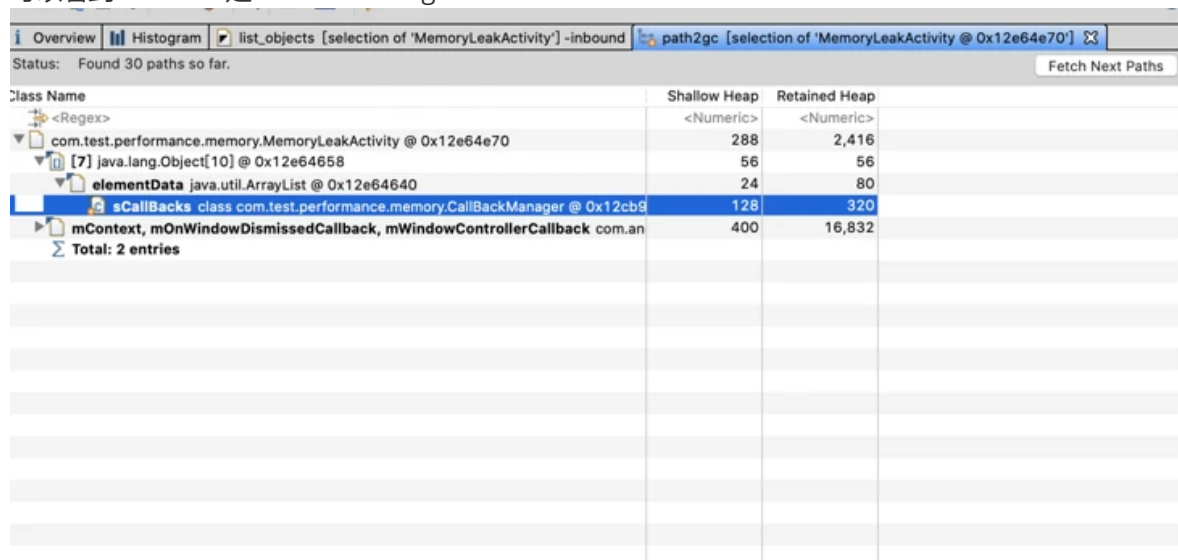
Overview Histogram

Class Name	Objects	Shallow Heap	Retained Heap
MemoryLeak	<Numeric>	<Numeric>	<Numeric>
byte[]	131,194	8,601,568	
java.lang.S...	121,335	2,912,040	
java.lang.C...	15,144	2,156,248	
long[]	12,973	1,324,344	
int[]	4,937	935,288	
char[]	3,942	800,456	
java.lang....	9,711	635,640	
java.util.H...	9,727	233,448	
java.util.H...	716	129,656	
java.lang.r...	3,505	112,160	
java.util.Li...	2,822	90,304	
java.lang.S...	1,100	70,664	
com.aliba...	1	65,552	
android.su...	30	63,168	
sun.misc....	1,547	61,880	
java.util.H...	1,141	45,640	
java.lang.l...	2,825	45,200	
float[]	974	44,176	
java.util.co...	1,805	43,320	
java.util.H...	1,798	43,152	
java.util.Ar...	1,756	42,144	
sun.util.io...	881	35,240	
libcore.util...	1,308	31,392	
java.util.Tr...	880	28,160	
sun.util.io...	860	27,520	
com.test.p...	200	27,200	
android.gr...	1,096	26,304	
android.su...	438	26,272	
java.util.co...	52	26,208	
java.lang.r...	643	25,720	
Total: 30...	371,308	20,231,752	

然后，查看所有引用对象，并得到相关的引用链，如下图。



可以看到GC Roots是CallbackManager



所以，我们在Activity销毁时将CallbackManager引用移除即可。

```
@Override
protected void onDestroy() {
    super.onDestroy();
    CallBackManager.removeCallBack(this);
}
```

当然，上面只是一个MAT分析工具使用的示例，其他的内存泄露都可以借助MAT分析工具解决。

3.3 大图内存优化

在Android开发中，经常会遇到加载大图导致内存泄露的问题，对于这种场景，有一个通用的解决方案，即使用ARTHook对不合理图片进行检测。我们知道，获取Bitmap占用的内存主要有两种方式：

- 通过getBytesCount方法，但是需要在运行时获取
- width * height * 一个像素所占内存 * 图片所在资源目录压缩比

通过ARTHook方法可以优雅的获取不合理图片，侵入性低，但是因为兼容性问题一般在线下使用。使用ARTHook需要安装以下依赖：

```
implementation 'me.weishu:epic:0.3.6'
```

然后自定义实现Hook方法，如下所示。

```
public class CheckBitmapHook extends XC_MethodHook {
    @Override protected void afterHookedMethod(MethodHookParam param) throws
    Throwable {
        super.afterHookedMethod(param);
        ImageView imageView = (ImageView)param.thisObject;
        checkBitmap(imageView, imageView.getDrawable());
    }
    private static void checkBitmap(Object o, Drawable drawable) {
        if(drawable instanceof BitmapDrawable && o instanceof View) {
            final Bitmap bitmap = ((BitmapDrawable) drawable).getBitmap();
            if(bitmap != null) {
                final View view = (View)o;
                int width = view.getWidth();
                int height = view.getHeight();
                if(width > 0 && height > 0) {
                    if(bitmap.getWidth() > (width << 1) && bitmap.getHeight() >
                    (height << 1)) {
                        warn(bitmap.getWidth(), bitmap.getHeight(), width, height,
                            new RuntimeException("Bitmap size is too
                            large"));
                    }
                } else {
                    final Throwable stacktrace = new RuntimeException();
                    view.getViewTreeObserver().addOnPreDrawListener(
                        new ViewTreeObserver.OnPreDrawListener() {
                            @Override public boolean onPreDraw() {
                                int w = view.getWidth();
                                int h = view.getHeight();
                                if(w > 0 && h > 0) {
                                    if (bitmap.getWidth() >= (w << 1)
                                        && bitmap.getHeight() >= (h <<
                                        1)) {
```

```

warn(bitmap.getWidth(),
bitmap.getHeight(), w, h, stacktrace);
    }

view.getViewTreeObserver().removeOnPreDrawListener(this);
    }
    return true;
}
});
}
}
}
}
private static void warn(int bitmapwidth, int bitmapHeight, int viewwidth,
int viewHeight, Throwable t) {
    String warnInfo = new StringBuilder("Bitmap size too large: ")
        .append("\n real size:
") .append(bitmapwidth).append(',').append(bitmapHeight).append(',')
        .append("\n desired size:
") .append(viewwidth).append(',').append(viewHeight).append(',')
        .append("\n call stack trace:
\n").append(Log.getStackTraceString(t)).append('\n')
        .toString();
    LogUtils.i(warnInfo);
}

```

最后，在Application初始化时注入Hook。

```

DexposedBridge.hookAllConstructors(ImageView.class, new XC_MethodHook() {
    @Override protected void afterHookedMethod(MethodHookParam param) throws
    Throwable {
        super.afterHookedMethod(param);
        DexposedBridge.findAndHookMethod(ImageView.class, "setImageBitmap",
        Bitmap.class,
            new CheckBitmapHook());
    }
});

```

3.4 线上监控

3.4.1 常规方案

方案一

在特定场景中获取当前占用内存大小，如果当前内存大小超过系统最大内存80%，对当前内存进行一次Dump（Debug.dumpHprofData()），选择合适时间将hprof文件进行上传，然后通过MAT工具手动分析该文件。

缺点：

- Dump文件比较大，和用户使用时间、对象树正相关。
- 文件较大导致上传失败率较高，分析困难。

方案二

将LeakCanary带到线上，添加预设怀疑点，对怀疑点进行内存泄露监控，发现内存泄露回传到服务端。

缺点：

- 通用性较低，需要预设怀疑点，对没有预设怀疑点的地方监控不到。

- LeakCanary分析比较耗时、耗内存，有可能会发生OOM。

3.4.2 LeakCannary改造

改造主要涉及以下几点：

- 将需要预设怀疑点改为自动寻找怀疑点，自动将前内存中所占内存较大的对象类中设置怀疑点。
- LeakCanary分析泄露链路比较慢，改造为只分析Retain size大的对象。
- 分析过程会OOM，是因为LeakCannary分析时会将分析对象全部加载到内存当中，我们可以记录下分析对象的个数和占用大小，对分析对象进行裁剪，不全部加载到内存当中。

完成的改造步骤如下：

1. 监控常规指标：待机内存、重点模块占用内存、OOM率
2. 监控APP一个生命周期内和重点模块界面的生命周期内的GC次数、GC时间等
3. 将定制的LeakCanary带到线上，自动化分析线上的内存泄露

4，网络优化

4.1 网络优化的影响

App的网络连接对于用户来说，影响很多，且多数情况下都很直观，直接影响用户对这个App的使用体验。其中较为重要的几点：

流量：App的流量消耗对用户来说是比较敏感的，毕竟流量是花钱的嘛。现在大部分人的手机上都有安装流量监控的工具App，用来监控App的流量使用。如果我们的App这方面没有控制好，会给用户不好的使用体验。

电量：电量相对于用户来说，没有那么明显。一般用户可能不会太注意。但是如电量优化中的那样，网络连接(radio)是对电量影响很大的一个因素。所以我们要加以注意。

用户等待：也就是用户体验，良好的用户体验，才是我们留住用户的第一步。如果App请求等待时间长，会给用户网络卡，应用反应慢的感觉，如果有对比，有替代品，我们的App很可能就会被用户无情抛弃。

4.2 网络分析工具

网络分析可以借助的工具具有Monitor、代理工具等。

4.2.1 Network Monitor

Android Studio内置的Monitor工具提供了一个Network Monitor，可以帮助开发者进行网络分析，下面是一个典型的Network Monitor示意图。



- Rx --- R(ceive) 表示下行流量，即下载接收。
- Tx --- T(ransmit) 表示上行流量，即上传发送。

Network Monitor实时跟踪选定应用的数据请求情况。我们可以连上手机，选定调试应用进程，然后在App上操作我们需要分析的页面请求。

4.2.2 代理工具

网络代理工具有两个作用，一个是截获网络请求响应包, 分析网络请求；另一个设置代理网络, 移动App开发中一般用来做不同网络环境的测试, 例如Wifi/4G/3G/弱网等。

现在，可以使用的代理工具有很多, 诸如Wireshark, Fiddler, Charles等。

4.3 网络优化方案

对于网络优化来说，主要从两个方面进行着手进行优化：

1. **减少活跃时间**：减少网络数据获取的频次，从而就减少了radio的电量消耗以及控制电量使用。
2. **压缩数据包的大小**：压缩数据包可以减少流量消耗，也可以让每次请求更快。

基于上面的方案，可以得到以下一些常见的解决方案：

4.3.1 接口设计

1, API设计

App与服务器之间的API设计要考虑网络请求的频次，资源的状态等。以便App可以以较少的请求来完成业务需求和界面的展示。

例如, 注册登录. 正常会有两个API, 注册和登录, 但是设计API时我们应该给注册接口包含一个隐式的登录. 来避免App在注册后还得请求一次登录接口。

2, 使用Gzip压缩

使用Gzip来压缩request和response, 减少传输数据量, 从而减少流量消耗。使用Retrofit等网络请求框架进行网络请求时，默认进行了Gzip的压缩。

3, 使用Protocol Buffer

以前，我们传输数据使用的是XML, 后来使用JSON代替了XML, 很大程度上也是为了可读性和减少数据量。而在游戏开发中，为了保证数据的准确和及时性，Google推出了Protocol Buffer数据交换格式。

4, 依据网络情况获取不同分辨率的图片

我们使用淘宝或者京东的时候，会看到应用会根据网络情况，获取不同分辨率的图片，避免流量的浪费以及提升用户的体验。

4.3.2 合理使用网络缓存

适当的使用缓存, 不仅可以让我们的应用看起来更快, 也能避免一些不必要的流量消耗，带来更好的用户体验。

1, 打包网络请求

当接口设计不能满足我们的业务需求时。例如，可能一个界面需要请求多个接口，或是网络良好，处于Wifi状态下时我们想获取更多的数据等。这时就可以打包一些网络请求, 例如请求列表的同时, 获取Header点击率较高的的item项的详情数据。

2, 监听设备状态

为了提升用户体验，我们可以对设备的使用状态进行监听，然后再结合JobScheduler来执行网络请求。比方说Splash闪屏广告图片, 我们可以在连接到Wifi时下载缓存到本地; 新闻类的App可以在充电, Wifi状态下做离线缓存。

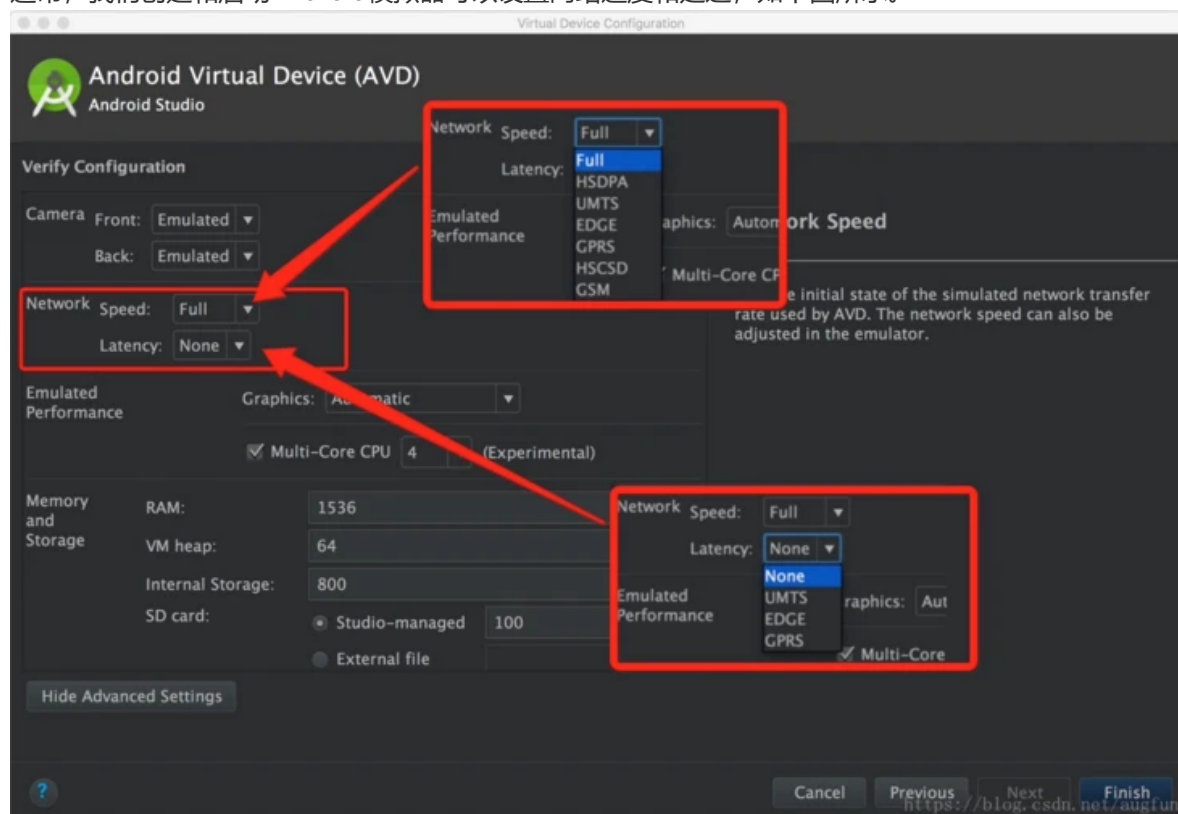
4.3.3 弱网测试&优化

1, 弱网测试

有几种方式来模拟弱网进行测试：

Android Emulator

通常，我们创建和启动Android模拟器可以设置网络速度和延迟，如下图所示。

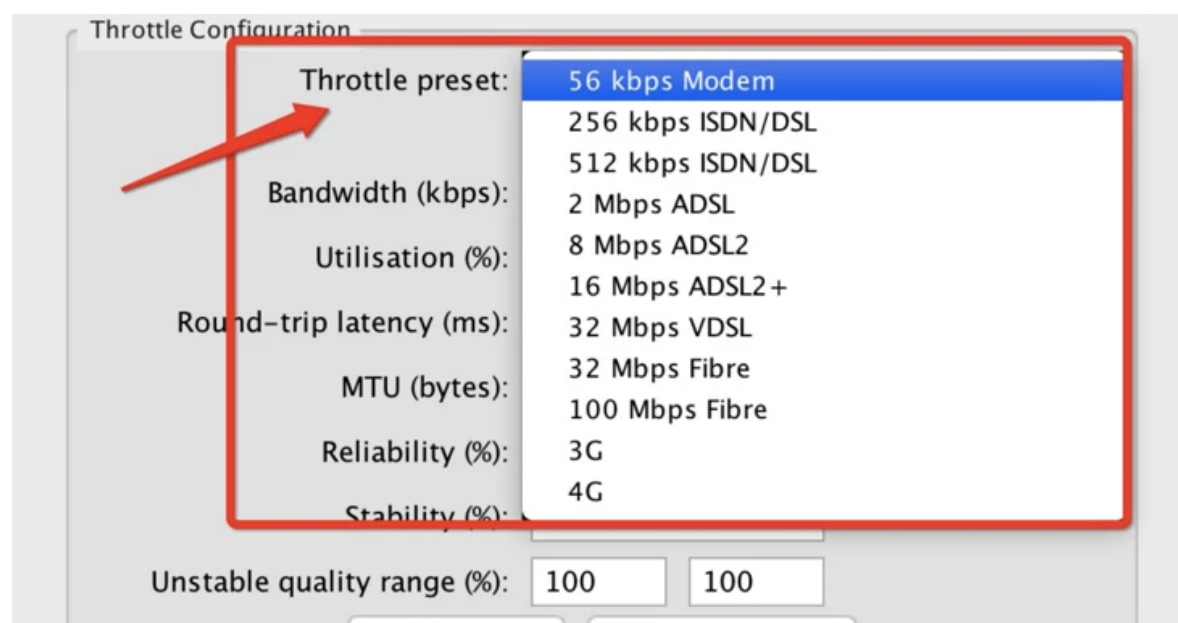


然后，我们在启动时使用的emulator命令如下。

```
$emulator -netdelay gprs -netspeed gsm -avd Nexus_5_API_22
```

2, 网络代理工具

使用网络代理工具也可以模拟网络情况。以Charles为例，保持手机和PC处于同一个局域网,在手机端wifi设置高级设置中设置代理方式为手动,代理ip填写PC端ip地址,端口号默认8888。



5, 耗电优化

事实上，如果我们的应用需要播放视频、需要获取 GPS 信息，亦或者是游戏应用，耗电都是比较严重的。如何判断哪些耗电是可以避免，或者是需要去优化的呢？我们可以打开手机自带的耗电排行榜，发现“王者荣耀”使用了 7 个多小时，这时用户对“王者荣耀”的耗电是有预期的。

10:32    4G  

← 耗电排行榜

应用耗电



王者荣耀

使用时间：7小时25分钟

60%



微信

使用时间：32分钟

7%



UC浏览器

使用时间：41分钟

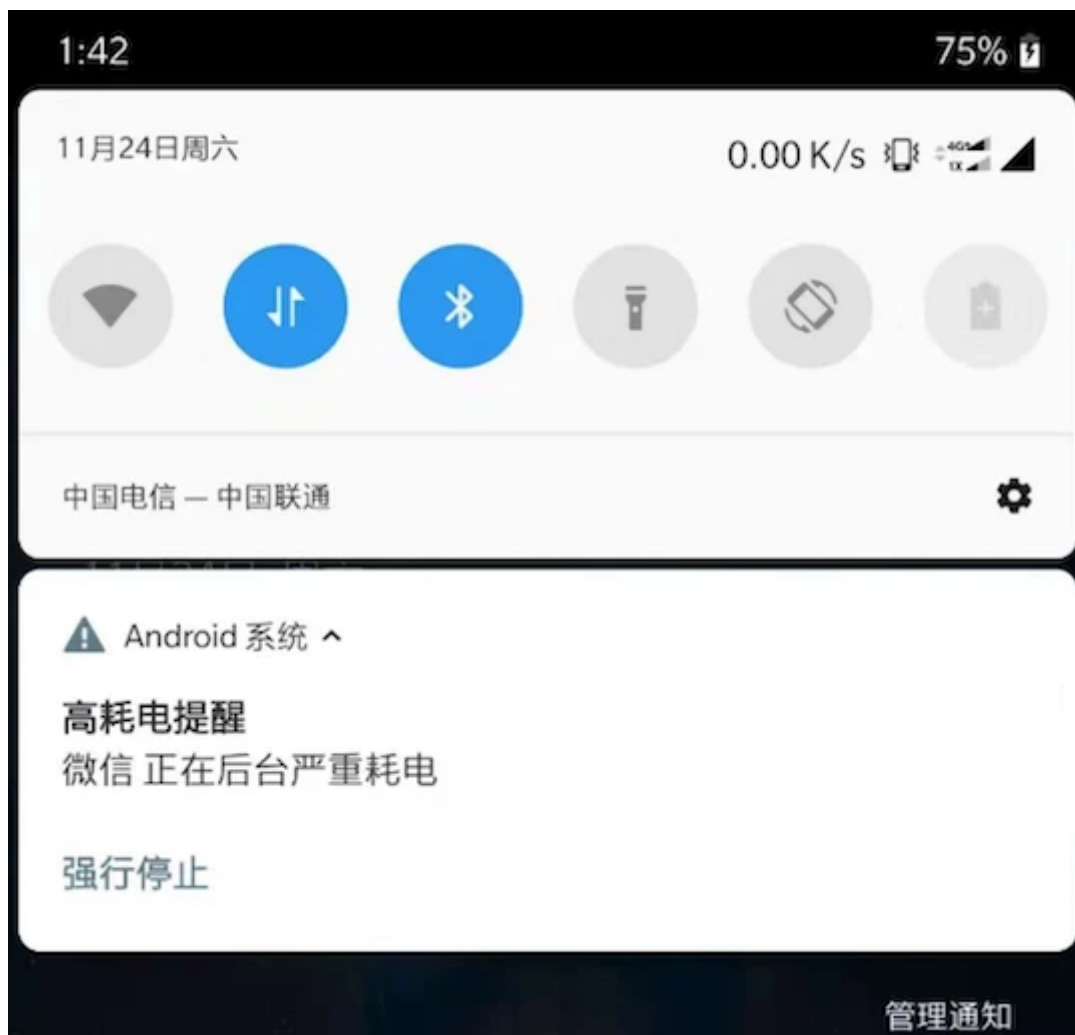
6%

5.1 优化方向

假设这个时候发现某个应用他根本没怎么使用，但是耗电却非常多，那么就会被系统无情的杀掉。所以耗电优化的第一个方向是优化应用的后台耗电。

知道了系统是如何计算耗电的，我们也就可以知道应用在后台不应该做什么，例如长时间获取 WakeLock、WiFi 和蓝牙的扫描等，以及后台服务。为什么说耗电优化第一个方向就是优化应用后台耗电，因为大部分厂商预装项目要求最严格的正是应用后台待机耗电。





当然前台耗电我们不会完全不管，但是标准会放松很多。再来看看下面这张图，如果系统对你的应用弹出这个对话框，可能对于微信来说，用户还可以忍受，但是对其他大多数的应用来说，可能很多用户就直接把你加入到后台限制的名单中了。

耗电优化的第二个方向是符合系统的规则，让系统认为你耗电是正常的。

而 Android P 及以上版本是通过 Android Vitals 监控后台耗电，所以我们需要符合 Android Vitals 的规则，目前它的具体规则如下。

描述	规则
Alarm Manager wakeup唤醒过多	当手机不在充电状态，每小时wakeup唤醒次数大于10次
频繁使用局部唤醒锁	当手机不在充电状态，partial wake lock持有超过1小时
后台网络使用量过高	当手机不在充电状态而且应用在后台，每小时网络使用量超过50MB
后台WiFi scans过多	当手机不在充电状态而且应用在后台，每小时大于4次WiFi scans

可以看到，Android系统目前比较关心是后台 Alarm 唤醒、后台网络、后台 WiFi 扫描以及部分长时间 WakeLock 阻止系统后台休眠，因为这些都有可能导致耗电问题。

5.2 耗电监控

5.2.1 Android Vitals

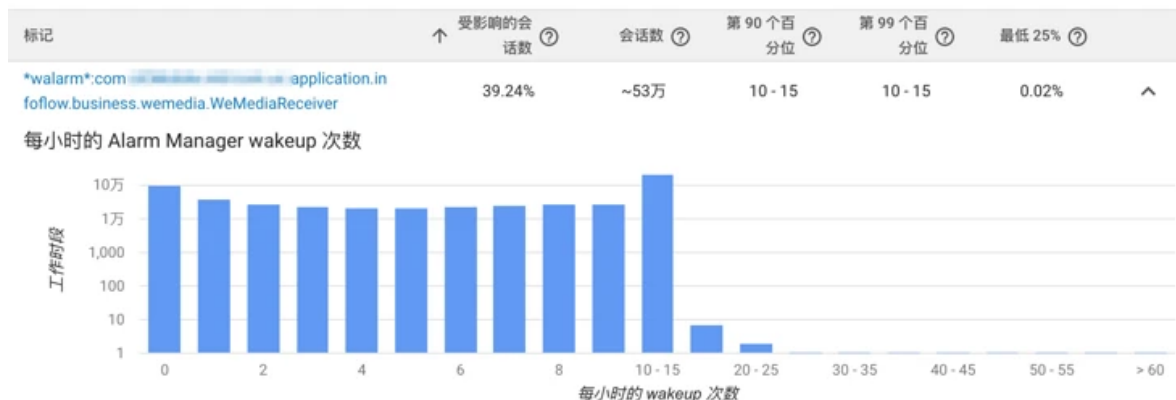
Android Vitals 的几个关于电量的监控方案与规则，可以帮助我们进行耗电监测。

- [Alarm Manager wakeup 唤醒过多](#)
- [频繁使用局部唤醒锁](#)

- [后台网络使用量过高](#)
- [后台 WiFi Scans 过多](#)

在使用了一段时间之后，我发现它并不是那么好用。以 Alarm wakeup 为例，Vitals 以每小时超过 10 次作为规则。由于这个规则无法做修改，很多时候我们可能希望针对不同的系统版本做更加细致的区分。其次跟 Battery Historian 一样，我们只能拿到 wakeup 的标记的组件，拿不到申请的堆栈，也拿不到当时手机是否在充电、剩余电量等信息。下图是wakeup拿到的信息。

按 wakeup 标记



对于网络、WiFi scans 以及 WakeLock 也是如此。虽然 Vitals 帮助我们缩小了排查的范围，但是依然没办法确认问题的具体原因。

5.3 如何监控耗电

前面说过，Android Vitals并不是那么好用，而且对于国内的应用来说其实也根本无法使用。那我们的耗电监控系统应该监控哪些内容，又应该如何做呢？首先，我们看一下耗电监控具体应该怎么做呢？

- **监控信息：**简单来说系统关心什么，我们就监控什么，而且应该以后台耗电监控为主。类似 Alarm wakeup、WakeLock、WiFi scans、Network 都是必须的，其他的可以根据应用的实际情况。如果是地图应用，后台获取 GPS 是被允许的；如果是计步器应用，后台获取 Sensor 也没有太大问题。
- **现场信息：**监控系统希望可以获得完整的堆栈信息，比如哪一行代码发起了 WiFi scans、哪一行代码申请了 WakeLock 等。还有当时手机是否在充电、手机的电量水平、应用前台和后台时间、CPU 状态等一些信息也可以帮助我们排查某些问题。
- **提炼规则：**最后我们需要将监控的内容抽象成规则，当然不同应用监控的事项或者参数都不太一样。由于每个应用的具体情况都不太一样，可以用来参考的简单规则。

	前台规则	后台规则
Alarm	单个Alarm每小时不能启动超过20次	单个Alarm每小时不能启动超过10次
WakeLock	单个WakeLock每小时不能超过30分钟或者超过20次	1. WakeLock每小时不能超过30分钟； 2. 单个WakeLock每小时不能超过10分钟或者超过12次
Sensor	不监控前台Sensor使用	每小时使用不能超过30分钟
WiFi scans	每小时扫描不允许超过10次	每小时扫描不允许超过4次
Bluetooth scans	每小时扫描不允许超过10次	每小时扫描不允许超过4次
GPS	每小时使用不能超过30分钟	每小时使用不能超过15分钟
Camera	不监控前台相机使用	后台不允许使用相机
Network	不监控前台网络耗电	每小时后台网络数据量不能超过10MB
CPU	不监控前台CPU	后台CPU持续运行超过30分钟

资源类型	后台资源使用红线标准
豁免正常业务使用资源	1. 用户经常使用的应用、默认设置应用； 2. 用户可感知业务：下载和上传（如手机克隆）、播放声音、录音、导航、计步、记录轨迹、骑行、蓝牙连接（穿戴设备）等
平均电流（亮灭屏）	IM应用<15mA/2min，其他应用<2mA/2min 注：如前台服务增加阈值（有常驻通知）
持锁阻止系统休眠（仅灭屏）	本次灭屏累计持锁超过5min
Alarm频繁唤醒系统（仅灭屏）	本次灭屏应用唤醒超过20次/h 注：如果应用同时（间隔小于50ms）算一次唤醒
应用频繁自启（亮灭屏）	应用进程后台自启超过20次/h或超过10次/10min
后台长时间使用GPS（亮灭屏）	本次灭屏（或亮屏）GPS累计使用超过5min
后台CPU高负载耗电（亮灭屏）	后台应用CPU负载持续10min超过40%
长时间网络定位（仅灭屏）	灭屏网络定位超过30min（非休眠时长）
后台WiFi扫描（亮屏）	本次亮屏WiFi扫描累计超过3min/h
后台蓝牙扫描（仅灭屏）	本次灭屏累计扫描超过5min/h
后台频繁收发数据包（仅灭屏）	以5s为周期，统计收发数据包次数。 IM/SMS/Email应用本次灭屏累计发送数据包超过60次/h； 其他类型应用，本次灭屏累计发送数据包超过30次/h
持锁阻止超时灭屏（仅亮屏）	没活动状态下，亮屏时长超过超时灭屏时间，提醒高耗电

5.3.2 Hook方案

明确了我们需要监控什么以及具体的规则之后，接下来我们来看一下电量监控的技术方案。这里首先来看一下Hook 方案。Hook 方案的好处在于使用者接入非常简单，不需要去修改代码，接入的成本比较低。下面我以几个比较常用的规则为例，看看如何使用 Java Hook 达到监控的目的。

1, WakeLock

WakeLock 用来阻止 CPU、屏幕甚至是键盘的休眠。类似 Alarm、JobService 也会申请 WakeLock 来完成后台 CPU 操作。WakeLock 的核心控制代码都在 PowerManagerService 中，实现的方法非常简单，如下所示。

```
// 代理 PowerManagerService
ProxyHook().proxyHook(context.getSystemService(Context.POWER_SERVICE),
    "mService", this);

@Override
public void beforeInvoke(Method method, Object[] args) {
    // 申请 wakelock
    if (method.getName().equals("acquirewakeLock")) {
        if (isAppBackground()) {
            // 应用后台逻辑，获取应用堆栈等等
        } else {
            // 应用前台逻辑，获取应用堆栈等等
        }
    }
    // 释放 wakelock
    } else if (method.getName().equals("releasewakeLock")) {
        // 释放的逻辑
    }
}
```

2, Alarm

Alarm 用来做一些定时的重复任务，它一共有四个类型，其中 ELAPSED_REALTIME_WAKEUP 和 RTC_WAKEUP 类型都会唤醒设备。同样，Alarm 的核心控制逻辑都在 AlarmManagerService 中，实现如下。

```
// 代理 AlarmManagerService
new ProxyHook().proxyHook(context.getSystemService
    (Context.ALARM_SERVICE), "mService", this);

public void beforeInvoke(Method method, Object[] args) {
    // 设置 Alarm
    if (method.getName().equals("set")) {
        // 不同版本参数类型的适配，获取应用堆栈等等
    }
    // 清除 Alarm
    } else if (method.getName().equals("remove")) {
        // 清除的逻辑
    }
}
```

除了 WakeLock 和 Alarm 外，对于后台 CPU，我们可以使用卡顿监控相关的方法；对于后台网络，同样我们可以通过网络监控相关的方法；对于 GPS 监控，我们可以通过 Hook 代理 LOCATION_SERVICE；对于 Sensor，我们通过 Hook SENSOR_SERVICE 中的 "mSensorListeners"，可以拿到部分信息。

最后，我们将申请资源到的堆栈信息保存起来。当我们触发某个规则上报问题的时候，可以将收集到的堆栈信息、电池是否充电、CPU 信息、应用前后台时间等辅助信息上传到后台即可。

5.3.3 插桩法

使用 Hook 方式虽然简单，但是某些规则可能不太容易找到合适的 Hook 点，而且在 Android P 之后，很多的 Hook 点都不支持了。出于兼容性考虑，我首先想到的是插桩法。以 WakeLock 为例：


```
public class wakelockMetrics {  
    // wakelock 申请  
    public void acquire(PowerManager.WakeLock wakelock) {  
        wakelock.acquire();  
        // 在这里增加 wakelock 申请监控逻辑  
    }  
    // wakelock 释放  
    public void release(PowerManager.WakeLock wakelock, int flags) {  
        wakelock.release();  
        // 在这里增加 wakelock 释放监控逻辑  
    }  
}
```

如果你对电量消耗又研究，那么肯定知道Facebook 的耗电监控的开源库Battery-Metrics，它监控的数据非常全，包括 Alarm、WakeLock、Camera、CPU、Network 等，而且也有收集电量充电状态、电量水平等信息。不过，遗憾的是Battery-Metrics 只是提供了一系列的基础类，在实际使用时开发者仍然需要修改大量的源码。

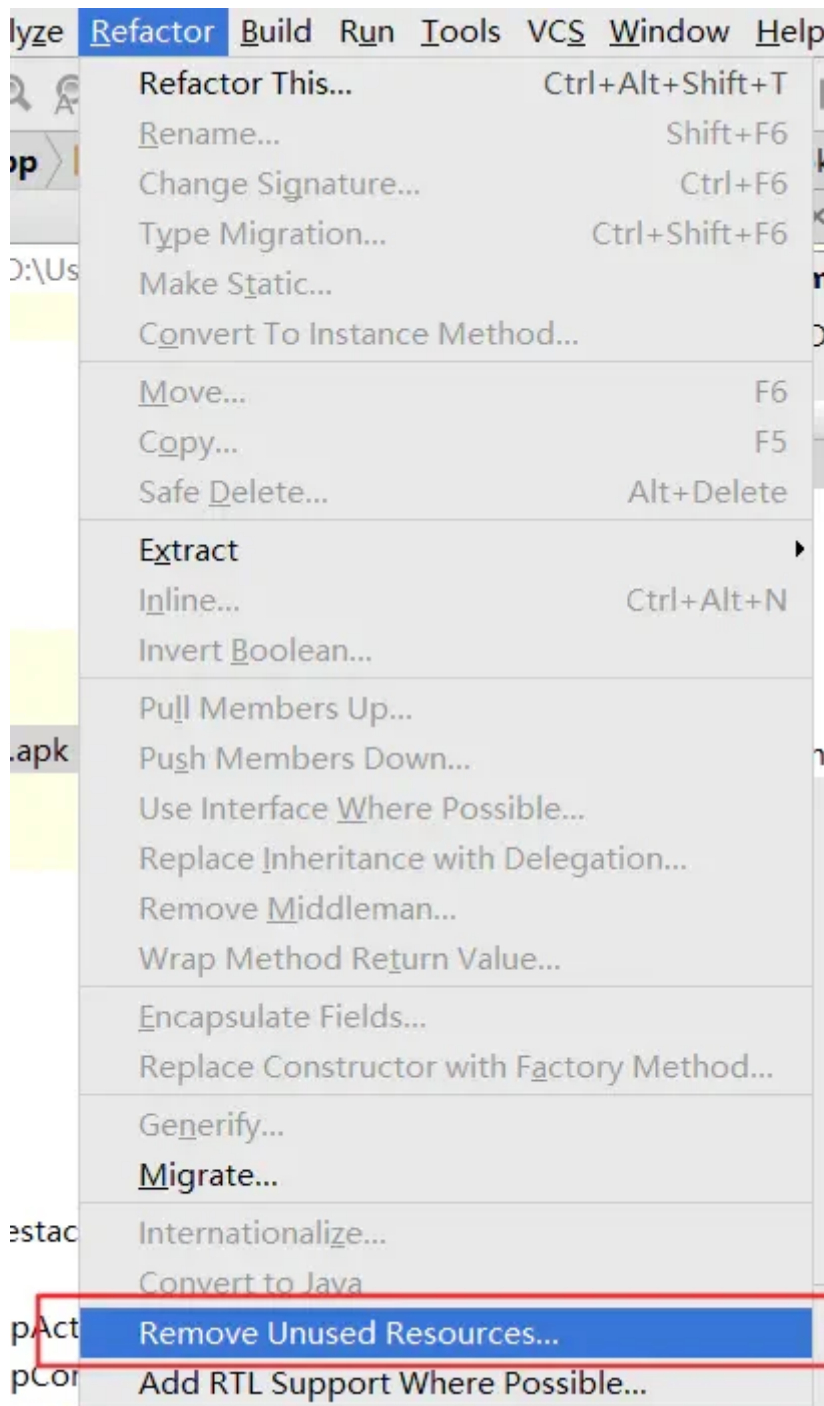
6，安装包优化

现在市面上的App，小则几十M，大则上百M。安装包越小，下载时省流量，用户好的体验，下载更快，安装更快。那么对于安装包，我们可以从哪些方面着手进行优化呢？

6，1 常用的优化策略

1，清理无用资源

在android打包过程中，如果代码有涉及资源和代码的引用，那么就会打包到App中，为了防止将这些废弃的代码和资源打包到App中，我们需要及时地清理这些无用的代码和资源来减小App的体积。清理的方法是，依次点击android Studio的【Refactor】->【Remove unused Resource】，如下图所示。



2, 使用Lint工具

Lint工具还是很有用的，它给我们需要优化的点：

- 检测没有用的布局并且删除
- 把未使用到的资源删除
- 建议String.xml有一些没有用到的字符也删除掉

3, 开启shrinkResources去除无用资源

在build.gradle 里面配置shrinkResources true，在打包的时候会自动清除掉无用的资源，但经过实验发现打出的包并不会，而是会把部分无用资源用更小的东西代替掉。注意，这里的“无用”是指调用图片的所有父级函数最终是废弃代码，而shrinkResources true 只能去除没有任何父函数调用的情况。

```

    android {
        buildTypes {
            release {
                shrinkResources true
            }
        }
    }
}

```

除此之外，大部分应用其实并不需要支持几十种语言的国际化支持，还可以删除语言支持文件。

6.2 资源压缩

在Android开发中，内置的图片是很多的，这些图片占用了大量的体积，因此为了缩小包的体积，我们可以对资源进行压缩。常用的方法有：

1. 使用压缩过的图片：使用压缩过的图片，可以有效降低App的体积。
2. 只用一套图片：对于绝对数APP来说，只需要取一套设计图就足够了。
3. 使用不带alpha值的jpg图片：对于非透明的大图，jpg将会比png的大小有显著的优势，虽然不是绝对的，但是通常会减小到一半都不止。
4. 使用tinypng有损压缩：支持上传PNG图片到官网上压缩，然后下载保存，在保持alpha通道的情况下对PNG的压缩可以达到1/3之内，而且用肉眼基本上分辨不出压缩的损失。
5. 使用webp格式：webp支持透明度，压缩比比，占用的体积比JPG图片更小。从Android 4.0+开始原生支持，但是不支持包含透明度，直到Android 4.2.1+才支持显示含透明度的webp，使用的时候要特别注意。
6. 使用svg：矢量图是由点与线组成,和位图不一样,它再放大也能保持清晰度，而且使用矢量图比位图设计方案能节约30~40%的空间。
7. 对打包后的图片进行压缩：使用7zip压缩方式对图片进行压缩，可以直接使用微信开源的AndResGuard压缩方案。

```

apply plugin: 'AndResGuard'
buildscript {
    dependencies {
        classpath 'com.tencent.mm:AndResGuard-gradle-plugin:1.1.7'
    }
}
andResGuard {
    mappingFile = null
    use7zip = true
    useSign = true
    keepRoot = false
    // add <your_application_id>.R.drawable.icon into whitelist.
    // because the launcher will get thgge icon with his name
    def packageName = <your_application_id>
    whiteList = [
    //for your icon
    packageName + ".R.drawable.icon",
    //for fabric
    packageName + ".R.string.com.crashlytics.*",
    //for umeng update
    packageName + ".R.string.umeng*",
    packageName + ".R.string.UM*",
    packageName + ".R.string.tb_.*",
    packageName + ".R.layout.umeng*",
    packageName + ".R.layout.tb_.*",
    packageName + ".R.drawable.umeng*",
    packageName + ".R.drawable.tb_.*",

```

```

        packageName + ".R.anim.umeng*",
        packageName + ".R.color.umeng*",
        packageName + ".R.color.tb_*",
        packageName + ".R.style.*UM*",
        packageName + ".R.style.umeng*",
        packageName + ".R.id.umeng*"
    ]
    compressFilePattern = [
        "*.png",
        "*.jpg",
        "*.jpeg",
        "*.gif",
        "resources.arsc"
    ]
    sevenzip {
        artifact = 'com.tencent.mm:SevenZip:1.1.7'
        //path = "/usr/local/bin/7za"
    }
}

```

6.3 资源动态加载

在前端开发中，动态加载资源可以有效减小apk的体积。除此之外，只提供对主流架构的支持，比如arm，对于mips和x86架构可以考虑不支持，这样可以大大减小APK的体积。

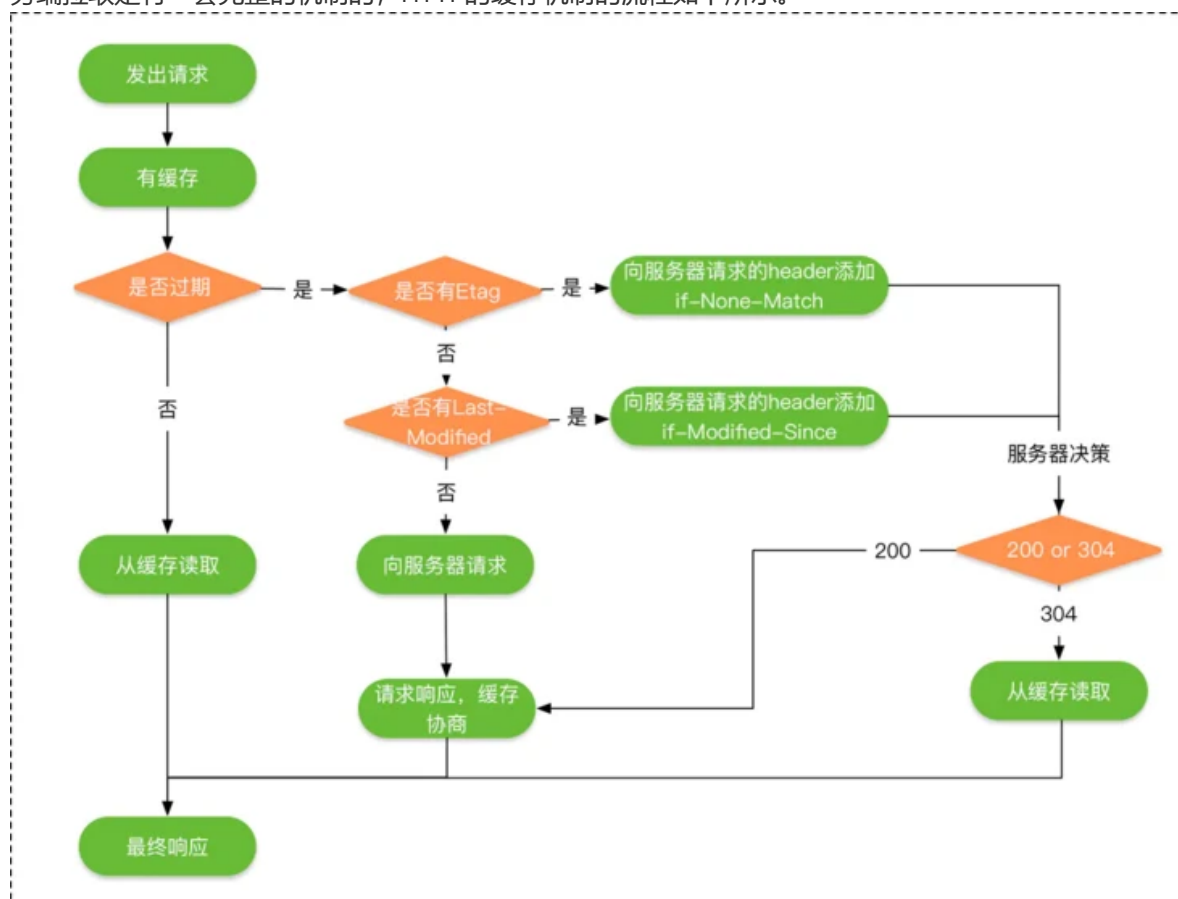
当然，除了上面提到的场景的优化场景外，Android App的优化还包括存储优化、多线程优化以及奔溃处理等方面。

第五章 Android 面试之开源库分析

1，HTTP与缓存理论

1.1 HTTP缓存策略

HTTP的缓存机制也是依赖于请求和响应header里的参数类实现的，最终的响应结果是从缓存还是从服务端拉取是有一套完整的机制的，HTTP的缓存机制的流程如下所示。



HTTP的缓存可以分为两种：强制缓存和对比缓存

1.2 强制缓存

要服务端参与判断是否继续使用缓存，当客户端第一次请求数据是，服务端返回了缓存的过期时间（Expires与Cache-Control），没有过期就可以继续使用缓存，否则则不适用，无需再向服务端询问。

强制缓存使用的两个标识：

- **Expires**：Expires的值为服务端返回的到期时间，即下一次请求时，请求时间小于服务端返回的到期时间，直接使用缓存数据。到期时间是服务端生成的，客户端和服务端的时间可能有误差。
- **Cache-Control**：Expires有个时间校验的问题，所以HTTP1.1采用Cache-Control替代Expires。

其中，Cache-Control的取值有如下一些：

- private:: 客户端可以缓存。
- public:: 客户端和代理服务器都可缓存。
- max-age=xxx: 缓存的内容将在 xxx 秒后失效
- no-cache:: 需要使用对比缓存来验证缓存数据。
- no-store: 所有内容都不会缓存；强制缓存，对比缓存都不会触发。

1.3 对比缓存

需要服务端参与判断是否继续使用缓存，当客户端第一次请求数据时，服务端会将缓存标识（Last-Modified/If-Modified-Since与Etag/If-None-Match）与数据一起返回给客户端，客户端将两者都备份到缓存中，再次请求数据时，客户端将上次备份的缓存标识发送给服务端，服务端根据缓存标识进行判断，如果返回304，则表示通知客户端可以继续使用缓存。

对比缓存的两个标识：

- Last-Modified：表示资源上次修改的时间。当客户端发送第一次请求时，服务端返回资源上次修改的时间，返回格式例子如下：Last-Modified: Tue, 12 Jan 2016 09:31:27 GMT。

- If-Modified-Since：服务端接收到客户端发来的资源修改时间，与自己当前的资源修改时间进行对比，如果自己的资源修改时间大于客户端发来的资源修改时间，则说明资源做过修改，则返回200表示需要重新请求资源，否则返回304表示资源没有被修改，可以继续使用缓存。不同于If-Unmodified-Since，If-Modified-Since只能与GET或HEAD一起使用。与组合使用时If-None-Match，将被忽略，除非服务器不支持If-None-Match。

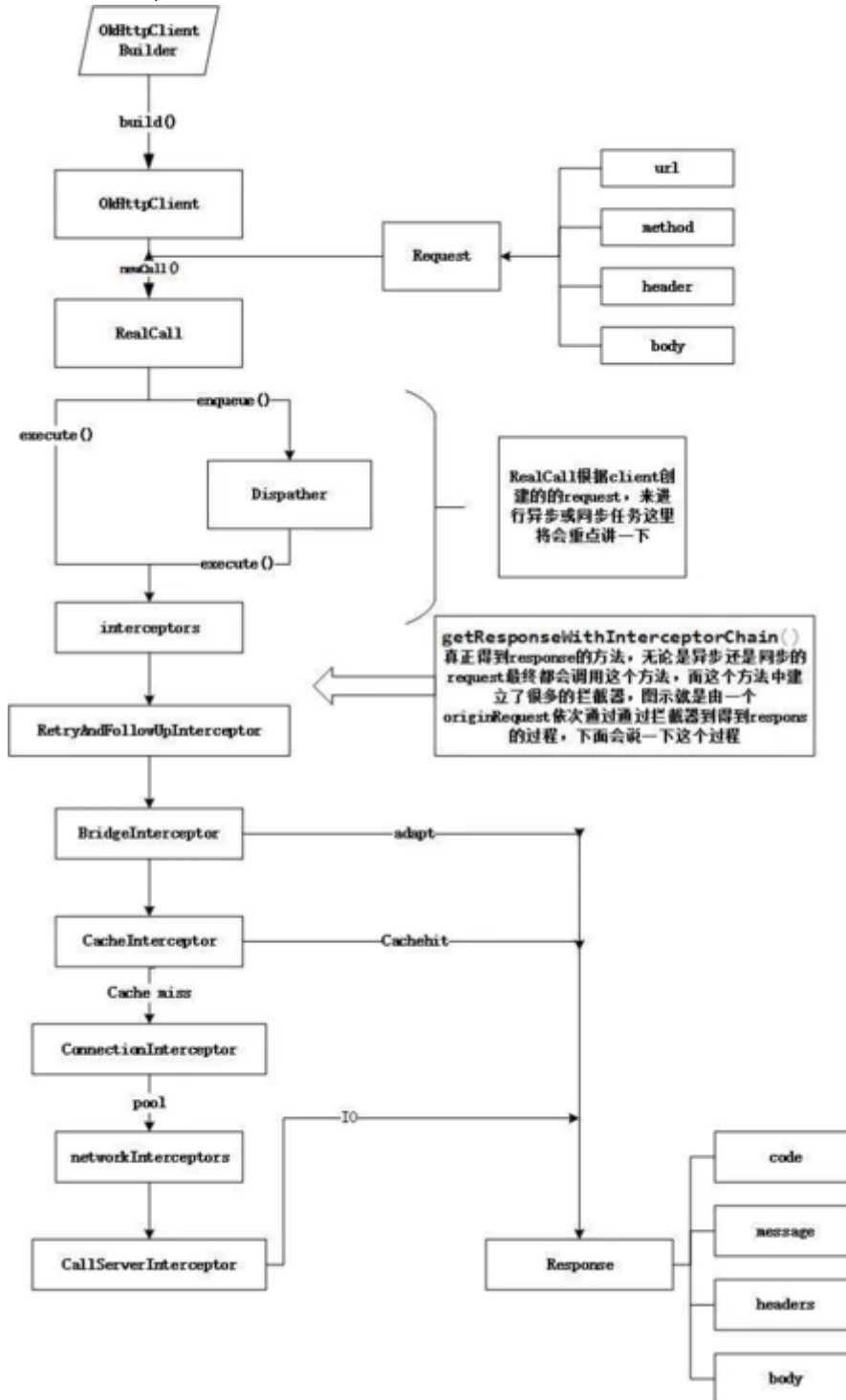
2, OKHttp

现在主流的Android网络请求框架有OKHttp和Retrofit，不过，Retrofit顶层使用的也是OKHttp。在正式介绍OKHttp之前，来看一下Http常见的一些状态码：

- 100~199：指示信息，表示请求已接收，继续处理。
- 200~299：请求成功，表示请求已被成功接收、理解。
- 300~399：重定向，要完成请求必须进行更进一步的操作。
- 400~499：客户端错误，请求有语法错误或请求无法实现。
- 500~599：服务器端错误，服务器未能实现合法的请求。

2.1 OKHttp请求流程

下面是OKHttp内部发起请求的大致流程图，如下图。



下面是使用OKHttp进行Get请求的代码。

```
//1. 新建OkHttpClient客户端
OkHttpClient client = new OkHttpClient();
//新建一个Request对象
Request request = new Request.Builder()
    .url(url)
    .build();
//2. Response为OkHttp中的响应
Response response = client.newCall(request).execute();
```

可以看到，使用OKHttp进行请求时，只需要创建一个OkHttpClient对象和Request对象，然后再调用execute()方法和enqueue()方法即可。其中，execute()方法是同步方法，enqueue()方法是异步方法。

2.2 OkHttpClient

使用OkHttpClient之前，需要先创建一个OkHttpClient客户端，OkHttpClient的构造方法如下。

```
OkHttpClient client = new OkHttpClient();

public OkHttpClient() {
    this(new Builder());
}

OkHttpClient(Builder builder) {
    ....
}
```

可以看到，OkHttpClient使用了建造者模式，Builder里面的可配置参数如下。

```
public static final class Builder {
    Dispatcher dispatcher; // 分发器
    @Nullable Proxy proxy;
    List<Protocol> protocols;
    List<ConnectionSpec> connectionSpecs; // 传输层版本和连接协议
    final List<Interceptor> interceptors = new ArrayList<>(); // 拦截器
    final List<Interceptor> networkInterceptors = new ArrayList<>();
    EventListener.Factory eventListenerFactory;
    ProxySelector proxySelector;
    CookieJar cookieJar;
    @Nullable Cache cache;
    @Nullable InternalCache internalCache; // 内部缓存
    SocketFactory socketFactory;
    @Nullable SslSocketFactory sslSocketFactory; // 安全套接层socket 工厂，用于HTTPS
    @Nullable CertificateChainCleaner certificateChainCleaner; // 验证确认响应证书
    // 适用 HTTPS 请求连接的主机名。
    HostnameVerifier hostnameVerifier; // 验证确认响应证书 适用 HTTPS 请求连接的主机名。
    CertificatePinner certificatePinner; // 证书锁定，使用CertificatePinner来约束哪些认证机构被信任。
    Authenticator proxyAuthenticator; // 代理身份验证
    Authenticator authenticator; // 身份验证
    ConnectionPool connectionPool; // 连接池
    Dns dns;
    boolean followSslRedirects; // 安全套接层重定向
    boolean followRedirects; // 本地重定向
    boolean retryOnConnectionFailure; // 重试连接失败
    int callTimeout;
    int connectTimeout;
    int readTimeout;
    int writeTimeout;
    int pingInterval;

    // 这里是默认配置的构建参数
    public Builder() {
        dispatcher = new Dispatcher();
        protocols = DEFAULT_PROTOCOLS;
        connectionSpecs = DEFAULT_CONNECTION_SPECS;
        ...
    }

    // 这里传入自己配置的构建参数
    Builder(OkHttpClient okHttpClient) {
```



```

        this.dispatcher = okHttpClient.dispatcher;
        this.proxy = okHttpClient.proxy;
        this.protocols = okHttpClient.protocols;
        this.connectionSpecs = okHttpClient.connectionSpecs;
        this.interceptors.addAll(okHttpClient.interceptors);
        this.networkInterceptors.addAll(okHttpClient.networkInterceptors);
        ...
    }

```

2.3 同步请求

同步请求使用的是execute()方法，使用方式如下。

```
Response response = client.newCall(request).execute();
```

下面是涉及到的一些源码。

```

/**
 * Prepares the {@code request} to be executed at some point in the future.
 */
@Override public Call newCall(Request request) {
    return RealCall.newRealCall(this, request, false /* for web socket */);
}

// RealCall为真正的请求执行者
static RealCall newRealCall(OkHttpClient client, Request originalRequest,
    boolean forWebSocket) {
    // Safely publish the Call instance to the EventListener.
    RealCall call = new RealCall(client, originalRequest, forWebSocket);
    call.eventListener = client.eventListenerFactory().create(call);
    return call;
}

@Override public Response execute() throws IOException {
    synchronized (this) {
        // 每个Call只能执行一次
        if (executed) throw new IllegalStateException("Already Executed");
        executed = true;
    }
    captureCallStackTrace();
    timeout.enter();
    eventListener.callStart(this);
    try {
        // 通知dispatcher已经进入执行状态
        client.dispatcher().executed(this);
        // 通过一系列的拦截器请求处理和响应处理得到最终的返回结果
        Response result = getResponseWithInterceptorChain();
        if (result == null) throw new IOException("Canceled");
        return result;
    } catch (IOException e) {
        e = timeoutExit(e);
        eventListener.callFailed(this, e);
        throw e;
    } finally {
        // 通知 dispatcher 自己已经执行完毕
        client.dispatcher().finished(this);
    }
}

```

```

    }
}

Response getResponseWithInterceptorChain() throws IOException {
    // Build a full stack of interceptors.
    List<Interceptor> interceptors = new ArrayList<>();
    // 在配置 OkHttpClient 时设置的 interceptors;
    interceptors.addAll(client.interceptors());
    // 负责失败重试以及重定向
    interceptors.add(retryAndFollowUpInterceptor);
    // 请求时, 对必要的Header进行一些添加, 接收响应时, 移除必要的Header
    interceptors.add(new BridgeInterceptor(client.cookieJar()));
    // 负责读取缓存直接返回、更新缓存
    interceptors.add(new CacheInterceptor(client.internalCache()));
    // 负责和服务端建立连接
    interceptors.add(new ConnectInterceptor(client));
    if (!forWebSocket) {
        // 配置 OkHttpClient 时设置的 networkInterceptors
        interceptors.addAll(client.networkInterceptors());
    }
    // 负责向服务器发送请求数据、从服务器读取响应数据
    interceptors.add(new CallServerInterceptor(forWebSocket));

    Interceptor.Chain chain = new RealInterceptorChain(interceptors, null, null,
null, 0,
        originalRequest, this, eventListener, client.connectTimeoutMillis(),
        client.readTimeoutMillis(), client.writeTimeoutMillis());

    // 使用责任链模式开启链式调用
    return chain.proceed(originalRequest);
}

// StreamAllocation 对象, 它相当于一个管理类, 维护了服务器连接、并发流
// 和请求之间的关系, 该类还会初始化一个 Socket 连接对象, 获取输入/输出流对象。
public Response proceed(Request request, StreamAllocation streamAllocation,
HttpCodec httpCodec,
    RealConnection connection) throws IOException {
    ...

    // call the next interceptor in the chain.
    // 实例化下一个拦截器对应的RealInterceptorChain对象
    RealInterceptorChain next = new RealInterceptorChain(interceptors,
streamAllocation, httpCodec,
        connection, index + 1, request, call, eventListener, connectTimeout,
readTimeout,
        writeTimeout);
    // 得到当前的拦截器
    Interceptor interceptor = interceptors.get(index);
    // 调用当前拦截器的intercept()方法, 并将下一个拦截器的RealInterceptorChain对象传递下
    去,最后得到响应
    Response response = interceptor.intercept(next);

    ...

    return response;
}

```

2.4 异步请求

异步请求使用的是enqueue(), 根据异步的写法, 我们可以在Builder后面跟很多的参数, 如下所示。

```
Request request = new Request.Builder()
    .url("http://publicobject.com/helloworld.txt")
    .build();

client.newCall(request).enqueue(new Callback() {
    @Override
    public void onFailure(Call call, IOException e) {
        e.printStackTrace();
    }

    @Override
    public void onResponse(Call call, Response response) throws IOException {
        ...
    }
})

void enqueue(AsyncCall call) {
    synchronized (this) {
        readyAsyncCalls.add(call);
    }
    promoteAndExecute();
}

// 正在准备中的异步请求队列
private final Deque<AsyncCall> readyAsyncCalls = new ArrayDeque<>();

// 运行中的异步请求
private final Deque<AsyncCall> runningAsyncCalls = new ArrayDeque<>();

// 同步请求
private final Deque<RealCall> runningSyncCalls = new ArrayDeque<>();

// Promotes eligible calls from {@link #readyAsyncCalls} to {@link
#runningAsyncCalls} and runs
// them on the executor service. Must not be called with synchronization because
executing calls
// can call into user code.
private boolean promoteAndExecute() {
    assert (!Thread.holdsLock(this));

    List<AsyncCall> executableCalls = new ArrayList<>();
    boolean isRunning;
    synchronized (this) {
        for (Iterator<AsyncCall> i = readyAsyncCalls.iterator(); i.hasNext(); ) {
            AsyncCall asyncCall = i.next();

            // 如果其中的runningAsyncCalls不满, 且call占用的host小于最大数量, 则将call加入到
runningAsyncCalls中执行,
            // 同时利用线程池执行call; 否者将call加入到readyAsyncCalls中。
            if (runningAsyncCalls.size() >= maxRequests) break; // Max capacity.
            if (runningCallsForHost(asyncCall) >= maxRequestsPerHost) continue; //
Host max capacity.

            i.remove();
```

```

        executableCalls.add(asyncCall);
        runningAsyncCalls.add(asyncCall);
    }
    isRunning = runningCallsCount() > 0;
}

for (int i = 0, size = executableCalls.size(); i < size; i++) {
    AsyncCall asyncCall = executableCalls.get(i);
    asyncCall.executeOn(executorService());
}

return isRunning;
}

```

2.5 CacheInterceptor网络请求缓存处理

okHttp的缓存原则是，缓存拦截器会根据请求的信息和缓存的响应的信息来判断是否存在缓存可用，如果有可以使用的缓存，那么就返回该缓存给用户，否则就继续使用责任链模式来从服务器中获取响应。当获取到响应的时候，又会把响应缓存到磁盘上面。涉及到的代码有：

```

@Override
public Response intercept(Chain chain) throws IOException {
    // 根据request得到cache中缓存的response
    Response cacheCandidate = cache != null
        ? cache.get(chain.request())
        : null;

    long now = System.currentTimeMillis();

    // request判断缓存的策略，是否要使用了网络，缓存或两者都使用
    CacheStrategy strategy = new CacheStrategy.Factory(now, chain.request(),
        cacheCandidate).get();
    Request networkRequest = strategy.networkRequest;
    Response cacheResponse = strategy.cacheResponse;

    if (cache != null) {
        cache.trackResponse(strategy);
    }

    if (cacheCandidate != null && cacheResponse == null) {
        closeQuietly(cacheCandidate.body()); // The cache candidate wasn't
        applicable. Close it.
    }

    // If we're forbidden from using the network and the cache is insufficient,
    fail.
    if (networkRequest == null && cacheResponse == null) {
        return new Response.Builder()
            .request(chain.request())
            .protocol(Protocol.HTTP_1_1)
            .code(504)
            .message("Unsatisfiable Request (only-if-cached)")
            .body(Util.EMPTY_RESPONSE)
            .sentRequestAtMillis(-1L)
            .receivedResponseAtMillis(System.currentTimeMillis())
            .build();
    }
}

```

```

// If we don't need the network, we're done.
if (networkRequest == null) {
    return cacheResponse.newBuilder()
        .cacheResponse(stripBody(cacheResponse))
        .build();
}

Response networkResponse = null;
try {
    // 调用下一个拦截器，决定从网络上得到response
    networkResponse = chain.proceed(networkRequest);
} finally {
    // If we're crashing on I/O or otherwise, don't leak the cache body.
    if (networkResponse == null && cacheCandidate != null) {
        closeQuietly(cacheCandidate.body());
    }
}

// If we have a cache response too, then we're doing a conditional get.
// 如果本地已经存在cacheResponse，那么让它和网络得到的networkResponse做比较，决定是否
// 来更新缓存的cacheResponse
if (cacheResponse != null) {
    if (networkResponse.code() == HTTP_NOT_MODIFIED) {
        Response response = cacheResponse.newBuilder()
            .headers(combine(cacheResponse.headers(),
networkResponse.headers()))
            .sentRequestAtMillis(networkResponse.sentRequestAtMillis())

.receivedResponseAtMillis(networkResponse.receivedResponseAtMillis())
            .cacheResponse(stripBody(cacheResponse))
            .networkResponse(stripBody(networkResponse))
            .build();
        networkResponse.body().close();

        // Update the cache after combining headers but before stripping the
        // Content-Encoding header (as performed by initContentStream()).
        cache.trackConditionalCacheHit();
        cache.update(cacheResponse, response);
        return response;
    } else {
        closeQuietly(cacheResponse.body());
    }
}

Response response = networkResponse.newBuilder()
    .cacheResponse(stripBody(cacheResponse))
    .networkResponse(stripBody(networkResponse))
    .build();

if (cache != null) {
    if (HttpHeaders.hasBody(response) && CacheStrategy.isCacheable(response,
networkRequest)) {
        // Offer this request to the cache.
        // 缓存未经缓存过的response
        CacheRequest cacheRequest = cache.put(response);
        return cachewritingResponse(cacheRequest, response);
    }
}

```

```

        if (HttpMethod.invalidatesCache(networkRequest.method())) {
            try {
                cache.remove(networkRequest);
            } catch (IOException ignored) {
                // The cache cannot be written.
            }
        }
    }

    return response;
}

```

2.6 ConnectInterceptor之连接池

ConnectInterceptor连接池拦截器包含两个方面的内容，一是网络连接角度发挥作用的网络拦截器，二是从连接池的操作角度发挥作用的拦截器。

```

@Override public Response intercept(Chain chain) throws IOException {
    RealInterceptorChain realChain = (RealInterceptorChain) chain;
    Request request = realChain.request();
    StreamAllocation streamAllocation = realChain.streamAllocation();

    // We need the network to satisfy this request.    Possibly for validating
    a conditional GET.
    boolean doExtensiveHealthChecks = !request.method().equals("GET");
    // HttpCodec是对 HTTP 协议操作的抽象，有两个实现：Http1Codec和Http2Codec，顾名思义，
    它们分别对应 HTTP/1.1 和 HTTP/2 版本的实现。在这个方法的内部实现连接池的复用处理
    HttpCodec httpCodec = streamAllocation.newStream(client, chain,
doExtensiveHealthChecks);
    RealConnection connection = streamAllocation.connection();

    return realChain.proceed(request, streamAllocation, httpCodec, connection);
}

// Returns a connection to host a new stream. This // prefers the existing
connection if it exists,
// then the pool, finally building a new connection.
// 调用 streamAllocation 的 newStream() 方法的时候，最终会经过一系列
// 的判断到达 StreamAllocation 中的 findConnection() 方法
private RealConnection findConnection(int connectTimeout, int readTimeout, int
writeTimeout,
    int pingIntervalMillis, boolean connectionRetryEnabled) throws IOException {
    ...

    // Attempt to use an already-allocated connection. We need to be careful
    here because our
    // already-allocated connection may have been restricted from creating new
    streams.
    // 尝试使用已分配的连接，已经分配的连接可能已经被限制创建新的流
    releasedConnection = this.connection;
    // 释放当前连接的资源，如果该连接已经被限制创建新的流，就返回一个Socket以关闭连接
    toClose = releaseIfNoNewStreams();
    if (this.connection != null) {
        // we had an already-allocated connection and it's good.

```

```

        result = this.connection;
        releasedConnection = null;
    }
    if (!reportedAcquired) {
        // If the connection was never reported acquired, don't report it as
released!
        // 如果该连接从未被标记为获得，不要标记为发布状态，reportedAcquired 通过
acquire() 方法修改
        releasedConnection = null;
    }

    if (result == null) {
        // Attempt to get a connection from the pool.
        // 尝试从连接池中获取一个连接
        Internal.instance.get(connectionPool, address, this, null);
        if (connection != null) {
            foundPooledConnection = true;
            result = connection;
        } else {
            selectedRoute = route;
        }
    }
}
// 关闭连接
closeQuietly(toClose);

if (releasedConnection != null) {
    eventListener.connectionReleased(call, releasedConnection);
}
if (foundPooledConnection) {
    eventListener.connectionAcquired(call, result);
}
if (result != null) {
    // If we found an already-allocated or pooled connection, we're done.
    // 如果已经从连接池中获取到了一个连接，就将其返回
    return result;
}

// If we need a route selection, make one. This is a blocking operation.
boolean newRouteSelection = false;
if (selectedRoute == null && (routeSelection == null ||
!routeSelection.hasNext())) {
    newRouteSelection = true;
    routeSelection = routeSelector.next();
}

synchronized (connectionPool) {
    if (canceled) throw new IOException("Canceled");

    if (newRouteSelection) {
        // Now that we have a set of IP addresses, make another attempt at
getting a connection from
        // the pool. This could match due to connection coalescing.
        // 根据一系列的 IP地址从连接池中获取一个链接
        List<Route> routes = routeSelection.getAll();
        for (int i = 0, size = routes.size(); i < size; i++) {
            Route route = routes.get(i);
            // 从连接池中获取一个连接

```



```

        Internal.instance.get(connectionPool, address, this, route);
        if (connection != null) {
            foundPooledConnection = true;
            result = connection;
            this.route = route;
            break;
        }
    }
}

if (!foundPooledConnection) {
    if (selectedRoute == null) {
        selectedRoute = routeSelection.next();
    }

    // Create a connection and assign it to this allocation immediately.
    This makes it possible
    // for an asynchronous cancel() to interrupt the handshake we're about
    to do.
    // 在连接池中如果没有该连接，则创建一个新的连接，并将其分配，这样我们就可以在握手之前
    进行终端
    route = selectedRoute;
    refusedStreamCount = 0;
    result = new RealConnection(connectionPool, selectedRoute);
    acquire(result, false);
}

// If we found a pooled connection on the 2nd time around, we're done.
if (foundPooledConnection) {
    // 如果我们在第二次的时候发现了一个池连接，那么我们就将其返回
    eventListener.connectionAcquired(call, result);
    return result;
}

// Do TCP + TLS handshakes. This is a blocking operation.
// 进行 TCP 和 TLS 握手
result.connect(connectTimeout, readTimeout, writeTimeout,
pingIntervalMillis,
connectionRetryEnabled, call, eventListener);
routeDatabase().connected(result.route());

Socket socket = null;
synchronized (connectionPool) {
    reportedAcquired = true;

    // Pool the connection.
    // 将该连接放进连接池中
    Internal.instance.put(connectionPool, result);

    // If another multiplexed connection to the same address was created
    concurrently, then
    // release this connection and acquire that one.
    // 如果同时创建了另一个到同一地址的多路复用连接，释放这个连接并获取那个连接
    if (result.isMultiplexed()) {
        socket = Internal.instance.deduplicate(connectionPool, address, this);
        result = connection;
    }
}
}

```

```

        closeQuietly(socket);

        eventListener.connectionAcquired(call, result);
        return result;
    }

```

从以上的源码分析可知：

- 判断当前的连接是否可以使用：流是否已经被关闭，并且已经被限制创建新的流；
- 如果当前的连接无法使用，就从连接池中获取一个连接；
- 连接池中也没有发现可用的连接，创建一个新的连接，并进行握手，然后将其放到连接池中。

在从连接池中获取一个连接的时候，使用了 Internal 的 get() 方法。Internal 有一个静态的实例，会在 OkHttpClient 的静态代码块中被初始化。我们会在 Internal 的 get() 中调用连接池的 get() 方法来得到一个连接。并且，从中我们明白了连接复用的一个好处就是省去了进行 TCP 和 TLS 握手的一个过程。因为建立连接本身也是需要消耗一些时间的，连接被复用之后可以提升我们网络访问的效率。

接下来详细分析下 ConnectionPool 是如何实现连接管理的。

OkHttp 的缓存管理分成两个步骤，一边当我们创建了一个新的连接的时候，我们要把它放进缓存里面；另一边，我们还要来对缓存进行清理。在 ConnectionPool 中，当我们向连接池中缓存一个连接的时候，只要调用双端队列的 add() 方法，将其加入到双端队列即可，而清理连接缓存的操作则交给线程池来定时执行。

```

private final Deque<RealConnection> connections = new ArrayDeque<>();

void put(RealConnection connection) {
    assert (Thread.holdsLock(this));
    if (!cleanupRunning) {
        cleanupRunning = true;
        // 使用线程池执行清理任务
        executor.execute(cleanupRunnable);
    }
    // 将新建的连接插入到双端队列中
    connections.add(connection);
}

private final Runnable cleanupRunnable = new Runnable() {
    @Override public void run() {
        while (true) {
            // 内部调用 cleanup() 方法来清理无效的连接
            long waitNanos = cleanup(System.nanoTime());
            if (waitNanos == -1) return;
            if (waitNanos > 0) {
                long waitMillis = waitNanos / 1000000L;
                waitNanos -= (waitMillis * 1000000L);
                synchronized (ConnectionPool.this) {
                    try {
                        ConnectionPool.this.wait(waitMillis, (int) waitNanos);
                    } catch (InterruptedException ignored) {}
                }
            }
        }
    }
};

long cleanup(long now) {

```

```

int inUseConnectionCount = 0;
int idleConnectionCount = 0;
RealConnection longestIdleConnection = null;
long longestIdleDurationNs = Long.MIN_VALUE;

// Find either a connection to evict, or the time that the next eviction is
due.
synchronized (this) {
    // 遍历所有的连接
    for (Iterator<RealConnection> i = connections.iterator(); i.hasNext(); )
    {
        RealConnection connection = i.next();

        // If the connection is in use, keep searching.
        // 遍历所有的连接
        if (pruneAndGetAllocationCount(connection, now) > 0) {
            inUseConnectionCount++;
            continue;
        }

        idleConnectionCount++;

        // If the connection is ready to be evicted, we're done.
        // 如果找到了一个可以被清理的连接，会尝试去寻找闲置时间最久的连接来释放
        long idleDurationNs = now - connection.idleAtNanos;
        if (idleDurationNs > longestIdleDurationNs) {
            longestIdleDurationNs = idleDurationNs;
            longestIdleConnection = connection;
        }
    }

    // maxIdleConnections 表示最大允许的闲置的连接的数量,keepAliveDurationNs表示连接
    // 允许存活的最长的时间。
    // 默认空闲连接最大数目为5个, keepalive 时间最长为5分钟。
    if (longestIdleDurationNs >= this.keepAliveDurationNs
        || idleConnectionCount > this.maxIdleConnections) {
        // We've found a connection to evict. Remove it from the list, then
        // below (outside
        // of the synchronized block).
        // 该连接的时长超出了最大的活跃时长或者闲置的连接数量超出了最大允许的范围，直接移除
        connections.remove(longestIdleConnection);
    } else if (idleConnectionCount > 0) {
        // A connection will be ready to evict soon.
        // 闲置的连接的数量大于0，停顿指定的时间（等会儿会将其清理掉，现在还不是时候）
        return keepAliveDurationNs - longestIdleDurationNs;
    } else if (inUseConnectionCount > 0) {
        // All connections are in use. It'll be at least the keep alive
        duration 'til we run again.
        // 所有的连接都在使用中，5分钟后再清理
        return keepAliveDurationNs;
    } else {
        // No connections, idle or in use.
        // 没有连接
        cleanupRunning = false;
        return -1;
    }
}
}

```

从以上的源码分析可知，首先会对缓存中的连接进行遍历，以寻找一个闲置时间最长的连接，然后根据该连接的闲置时长和最大允许的连接数量等参数来决定是否应该清理该连接。同时注意上面的方法的返回值是一个时间，如果闲置时间最长的连接仍然需要一段时间才能被清理的时候，会返回这段时间的时间差，然后会在这段时间之后再次对连接池进行清理。

3, Retrofit

Retrofit 是一个 RESTful 的 HTTP 网络请求框架的封装，本质上网络请求是 OkHttp 完成的，而 Retrofit 仅负责网络请求接口的封装。客户端使用 Retrofit，实际上是使用 Retrofit 接口层封装请求参数、Header、Url 等信息，之后由 OkHttp 完成后续的请求操作，当服务端返回数据之后，OkHttp 再将原始的结果交给 Retrofit，Retrofit 然后根据用户的需求，对结果进行解析。

3.1 基本使用

首先，定义一个 HTTP API，用于描述请求，比如下面是一个 Get 请求。

```
public interface GitHubService {  
  
    @GET("users/{user}/repos")  
    Call<List<Repo>> listRepos(@Path("user") String user);  
}
```

然后，创建一个 Retrofit 并生成 API 的实现，返回类型是请求的返回值类型，方法的参数即是请求的参数。

```
// 1.Retrofit构建过程  
Retrofit retrofit = new Retrofit.Builder()  
    .baseUrl("https://api.github.com/")  
    .build();  
  
// 2.创建网络请求接口类实例过程  
GitHubService service = retrofit.create(GitHubService.class);
```

最后，再调用 API 方法生成 Call 完成请求。

```
// 3.生成并执行请求过程  
Call<List<Repo>> repos = service.listRepos("octocat");  
repos.execute() or repos.enqueue()
```

上面是一个简单的 Get 请求的事例，POST 请求只需要将 API 定义改为 POST 即可。Retrofit 的基本使用流程很简洁，但是简洁并不代表简单，Retrofit 为了实现这种简洁的使用流程，内部使用了优秀的架构设计和大量的设计模式，仔细阅读 Retrofit 最新版的源码会发现用到大量的设计模式。比如，Retrofit 构建过程会用到建造者模式、工厂方法模式，建网络请求接口实例过程会用到外观模式、代理模式、单例模式、策略模式、装饰模式（建造者模式），生成并执行请求过程适配器模式（代理模式、装饰模式）。

3.2 源码分析

3.2.1 Retrofit 构建过程

1, Retrofit 核心对象解析

首先 Retrofit 中有一个全局变量非常关键，在 V2.5 之前的版本，使用的是 LinkedHashMap()，它是一个网络请求配置对象，是由网络请求接口中方法注解进行解析后得到的。

```

public final class Retrofit {
    private final Map<Method, ServiceMethod<?>> serviceMethodCache = new
    ConcurrentHashMap<>();

    ...
}

```

Retrofit使用了建造者模式通过内部类Builder类建立一个Retrofit实例，如下所示。

```

public static final class Builder {

    // 平台类型对象 (Platform -> Android)
    private final Platform platform;
    // 网络请求工厂，默认使用OkHttpClient (工厂方法模式)
    private @Nullable OkHttpClient callFactory;
    // 网络请求的url地址
    private @Nullable HttpUrl baseUrl;
    // 数据转换器工厂的集合
    private final List<Converter.Factory> converterFactories = new ArrayList<>
    ();
    // 网络请求适配器工厂的集合，默认是ExecutorsCallAdapterFactory
    private final List<CallAdapter.Factory> callAdapterFactories = new
    ArrayList<>();
    // 回调方法执行器，在 Android 上默认是封装了 handler 的 MainThreadExecutor，默认作
    用是：切换线程 (子线程 -> 主线程)
    private @Nullable Executor callbackExecutor;

    private boolean validateEagerly;
}

```

2, Builder内部构造

```

public static final class Builder {

    ...

    Builder(Platform platform) {
        this.platform = platform;
    }

    public Builder() {
        this(Platform.get());
    }

    ...
}

class Platform {

    private static final Platform PLATFORM = findPlatform();

    static Platform get() {
        return PLATFORM;
    }
}

```

```

private static Platform findPlatform() {
    try {
        // 使用JVM加载类的方式判断是否是Android平台
        Class.forName("android.os.Build");
        if (Build.VERSION.SDK_INT != 0) {
            return new Android();
        }
    } catch (ClassNotFoundException ignored) {
    }
    try {
        // 同时支持Java平台
        Class.forName("java.util.Optional");
        return new Java8();
    } catch (ClassNotFoundException ignored) {
    }
    return new Platform();
}

static class Android extends Platform {

    ...

    @Override public Executor defaultCallbackExecutor() {
        //切换线程（子线程 -> 主线程）
        return new MainThreadExecutor();
    }

    // 创建默认的网络请求适配器工厂，如果是Android7.0或Java8上，则使
    // 用了并发包中的CompletableFuture保证了回调的同步
    // 在Retrofit中提供了四种CallAdapterFactory(策略模式):
    // ExecutorCallAdapterFactory（默认）、GuavaCallAdapterFactory、
    // va8CallAdapterFactory、RxJavaCallAdapterFactory
    @Override List<? extends CallAdapter.Factory> defaultCallAdapterFactories(
        @Nullable Executor callbackExecutor) {
        if (callbackExecutor == null) throw new AssertionError();
        ExecutorCallAdapterFactory executorFactory = new
ExecutorCallAdapterFactory(callbackExecutor);
        return Build.VERSION.SDK_INT >= 24
            ? asList(CompletableFutureCallAdapterFactory.INSTANCE, executorFactory)
            : singletonList(executorFactory);
    }

    ...

    @Override List<? extends Converter.Factory> defaultConverterFactories() {
        return Build.VERSION.SDK_INT >= 24
            ? singletonList(OptionalConverterFactory.INSTANCE)
            : Collections.<Converter.Factory>emptyList();
    }

    ...

    static class MainThreadExecutor implements Executor {

        // 获取Android 主线程的Handler
        private final Handler handler = new Handler(Looper.getMainLooper());
    }
}

```

```

@Override public void execute(Runnable r) {

    // 在UI线程对网络请求返回数据处理
    handler.post(r);
}
}
}

```

3, 添加baseUrl

baseUrl最基本的功能就是将String类型的url转换为OkHttp的HttpUrl的过程，涉及的代码如下。

```

public Builder baseUrl(String baseUrl) {
    checkNotNull(baseUrl, "baseUrl == null");
    return baseUrl(HttpUrl.get(baseUrl));
}

public Builder baseUrl(HttpUrl baseUrl) {
    checkNotNull(baseUrl, "baseUrl == null");
    List<String> pathSegments = baseUrl.pathSegments();
    if (!"".equals(pathSegments.get(pathSegments.size() - 1))) {
        throw new IllegalArgumentException("baseUrl must end in /: " + baseUrl);
    }
    this.baseUrl = baseUrl;
    return this;
}

```

4, build过程

build()主要任务是执行Retrofit对象的创建，涉及代码如下。

```

public Retrofit build() {

    if (baseUrl == null) {
        throw new IllegalStateException("Base URL required.");
    }

    okhttp3.Call.Factory callFactory = this.callFactory;
    if (callFactory == null) {
        // 默认使用okhttp
        callFactory = new OkHttpClient();
    }

    Executor callbackExecutor = this.callbackExecutor;
    if (callbackExecutor == null) {
        // Android默认的callbackExecutor
        callbackExecutor = platform.defaultCallbackExecutor();
    }

    // Make a defensive copy of the adapters and add the defaultCall adapter.
    List<CallAdapter.Factory> callAdapterFactories = new ArrayList<>
        (this.callAdapterFactories);
    // 添加默认适配器工厂在集合尾部
    callAdapterFactories.addAll(platform.defaultCallAdapterFactoryisca
        llbackExecutor));
}

```



```

        // Make a defensive copy of the converters.
        List<Converter.Factory> converterFactories = new ArrayList<>(
            1 + this.converterFactories.size() +
platform.defaultConverterFactoriesSize());
        // Add the built-in converter factory first. This prevents overriding its
behavior but also
        // ensures correct behavior when using converters that consume all types.
        converterFactories.add(new BuiltInConverters());
        converterFactories.addAll(this.converterFactories);
        converterFactories.addAll(platform.defaultConverterFactories());

        return new Retrofit(callFactory, baseUrl,
            unmodifiableList(converterFactories),
                unmodifiableList(callAdapterFactories), callbackExecutor,
            validateEagerly);
    }
}

```

3.2.2 创建网络请求接口实例

Retrofit.create()使用了外观模式和代理模式创建了网络请求的接口实例，创建create()的方法如下。

```

public <T> T create(final Class<T> service) {
    Utils.validateServiceInterface(service);
    if (validateEagerly) {
        // 判断是否需要提前缓存ServiceMethod对象
        eagerlyValidateMethods(service);
    }

    // 使用动态代理拿到请求接口所有注解配置后，创建网络请求接口实例
    return (T) Proxy.newProxyInstance(service.getClassLoader(), new Class<?>[]
{ service },
        new InvocationHandler() {
            private final Platform platform = Platform.get();
            private final Object[] emptyArgs = new Object[0];

            @Override public Object invoke(Object proxy, Method method, @Nullable
Object[] args)
                throws Throwable {
                // If the method is a method from Object then defer to normal
invocation.
                if (method.getDeclaringClass() == Object.class) {
                    return method.invoke(this, args);
                }
                if (platform.isDefaultMethod(method)) {
                    return platform.invokeDefaultMethod(method, service, proxy, args);
                }
                return loadServiceMethod(method).invoke(args != null ? args :
emptyArgs);
            }
        });
}

private void eagerlyValidateMethods(Class<?> service) {

    Platform platform = Platform.get();
    for (Method method : service.getDeclaredMethods()) {

```

```

        if (!platform.isDefaultMethod(method)) {
            loadServiceMethod(method);
        }
    }
}

```

然后，我们再看一下loadServiceMethod()方法。

```

ServiceMethod<?> loadServiceMethod(Method method) {

    ServiceMethod<?> result = serviceMethodCache.get(method);
    if (result != null) return result;

    synchronized (serviceMethodCache) {
        result = serviceMethodCache.get(method);
        if (result == null) {
            // 解析注解配置得到了ServiceMethod
            result = ServiceMethod.parseAnnotations(this, method);
            // 可以看到，最终加入到ConcurrentHashMap缓存中
            serviceMethodCache.put(method, result);
        }
    }
    return result;
}

abstract class ServiceMethod<T> {
    static <T> ServiceMethod<T> parseAnnotations(Retrofit retrofit, Method
method) {
        // 通过RequestFactory解析注解配置（工厂模式、内部使用了建造者模式）
        RequestFactory requestFactory =
RequestFactory.parseAnnotations(retrofit, method);

        Type returnType = method.getGenericReturnType();
        if (Utils.hasUnresolvableType(returnType)) {
            throw methodError(method,
                "Method return type must not include a type variable or wildcard:
%s", returnType);
        }
        if (returnType == void.class) {
            throw methodError(method, "Service methods cannot return void.");
        }

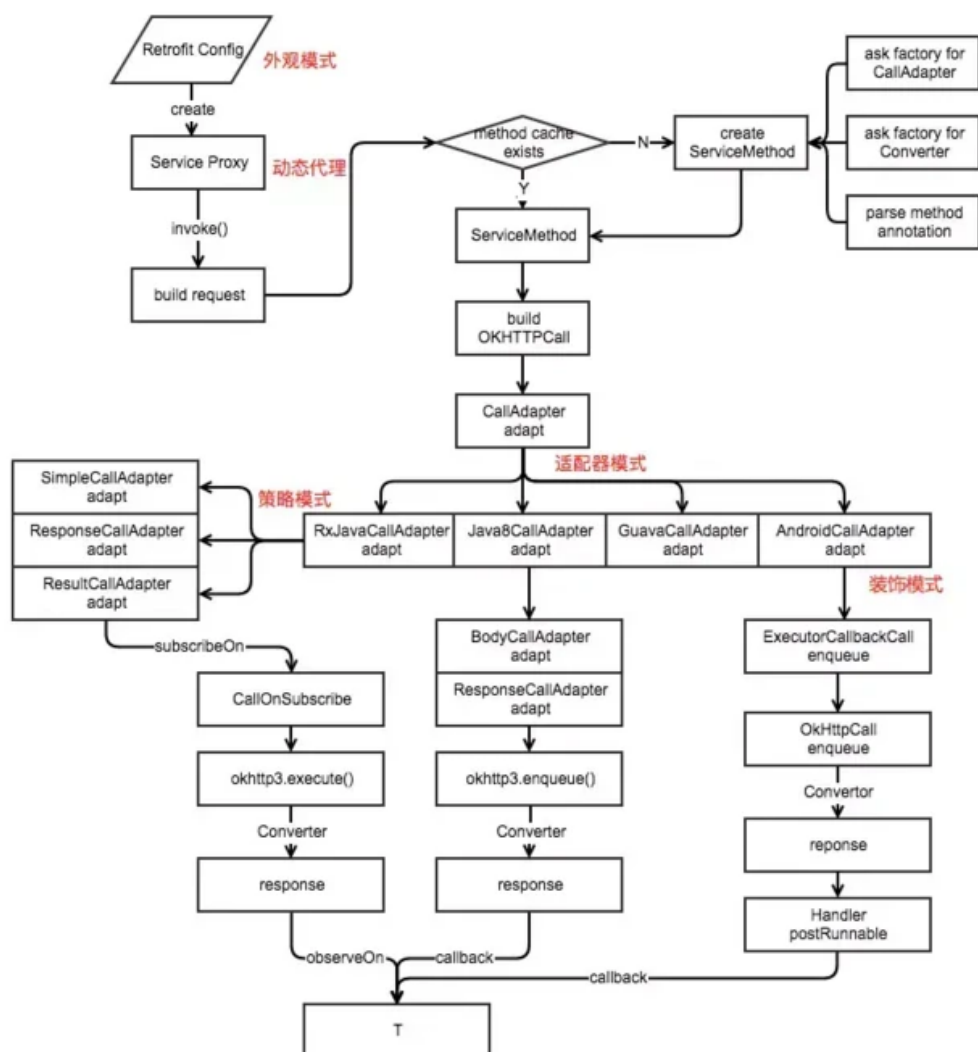
        // 最终是通过HttpServiceMethod构建的请求方法
        return HttpServiceMethod.parseAnnotations(retrofit, method,
requestFactory);
    }

    abstract T invoke(Object[] args);
}

```

3.3 Retrofit流程图

Retrofit最新的版本是2.9.0，已经大半年没有更新了。Retrofit虽然只是一个RESTful 的HTTP 网络请求框架的封装库。但是，它内部通过 大量的设计模式 封装了 OkHttp，让使用者感到它非常简洁、易懂。它内部主要是用动态代理的方式，动态将网络请求接口的注解解析成HTTP请求，最后执行请求的过程。Retrofit完整的流程如下图所示。



4, Glide

4.1 基本使用

作为一个Android图片加载框架，Glide具有功能全、性能高，使用简单等优点。使用下面这一行代码就可以完成图片的加载与展示。

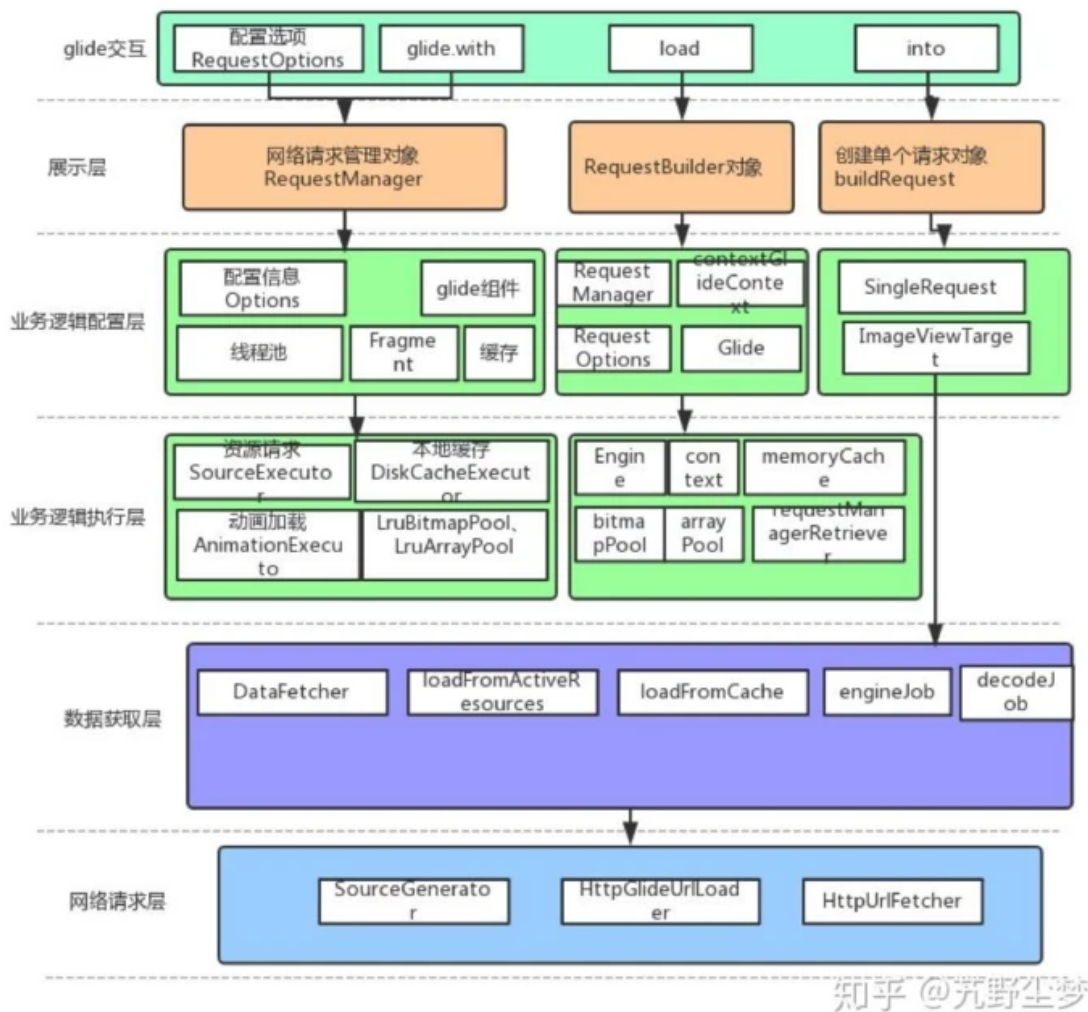
```
glide.with(context).load(url).into(iv);
```

除此之外，我们还可以在图片的加载过程中指定一个占位图。

```
glide.with(this)
    .load(url)
    .placeholder(R.drawable.noimage)
    .into(iv);
```

4.2 源码分析

下面是一个完整的Glide框架的架构图。



4.2.1 with(context)

我们使用Glide，都是从Glide.with()方法开始的，源码如下。

```
public static RequestManager with(Context context) {
    RequestManagerRetriever retriever = RequestManagerRetriever.get();
    return retriever.get(context);
}

public static RequestManager with(Activity activity) {
    RequestManagerRetriever retriever = RequestManagerRetriever.get();
    return retriever.get(activity);
}

public static RequestManager with(FragmentActivity activity) {
    RequestManagerRetriever retriever = RequestManagerRetriever.get();
    return retriever.get(activity);
}

public static RequestManager with(android.app.Fragment fragment) {
    RequestManagerRetriever retriever = RequestManagerRetriever.get();
    return retriever.get(fragment);
}

public static RequestManager with(Fragment fragment) {
```

```

    RequestManagerRetriever retriever = RequestManagerRetriever.get();
    return retriever.get(fragment);
}

```

可以看到，with()方法有很多，但内容基本一致，都是通过RequestManagerRetriever.get()获取RequestManagerRetriever对象retriever，然后通过retriever.get(context)获取一个RequestManager对象并返回。这些with()方法关键的不同在于传入的参数不同，可以是Context、Activity、Fragment等等。

之所以Glide在加载图片的时候要绑定with(context)方法中传入的context的生命周期，如果传入的是Activity，那么在这个Activity销毁的时候Glide会停止图片的加载。这样做的好处在于：避免了消耗多余的资源，也避免了在Activity销毁之后加载图片从而导致空指针问题。

接下来，我们看一下RequestManagerRetriever类的源码。

```

public class RequestManagerRetriever implements Handler.Callback {
    //饿汉式创建单例
    private static final RequestManagerRetriever INSTANCE = new
RequestManagerRetriever();

    //返回单例对象
    public static RequestManagerRetriever get() {
        return INSTANCE;
    }

    //根据传入的参数，获取不同的RequestManager
    public RequestManager get(Context context) {
        if (context == null) {
            throw new IllegalArgumentException("You cannot start a load on a
null Context");
        } else if (Util.isOnMainThread() && !(context instanceof Application)) {
            if (context instanceof FragmentActivity) {
                return get((FragmentActivity) context);
            } else if (context instanceof Activity) {
                return get((Activity) context);
            } else if (context instanceof ContextWrapper) {
                return get(((ContextWrapper) context).getBaseContext());
            }
        }

        return getApplicationManager(context);
    }

    //省略无关代码.....
}

```

上面的代码是一个饿汉式单例模式，核心是根据传入的参数获取不同的RequestManager。接下来，我们看一下getApplicationManager方法。

```

private RequestManager getApplicationManager(Context context) {
    // 返回一个单例
    if (applicationManager == null) {
        synchronized (this) {
            if (applicationManager == null) {
                applicationManager = new
RequestManager(context.getApplicationContext(),
                    new ApplicationLifecycle(), new
EmptyRequestManagerTreeNode());
            }
        }
    }
    return applicationManager;
}

```

getApplicationManager(Context context)通过双检查单例模式创建并返回applicationManager。如果传入的context是Activity时，操作如下。

```

public RequestManager get(Activity activity) {
    //如果不在主线程或者Android SDK的版本低于HONEYCOMB，传入的还是Application类型的
context
    if (Util.isOnBackgroundThread() || Build.VERSION.SDK_INT <
Build.VERSION_CODES.HONEYCOMB) {
        return get(activity.getApplicationContext());
    } else {
        //判断当前activity是否被销毁
        assertNotDestroyed(activity);
        android.app.FragmentManager fm = activity.getFragmentManager();
        //通过fragmentGet(activity, fm)获取RequestManager
        return fragmentGet(activity, fm);
    }
}

```

如果传入的context是Activity时，操作如下。

```

public RequestManager get(Activity activity) {
    //如果不在主线程或者Android SDK的版本低于HONEYCOMB，传入的还是Application类型的
context
    if (Util.isOnBackgroundThread() || Build.VERSION.SDK_INT <
Build.VERSION_CODES.HONEYCOMB) {
        return get(activity.getApplicationContext());
    } else {
        //判断当前activity是否被销毁
        assertNotDestroyed(activity);
        android.app.FragmentManager fm = activity.getFragmentManager();
        //通过fragmentGet(activity, fm)获取RequestManager
        return fragmentGet(activity, fm);
    }
}

```

下面是with()方法的一个简单的总结。

- 通过RequestManagerRetriever.get()获取RequestManagerRetriever单例对象。
- 通过retriever.get(context)获取RequestManager，在get(context)方法中通过对context类型的判断做不同的处理。

- 当context是Application, 通过getApplicationManager(Context context)创建并返回一个RequestManager对象。
- 当context是Activity, 通过fragmentGet(activity, fm)在当前activity创建并添加一个没有界面的fragment, 从而实现图片加载与activity的生命周期相绑定, 之后创建并返回一个RequestManager对象。

4.2.2 load(url)

load(url)主要是用于加载网络的图片, 如下所示。

```

Glide.with(context)
    .load(url)
    .placeholder(R.drawable.place_image)
    .error(R.drawable.error_image)
    .into(imageView);

```

load(url)方法的源码如下。

```

public DrawableTypeRequest<String> load(String string) {
    return (DrawableTypeRequest<String>) fromString().load(string);
}

```

load()方法涉及fromString()、load(string)两个方法, 接下来看下这两个方法。

fromString()

```

public DrawableTypeRequest<String> fromString() {
    return loadGeneric(String.class);
}

private <T> DrawableTypeRequest<T> loadGeneric(Class<T> modelClass) {
    ModelLoader<T, InputStream> streamModelLoader =
        Glide.buildStreamModelLoader(modelClass, context);
    ModelLoader<T, ParcelFileDescriptor> fileDescriptorModelLoader =
        Glide.buildFileDescriptorModelLoader(modelClass, context);
    if (modelClass != null && streamModelLoader == null &&
        fileDescriptorModelLoader == null) {
        throw new IllegalArgumentException("Unknown type " + modelClass + ".
You must provide a Model of a type for"
            + " which there is a registered ModelLoader, if you are
using a custom model, you must first call"
            + " Glide#register with a ModelLoaderFactory for your custom
model class");
    }

    //这句是核心, 本质是创建并返回了一个DrawableTypeRequest
    return optionsApplier.apply(
        new DrawableTypeRequest<T>(modelClass, streamModelLoader,
            fileDescriptorModelLoader, context,
            glide, requestTracker, lifecycle, optionsApplier));
}

```

loadGeneric()方法本质是创建并返回一个DrawableTypeRequest, Drawable类型的请求。

load(string)

load(string)方法的代码如下。

```

@Override
public DrawableRequestBuilder<ModelType> load(ModelType model) {
    super.load(model);
    return this;
}

```

此方法首先调用DrawableRequestBuilder的父类GenericRequestBuilder的load()方法，然后返回自身。接下来，看一下DrawableRequestBuilder父类中的load()方法，如下所示。

```

public GenericRequestBuilder<ModelType, DataType, ResourceType, TranscodeType>
load(ModelType model) {
    this.model = model;
    isModelSet = true;
    return this;
}

```

DrawableRequestBuilder的父类是GenericRequestBuilder，从名字中我们也可以看出来，前者是Drawable请求的构建者，后者是通用的请求构建者，他们是子父关系。

经过上面的分析我们可以知道，在Glide.with(context).load(url)之后会返回一个DrawableTypeRequest的对象，它的父类是DrawableRequestBuilder，DrawableRequestBuilder的父类是GenericRequestBuilder，我们写的placeholder()、error()等等相关图片请求配置的方法都定义在GenericRequestBuilder中。

4.2.3 into(imageView)

Glide中的前两部是创建了一个Request，这个Request可以理解为对图片加载的配置请求，需要注意的是仅仅是创建了一个请求，而没有去执行。只有当调用into()方法时，这个请求才会真正的被执行。into(imageView)的源码如下。

```

public Target<GlideDrawable> into(ImageView view) {
    return super.into(view);
}

```

发现它调用的是父类GenericRequestBuilder的into()方法，那我们继续看GenericRequestBuilder中的into()方法。

```

public Target<TranscodeType> into(ImageView view) {
    //确保在主线程
    Util.assertMainThread();
    //确保view不为空
    if (view == null) {
        throw new IllegalArgumentException("You must pass in a non null
view");
    }
    //对ScaleType进行配置
    if (!isTransformationSet && view.getScaleType() != null) {
        switch (view.getScaleType()) {
            case CENTER_CROP:
                applyCenterCrop();
                break;
            case FIT_CENTER:
            case FIT_START:
            case FIT_END:
                applyFitCenter();

```



```

        break;
        //$CASES-OMITTED$
        default:
            // Do nothing.
    }
}

//核心
return into(glide.buildImageViewTarget(view, transcodeClass));
}

```

可以看到，上面的方法就是into()的核心代码，它定义在GenericRequestBuilder这个通用的请求构建者中。方法的核心是最后一行：into(glide.buildImageViewTarget(view, transcodeClass))，首先通过glide.buildImageViewTarget(view, transcodeClass)创建出一个Target类型的对象，然后把这个target传入GenericRequestBuilder中的into()方法中。

关于buildImageViewTarget(view, transcodeClass)方法，我们就不再赘述了。

6, EventBus

6.1 基本使用

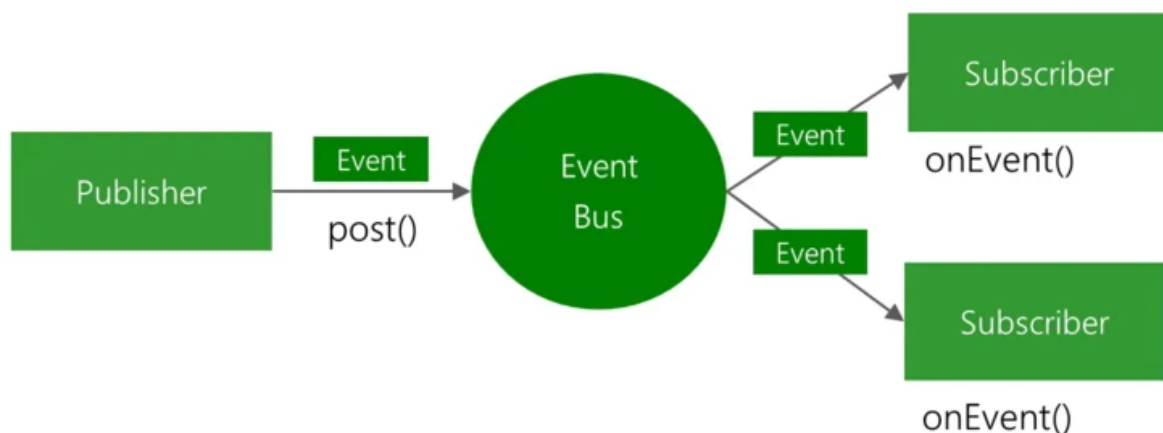
6.1.1 基本概念

EventBus是一种用于Android的事件发布-订阅的事件总线。它简化了应用程序内各个组件之间进行通信的复杂度，尤其是碎片之间进行通信的问题，可以避免由于使用广播通信而带来的诸多不便。

EventBus由三个角色构成：Publisher、Event和Subscriber。

- **Event**：事件，它可以是任意类型，EventBus会根据事件类型进行全局的通知。
- **Subscriber**：事件订阅者，在EventBus 3.0之前我们必须定义以onEvent开头的那几个方法，分别是onEvent、onEventMainThread、onEventBackgroundThread和onEventAsync，而在3.0之后事件处理的方法名可以随意取，不过需要加上注解@subscribe，并且指定线程模型，默认是POSTING。
- **Publisher**：事件的发布者，可以在任意线程里发布事件。一般情况下，使用EventBus.getDefault()就可以得到一个EventBus对象，然后再调用post(Object)方法即可。

EventBus是一种典型的事件发布-订阅模式，事件由发布者通过EventBus传递给订阅者，总体框架如下。



EventBus提供了四种线程模型，分别是：

- **POSTING**：默认，表示事件处理函数的线程跟发布事件的线程在同一个线程。
- **MAIN**：表示事件处理函数的线程在主线程(UI)线程，因此在这里不能进行耗时操作。
- **BACKGROUND**：表示事件处理函数的线程在后台线程，因此不能进行UI操作。如果发布事件的线程是主线程(UI线程)，那么事件处理函数将会开启一个后台线程，如果发布事件的线程是在后台

线程，那么事件处理函数就使用该线程。

- ASYNC：表示无论事件发布的线程是哪一个，事件处理函数始终会新建一个子线程运行，同样不能进行UI操作。

6.1.2 基本使用

EventBus使用流程上分为3步。首先，定义一个事件类。

```
public class SayHelloEvent {
    private String message;

    public void sayHello(String message) {
        this.message = message;
    }
}
```

然后，准备一个事件的订阅者，为了防止事件带来的性能消耗问题，还需要在onStop生命周期中注销事件的订阅。

```
@Subscribe(threadMode = ThreadMode.MAIN)
public void onMessageEvent(SayHelloEvent event) {
    String message = event.getMessage();
    ...
}

@Override
public void onStart() {
    super.onStart();
    EventBus.getDefault().register(this);
}

@Override
public void onStop() {
    EventBus.getDefault().unregister(this);
    super.onStop();
}
```

最后，在需要发送事件的地方调用EventBus.getDefault().post方法发送事件。

```
EventBus.getDefault().post(new SayHelloEvent("Hello, EventBus!! "));
```

6.2 源码分析

6.2.1 EventBus.getDefault().register(this)

首先，我们从获取EventBus实例的方法getDefault()开发看EventBus。

```

public static EventBus getDefault() {
    if (defaultInstance == null) {
        synchronized (EventBus.class) {
            if (defaultInstance == null) {
                defaultInstance = new EventBus();
            }
        }
    }
    return defaultInstance;
}

```

可以看到，在getDefault()中使用了双重校验并加锁的单例模式来创建EventBus实例，然后我们看一下构造方法。

```

private static final EventBusBuilder DEFAULT_BUILDER = new EventBusBuilder();

public EventBus() {
    this(DEFAULT_BUILDER);
}

```

在EventBus的默认构造方法中又调用了它的另一个有参构造方法，将一个类型为EventBusBuilder的DEFAULT_BUILDER对象传递进去了。再看一下EventBusBuilder的构造方法。

```

public class EventBusBuilder {
    ...

    EventBusBuilder() {
    }
    ...
}

```

里面基本什么也没做，继续查看EventBus的这个有参构造方法。

```

private final Map<Class<?>, CopyOnWriteArrayList<Subscription>>
subscriptionsByEventType;
private final Map<Object, List<Class<?>>> typesBySubscriber;
private final Map<Class<?>, Object> stickyEvents;

EventBus(EventBusBuilder builder) {
    ...

    // 1
    subscriptionsByEventType = new HashMap<>();

    // 2
    typesBySubscriber = new HashMap<>();

    // 3
    stickyEvents = new ConcurrentHashMap<>();

    // 4
    mainThreadSupport = builder.getMainThreadSupport();
    mainThreadPoster = mainThreadSupport != null ?
    mainThreadSupport.createPoster(this) : null;
}

```

```

backgroundPoster = new BackgroundPoster(this);
asyncPoster = new AsyncPoster(this);

...

// 5
subscriberMethodFinder = new
SubscriberMethodFinder(builder.subscriberInfoIndexes,
    builder.strictMethodVerification, builder.ignoreGeneratedIndex);

// 从builder取中一些列订阅相关信息进行赋值
...

// 6
executorService = builder.executorService;
}

```

在注释1处，创建了一个subscriptionsByEventType对象，可以看到它是一个类型为HashMap的subscriptionsByEventType对象，并且其key为 Event 类型，value为 Subscription链表。这里的Subscription是一个订阅信息对象，它里面保存了两个重要的字段，一个是类型为 Object 的 subscriber，该字段即为注册的对象（在 Android 中时通常是 Activity对象）；另一个是类型为 SubscriberMethod 的 subscriberMethod，它就被@Subscribe注解的那个订阅方法，里面保存了一个重要的字段eventType，它是 Class<?> 类型的，代表了 Event 的类型。在注释2处，新建了一个类型为 Map 的typesBySubscriber对象，它的key为subscriber对象，value为subscriber对象中所有的 Event 类型链表，日常使用中仅用于判断某个对象是否注册过。在注释3处新建了一个类型为 ConcurrentHashMap的stickyEvents对象，它是专用于粘性事件处理的一个字段，key为事件的Class对象，value为当前的事件。

可能有的同学还不知道粘性事件，所谓粘性事件是相对普通事件来说的。普通事件是先注册，然后发送事件才能收到；而粘性事件，在发送事件之后再订阅该事件也能收到。并且，粘性事件会保存在内存中，每次进入都会去内存中查找获取最新的粘性事件，除非你手动解除注册。

然后，我们看注释5这行代码新建了一个subscriberMethodFinder对象，这是从EventBus中抽离出的订阅方法查询的一个对象。在注释6处，从builder中取出了一个默认的线程池对象，它由Executors的newCachedThreadPool()方法创建，它是一个有则用、无则创建、无数量上限的线程池。

接下来，我们再看一下EventBus的regist()方法。

```

public void register(Object subscriber) {
    Class<?> subscriberClass = subscriber.getClass();

    // 1
    List<SubscriberMethod> subscriberMethods =
    subscriberMethodFinder.findSubscriberMethods(subscriberClass);
    synchronized (this) {
        for (SubscriberMethod subscriberMethod : subscriberMethods) {
            // 2
            subscribe(subscriber, subscriberMethod);
        }
    }
}
}

```

上面代码的主要作用是，根据当前注册类获取 subscriberMethods这个订阅方法列表，然后使用了增强for循环令subscriber对象 对 subscriberMethods 中每个 SubscriberMethod 进行订阅。

更深层的代码，我们就不做分析了，regist()主要完成了如下一些事情。

1. 根据单例设计模式创建一个EventBus对象，同时创建一个EventBus.Builder对象对EventBus进行初始化，其中有三个比较重要的集合和一个SubscriberMethodFinder对象。
2. 调用register方法,首先通过反射获取到订阅者的Class对象。
3. 通过SubscriberMethodFinder对象获取订阅者中所有订阅的事件集合,它先从缓存中获取，如果缓存中有，直接返回；如果缓存中没有，通过反射的方式去遍历订阅者内部被注解的方法，将这些方法放入到集合中进行返回。
4. 遍历第三步获取的集合，将订阅者和事件进行绑定。
5. 在绑定之后会判断绑定的事件是否是粘性事件，如果是粘性事件，直接调用postToSubscription方法，将之前发送的粘性事件发送给订阅者。其实这也很好理解，在讲粘性事件时说过，如果在粘性事件发送之前注册的订阅者，当发送粘性事件时，会接收到该事件；如果是粘性事件发送之后注册的订阅者，同样也能接收到事件。

6.2.2 EventBus.getDefault().post()

post()方法的代码如下。

```
public void post(Object event) {

    PostingThreadState postingState = currentPostingThreadState.get();
    List<Object> eventQueue = postingState.eventQueue;
    eventQueue.add(event);

    if (!postingState.isPosting) {
        postingState.isMainThread = isMainThread();
        postingState.isPosting = true;
        if (postingState.canceled) {
            throw new EventBusException("Internal error. Abort state was not reset");
        }
        try {
            while (!eventQueue.isEmpty()) {
                postSingleEvent(eventQueue.remove(0), postingState);
            }
        } finally {
            postingState.isPosting = false;
            postingState.isMainThread = false;
        }
    }
}
```

此方法的作用是，获取当前线程的事件集合，将要发送的事件加入到集合中。然后通过循环，只要事件集合中还有事件，就一直发送。这里的currentPostingThreadState 是一个 ThreadLocal 类型的对象，里面存储了 PostingThreadState，而 PostingThreadState 中包含了一个 eventQueue 和其他一些标志位，相关的源码如下。

```
private final ThreadLocal<PostingThreadState> currentPostingThreadState = new
ThreadLocal<PostingThreadState> () {
    @Override
    protected PostingThreadState initialValue() {
        return new PostingThreadState();
    }
};

final static class PostingThreadState {
    final List<Object> eventQueue = new ArrayList<>();
    boolean isPosting;
```

```

        boolean isMainThread;
        Subscription subscription;
        Object event;
        boolean canceled;
    }

```

然后，我们看一下postSingleEvent() 方法，代码如下。

```

private void postSingleEvent(Object event, PostingThreadState postingState)
throws Error {
    Class<?> eventClass = event.getClass();
    boolean subscriptionFound = false;

    if (eventInheritance) {
        List<Class<?>> eventTypes = lookupAllEventTypes(eventClass);
        int countTypes = eventTypes.size();
        for (int h = 0; h < countTypes; h++) {
            Class<?> clazz = eventTypes.get(h);
            subscriptionFound |=
                postSingleEventForEventType(event, postingState, clazz);
        }
    } else {
        subscriptionFound = postSingleEventForEventType(event, postingState,
eventClass);
    }
    if (!subscriptionFound) {
        ...
    }
}

```

上面代码的作用是，获取事件的Class对象，找到当前的event的所有父类和实现的接口的class集合。遍历这个集合，调用发送单个事件的方法进行发送。

可以看到，上面的代码首先取出 Event 的 class 类型，接着会对 eventInheritance 标志位判断，它默认为true，如果设为 true 的话，它会在发射事件的时候判断是否需要发射父类事件，设为 false，能够提高一些性能。接着，调用lookupAllEventTypes() 方法，它的作用就是取出 Event 及其父类和接口的 class 列表，当然重复取的话会影响性能，所以它也做了一个 eventTypesCache 的缓存，这样就不用重复调用 getSuperclass() 方法。最后，调用postSingleEventForEventType()方法。

```

private boolean postSingleEventForEventType(Object event, PostingThreadState
postingState, Class <?> eventClass) {
    CopyOnWriteArrayList <Subscription> subscriptions;
    synchronized(this) {
        subscriptions = subscriptionsByEventType.get(eventClass);
    }
    if (subscriptions != null && !subscriptions.isEmpty()) {
        for (Subscription subscription: subscriptions) {
            postingState.event = event;
            postingState.subscription = subscription;
            boolean aborted = false;
            try {
                postToSubscription(subscription, event,
postingState.isMainThread);
                aborted = postingState.canceled;
            } finally {
                postingState.event = null;
            }
        }
    }
}

```

```

        postingState.subscription = null;
        postingState.canceled = false;
    }
    if (aborted) {
        break;
    }
}
return true;
}
return false;
}

```

根据事件获取所有订阅它的订阅者集合，遍历集合，将事件发送给订阅者。方法最后又调用了 postToSubscription()方法。

```

private void postToSubscription(Subscription subscription, Object event, boolean
isMainThread) {
    switch (subscription.subscriberMethod.threadMode) {
        case POSTING:
            invokeSubscriber(subscription, event);
            break;
        case MAIN:
            if (isMainThread) {
                invokeSubscriber(subscription, event);
            } else {
                mainThreadPoster.enqueue(subscription, event);
            }
            break;
        case MAIN_ORDERED:
            if (mainThreadPoster != null) {
                mainThreadPoster.enqueue(subscription, event);
            } else {
                invokeSubscriber(subscription, event);
            }
            break;
        case BACKGROUND:
            if (isMainThread) {
                backgroundPoster.enqueue(subscription, event);
            } else {
                invokeSubscriber(subscription, event);
            }
            break;
        case ASYNC:
            asyncPoster.enqueue(subscription, event);
            break;
        default:
            throw new IllegalStateException("Unknow thread mode: " +
subscription.subscriberMethod.threadMode);
    }
}
}

```

可以看出，通过threadMode 来判断在哪个线程中去执行订阅消息。

- **POSTING**：执行 invokeSubscriber() 方法，内部直接采用反射调用。
- **MAIN**：首先去判断当前是否在 UI 线程，如果是的话则直接反射调用，否则调用 mainThreadPoster的enqueue()方法，即把当前的方法加入到队列之中，然后通过 handler 去发

送一个消息，在 handler 的 handleMessage 中去执行方法。

- **MAIN_ORDERED**: 与MAIN类似，不过是确保是顺序执行的。
- **BACKGROUND**: 判断当前是否在 UI 线程，如果不是的话则直接反射调用，是的话通过 backgroundPoster的enqueue()方法 将方法加入到后台的一个队列，最后通过线程池去执行。注意，backgroundPoster在 Executors的execute()方法 上添加了 synchronized关键字 并设立了 控制标记flag，保证任一时间只且仅能有一个任务会被线程池执行。
- **ASYNC**: 逻辑实现类似于BACKGROUND，将任务加入到后台的一个队列，最终由Eventbus 中的一个线程池去调用，这里的线程池与 BACKGROUND 逻辑中的线程池用的是同一个，即使用 Executors的newCachedThreadPool()方法创建的线程池，它是一个有则用、无则创建、无数量上限的线程池。不同于backgroundPoster的保证任一时间只且仅能有一个任务会被线程池执行的特性，这里asyncPoster则是异步运行的，可以同时接收多个任务。

可以看到，EventBus.getDefault().post()主要做了如下一些事情：

1. 获取当前线程的事件集合，将要发送的事件加入到集合中。
2. 通过循环，只要事件集合中还有事件，就一直发送。
3. 获取事件的Class对象，找到当前的event的所有父类和实现的接口的class集合。遍历这个集合，调用发送单个事件的方法进行发送。
4. 根据事件获取所有订阅它的订阅者集合，遍历集合，将事件发送给订阅者。
5. 发送给订阅者时，根据订阅方法的线程模式调用订阅方法，如果需要线程切换，则切换线程进行调用；否则，直接调用。

6.2.3 EventBus.getDefault().unregister(this)

前面已经介绍了订阅者注册和消息的发送，接下来我们再来看一下消息的解绑。

```
public synchronized void unregister(Object subscriber) {
    List<Class<?>> subscribedTypes = typesBySubscriber.get(subscriber);
    if (subscribedTypes != null) {
        for (Class<?> eventType : subscribedTypes) {
            unsubscribeByEventType(subscriber, eventType);
        }
        typesBySubscriber.remove(subscriber);
    }
}
```

可以看到，在unsubscribeByEventType() 方法中对 subscriptionsByEventType 移除了该 subscriber 的所有订阅信息。最后，在移除了注册对象和其对应的所有 Event 事件链表。

6.2.4 EventBus.getDefault().postSticky()

如果想要发射 sticky 事件，需要通过 EventBus的postSticky() 方法，如下所示。

```
public void postSticky(Object event) {
    synchronized (stickyEvents) {
        stickyEvents.put(event.getClass(), event);
    }
    post(event);
}
```

上面代码的主要作用是先将该事件放入StickyEvents 中，然后再使用post()方法发送事件。前面我们在分析register()方法的时候，有一段粘性事件的分析。


```
if (subscriberMethod.sticky) {
    Object stickyEvent = stickyEvents.get(eventType);
    if (stickyEvent != null) {
        postToSubscription(newSubscription, stickyEvent, isMainThread());
    }
}
```

可以看到，在注册事件时，熟悉会判断当前事件是否是 sticky 事件，如果是则从 stickyEvents 中拿出该事件并执行 postToSubscription() 方法。

第六章 算法面试题汇总

6.1 排序

所谓排序算法，即通过特定的算法因式将一组或多组数据按照既定模式进行重新排序。这种新序列遵循着一定的规则，体现出一定的规律，因此，经处理后的数据便于筛选和计算，大大提高了计算效率。

对于排序：

- 我们首先要求其具有一定的稳定性
- 即当两个相同的元素同时出现于某个序列之中
- 则经过一定的排序算法之后
- 两者在排序前后的相对位置不发生变化。

所以，就让我们先来看看，面试中，有哪些超高频的排序算法

6.1.1 冒泡排序 ★★★★★

- 冒泡排序可以说是最基础的了，无非就是两个 for 循环嵌套，然后两两比较交换罢了。这就不多说了。

6.1.1.1 步骤：

- 1、比较相邻的元素。如果第一个比第二个大（小），就交换他们两个。
- 2、对每一对相邻元素作同样的工作，从开始第一对到结尾的最后一对。这步做完后，最后的元素会是最大（小）的数。
- 3、针对所有的元素重复以上的步骤，除了最后已经选出的元素（有序）。
- 4、持续每次对越来越少的元素（无序元素）重复上面的步骤，直到没有任何

6.1.1.2 视频：

- [数据结构排序算法之冒泡排序演示](#)

6.1.1.3 示例代码：

```
public void bubbleSort(int[] arr) {
    int temp = 0;
    boolean swap;
    for (int i = arr.length - 1; i > 0; i--) { // 每次需要排序的长度
        // 增加一个swap的标志，当前一轮没有进行交换时，说明数组已经有序
        swap = false;
        for (int j = 0; j < i; j++) { // 从第一个元素到第i个元素
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
```

```

        arr[j] = arr[j + 1];
        arr[j + 1] = temp;
        swap = true;
    }
}
if (!swap){
    break;
}
}
}

```

6.1.2 归并排序 ★★★★★

- 对于归并排序而言，思想可以概括为：分而治之。也就是将一个数组，首先划分为一堆单个的数，然后再一个接一个的，进行两两有序合并，最后就得到了一个有序数组。

6.1.2.1 步骤：

1. 将待排序的数列分成若干个长度为1的子数列
2. 然后将这些数列两两合并；得到若干个长度为2的有序数列
3. 再将这些数列两两合并；得到若干个长度为4的有序数列
4. 再将它们两两合并；直接合并成一个数列为止
5. 这样就得到了我们想要的排序结果

6.1.2.2 视频：

- [归并排序](#)

6.1.2.3 示例代码：

```

// 入口
public void mergeSort(int[] arr) {
    int[] temp = new int[arr.length];
    internalMergeSort(arr, temp, 0, arr.length - 1);
}

private void internalMergeSort(int[] arr, int[] temp, int left, int right) {
    // 当left == right时，不需要再划分
    if (left < right) {
        int mid = (left + right) / 2;
        // 左右往下拆分
        internalMergeSort(arr, temp, left, mid);
        internalMergeSort(arr, temp, mid + 1, right);
        // 拆分结束后返回结果进行合并
        mergeSortedArray(arr, temp, left, mid, right);
    }
}

// 合并两个有序子序列
public void mergeSortedArray(int[] arr, int[] temp, int left, int mid, int right) {
    int i = left;
    int j = mid + 1;
    int k = 0;

```

```

while (i <= mid && j <= right) {
    temp[k++] = arr[i] < arr[j] ? arr[i++] : arr[j++];
}
// 合并完，将非空的那列拼入
while (i <= mid) {
    temp[k++] = arr[i++];
}
while (j <= right) {
    temp[k++] = arr[j++];
}
// 把temp数据复制回原数组
for (i = 0; i < k; i++) {
    arr[left + i] = temp[i];
}
}

```

6.1.3 快速排序 ☆☆☆☆☆

快速排序的思想，可以简单的概括为：两边包抄、一次一个。每选一个基准点，一次排序后确定它的最最终位置，一步到位。

6.1.3.1 步骤：

- 1、先从数列中取出一个数作为基准数
- 2、分区过程，将比这个数大的数全放到它的右边，小于或等于它的数全放到它的左边
- 3、再对左右区间重复第二步，直到各区间只有一个数

概括来说为 挖坑填数+分治法

注：快排算法不唯一，到目前为止我已经看到三种排法，这里我用最老的，就是很多教材上的排法解析

6.1.3.2 视频：

- [快速排序算法](#)

6.1.3.3 示例代码：

```

public void quickSort(int[] arr){
    quickSort(arr, 0, arr.length-1);
}

private void quickSort(int[] arr, int low, int high){
    if (low >= high)
        return;
    int pivot = partition(arr, low, high);           //将数组分为两部分
    quickSort(arr, low, pivot - 1);                 //递归排序左子数组
    quickSort(arr, pivot + 1, high);                 //递归排序右子数组
}

private int partition(int[] arr, int low, int high){
    int pivot = arr[low];           //基准
    while (low < high){
        // 注意 low 设为 pivot 后，必须先走 high

```

```

        while (low < high && arr[high] >= pivot) {
            high--;
        }
        arr[low] = arr[high];           //交换比基准大的记录到左端
        while (low < high && arr[low] <= pivot) {
            low++;
        }
        arr[high] = arr[low];           //交换比基准小的记录到右端
    }
    //扫描完成，基准到位
    arr[low] = pivot;
    //返回的是基准的位置
    return low;
}

```

6.1.4 计数排序 ☆☆☆☆☆

- 求大小 - 初始化 - 刷新 - 附值

计数排序顾名思义，其思想就在于记录各个数的出现次数，最后按顺序取出即可。

6.1.4.1 步骤：

1. 建一个长度为K+1的的数组C，里面的每一个元素初始都置为0(Java里面默认就是0)。
2. 遍历待排序的数组，计算其中的每一个元素出现的次数，比如一个key为i的元素出现了3次，那么C[i]=3。
3. 累加C数组，获得元素的排位，从0开始遍历C, C[i+1]=C[i]+C[i-1]
4. 建一个临时数组T，长度与待排序数组一样。从数组末尾遍历待排序数组，把元素都安排到T里面，直接从C里面就可以得到元素的具体位置，不过记得每处理过一个元素之后都要把C里面对应位置的计数减1。

6.1.4.2 视频：

- [计数排序算法可视化解读](#)

6.1.4.3 示例代码：

- 我在网上看了巨多代码，但基本都是用来处理 0 以上数的计数排序。下面介绍的这个算法，可以适应小于 0 的数的计数排序，我加了很多注释，也很好理解：

```

public void countSort(int[] arr) {
    // 找到最大值和最小值
    int max = Integer.MIN_VALUE;
    int min = Integer.MAX_VALUE;
    for(int i = 0; i < arr.length; i++){
        max = Math.max(max, arr[i]);
        min = Math.min(min, arr[i]);
    }

    int[] b = new int[arr.length]; // 存储数组
    int[] count = new int[max - min + 1]; // 计数数组

    for (int i = 0; i < arr.length; i++) {
        int num = arr[i];

```

```

        count[num - min]++; // 每出现一个值，计数数组对应元素的值+1
        // 此时count[i]表示数值等于i的元素的个数
    }

    for (int i = 1; i < count.length; i++) {
        count[i] += count[i-1];
    }

    for (int i = 0; i < arr.length; i++) {
        int num = arr[i]; // 原数组第i位的值
        int index = count[num - min] - 1; //加总数组中对应元素的下标
        b[index] = num; // 将该值存入存储数组对应下标中
        count[num - min]--; // 加总数组中，该值的总和减少1。
    }

    // 将存储数组的值替换给原数组
    for(int i=0; i < arr.length;i++){
        arr[i] = b[i];
    }
}

```

6.1.5 桶排序 ★★★★★

桶排序的思想是，首先按特定规则，划分出若干个‘桶’，每个‘桶’有个范围，将大小在对应‘桶’范围内的数，对号入座。再依次将每个‘桶’内的数有序排列，最后按顺序拼接各个‘桶’即可。

6.1.5.1 步骤：

1. 根据待排序集合中最大元素和最小元素的差值范围和映射规则，确定申请的桶个数；
2. 遍历待排序集合，将每一个元素移动到对应的桶中；
3. 对每一个桶中元素进行排序，并移动到已排序集合中。

步骤 3 中提到的已排序集合，和步骤 1、2 中的待排序集合是同一个集合。与计数排序不同，桶排序的步骤 2 完成之后，所有元素都处于桶中，并且对桶中元素排序后，移动元素过程中不再依赖原始集合，所以可以将桶中元素移动回原始集合即可。

6.1.5.2 视频：

- [桶排序](#)

6.1.5.3 示例代码：

上面讲的计数排序其实一定程度上，也可以看作一种特殊的桶排序，同样的，网上桶排序代码大一堆，啥语言都有。但却没有一个解决小于 0 数排序问题的，要么就不处理要么就抛出异常，下面这个算法，有效的解决了，小于 0 数排序的难题

```

public static void bucketSort(int[] arr){
    // 首先还是找出最大、最小值
    int max = Integer.MIN_VALUE;
    int min = Integer.MAX_VALUE;
    for(int i = 0; i < arr.length; i++){
        max = Math.max(max, arr[i]);
        min = Math.min(min, arr[i]);
    }
}

```

```

// 桶数
// 在桶排序中，对桶的划分个数是随意的
// 这个方法划分的桶数量随带划分数列的密集程度改变而改变
int bucketNum = (max - min) / arr.length + 1;
ArrayList<ArrayList<Integer>> bucketArr = new ArrayList<>(bucketNum);
// 初始化各个桶
for(int i = 0; i < bucketNum; i++){
    bucketArr.add(new ArrayList<Integer>());
}

// 将每个元素放入相应的桶
for(int i = 0; i < arr.length; i++){
    int num = (arr[i] - min) / (arr.length);
    bucketArr.get(num).add(arr[i]);
}

// 对每个桶进行排序
for(int i = 0; i < bucketArr.size(); i++){
    Collections.sort(bucketArr.get(i));
    for (int j = 0; j < bucketArr.get(i).size(); j++) {
        arr[j] = bucketArr.get(i).get(j);
    }
}
}
}

```

6.2 二叉树

在计算机科学中，二叉树是每个结点最多有两个子树的树结构。通常子树被称作“左子树”（left subtree）和“右子树”（right subtree）。二叉树常被用于实现二叉查找树和二叉堆。

这里我们就来看看，面试中会怎么样来考察我们有关二叉树的问题，首先我们先定义一个节点类：

后文测试所使用的节点类如下：ps：解释 [LeetCode](#) 上那种

```

class TreeNode {
    public TreeNode left, right;
    public int val;

    public TreeNode(int val) {
        this.val = val;
    }
}

```

6.2.1 顺序遍历 ☆☆☆

二叉树的遍历分为以下三种：

- 先序遍历：遍历顺序规则为【根左右】
- 中序遍历：遍历顺序规则为【左根右】
- 后序遍历：遍历顺序规则为【左右根】

下面以上图为例，我们通过代码实现三种基本遍历：

6.3.1.1 先序遍历：

首先是代码实现：

```
// 先序遍历
public void preTraverse(TreeNode root) {
    if (root != null) {
        System.out.println(root.val);
        preTraverse(root.left);
        preTraverse(root.right);
    }
}
```

遍历结果: ABCDEFGHK

6.3.1.2 中序遍历:

首先是代码实现:

```
// 中序遍历
public void inTraverse(TreeNode root) {
    if (root != null) {
        inTraverse(root.left);
        System.out.println(root.val);
        inTraverse(root.right);
    }
}
```

遍历结果: BDCAEHGKF

6.3.1.3 后序遍历:

首先是代码实现:

```
// 后序遍历
public void postTraverse(TreeNode root) {
    if (root != null) {
        postTraverse(root.left);
        postTraverse(root.right);
        System.out.println(root.val);
    }
}
```

遍历结果: DCBHKGF EA

- 视频

[一节课搞定计算机二级难题: 二叉树遍历结构](#)

####

6.2.2 层次遍历 - [二叉树的层次遍历](#)

二叉树的层次遍历很好理解, 在这里我举个例子。首先我们先给出一棵二叉树:

层次遍历顾名思义, 就是从上到下, 逐层遍历, 每层从左往右输出

计算结果: 5 - 4 - 8 - 11 - 13 - 4 - 7 - 2 - 1

关于遍历算法, 常见的有:

- 深度优先遍历 (DFS)
- 广度优先遍历 (BFS)

在我刷题的过程中遇到过这样一道题:

- Given a binary tree, return the zigzag level order traversal of its nodes' values. (ie, from left to right, then right to left for the next level and alternate between).

即 Z 字型遍历，所以这里再加上一种：

- Z 字形遍历

下面我们来看看代码上的实现：

6.2.2.1 深度优先遍历 (DFS) ☆☆☆☆

我们所学的层次遍历只有 BFS（广搜），DFS 深搜本身是用于顺序排序的非递归实现。‘用DFS’来解决层次遍历这种题我也是第一次见。

它的步骤可以简要概括为：

1. 常规深度搜索，记录下当前节点所在层 level
2. 将当前节点加入 List 中对应的层
3. 由于是从左往右搜索，所以也是从左往右加入
4. 最后得到一个类似下面的结构

0 -- 5 1 -- 4 -> 8 2 -- 11 -> 13 -> 4 3 -- 7 -> 2 -> 1

这种遍历方式太少见，找不到相关的视频，好在原理容易理解

```
// 层次遍历(DFS)
public static List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> res = new ArrayList<>();    if (root == null) {
        return res;    }    dfs(root, res, 0);    return res;}
private void dfs(TreeNode root, List<List<Integer>> res, int level) {
    if (root == null) {
        return;    }    if (level == res.size()) {
        res.add(new ArrayList<>());    }    res.get(level).add(root.val);    dfs(root.left, res, level + 1);
    dfs(root.right, res, level + 1);}
```

6.2.2.2 广度优先遍历 (BFS) ☆☆☆☆

与 DFS 用递归去实现不同，BFS需要用队列去实现。

层次遍历的步骤是：

1. 对于不为空的结点，先把该结点加入到队列中
2. 从队中拿出结点，如果该结点的左右结点不为空，就分别把左右结点加入到队列中
3. 重复以上操作直到队列为空

说白了就是：父节点入队，父节点出队列，先左子节点入队，后右子节点入队。递归遍历全部节点即可

- 视频

[二叉树的遍历算法--层次遍历算法](#)

```
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();

    if (root == null) {
        return result;
    }

    Queue<TreeNode> queue = new LinkedList<TreeNode>();
    queue.offer(root);
```



```

while (!queue.isEmpty()) {
    ArrayList<Integer> level = new ArrayList<Integer>();
    int size = queue.size();
    for (int i = 0; i < size; i++) {
        TreeNode head = queue.poll(); //返回第一个元素，并在队列中删除
        level.add(head.val);
        if (head.left != null) {
            queue.offer(head.left);
        }
        if (head.right != null) {
            queue.offer(head.right);
        }
    }
    result.add(level);
}

return result;
}

```

1. queue.poll(); //返回第一个元素，并在队列中删除
2. queue.element(); //返回第一个元素
3. queue.peek(); //返回第一个元素
4. queue.offer("a"); //添加元素
5. add()和remove() //方法在失败的时候会抛出异常(不推荐)

6.2.2.3 Z 字形遍历 ★★★

这个题型也很罕见，源于 LeetCode 上一道面试题：

Given a binary tree, return the zigzag level order traversal of its nodes' values. (ie, from left to right, then right to left for the next level and alternate between). 给定一棵二叉树，从顶向下，进行Z字形分层遍历，即：如果本层是从左向右的，下层就是从右向左。([103. 二叉树的锯齿形层次遍历](#))

流程与 BFS 类似，就是多了个用于区分左右的 flag

1. 对于不为空的结点，先把该结点加入到队列中
2. 从队中拿出结点，如果该结点的左右结点不为空，就分别把左右结点加入到队列中
3. 将 isFromLeft 值取反
4. 重复以上操作直到队列为空

- 视频

同样的这个题型太特殊，所以没有相关视频解析，不过好在算法过程也很好理解

```

public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();    if (root == null){
    return result;    }    Deque<TreeNode> queue = new ArrayDeque<>();
    queue.offer(root);    boolean isFromLeft = false;    while(!queue.isEmpty()){
        int size = queue.size();    isFromLeft = !isFromLeft;
        List<Integer> list = new ArrayList<>();    for(int i = 0; i < size; i++){
            TreeNode node;    if (isFromLeft){        node =
            queue.pollFirst();    }else{        node = queue.pollLast();
            }    list.add(node.val);    if (isFromLeft){
                if (node.left != null){
                    queue.offerLast(node.left);    }    if (node.right !=
                    null){        queue.offerLast(node.right);    }
                }else{    if (node.right != null){
                    queue.offerFirst(node.right);    }    if (node.left !=
                    null){        queue.offerFirst(node.left);    }
                }    }    result.add(list);    }    return result;}

```

6.2.2.4 复原IP地址

- 给定一个只包含数字的字符串，复原它并返回所有可能的 IP 地址格式。
- 示例:

输入: "25525511135"输出: ["255.255.11.135", "255.255.111.35"]

- 代码:

```

@Test
public void test() {
    dfs("19216801", -1, 1, new ArrayList<String>());
}

private void dfs(String str, int index, int level, ArrayList<String> res) {
    if (level == 5 || index == str.length() - 1) {
        if (level == 5 && index == str.length() - 1) {
            System.out.println(res.toString());
        }
    }
    for (int i = 1; i < 4; i++) {
        if (index + 1 + i > str.length()) {
            continue;
        }
        String x = str.substring(index + 1, index + 1 + i);
        if (Integer.parseInt(x) < 256 && (x.equals("0") || !(x.charAt(0) ==
'0')))) {
            res.add(x);
            dfs(str, index + i, level + 1, res);
            res.remove(( res.size()-1 ));
        }
    }
}

```

6.2.3 左右翻转

这是一道华为面试原题（[226. 翻转二叉树](#)），题目大意是：

输入二叉树如下：

反转后输出：

乍一看很难办，其实想一个解决方案很简单，这里我直接举三个方案：

6.2.3.1 方法一：比如我们用递归的思路，本质思想是：★

1. 本质思想也是左右节点进行交换
2. 交换前递归调用对根结点的左右节点分别进行处理
3. 保证交换前左右节点已经翻转。

三步搞定，我们看下代码实现：

```
public TreeNode invertTree(TreeNode root) {
    if (root == null) {
        return null;
    }
    Stack<TreeNode> stack = new Stack<>();
    stack.push(root);
    while (!stack.isEmpty()) {
        final TreeNode node = stack.pop();
        final TreeNode left = node.left;
        node.left = node.right;
        node.right = left;
        if (node.left != null) {
            stack.push(node.left);
        }
        if (node.right != null) {
            stack.push(node.right);
        }
    }
    return root;
}
```

6.2.3.2 方法二：循环，队列存储（BFS，非递归）★

本质思想是：

1. 左右节点进行交换
2. 循环翻转每个节点的左右子节点
3. 将未翻转的子节点存入队列中
4. 循环直到栈里所有节点都循环交换完为止。

```
public TreeNode invertTree(TreeNode root) {
    if (root == null) {
        return null;
    }
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
        TreeNode node = queue.poll();
        TreeNode left = node.left;
        node.left = node.right;
        node.right = left;
        if (node.left != null) {
            queue.offer(node.left);
        }
        if (node.right != null) {
            queue.offer(node.right);
        }
    }
    return root;
}
```

6.2.3.3 方法三：「压轴出场，三步秒杀」递归★★

本质思想是：

1. 左右节点进行交换
2. 交换前递归调用对根结点的左右节点分别进行处理
3. 保证交换前左右节点已经翻转。

同样三步搞定，我们看下代码：

```
public void invert(TreeNode root) {
    if (root == null) {
        return;
    }
    TreeNode temp = root.left;
    root.left = root.right;
    root.right = temp;
    invert(root.left);
    invert(root.right);
}
```

6.2.4 重建二叉树

- 题目
- 输入某二叉树的前序遍历和中序遍历的结果，请重建该二叉树。假设输入的前序遍历和中序遍历的结果中都不含重复的数字。
- 具体数据如下：

前序：A B D E H I C F G 中序：D B H E I A F C G

- 思路
- 在上面的前序序列中，我们可以很容易地获得A就是根节点。此时，我们可以在后序序列中找到这个A，那么在A的左边就是A的左孩子及其子节点，在A的右边就是A的右孩子及其子节点。
- 假设，我们目前在A的左边。在遍历前序序列到B的时候，我就知道了B就是A的左孩子，而在B的左边(中序序列)的都是B的左孩子及其子节点，在B的右边(同是也在A的左边)的就是B的右孩子及其子节点...以此类推.这就是利用递归来重建二叉树。
- 代码

```
class Solution {
    HashMap<Integer, Integer> dic = new HashMap<>();
    int[] po;
    public TreeNode buildTree(int[] preorder, int[] inorder) {
        po = preorder;
        for(int i = 0; i < inorder.length; i++)
            dic.put(inorder[i], i);
        return recur(0, 0, inorder.length - 1);
    }
    TreeNode recur(int pre_root, int in_left, int in_right) {
        if(in_left > in_right) return null;
        TreeNode root = new TreeNode(po[pre_root]);
        int i = dic.get(po[pre_root]);
        root.left = recur(pre_root + 1, in_left, i - 1);
        root.right = recur(pre_root + i - in_left + 1, i + 1, in_right);
        return root;
    }
}
```

####

6.2.5 最大值 ★

乍一看，是到送分题。我第一眼的想法就是：在方法中定义max用来保存遍历得到的最大值，结果每次递归时，都等于在重新定义max，这种方法不对。所以怎么办？

1. 采用分治思想
2. 从整棵树的底部开始
3. 两两比较，放回最大值

看一眼算法你就懂了

```
public int getMax(TreeNode root) {
    if (root == null) {
        return Integer.MIN_VALUE;
    } else {
        int left = getMax(root.left);
        int right = getMax(root.right);
        return Math.max(Math.max(left, right), root.val);
    }
}
```

####

6.2.6 最大深度 ★

深度问题和最大值一样，容易想复杂，其实非常简单，也可以看作一种分治的思想

1. 二叉树的最大深度是距根节点路径最长的某一树叶节点的深度。
 2. 二叉树的深度等于二叉树的高度，也就等于根节点的高度。根节点的高度为左右子树的高度较大者 +1。
- 视频

[【算法面试题】求二叉树最大的深度](#)

```
public int maxDepth(TreeNode root) {
    if (root == null) {
        return 0;
    }
    int left = maxDepth(root.left);
    int right = maxDepth(root.right);
    return Math.max(left, right) + 1;
}
```

####

6.2.7 最小深度

这道题目太常见了，当我一看到题目时就错了：

题目：最小深度是从根节点到最近叶子节点的最短路径上的节点数量。说明：叶子节点是指没有子节点的节点。

看到了吧，这时就得明确正确的递归结束条件

- 举个例子：很多人写出的代码都不符合 1,2 这个测试用例，是因为没搞清楚题意

题目中说明：叶子节点是指没有子节点的节点，这句话的意思是 1 不是叶子节点

题目问的是到叶子节点的最短距离，所以所有返回结果为 1 当然不是这个结果

另外这道题的关键是搞清楚递归结束条件

- 叶子节点的定义是左孩子和右孩子都为 null 时叫做叶子节点
- 当 root 节点左右孩子都为空时，返回 1
- 当 root 节点左右孩子有一个为空时，返回不为空的孩子节点的深度
- 当 root 节点左右孩子都不为空时，返回左右孩子较小深度的节点值
- 视频

二叉树的最小深度

```
public int minDepth(TreeNode root) {    if (root == null) {        return 0;    }    int left = minDepth(root.left);    int right = minDepth(root.right);    if (left == 0) {        return right + 1;    } else if (right == 0) {        return left + 1;    } else {        return Math.min(left, right) + 1;    }}
```

####

6.2.8 平衡二叉树

概念：平衡二叉树每一个节点的左右两个子树的高度差不超过 1

1. 设一个 flag
2. 如果发现不平衡则就返回非 flag

- 视频

平衡二叉树

```
public boolean isBalanced(TreeNode root) {    return maxDepth(root) != -1;}private int maxDepth(TreeNode root) {    if (root == null) {        return 0;    }    int left = maxDepth(root.left);    int right = maxDepth(root.right);    if (left == -1 || right == -1 || Math.abs(left - right) > 1) {        return -1;    }    return Math.max(left, right) + 1;}
```

6.3 链表

链表是最基本的数据结构，面试官也常常用链表来考察面试者的基本能力，而且链表相关的操作相对而言比较简单，也适合考察写代码的能力。链表的操作也离不开指针，指针又很容易导致出错。

综合多方面的原因，链表题目在面试中占据着很重要的地位。

```
public class ListNode {    int val;    ListNode next;    ListNode(int x) {        val = x;        next = null;    }}
```

6.3.1 删除节点

6.3.1.1 思路:

- 将 下一个节点 复制到 当前

```
public void deleteNode(ListNode node) {  
    if (node.next == null){  
        node = null;  
        return;  
    }  
    // 取缔下一节点  
    node.val = node.next.val  
    node.next = node.next.next  
}
```

6.3.2 翻转链表

6.3.2.1 思路

思路: 每次都原第一个结点之后的那个结点放在新的表头后面。比如 1,2,3,4,5

- 第一次: 把第一个 结点1 后边的 结点2 放到 新表头 后面, 变成 2,1,3,4,5
- 第二次: 把第一个 结点1 后边的 结点3 放到 新表头 后面, 变成 3,2,1,4,5
-
- 直到: 第一个 结点1, 后边没有结点为止。

6.3.2.2 视频

[大圣算法 翻转链表\(Reverse Linked List\) -- LeetCode 206](#)

```
public ListNode reverse(ListNode head) {    //prev表示前继节点    ListNode prev =  
null;    while (head != null) {        //temp记录下一个节点, head是当前节点  
ListNode temp = head.next;        head.next = prev;        prev = head;  
head = temp;    }    return prev;}
```

6.3.3 中间元素

6.3.3.1 思路

我总结了一下, 可以称为 田忌赛马' 法

```
public ListNode findMiddle(ListNode head){    if(head == null){        return  
null;    }    ListNode slow = head;    ListNode fast = head;    //  
fast.next = null 表示 fast 是链表的尾节点    while(fast != null && fast.next !=  
null){        fast = fast.next.next;        slow = slow.next;    }    return  
slow;}
```

6.3.4 合并两个已排序链表

6.3.4.1 思路

- 递归方法: 首先比较给新链表 接上 一个结点, 然后这个结点的 next 就是剩下的两条链表合并的结果。

```
public ListNode mergeTwoLists(ListNode l1, ListNode l2) {    ListNode dummy =
new ListNode(0);    ListNode lastNode = dummy;    while (l1 != null && l2 !=
null) {        if (l1.val < l2.val) {            lastNode.next = l1;
l1 = l1.next;        } else {            lastNode.next = l2;            l2 =
l2.next;        }        lastNode = lastNode.next;    }    if (l1 != null) {
lastNode.next = l1;    } else {        lastNode.next = l2;    }
return dummy.next;}
```

6.3.5 链表排序

6.3.5.1 归并排序

- 归并排序 的也是基于 分治 的思想，但是与快排不同的是归并是 先划分，然后从底层开始 向上合并。
- 归并排序的主要思想是将两个 已经排好序的分段 合并成一个 有序的分段。除了找到中间节点的操作必须遍历链表外，其它操作与数组的归并排序基本相同。

视频

[合并两个排序链表](#)

```
public ListNode sortList(ListNode head) {    if (head == null || head.next ==
null) {        return head;    }    // 取得中间节点，将链表一分为二    ListNode mid =
findMiddle(head);    ListNode right = sortList(mid.next);    mid.next = null;
ListNode left = sortList(head);    return mergeTwoLists(left, right);} // 查找中间
元素算法public ListNode findMiddle(ListNode head){    if(head == null){
return null;    }    ListNode slow = head;    ListNode fast = head;
// fast.next = null 表示 fast 是链表的尾节点    while(fast != null && fast.next !=
null){        fast = fast.next.next;        slow = slow.next;    }    return
slow;} // 合并两个有序链表public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0);    ListNode lastNode = dummy;    while
(l1 != null && l2 != null) {        if (l1.val < l2.val) {
lastNode.next = l1;        l1 = l1.next;        } else {
lastNode.next = l2;        l2 = l2.next;        }        lastNode =
lastNode.next;    }    if (l1 != null) {        lastNode.next = l1;    }
else {        lastNode.next = l2;    }    return dummy.next;}
```

6.3.5.2 快速排序

快速排序的主要思想是：

1. 选定一个 基准元素
2. 经过一趟排序，将所有元素分成 两部分
3. 分别对两部分重复上述操作，直到所有元素都已排序成功

因为单链表只能从链表头节点向后遍历，没有 prev 指针，因此必须选择头节点作为基准元素。这样第二步操作的时间复杂度就为 $O(n)$ 。由于之后都是分别对两部分完成上述操作，因此会将链表划分为 $\lg n$ 个段，因此时间复杂度为 $O(n \lg n)$


```

public ListNode sortList(ListNode head) {    quickSort(head, null);    return
head;}private void quickSort(ListNode start, ListNode end) {    if (start ==
end) {        return;    }    ListNode pt = partition(start, end);
quickSort(start, pt);    quickSort(pt.next, end);} // 快排 轮状法private ListNode
partition(ListNode start, ListNode end) {    int pivotKey = start.val;
ListNode p1 = start, p2 = start.next;    while (p2 != end) {        if (p2.val <
pivotKey) {            p1 = p1.next;            swapValue(p1, p2);        }
p2 = p2.next;    }    swapValue(start, p1);    return p1;}private void
swapValue(ListNode node1, ListNode node2) {    int tmp = node1.val;    node1.val
= node2.val;    node2.val = tmp;}

```

6.3.6 两个链表是否相交

6.3.6.1 思路

1. 如果两个单链表有 共同的节点
2. 那么从第一个节点开始，后面的节点都会 重叠，直至链表结束
3. 因为两个链表中有一个 共同 节点
4. 则从这个节点里的 指针域 指向下一个节点的地址就相同
5. 所以 相交以后 的节点就会 相同，直至链表结束，总的模型就像一个“Y”

6.15.6.2 视频

[【一起玩算法】交叉链表练习题讲解](#)

```

public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
    if (headA == null || headB == null) {
        return null;
    }

    ListNode curra = headA;
    ListNode currb = headB;
    int lengthA = 0;
    int lengthB = 0;

    // 让长的先走到剩余长度和短的一样
    while (curra != null) {
        curra = curra.next;
        lengthA++;
    }
    while (currb != null) {
        currb = currb.next;
        lengthB++;
    }

    curra = headA;
    currb = headB;
    while (lengthA > lengthB) {
        curra = curra.next;
        lengthA--;
    }
    while (lengthB > lengthA) {
        currb = currb.next;
        lengthB--;
    }
}

```

```

// 然后同时走到第一个相同的地方
while (currA != currB) {
    currA = currA.next;
    currB = currB.next;
}

// 返回交叉开始的节点
return currA;
}

```

6.4 栈 / 队列

- 栈 (stack) 又名堆栈:

它是一种运算受限的线性表。限定仅在表尾进行插入和删除操作的线性表。这一端被称为栈顶，相对地，把另一端称为栈底。向一个栈插入新元素又称作进栈、入栈或压栈，它是把新元素放到栈顶元素的上面，使之成为新的栈顶元素；从一个栈删除元素又称作出栈或退栈，它是把栈顶元素删除掉，使其相邻的元素成为新的栈顶元素。

- 队列是一种特殊的线性表

特殊之处在于它只允许在表的前端 (front) 进行删除操作，而在表的后端 (rear) 进行插入操作，和栈一样，队列是一种操作受限制的线性表。进行插入操作的端称为队尾，进行删除操作的端称为队头。

6.4.1 带最小值操作的栈

这道面试题主要考察我们对于辅助栈的使用。

常见的辅助栈包括两种:

1. 辅助栈和数据栈同步
2. 辅助栈和数据栈不同步

我们这里采用辅助栈和数据栈同步的方式:

特点: 编码简单, 不用考虑一些边界情况, 就有一点不好: 辅助栈可能会存一些“不必要”的元素。

1. 辅助栈为空的时候, 必须放入新进来的数;
2. 新来的数小于或者等于辅助栈栈顶元素的时候, 才放入, 特别注意这里“等于”要考虑进去, 因为出栈的时候, 连续的、相等的并且是最小值的元素要同步出栈;
3. 出栈的时候, 辅助栈的栈顶元素等于数据栈的栈顶元素, 才出栈。

总结一下: 出栈时, 最小值出栈才同步; 入栈时, 最小值入栈才同步。

```

public class MinStack {
    private Stack<Integer> stack;
    private Stack<Integer> minStack; // 维护一个辅助栈, 传入当前栈的最小值

    public MinStack() {
        stack = new Stack<Integer>();
        minStack = new Stack<Integer>();
    }

    public void push(int number) {
        stack.push(number);
        if (minStack.isEmpty()) {
            minStack.push(number);
        }
    }
}

```

```

        } else {
            minStack.push(Math.min(number, minStack.peek()));
        }
    }

    public int pop() {
        minStack.pop();
        return stack.pop();
    }

    public int min() {
        return minStack.peek();
    }
}

```

6.4.2 有效括号

6.4.2.1 思路：

1. 初始化栈 S。
2. 一次处理表达式的每个括号。
3. 如果遇到开括号，我们只需将其推到栈上即可。这意味着我们将稍后处理它，让我们简单地转到前面的子表达式。
4. 如果我们遇到一个闭括号，那么我们检查栈顶的元素。如果栈顶的元素是一个相同类型的左括号，那么我们将它从栈中弹出并继续处理。否则，这意味着表达式无效。
5. 如果到最后我们剩下的栈中仍然有元素，那么这意味着表达式无效。

```

public boolean isValidParentheses(String s) {
    Stack<Character> stack = new Stack<Character>();
    for (Character c : s.toCharArray()) {
        if ("({[".contains(String.valueOf(c))) {
            stack.push(c);
        } else {
            if (!stack.isEmpty() && isValid(stack.peek(), c)) {
                stack.pop();
            } else {
                return false;
            }
        }
    }
    return stack.isEmpty();
}

private boolean isValid(char c1, char c2) {
    return (c1 == '(' && c2 == ')') || (c1 == '{' && c2 == '}')
        || (c1 == '[' && c2 == ']');
}

```

6.4.3 用栈实现队列

6.4.3.1 思路：

1. 思路是有两个栈，一个用来放数据（数据栈），一个用来辅助（辅助栈）。
2. 数据添加时，会依次压入栈，取数据时肯定会取栈顶元素，但我们想模拟队列的先进先出，所以就取得栈底元素，那么辅助栈就派上用场了

3. 把数据栈的元素依次弹出到辅助栈，但保留最后一个元素，最后数据栈就剩下了最后一个元素，直接把元素返回，这时数据栈已经没有了数据。
4. 最后呢，把辅助栈的元素依次压入数据栈，这样，我们成功取到了栈底元素。

```
public class MyQueue {    private Stack<Integer> outStack;    private Stack<Integer> inStack;    public MyQueue() {        outStack = new Stack<Integer>();        inStack = new Stack<Integer>();    }    private void in2outStack(){        while(!inStack.isEmpty()){            outStack.push(inStack.pop());        }    }    public void push(int element) {        inStack.push(element);    }    public int pop() {        if(outStack.isEmpty()){            this.in2outStack();        }        return outStack.pop();    }    public int top() {        if(outStack.isEmpty()){            this.in2outStack();        }        return outStack.peek();    } }
```

6.4.4 逆波兰表达式求值 (后缀)

6.4.4.1 思路:

1. 逆波兰表达式求解,定义一个栈辅助计算
2. 当遇到运算符"+", "-", "*", "/"时,从栈中pop出两个数字计算,否则将数字入栈;

```
public int evalRPN(String[] tokens) {    Stack<Integer> s = new Stack<Integer>();    String operators = "+-*/";    for (String token : tokens) {        if (!operators.contains(token)) {            s.push(Integer.valueOf(token));            continue;        }        // 这里有个坑        int a = s.pop();        int b = s.pop();        // 先出的在运算符后        // 后出的在运算符前        if (token.equals("+")) {            s.push(b + a);        } else if (token.equals("-")) {            s.push(b - a);        } else if (token.equals("*")) {            s.push(b * a);        } else {            s.push(b / a);        }    }    return s.pop(); }
```

6.5 二分搜索

6.5.1 二分搜索

给定一个 n 个元素有序的（升序）整型数组 nums 和一个目标值 target，写一个函数搜索 nums 中的 target，如果目标值存在返回下标，否则返回 -1。

示例 1:

输入: nums = [-1,0,3,5,9,12], target = 9 输出: 4 解释: 9 出现在 nums 中并且下标为 4

6.5.1.1 技巧

分析二分查找的一个技巧是：

- 不要出现 else，而是把所有情况用 if / else if 写清楚
- 这样可以清楚地展现所有细节。

这里我们以 递归 和 非递归 方式，解决面试中的二分搜索题

6.5.1.2 递归

思路很简单：

- 判断起始点是否大于终止点
- 比较 nums[mid] 与目标值大小
- 如果 nums[mid] 大，说明目标值 target 在前面

- 反之如果 `nums[mid]` 小, 说明目标值 `target` 在前面后面
- 如果既不大也不小, 说明相等, 则返回 `当前位置`

```
class Solution {
    public int search(int[] nums, int target) {
        return
        binarySearch(nums, 0, nums.length - 1, target);
    }
    private int
    binarySearch(int[] nums, int start, int end, int target) {
        if(start >
        end) {
            return -1;
        }
        int mid = (end + start) / 2;
        if(nums[mid] < target) {
            return binarySearch(nums, mid + 1, end,
            target);
        }
        if(nums[mid] > target) {
            return
            binarySearch(nums, start, mid - 1, target);
        }
        return mid;
    }
}
```

6.5.1.3 非递归

这个场景是最简单的:

- 搜索一个数
- 如果存在, 返回其索引
- 否则返回 -1

```
int binarySearch(int[] nums, int target) {
    int left = 0;        // 注意减 1
    int right = nums.length - 1;
    while(left <= right) {
        int mid = (right
        + left) / 2;
        if(nums[mid] == target)
            return mid;
        else
        if (nums[mid] < target)
            left = mid + 1; // 注意
        else if
        (nums[mid] > target)
            right = mid - 1; // 注意
    }
    return -1;
}
```

6.5.1.4 相关视频

[分钟教你二分查找\(python版\)](#)

6.5.2 X的平方根

计算并返回 `x` 的平方根, 其中 `x` 是非负整数。

由于返回类型是整数, 结果只保留整数的部分, 小数部分将被舍去。

示例 2:

输入: 8 输出: 2 说明: 8 的平方根是 2.82842..., 由于返回类型是整数, 小数部分将被舍去。

6.5.2.1 解题思路

使用二分法搜索平方根的思想很简单:

- 就类似于小时候我们看的电视节目中的“猜价格”游戏
- 高了就往低了猜
- 低了就往高了猜
- 范围越来越小。

注: 一个数的平方根最多不会超过它的一半, 例如 8 的平方根, 8 的一半是 4, 如果这个数越大越是如此

6.5.2.2 注意:

对于判断条件:

- 比如说: 我们很容易想当然觉得

- `mid == x / mid` 和 `mid * mid == x` 是等价的, 实际却不然
- 比如 `mid = 2, x = 5`
- 对于 `mid == x / mid` 就是: `2 == 2` 返回 `true`
- 而对于 `mid * mid == x` 就是: `4 == 5` 返回 `false`

对于边界条件有个坑:

- 要注意此处耍了一下小技巧, 在二分左值和右值相差为1的时候就停止查找; 因为在这里, 有个对中值取整数的操作, 在取整后始终有 `start == mid == end` 则会死循环。

取整操作的误差为1, 故而在这里限制条件改成包含1在内的范围 `start + 1 < end`; 这里减一很精髓

```
public int sqrt(int x) {
    if (x < 0) {
        throw new
        IllegalArgumentException();
    } else if (x <= 1) {
        return x;
    }
    int start = 1, end = x;
    // 直接对答案可能存在的区间进行二分 => 二分答案
    while (start + 1 < end) {
        int mid = start + (end - start) / 2;
        if (mid == x / mid) {
            return mid;
        } else if (mid < x / mid) {
            start = mid;
        } else {
            end = mid;
        }
    }
    if (end > x / end) {
        return start;
    }
    return end;
}
```

6.6 哈希表

- **概念** 散列表 (Hash table, 也叫哈希表), 是根据关键码值(Key value)而直接进行访问的数据结构。也就是说, 它通过把关键码值映射到表中一个位置来访问记录, 以加快查找的速度。这个映射函数叫做散列函数, 存放记录的数组叫做散列表。
- **数据结构** 给定表 `M`, 存在函数 `f(key)`, 对任意给定的关键字值 `key`, 代入函数后若能得到包含该关键字的记录在表中的地址, 则称表 `M` 为 哈希(Hash) 表, 函数 `f(key)` 为哈希(Hash) 函数。

6.6.1 两数之和

给一个整数数组, 找到两个数使得他们的和等于一个给定的数 `target`。需要实现的函数 `twoSum` 需要返回这两个数的下标。

示例:

给定 `nums = [2, 7, 11, 15]`, `target = 9`

因为 `nums[0] + nums[1] = 2 + 7 = 9` 所以返回 `[0, 1]`

6.6.1.1 解题思路

- 用一个 `hashmap` 来记录
- `key` 记录 `target - numbers[i]` 的值, `value` 记录 `numbers[i]` 的 `i` 的值
- 如果碰到一个 `numbers[j]` 在 `hashmap` 中存在
- 那么说明前面的某个 `numbers[i]` 和 `numbers[j]` 的和为 `target`
- 那么当前的 `i` 和 `j` 即为答案

```
public int[] twoSum(int[] numbers, int target) {

    HashMap<Integer,Integer> map = new HashMap<>();

    for (int i = 0; i < numbers.length; i++) {
        // 判断 map 中是否有需要该值的项
        if (map.containsKey(target - numbers[i])) {
            return new int[]{map.get(target - numbers[i]), i};
        }
        // 意思可理解为第 i 项, 需要 target - numbers[i]
        map.put(target - numbers[i], i);
    }
}
```

```

        map.put(target - numbers[i], i);
    }

    return new int[]{};
}

```

6.6.2 连续数组

给一个 **二进制** 数组，找到 0 和 1 **数量相等** 的子数组的 **最大长度**

示例 2:

输入: [0,1,0] 输出: 2 说明: [0, 1] (或 [1, 0]) 是具有相同数量0和1的最长连续子数组。

6.6.2.1 步骤

1. 使用一个数字 **sum** 维护到 **i** 为止 1 的数量与 0 的数量的差值
2. 在 **loop i** 的同时维护 **sum** 并将其插入 **hashmap** 中
3. 对于某一个**sum**值，若**hashmap**中已有这个值
4. 则当前的 **i** 与 **sum** 上一次出现的位置之间的序列 0 的数量与 1 的数量相同

```

public int findMaxLength(int[] nums) {    Map<Integer, Integer> prefix = new
HashMap<>();    int sum = 0;    int max = 0;    // 因为在开始时 0 、 1 的数量都为 0
, 所以必须先存 0    // 否则第一次为 0 的时候, <- i - prefix.get(sum) -> 找不到
prefix.get(0)    prefix.put(0, -1);    // 当第一个 0 1 数量相等的情况出现时, 数组下标
减去-1得到正确的长度    for (int i = 0; i < nums.length; i++) {        int num =
nums[i];        if (num == 0) {            sum--;        } else {
sum++;        }        // 判断是否已存在 sum 值        // 存在则说明之前存过        if
(prefix.containsKey(sum)) {            // 只做判断, 不做存储            max =
Math.max(max, i - prefix.get(sum));        } else {            prefix.put(sum,
i);        }    }    return max;}

```

6.6.3 最长无重复字符的子串

给定一个字符串，请你找出其中不含有重复字符的 **最长子串** 的长度。

输入: "abcabcbb" 输出: 3 解释: 因为无重复字符的最长子串是 "abc", 所以其长度为 3。

6.6.3.1 解题思路

用 **HashMap** 记录每一个字母出现的位置:

1. 设定一个 **左边界**，到当前枚举到的位置之间的字符串为不含重复字符的子串。
2. 若新碰到的字符的上一次的位置在左边界右边, 则需要向右移动左边界。

6.6.3.2 视频

[太圣算法- 最长无重复字符的子串](#)

```

public int lengthOfLongestSubstring(String s) {
    if (s == null || s.length() == 0) {

```

```

        return 0;
    }
    HashMap<Character, Integer> map = new HashMap<>();
    int max = Integer.MIN_VALUE;
    // 计算无重复字符串开始的位置
    int start = -1;
    int current = 0;
    for (int i = 0; i < s.length(); i++) {
        if (map.containsKey(s.charAt(i))) {
            int tmp = map.get(s.charAt(i));
            // 上一次的位置在左边界右边，则需要向右移动左边界
            if (tmp >= start) {
                start = tmp;
            }
        }

        map.put(s.charAt(i), i);
        max = Math.max(max, i - start);
    }
    return max;
}

```

6.6.4 最多点在一条直线上

给出二维平面上的n个点，求最多有多少点在同一条直线上

首先点的定义如下

```

class Point {
    int x;
    int y;
    Point() {
        x = 0; y = 0;
    }
    Point(int a, int b) {
        x = a; y = b;
    }
}

```

示例：

输入: [[1,1],[3,2],[5,3],[4,1],[2,3],[1,4]] 输出: 4 解释: ^ | | o | o o | o | o o +-----> 0 1 2
3 4 5 6

6.6.4.1 解题思路

提示：我们会发现，其实只需要考虑当前点之后出现的点 $i + 1 \dots N - 1$ 即可，因为通过点 $i - 2$ 的直线已经在搜索点 $i - 2$ 的过程中考虑过了。

- 画一条通过点 i 和之后出现的点的直线，在哈希表中存储这条边并计数为 2 = 当前这条直线上有两个点。
- 存储时，以斜率来区分线与线之间的关系
- 假设现在 $i < i + k < i + 1$ 这三个点在同一条直线上，当画出一条通过 i 和 $i + 1$ 的直线会发现已经记录过了，因此对更新这条边对应的计数：count++。

通过 `HashMap` 记录下两个点之间的斜率相同出现的次数，注意考虑点重合的情况

```
public int maxPoints(int[][] points) {
    if (points == null) {
        return 0;
    }
    int max = 0;
    for (int i = 0; i < points.length; i++) {
        Map<String, Integer> map = new HashMap<>();
        int maxPoints = 0;
        int overlap = 0;
        for (int j = i + 1; j < points.length; j++) {
            int dy = points[i][1] - points[j][1];
            int dx = points[i][0] - points[j][0];
            // 两个点重合的情况记录下来
            if (dy == 0 && dx == 0) {
                overlap++;
                continue;
            }
            // 防止 x 相同 y 不同, 但 rate 都为 0
            // 防止 y 相同 x 不同, 但 rate 都为 0
            // 以及超大数约等于 0 的情况: [[0,0],[94911151,94911150],[94911152,94911151]]
            String rate = "";
            if(dy == 0)
                rate = "yy";
            else if (dx == 0)
                rate = "xx";
            else
                rate = ((dy * 1.0) / dx) + "";
            map.put(rate, map.getOrDefault(rate, 0) + 1);
            maxPoints = Math.max(maxPoints, map.get(rate));
        }
        max = Math.max(max, overlap + maxPoints + 1);
    }
    return max;
}
```

6.7 堆 / 优先队列

- **堆** (英语: heap)是计算机科学中一类特殊的数据结构的统称。将根节点最大的堆叫做最大堆或大根堆，根节点最小的堆叫做最小堆或小根堆。

堆通常是一个可以被看做一棵树的数组对象。堆总是满足下列性质：

1. 堆中某个节点的值总是不大于或不小于其父节点的值；
2. 堆总是一棵完全二叉树。

如下图这是一个最大堆，，因为每一个父节点的值都比其子节点要大。10 比 7 和 2 都大。7 比 5 和 1 都大。

- **优先队列**(priority queue) 优先队列是一种抽象数据类型，它是一种排序的机制，它有两个核心操作：找出键值最大(优先级最高)的元素、插入新的元素，效果就是他在维护一个动态的队列。可以收集一些元素，并快速取出键值最大的元素，对其操作后移出队列，然后再收集更多的元素，再处理当前键值最大的元素，如此这般。
- 例如，我们有一台能够运行多个程序的计算机。计算机通过给每个应用一个优先级属性，将应用根据优先级进行排列，计算机总是处理下一个优先级最高的元素。

6.7.1 前K大的数

PriorityQueue 优先队列：Java 的优先队列，保证了每次取最小元素

```
// 维护一个 PriorityQueue, 以返回前K大的数
public int[] topk(int[] nums, int k) {
    int[] result = new int[k];
    if (nums == null || nums.length < k) {
        return result;
    }
    Queue<Integer> pq = new PriorityQueue<>();
    for (int num : nums) {
        pq.add(num);
        if (pq.size() > k) {
            // poll() 方法用于检索或获取和删除队列的第一个元素或队列头部的元素
            pq.poll();
        }
    }
    for (int i = k - 1; i >= 0; i--) {
        result[i] = pq.poll();
    }
    return result;
}
```

6.7.2 前K大的数II

实现一个数据结构，提供下面两个接口：

1. add(number) 添加一个元素
2. topk() 返回前K大的数

```
public class Solution {    private int maxSize;    private Queue<Integer> minheap;    public Solution(int k) {        minheap = new PriorityQueue<>();        maxSize = k;    }    public void add(int num) {        if (minheap.size() < maxSize) {            // add(E e)和offer(E e)的语义相同，都是向优先队列中插入元素            // 只是Queue接口规定二者对插入失败时的处理不同            // 前者在插入失败时抛出异常，后则则会返回false            minheap.offer(num);            return;        }        if (num > minheap.peek()) {            minheap.poll();            minheap.offer(num);        }    }    public List<Integer> topk() {        // 将队列中的数存到数组中        Iterator it = minheap.iterator();        List<Integer> result = new ArrayList<Integer>();        while (it.hasNext()) {            result.add((Integer) it.next());        }        // 调用数组排序法后返回        Collections.sort(result, Collections.reverseOrder());        return result;    }
}
```

6.7.3 数组中的第K个最大元素

在未排序的数组中找到 第 k 个最大的元素。请注意，你需要找的是数组 排序后 的第 k 个最大的元素，而不是第 k 个不同的元素。

示例 2:

输入: [3,2,3,1,2,4,5,5,6] 和 $k = 4$ 输出: 4

我的第一个想法：暴力法

```
public int findKthLargest(int[] nums, int k) {    Queue<Integer> que = new PriorityQueue<>();    for(int num : nums) {        if(que.size() < k) {            que.offer(num);        } else {            if(que.peek() < num) {                que.poll();                que.offer(num);            }        }    }    return que.peek();
}
```

6.7.3.1 这里举个无关的算法：

使用 快速排序，思路极其简单：

1. 首先对数组进行快速排序
2. 最后返回 第 k 个数即可

具体实现：

```
public int kthLargestElement(int k, int[] nums) {    if (nums == null || nums.length == 0 || k < 1 || k > nums.length){        return -1;    }    return partition(nums, 0, nums.length - 1, nums.length - k);
}

private int partition(int[] nums, int start, int end, int k) {    if (start >= end) {        return nums[k];    }
}
```

```

int left = start, right = end;
int pivot = nums[(start + end) / 2];

while (left <= right) {
    while (left <= right && nums[left] < pivot) {
        left++;
    }
    while (left <= right && nums[right] > pivot) {
        right--;
    }
    if (left <= right) {
        swap(nums, left, right);
        left++;
        right--;
    }
}

if (k <= right) {
    return partition(nums, start, right, k);
}
if (k >= left) {
    return partition(nums, left, end, k);
}
return nums[k];
}

private void swap(int[] nums, int i, int j) {
    int tmp = nums[i];
    nums[i] = nums[j];
    nums[j] = tmp;
}

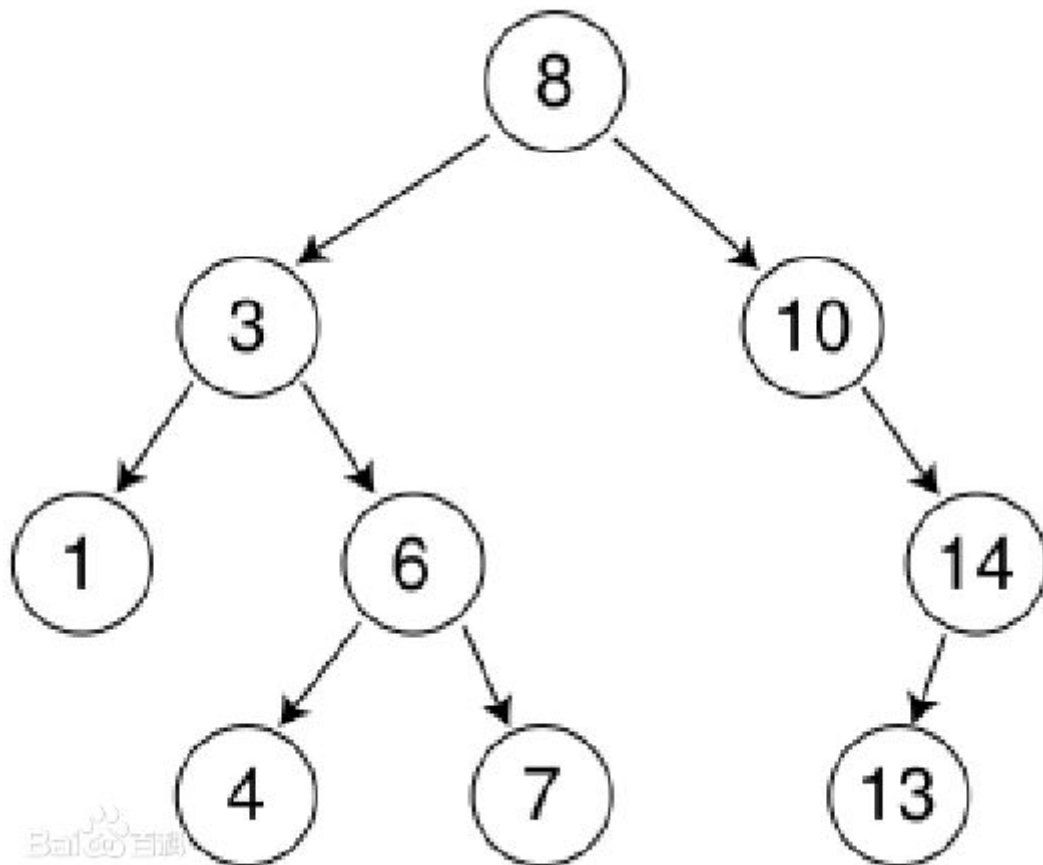
```

6.8 二叉搜索树

6.8.1 二叉搜索树

二叉搜索树（Binary Search Tree），它或者是一棵空树，或者是具有下列性质的二叉树：

1. 若它的左子树不空，则左子树上所有结点的值均小于它的根结点的值；
2. 若它的右子树不空，则右子树上所有结点的值均大于它的根结点的值；
3. 它的左、右子树也分别为二叉搜索树。



6.8.2 验证二叉搜索树

给定一个二叉树，判断其是否是一个有效的二叉搜索树。

假设一个二叉搜索树具有如下特征：

1. 节点的左子树只包含小于当前节点的数。
2. 节点的右子树只包含大于当前节点的数。
3. 所有左子树和右子树自身必须也是二叉搜索树。

示例：

输入： 5 / \ 1 4 / \ 3 6 输出： false 解释： 输入为：
[5,1,4,null,null,3,6]。 根节点的值为 5，但是其右子节点值为 4。

6.8.2.1 解题思路

乍一看，这是一道很简单的题。只需要遍历整棵树，检查 `node.right.val > node.val` 和 `node.left.val < node.val` 对每个结点是否成立。

问题是，这种方法并不总是正确。不仅右子结点要大于该节点，整个右子树的元素都应该大于该节点。例如：这意味着我们需要在遍历树的同时保留结点的上界与下界，在比较时不仅比较子结点的值，也要与上下界比较。

上述思路可以用递归法实现：

- 首先将结点的值与上界和下界（如果有）比较。然后，对左子树和右子树递归进行该过程。

6.8.2.2 视频

```
public boolean isValidBST(TreeNode root) {    return isValidBST(root, Long.MIN_VALUE, Long.MAX_VALUE);}private boolean isValidBST(TreeNode root, long min, long max){    if (root == null) {        return true;    }    if (root.val <= min || root.val >= max){        return false;    }    return isValidBST(root.left, min, root.val) && isValidBST(root.right, root.val, max);}
```

6.8.3 二叉搜索树中第K小的元素

给定一个二叉搜索树，编写一个函数 `kthSmallest` 来查找其中第 `k` 个最小的元素。

说明： 你可以假设 `k` 总是有效的， $1 \leq k \leq$ 二叉搜索树元素个数。

示例：

输入：root = [5,3,6,2,4,null,null,1], k = 3 5 / \ 3 6 / \ 2
4 / 1 输出：3

6.8.3.1 解题思路

1. 增加 `getCount` 方法来获取传入节点的子节点数（包括自己）
2. 从 `root` 节点开始判断 `k` 值和子节点数的大小决定递归路径是往左还是往右。

```
public int kthSmallest(TreeNode root, int k) {    if (root == null) {        return 0;    }    int leftCount = getCount(root.left);    if (leftCount >= k) {        return kthSmallest(root.left, k);    } else if (leftCount + 1 == k) {        return root.val;    } else {        //注(1)        return kthSmallest(root.right, k - leftCount - 1);    }}private int getCount(TreeNode root) {    if (root == null) {        return 0;    }    return getCount(root.left) + getCount(root.right) + 1;}
```

注：（1）为什么是 `k - leftCount - 1` 而不是 `k`，我们可以把当前的二叉树看成左右两部分。在执行到这个条件的时候，很明显，左边 `leftCount` 个数，加上根节点，都小于所要求的元素。接着，现在要从右子树搜索，很明显，搜索是往下的，不可能往上（原根节点的方向）搜索，故，之前 `leftCount + 1` 个数作废，所以所传入 `k - leftCount - 1`

6.9 数组 / 双指针

所谓双指针

指的是在遍历对象的过程中，不是普通的使用单个指针进行访问，而是使用两个相同方向或者相反方向的指针进行扫描，从而达到相应的目的。

换言之，双指针法充分使用了数组有序这一特征，从而在某些情况下能够简化一些运算。

6.9.1 加一

给定一个非负数，表示一个数字数组，在该数的基础上+1，返回一个新的数组。该数字按照数位高低进行排列，最高位的数在列表的最前面。

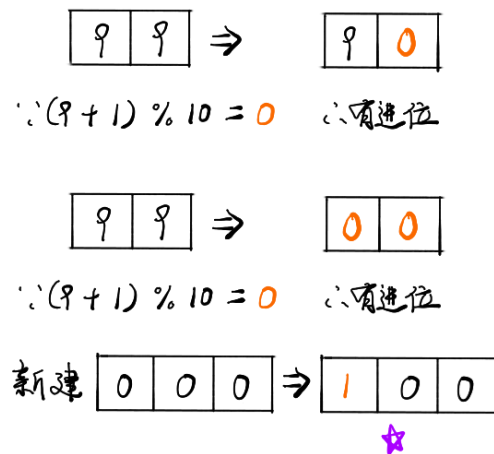
示例：

输入: [4,3,2,1] 输出: [4,3,2,2] 解释: 输入数组表示数字 4321。

6.9.1.1 解题思路

只需要判断有没有进位并模拟出它的进位方式, 如十位数加 11 个位数置为 00, 如此循环直到判断没有再进位就退出循环返回结果。

然后还有一些特殊情况就是当出现 9999、999999 之类的数字时, 循环到最后也需要进位, 出现这种情况时需要手动将它进一位。



6.9.1.2 视频

[给定一个由整数组成的非空数组所表示的非负整数，在该数的基础上加一](#)

```
public int[] plusOne(int[] digits) {
    for (int i = digits.length - 1; i >= 0; i--) {
        digits[i]++;
        digits[i] = digits[i] % 10;
        if (digits[i] != 0) return digits;
    }
    digits = new int[digits.length + 1];
    digits[0] = 1;
    return digits;
}
```

6.9.2 删除元素

给定一个数组和一个值，在原地删除与值相同的数字，返回新数组的长度。

6.9.2.1 解题思路

1. 定义一个 index 用于记录新数组下标, 遍历数组
2. 如果与传入值不同, 则其应存在于新数组中 index++ 并存入
3. 如果与传入值相同, 说明重复, 则直接跳过该数
4. 最后返回 index 即可

```
public int removeElement(int[] A, int elem) {
    if (A == null || A.length == 0) {
        return 0;
    }

    int index = 0;
    for (int i = 0; i < A.length; i++) {
        if (A[i] != elem) {
            A[index++] = A[i];
        }
    }

    return index;
}
```

6.9.3 删除排序数组中的重复数字

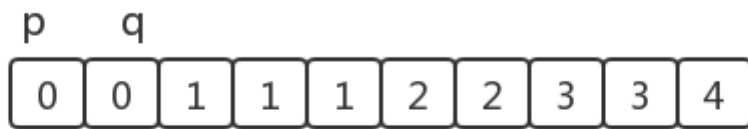
在原数组中“删除”重复出现的数字，使得每个元素只出现一次，并且返回“新”数组的长度。

示例：

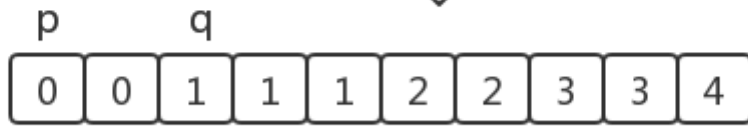
给定 `nums = [0,0,1,1,1,2,2,3,3,4]`，函数应该返回新的长度 `5`，并且原数组 `nums` 的前五个元素被修改为 `0, 1, 2, 3, 4`。你不需要考虑数组中超出新长度后面的元素。

6.9.3.1 解题步骤

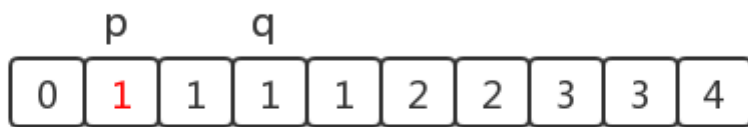
1. 数组完成排序后，我们可以放置两个指针 `size` 和 `i`，其中 `size` 是慢指针，而 `i` 是快指针。
2. 只要 `nums[size] = nums[i]`，我们就增加 `i` 以跳过重复项。
3. 当我们遇到 `nums[i] != nums[size]` 时，跳过重复项的运行已经结束
4. 因此我们必须把它 (`nums[i]`) 的值复制到 `nums[size+1]`。
5. 然后递增 `i` 接着我们将再次重复相同的过程，直到 `size` 到达数组的末尾为止。



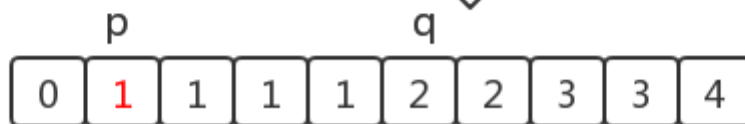
when $\text{nums}[p] == \text{nums}[q]$
 $q++$;



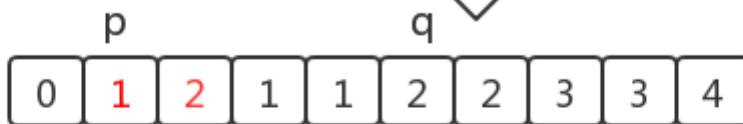
when $\text{nums}[p] != \text{nums}[q]$
 $\text{nums}[p + 1] = \text{nums}[q];$
 $q++;$
 $p++;$



when $\text{nums}[p] == \text{nums}[q]$
 $q++$;



when $\text{nums}[p] != \text{nums}[q]$
 $\text{nums}[p + 1] = \text{nums}[q];$
 $q++;$
 $p++;$




```
public int removeDuplicates(int[] A) {    if (A == null || A.length == 0) {
    return 0;    }    int size = 0;    for (int i = 0; i < A.length; i++) {
        if (A[i] != A[size]) {            A[++size] = A[i];            }        }    // (1)
    return size + 1;}

```

注：因为 size 为下标，所以返回长度要加一

6.9.4 我的日程安排表 I

实现MyCalendar类来存储活动。如果新添加的活动没有重复，则可以添加。类将有方法book(int start, int end)。这代表左闭右开的间隔[start, end)有了预定，范围内的实数x，都满足start <= x < end，返回true。否则，返回false，并且事件不会添加到日历中。

示例：

```
MyCalendar();MyCalendar.book(10, 20); // returns trueMyCalendar.book(15, 25); //
returns falseMyCalendar.book(20, 30); // returns true

```

解释：第一个日程安排可以添加到日历中。第二个日程安排不能添加到日历中，因为时间 15 已经被第一个日程安排预定了。第三个日程安排可以添加到日历中，因为第一个日程安排并不包含时间 20。

6.9.4.1 解题步骤

1. TreeMap 是一个有序的key-value集合，它通过 红黑树 实现，继承于AbstractMap，所以它是一个Map，即一个key-value集合。
 2. TreeMap可以查询小于等于某个值的最大的key，也可查询大于等于某个值的最小的key。
 3. 元素的顺序可以改变，并且对新的数组不会有影响。
- floorKey(K key) 方法用于返回小于或等于给定的键的所有键中，的最大键，或null，如果不存在这样的键
 - ceilingKey(K key) 方法用于返回大于或等于返回到给定的键中，的最小键，或null，如果不存在这样的键

```
class MyCalendar {
    TreeMap<Integer, Integer> calendar;

    MyCalendar() {
        calendar = new TreeMap();
    }

    public boolean book(int start, int end) {
        Integer previous = calendar.floorKey(start), next =
calendar.ceilingKey(start);
        if ((previous == null || calendar.get(previous) <= start) && (next ==
null || end <= next)) {
            calendar.put(start, end);
            return true;
        }
        return false;
    }
}

```

6.9.5 合并两个有序数组

合并两个排序的整数数组A和B变成一个新的数组。可以假设A具有足够的空间去添加B中的元素。

说明:

初始化 A 和 B 的元素数量分别为 m 和 n。你可以假设 A 有足够的空间（空间大小大于或等于 $m + n$ ）来保存 B 中的元素。

示例:

输入:

nums1 = [1,2,3,0,0,0], m = 3

nums2 = [2,5,6], n = 3

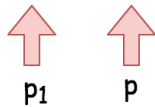
输出: [1,2,2,3,5,6]

6.9.5.1 解题思路

- 双指针 / 从后往前
- 这里的指针 p 用于追踪添加元素的位置。

Get pointers: start from the **end**

nums1 = [1, 2, 3, 0, 0, 0]



nums2 = [2, 5, 6]

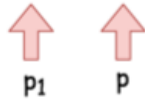


$3 < 6 \Rightarrow \text{set } \text{nums1}[p] = 6$

Get pointers: start from the **end**

1. $3 < 6 \Rightarrow \text{set } \text{nums1}[p = 5] = 6$ and move p2

nums1 = [1, 2, 3, 0, 0, 0]



nums2 = [2, 5, 6]



```
public void mergeSortedArray(int[] A, int m, int[] B, int n) {  
    int i = m - 1, j = n - 1, index = m + n - 1;  
    while (i >= 0 && j >= 0) {  
        if (A[i] > B[j]) {
```

```

        A[index--] = A[i--];
    } else {
        A[index--] = B[j--];
    }
}
while (i >= 0) {
    A[index--] = A[i--];
}
while (j >= 0) {
    A[index--] = B[j--];
}
}

```

6.10 贪心

顾名思义，贪心算法总是作出在当前看来最好的选择。也就是说贪心算法并不从整体最优考虑，它所作出的选择只是在某种意义上的局部最优选择。当然，希望贪心算法得到的最终结果也是整体最优的。虽然贪心算法不能对所有问题都得到整体最优解，但对许多问题它能产生整体最优解。如单源最短路径问题，最小生成树问题等。在一些情况下，即使贪心算法不能得到整体最优解，其最终结果却是最优解的很好近似。

视频

[贪心算法 - 2 理论基础](#)

6.10.1 买卖股票的最佳时机

假设有一个数组，它的第*i*个元素是一支给定的股票在第*i*天的价格。如果你最多只允许完成一次交易(例如，一次买卖股票)，设计一个算法来找出最大利润。

注意:

- 你不能在买入股票前卖出股票。

示例：

输入：[7,1,5,3,6,4]

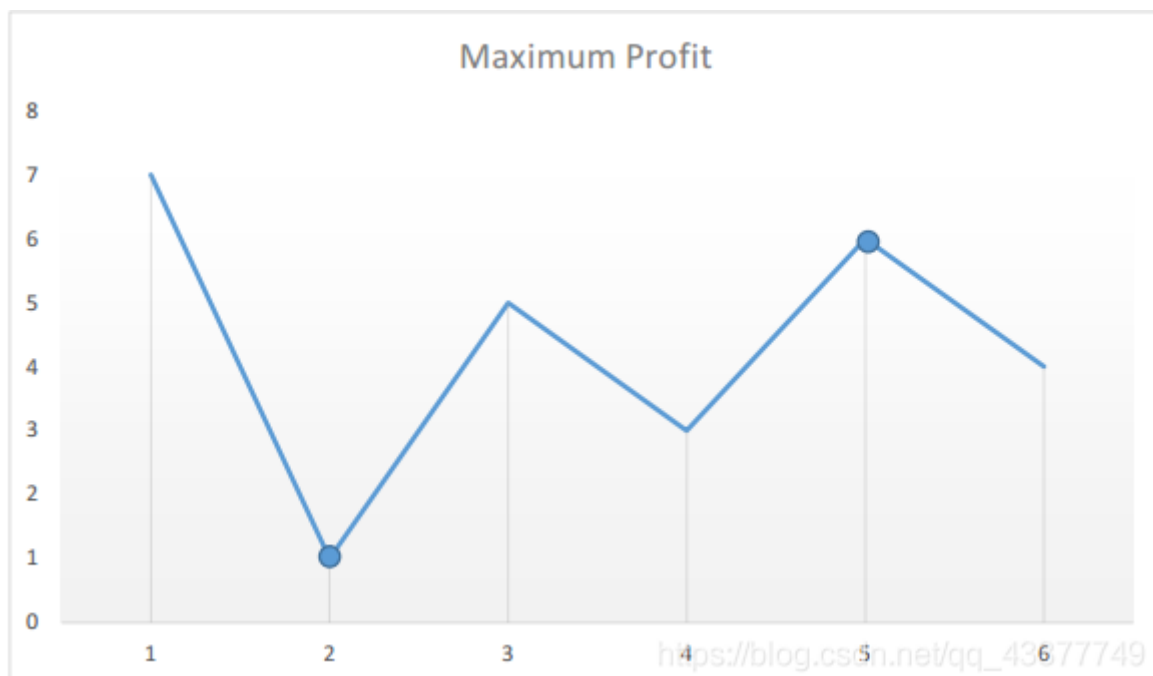
输出：5

解释：在第 2 天（股票价格 = 1）的时候买入，在第 5 天（股票价格 = 6）的时候卖出，最大利润 = 6-1 = 5 。

注意利润不能是 7-1 = 6，因为卖出价格需要大于买入价格。

如果将测试范例 [7, 1, 5, 3, 6, 4]

绘制成图，我们发现：



1. 我们需要找到最小的谷之后的最大的峰。
2. 我们可以维持两个变量 —— min 和 profit，它们分别对应迄今为止所得到的最小的谷值和最大的利润（卖出价格与最低价格之间的最大差值）。

```
public int maxProfit(int[] prices) {    if (prices == null || prices.length == 0) {        return 0;    }    int min = Integer.MAX_VALUE;    //记录最低的价格    int profit = 0;    for (int price : prices) {        min = Math.min(price, min);        profit = Math.max(price - min, profit);    }    return profit;}
```

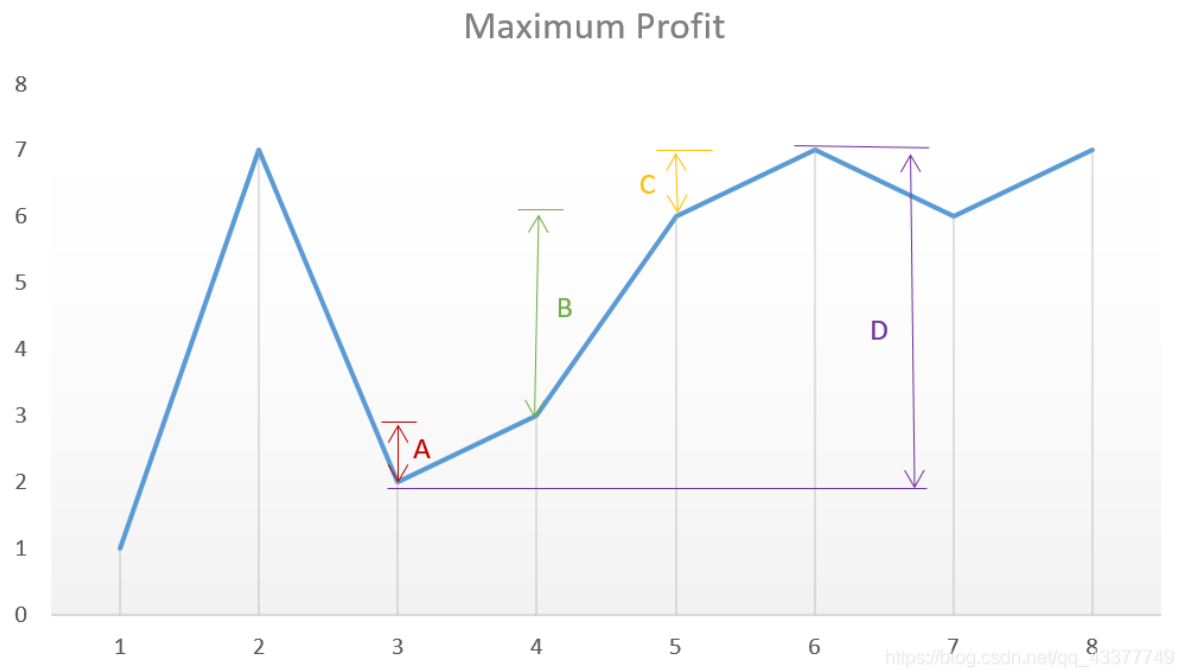
6.10.2 买卖股票的最佳时机 II

给定一个数组 prices 表示一支股票每天的价格。可以完成任意次数的交易, 不过不能同时参与多个交易, 设计一个算法求出最大的利润。

6.10.2.1 解题思路

贪心:

1. 只要相邻的两天股票的价格是上升的,
2. 我们就进行一次交易, 获得一定利润。



6.10.2.2 视频

[买卖股票的最佳时机 II by 代码会说话](#)

```
public int maxProfit(int[] prices) {  
    int profit = 0;  
    for(int i = 0 ; i < prices.length -1; i++) {  
        if(prices[i + 1] > prices[i]) {  
            profit += prices[i + 1] - prices[i];  
        }  
    }  
    return profit;  
}
```

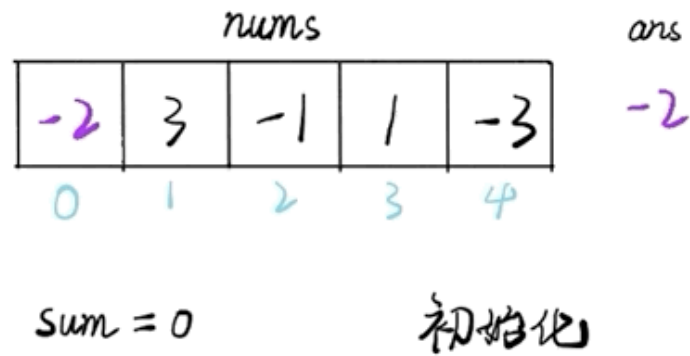
6.10.3 最大子数组

给定一个整数数组，找到一个具有最大和的子数组，返回其最大和。

示例:

输入: $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$,
输出: 6
解释: 连续子数组 $[4, -1, 2, 1]$ 的和最大, 为 6。

6.10.3.1 解题思路



```
public int maxSubArray(int[] nums) {  
    if(nums == null || nums.length == 0) {  
        return 0;  
    }  
    int max = Integer.MIN_VALUE;  
    int sum = 0;  
    for (int num : nums) {  
        sum += num;  
        max = Math.max(sum, max);  
        sum = Math.max(sum, 0);  
    }  
    return max;  
}
```

6.10.4 主元素

给定一个整型数组，找出主元素，它在数组中的出现次数严格大于数组元素个数的二分之一(可以假设数组非空，且数组中总是存在主元素)。

6.10.4.1 解题思路

1. 重点在于：主元素数量大于数组所有元素的二分之一
2. 所以我们要做的是，选出一个出现次数大于其他所有数，出现次数和的数即可
3. 设一个计数器 *currentMajor* 候选数 和 一个 *count* 用于记录次数
4. 每当当前数和 *currentMajor* 值相同时，*count* 值 +1
5. 每当当前数和 *currentMajor* 值不同时，*count* 值 -1
6. 每次 *count* 等于 0 时，说明在之前访问的数组里 *currentMajor* 的数量小于或等于一半
7. 则将 *currentMajor* 赋值为当前数，继续寻找。

```

public int majorityNumber(List<Integer> nums) {    int currentMajor = 0;    int
count = 0;    for(Integer num : nums) {        if(count == 0) {
currentMajor = num;        }        if(num == currentMajor) {
count++;        } else {        count--;        }    }    return
currentMajor;}

```

6.11 字符串处理

字符串广泛应用在Java编程中，在Java中字符串属于对象，Java提供了String类来创建和操作字符串。面试中的字符串处理问题，主要是对于字符串各种方法的灵活应用。下面结合实例，讲讲常见的考点：

方法	说明
int length()	返回当前字符串的长度
int indexOf(int ch)	查找ch字符在该字符串中第一次出现的位置
int indexOf(String str)	查找str子字符串在该字符串中第一次出现的位置
int lastIndexOf(int ch)	查找ch字符在该字符串中最后一次出现的位置
int lastIndexOf(String str)	查找str子字符串在该字符串中最后一次出现的位置
String substring(int beginIndex)	获取从beginIndex位置开始到结束的子字符串
String substring(int beginIndex, int endIndex)	获取从beginIndex位置开始到endIndex位置的子字符串
String trim()	返回去除了前后空格的字符串
boolean equals(Object obj)	将该字符串与指定对象比较，返回true或false
String toLowerCase()	将字符串转换为小写
String toUpperCase()	将字符串转换为大写
char charAt(int index)	获取字符串中指定位置的字符
String[] split(String regex, int limit)	将字符串分割为子字符串，返回字符串数组
byte[] getBytes()	将该字符串转换为byte数组

6.11.1 括号生成

给定 n ，表示有 n 对括号，请写一个函数以将其生成所有的括号组合，并返回组合结果。

例如

给出 $n = 3$ ，生成结果为：["(())()", "(()())", "(()())", "()(())", "()()()"]

6.11.2 解题思路

使用 回溯法

只有在我们知道序列仍然保持有效时才添加 '(' or ')'，而不是像 方法一 那样每次添加。我们可以通过跟踪到目前为止放置的左括号和右括号的数目来做到这一点，

如果我们还剩一个位置，我们可以开始放一个左括号。如果它不超过左括号的数目，我们可以放一个右括号。

6.11.3 视频

[视频讲解和源码-生成括号](#)

```
public List<String> generateParenthesis(int n) {    List<String> res = new
ArrayList<>();    helper(n, n, "", res);    return res;}// DFSprivate void
helper(int nL, int nR, String parenthesis, List<String> res) {    // nL 和 nR 分
别代表左右括号剩余的数量    if (nL < 0 || nR < 0) {        return;    }    if
(nL == 0 && nR == 0) {        res.add(parenthesis);        return;    }
helper(nL - 1, nR, parenthesis + "(", res);    if (nL >= nR) {        return;
    }    helper(nL, nR - 1, parenthesis + ")", res);}
```

6.11.3.1 复杂度

我们的复杂度分析依赖于理解 `generateParenthesis(n)` 中有多少个元素。这个分析超出了本文的范畴，但事实证明这是第 `n` 个卡特兰数 $\frac{1}{n+1} \binom{2n}{n}$ ，这是由 $\frac{4^n}{n\sqrt{n}}$ 渐近界定的。

- 时间复杂度： $O(\frac{4^n}{\sqrt{n}})$ ，在回溯过程中，每个有效序列最多需要 `n` 步。
- 空间复杂度： $O(\frac{4^n}{\sqrt{n}})$ ，如上所述，并使用 $O(n)$ 的空间来存储序列。

6.11.2 Excel表列标题

给定一个正整数，返回相应的列标题，如Excel表中所示。如: 1 -> A, 2 -> B ... 26 -> Z, 27 -> AA

示例：

输入：28 输出："AB"

6.11.2.1 解题思路

- 网上看了 n 多人的方法，感觉很多都做麻烦了。大多数人都困在这个 'A' 或者说 $n = 0$ 上
- 举个例子，如果输入 26，我们一般会直接把它 %26 这样得到的就是一个 0
- 然而很多人得到字符的方式都是 $\%26 + 64$ ，也就是 $0 + 'A' = 'A'$ ，正确答案当然是 'Z'，于是加了一堆判断
- 其实不用那么麻烦，一个 $n--$ 就能搞定。

```
public String convertToTitle (int n) {    StringBuilder str = new
StringBuilder();    while (n > 0) {        n--;        str.append ( (char) ( (n
% 26) + 'A'))        n /= 26;    }    return str.reverse().toString();}
```

6.11.3 翻转游戏

给定一个只包含两种字符的字符串：+和-，你和你的小伙伴轮流翻转"++"变成"--"。当一个人无法采取行动时游戏结束，另一个人将是赢家。编写一个函数，计算字符串在一次有效移动后的所有可能状态。

示例：

输入：s = "++++" ["--++", "-+-+", "+--+", "++--"]

6.11.3.1 解题思路

1. 我们从第二个字母开始遍历
2. 每次判断当前字母是否为+, 和之前那个字母是否为+
3. 如果都为加, 则将翻转后的字符串存入结果中即可

```
public List<String> generatePossibleNextMoves (String s) {
    List list = new ArrayList();
    // indexOf 方法使用 看下方拓展
    for (int i = -1; (i = s.indexOf ("++", i + 1)) >= 0;) {
        list.add (s.substring (0, i) + "--" + s.substring (i + 2));
    }
    return list;
}
```

6.11.3.2 拓展:

Java中字符串中子串的查找共有四种方法, 如下:

1. int indexOf(String str): 返回第一次出现的指定子字符串在此字符串中的索引。
2. int indexOf(String str, int startIndex): 从指定的索引处开始, 返回第一次出现的指定子字符串在此字符串中的索引。
3. int lastIndexOf(String str): 返回在此字符串中最右边出现的指定子字符串的索引。
4. int lastIndexOf(String str, int startIndex): 从指定的索引处开始向后搜索, 返回在此字符串中最后一次出现的指定子字符串的索引。

substring() 方法返回字符串的子字符串。

1. public String substring(int beginIndex) 返回 beginIndex 后的字符串
2. public String substring(int beginIndex, int endIndex) 返回 beginIndex 到 endIndex 之间的字符串

6.11.4 翻转字符串中的单词

给定一个字符串, 逐个翻转字符串中的每个单词。

示例:

输入: "a good example" 输出: "example good a" 解释: 如果两个单词间有多余的空格, 将反转后单词间的空格减少到只含一个。

6.11.4.1 解题思路

1. 通过 split 方法, 以 " " 为标识符为基准拆分字符串
2. 将拆分后的字符串倒序插入数组中即可

```
public String reverseWords(String s) {
    if(s.length() == 0 || s == null){
        return " ";
    } //按照空格将s切分
    String[] array = s.split(" ");
    StringBuilder sb = new StringBuilder(); //从后往前遍历array, 在sb中插入单词
    for(int i = array.length - 1; i >= 0; i--){
        if(!array[i].equals("")) {
            // 为防止字符串首多一个 " " 判断当前是不是空字符串 // 是字符串第一个就不
            // 输出空格
            if (sb.length() > 0) {
                sb.append(" ");
            }
            sb.append(array[i]);
        }
    }
    return sb.toString();
}
```

6.11.5 字符串转换整数 (atoi)

实现atoi这个函数，将一个字符串转换为整数。如果没有合法的整数，返回0。如果整数超出了32位整数的范围，返回 INT_MAX(2147483647) 如果是正整数，或者 INT_MIN(-2147483648) 如果是负整数。

示例：

输入："4193 with words"输出：4193解释：转换截止于数字 '3'，因为它的下一个字符不为数字。
例 4:输入："words and 987"输出：0解释：第一个非空字符是 'w'，但它不是数字或正、负号。
因此无法执行有效的转换。

6.11.5.1 解题思路

1. 首先我们要知道该数正负
2. 根据题意调用 trim() 去掉空格
3. 去完多余空格之后，首位有三种情况 '+' '-' 其他
4. 设一个 flag 叫做 sign 默认值为一，如果监测到 '-' 则设为 -1
5. 这样一来后面求出的结果乘以 sign 就能带上正负值
6. 在定义一个 num 值用于保存答案数值
7. for 循环从头到尾访问字符串
8. 先判断当前位是否为数字，这时分两种情况
9. 如果字符串首位就不是数字和 +- 号，根据题意直接退出循环
10. 如果为数字就将 sum 的值 *10 倍，再将其加入 sum 中
11. 如果值超过 MAX_VALUE 跳出循环
12. 对应 *sign 输出正负值，或者 MAX_VALUE 或 MIN_VALUE 即可

6.11.5.2 视频

[视频讲解和源码-字符串转换整数](#)

```
public int myAtoi(String str) {    if(str == null) {        return 0;    }
str = str.trim();    if (str.length() == 0) {        return 0;    }
int sign = 1;    int index = 0;    if (str.charAt(index) == '+') {
index++;    } else if (str.charAt(index) == '-') {        sign = -1;
index++;    }    long num = 0;    for (; index < str.length(); index++) {
if (str.charAt(index) < '0' || str.charAt(index) > '9') {        break;
    }        num = num * 10 + (str.charAt(index) - '0');        if (num >
Integer.MAX_VALUE ) {            break;        }    }    if (num * sign >=
Integer.MAX_VALUE) {        return Integer.MAX_VALUE;    }    if (num * sign <=
Integer.MIN_VALUE) {        return Integer.MIN_VALUE;    }    return (int)num *
sign;}
```

注：trim() 函数是去掉String字符串的首尾空格;

6.11.6 字符串中是否含有子串

- 暴力算法：O(mn)
- KMP:
-

6.11.7 最长公共前缀

编写一个函数来查找字符串数组中的最长公共前缀。

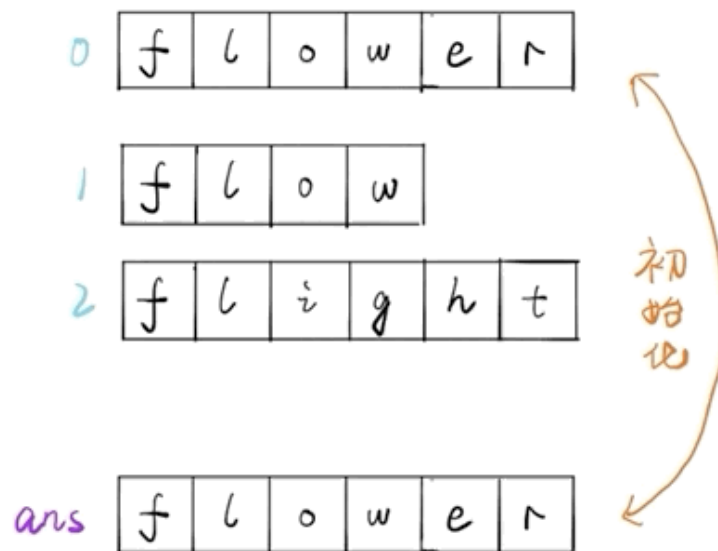
如果不存在公共前缀，返回空字符串 ""。

示例：

输入：["flower","flow","flight"]
输出："fl"

6.11.7.1 解题思路

标签：链表 当字符串数组长度为 0 时则公共前缀为空，直接返回 令最长公共前缀 ans 的值为第一个字符串，进行初始化 遍历后面的字符串，依次将其与 ans 进行比较，两两找出公共前缀，最终结果即为最长公共前缀 如果查找过程中出现了 ans 为空的情况，则公共前缀不存在直接返回 s 为所有字符串的长度之和



6.11.7.2 视频

[最长公共前缀](#)

```
public String longestCommonPrefix(String[] strs) {    if (strs == null ||  
strs.length == 0) {        return "";    }    String prefix = strs[0];  
for(int i = 1; i < strs.length; i++) {        int j = 0;        while (j <  
strs[i].length() && j < prefix.length() && strs[i].charAt(j) ==  
prefix.charAt(j)) {            j++;        }        if( j == 0) {  
return "";        }        prefix = prefix.substring(0, j);    }    return  
prefix;}
```

6.11.7.3 时间复杂度：

$O(s)$

6.11.8 回文数

判断一个正整数是不是回文数。回文数的定义是，将这个数反转之后，得到的数仍然是同一个数。

示例：

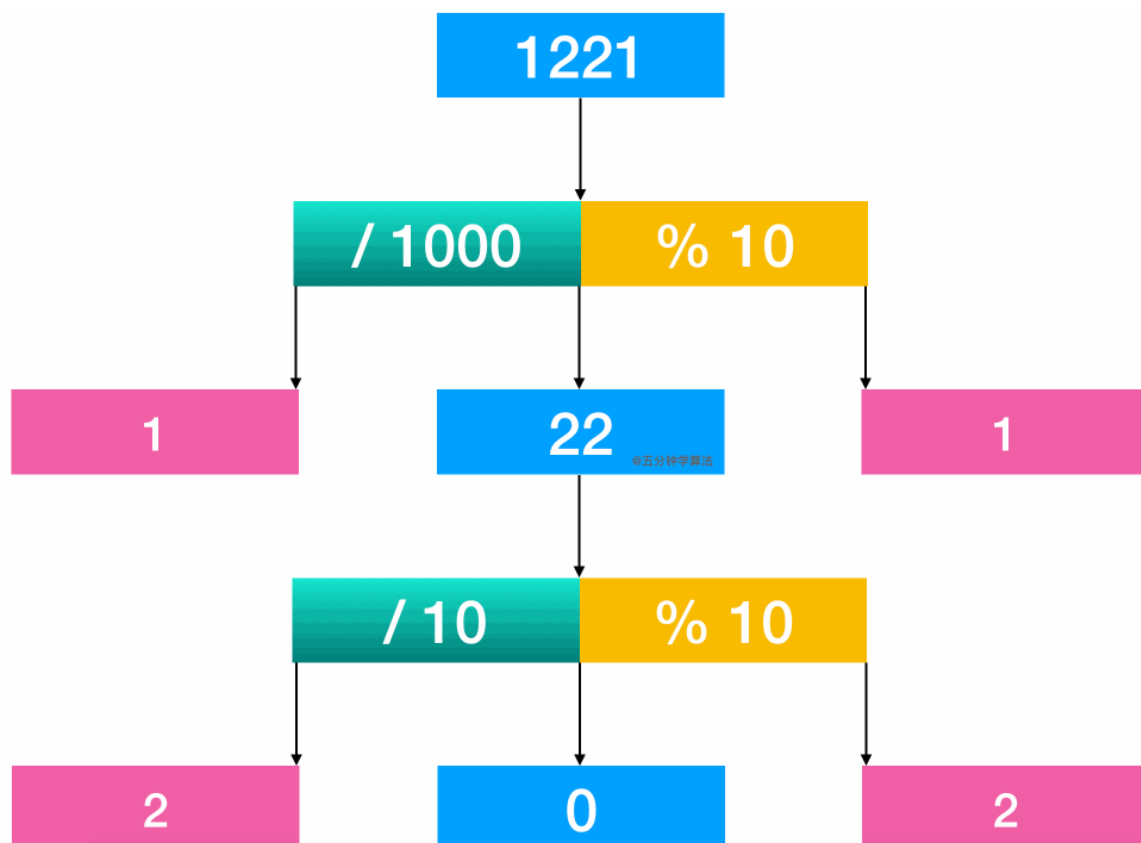
输入：121 输出：true

6.11.8.1 解题思路

通过取整和取余操作获取整数中对应的数字进行比较。

举个例子：1221 这个数字。

通过计算 $1221 / 1000$ ，得首位1 通过计算 $1221 \% 10$ ，可得末位 1 进行比较 再将 22 取出来继续比较



6.11.8.2 视频

[回文数](#)

```
public boolean palindromeNumber(int num) {  
    // write your code here  
    if(num < 0){  
        return false;  
    }  
    int div = 1;  
    while(num / div >= 10){  
        div *= 10;  
    }  
    while(num > 0){  
        if(num / div != num % 10){  
            return false;  
        }  
        num = (num % div) / 10;  
    }  
}
```

```
        div /= 100;
    }
    return true;
}
```

6.12 动态规划

动态规划常常适用于有重叠子问题和最优子结构性质的问题，动态规划方法所耗时间往往远少于朴素解法。其背后的基本思想非常简单。大致上，若要解一个给定问题，我们需要解其不同部分（即子问题），再根据子问题的解以得出原问题的解。

通常许多子问题非常相似，为此动态规划法试图仅仅解决每个子问题一次，从而减少计算量：一旦某个给定子问题的解已经算出，则将其记忆化存储，以便下次需要同一个子问题解之时直接查表。这种做法在重复子问题的数目关于输入的规模呈指数增长时特别有用。

6.12.1 单词拆分

给定字符串 s 和单词字典 dict ，确定 s 是否可以分成一个或多个以空格分隔的子串，并且这些子串都在字典中存在。

示例：

```
输入: s = "applepenapple", wordDict = ["apple", "pen"]
输出: true
解释: 返回 true 因为 "applepenapple" 可以被拆分成 "apple pen apple"。
      注意你可以重复使用字典中的单词。
```

6.12.1.1 解题思路

这个方法的想法是对于给定的字符串 s 可以被拆分成子问题 s_1 和 s_2 。如果这些子问题都可以独立地被拆分成符合要求的子问题，那么整个问题 s 也可以满足。也就是，如果 "catsanddog" 可以拆分成两个子字符串 "catsand" 和 "dog" 。子问题 "catsand" 可以进一步拆分成 "cats" 和 "and" ，这两个独立的部分都是字典的一部分，所以 "catsand" 满足题意条件，再往前， "catsand" 和 "dog" 也分别满足条件，所以整个字符串 "catsanddog" 也满足条件。

现在，我们考虑 dp 数组求解的过程：

1. 我们使用 $n+1$ 大小数组的 dp ，其中 n 是给定字符串的长度。
2. 我们也使用 2 个下标指针 i 和 j ，其中 i 是当前字符串从头开始的子字符串 s' 的长度， j 是当前子字符串 s' 的拆分位置，拆分成 $s'(0,j)$ 和 $s'(j+1,i)$ 。
3. 为了求出 dp 数组，我们初始化 $\text{dp}[0]$ 为 true ，这是因为空字符串总是字典的一部分。 dp 数组剩余的元素都初始化为 false 。
4. 我们用下标 i 来考虑所有从当前字符串开始的可能的子字符串。对于每一个子字符串，我们通过下标 j 将它拆分成 s_1' 和 s_2' （注意 i 现在指向 s_2' 的结尾）。
5. 为了将 $\text{dp}[i]$ 数组求出来，我们依次检查每个 $\text{dp}[j]$ 是否为 true ，也就是子字符串 s_1' 是否满足题目要求。如果满足，我们接下来检查 s_2' 是否在字典中。如果包含，我们接下来检查 s_2' 是否在字典中，如果两个字符串都满足要求，我们让 $\text{dp}[i]$ 为 true ，否则令其为 false 。

```
public boolean wordBreak(String s, List<String> wordDict) {
    Set<String> wordDictSet=new HashSet(wordDict);
    boolean[] dp = new boolean[s.length() + 1];
```

```

    dp[0] = true;
    for (int i = 1; i <= s.length(); i++) {
        for (int j = 0; j < i; j++) {
            if (dp[j] && wordDictSet.contains(s.substring(j, i))) {
                dp[i] = true;
                break;
            }
        }
    }
    return dp[s.length()];
}

```

6.12.1.2 复杂度分析

时间复杂度： $O(n^2)$ 。求出 dp 数组需要两重循环。

空间复杂度： $O(n)$ 。 dp 数组的长度是 $n+1$ 。

6.12.2 爬楼梯

假设你正在爬楼梯。需要 n 阶你才能到达楼顶。

每次你可以爬 1 或 2 个台阶。你有多少种不同的方法可以爬到楼顶呢？

注意：给定 n 是一个正整数。

示例：

输入：3 输出：3 解释：有三种方法可以爬到楼顶。 1 阶 + 1 阶 + 1 阶 1 阶 + 2 阶 2 阶 + 1 阶

6.12.2.1 解题思路

感觉这题类似斐波那契数列。不难发现，这个问题可以被分解为一些包含最优子结构的子问题，即它的最优解可以从其子问题的最优解来有效地构建，我们可以使用动态规划来解决这一问题。

第 i 阶可以由以下两种方法得到：

在第 $(i-1)$ 阶后向上爬 1 阶。

在第 $(i-2)$ 阶后向上爬 2 阶。

所以到达第 i 阶的方法总数就是到第 $(i-1)$ 阶和第 $(i-2)$ 阶的方法数之和。

令 $dp[i]$ 表示能到达第 i 阶的方法总数：

$dp[i] = dp[i-1] + dp[i-2]$ $dp[i] = dp[i-1] + dp[i-2]$

N=6

DP

0	1					
0	1	2	3	4	5	6

```
public int climbStairs(int n) {    if (n == 0) return 0;    int[] array = new
int[n + 1];    array[0] = 1;    if (array.length > 1) {        array[1] = 1;
    }    for(int i = 2; i < array.length; i++) {        array[i] = array[i - 1]
+ array[i - 2];    }    return array[n];}
```

6.12.3 打家劫舍

假设你是一个专业的窃贼，准备沿着一条街打劫房屋。每个房子都存放着特定金额的钱。你面临的唯一约束条件是：相邻的房子装着相互联系的防盗系统，且当相邻的两个房子同一天被打劫时，该系统会自动报警。给定一个非负整数列表，表示每个房子中存放的钱，算一算，如果今晚去打劫，在不触动报警装置的情况下，你最多可以得到多少钱。

示例：

输入：[2,7,9,3,1] 输出：12 解释：偷窃 1 号房屋（金额 = 2），偷窃 3 号房屋（金额 = 9），接着偷窃 5 号房屋（金额 = 1）。偷窃到的最高金额 = 2 + 9 + 1 = 12。

6.12.3.1 解题思路

考虑所有可能的抢劫方案过于困难。一个自然而然的想法是首先从最简单的情况开始。记：

$f(k)$ = 从前 k 个房屋中能抢劫到的最大数额， A_i = 第 i 个房屋的钱数。

首先看 $n = 1$ 的情况，显然 $f(1) = A_1$ 。

再看 $n = 2$ ， $f(2) = \max(A_1, A_2)$ 。

对于 $n = 3$ ，有两个选项：

抢第三个房子，将数额与第一个房子相加。

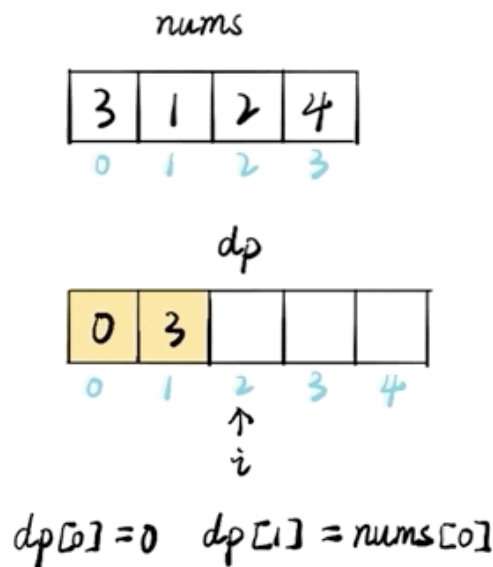
不抢第三个房子，保持现有最大数额。

显然，你想选择数额更大的选项。于是，可以总结出公式：

$$f(k) = \max(f(k-2) + A_k, f(k-1))$$

我们选择 $f(-1) = f(0) = 0$ 为初始情况，这将极大地简化代码。

答案为 $f(n)$ 。可以用一个数组来存储并计算结果。不过由于每一步你只需要前两个最大值，两个变量就足够用了。



```
public long houseRobber(int[] A) {    if (A.length == 0) return 0;    long[] res
= new long[A.length + 1];    res[0] = 0;    res[1] = A[0];    for (int i = 2; i
< res.length; i++) {        res[i] = Math.max(res[i - 2] + A[i - 1], res[i -
1]);    }    return res[A.length];}
```

6.12.3.2 复杂度分析

时间复杂度： $O(n)$ 。其中 n 为房子的数量。空间复杂度： $O(1)$ 。

6.12.4 编辑距离

给出两个单词 word1 和 word2，计算出将 word1 转换为 word2 的最少操作次数。你总共三种操作方法：插入一个字符、删除一个字符、替换一个字符。

示例：

输入：word1 = "horse", word2 = "ros"

输出：3

解释：

horse -> rorse (将 'h' 替换为 'r')

rorse -> rose (删除 'r')

rose -> ros (删除 'e')

输入：word1 = "intention", word2 = "execution"

输出: 5

解释:

intention -> inention (删除 't')

inention -> enention (将 'i' 替换为 'e')

enention -> exention (将 'n' 替换为 'x')

exention -> exection (将 'n' 替换为 'c')

exection -> execution (插入 'u')

6.12.4.1 解题思路

我们的目的是让问题简单化, 比如说两个单词 `horse` 和 `ros` 计算他们之间的编辑距离 D , 容易发现, 如果把单词变短会让这个问题变得简单, 很自然的想到用 $D[n][m]$ 表示输入单词长度为 n 和 m 的编辑距离。

具体来说, $D[i][j]$ 表示 `word1` 的前 i 个字母和 `word2` 的前 j 个字母之间的编辑距离。



$D[i][j]$ = the edit distance between
`word1[1..i]` and `word2[1..j]`
i.e. between **HOR** and **RO**

当我们获得 $D[i-1][j]$, $D[i][j-1]$ 和 $D[i-1][j-1]$ 的值之后就可以计算出 $D[i][j]$ 。

每次只可以往单个或者两个字符串中插入一个字符

那么递推公式很显然了

如果两个子串的最后一个字母相同, $word1[i] = word2[j]$ 的情况下:

$$D[i][j] = 1 + \min(D[i-1][j], D[i][j-1], D[i-1][j-1] - 1) \quad D[i][j] = 1 + \min(D[i-1][j], D[i][j-1], D[i-1][j-1] - 1)$$

否则, $word1[i] \neq word2[j]$ 我们将考虑替换最后一个字符使得他们相同:

$$D[i][j] = 1 + \min(D[i-1][j], D[i][j-1], D[i-1][j-1]) \quad D[i][j] = 1 + \min(D[i-1][j], D[i][j-1], D[i-1][j-1])$$

所以每一步结果都将基于上一步的计算结果, 示意如下:


word1[1..i]


word2[1..j]

$D[i][j] = ?$ the edit distance between HOR and RO
 $D[i][j] = 1 + \min(D[i-1][j], D[i][j-1], D[i-1][j-1])$, since R != O

$D[i-1][j] = 1$ the edit distance between HO and RO
 $D[i][j-1] = 2$ the edit distance between HOR and R
 $D[i-1][j-1] = 2$ the edit distance between HO and R

$D[i][j] = 2$

同时，对于边界情况，一个空串和一个非空串的编辑距离为 $D[i][0] = i$ 和 $D[0][j] = j$ 。

综上所述我们得到了算法的全部流程。

E				
S				
R				
O				
H				
#				
	#	R	O	S

温馨提示，如果思维不好理解的话，把解题思路记清楚就行

```

public int minDistance(String word1, String word2) {
    // write your code here
    int n = word1.length();
    int m = word2.length();
    int[][] dp = new int[n + 1][m + 1];
    for (int i = 0; i < n + 1; i++){
        dp[i][0] = i;
    }
    for (int j = 0; j < m + 1; j++){
        dp[0][j] = j;
    }
    for (int i = 1; i < n + 1; i++){
        for (int j = 1; j < m + 1; j++){
            if (word1.charAt(i - 1) == word2.charAt(j - 1)){
                dp[i][j] = dp[i - 1][j - 1];
            } else {

```

```

        dp[i][j] = 1 + Math.min(dp[i - 1][j - 1], Math.min(dp[i][j - 1],
dp[i - 1][j]));
    }
}
}
return dp[n][m];
}

```

6.12.4.2 复杂度分析

时间复杂度： $O(mn)$ ，两层循环显而易见。空间复杂度： $O(mn)$ ，循环的每一步都要记录结果。

6.12.5 乘积最大子序列

给定一个整数数组 `nums`，找出一个序列中乘积最大的连续子序列（该序列至少包含一个数）。

示例：

输入：`[-2,0,-1]` 输出：`0` 解释：结果不能为 `2`，因为 `[-2,-1]` 不是子数组。

6.12.5.1 解题思路

1. 遍历数组时计算当前最大值，不断更新
2. 令 `imax` 为当前最大值，则当前最大值为 `imax = max(imax * nums[i], nums[i])`
3. 由于存在负数，那么会导致最大的变最小的，最小的变最大的。因此还需要维护当前最小值 `imin`，`imin = min(imin * nums[i], nums[i])`
4. 当负数出现时则 `imax` 与 `imin` 进行交换再进行下一步计算



```

public int maxProduct(int[] nums) {
    int max = Integer.MIN_VALUE, imax = 1, imin = 1;
    for(int i=0; i<nums.length; i++){
        if(nums[i] < 0){
            int tmp = imax;
            imax = imin;

```

```

        imin = tmp;
    }
    imax = Math.max(imax*nums[i], nums[i]);
    imin = Math.min(imin*nums[i], nums[i]);

    max = Math.max(max, imax);
}
return max;
}

```

6.12.5.2 时间复杂度:

- $O(n)$

6.12.6 硬币问题

6.12.6.1 思路

- 令 $dp[i][j]$ 为遍历到当下这个硬币时，组成金额 j 的方法数目
- 有两种可能性 (1) 当前这个硬币没有取， $dp[i][j]=dp[i-1][j]$ ；(2) 当前这个硬币取了， $dp[i][j]=dp[i][j-coins[i]]$ 。最后的结果是两者的和
- 将状态转移方程翻译成代码，并处理边界条件

6.12.6.2 代码

```

public int waysToChange(int n) {
    int[][] dp=new int[4][n+1];
    int[] coins={1,5,10,25};

    for(int i=0;i<=n;i++)
        dp[0][i]=1;
    for(int i=0;i<4;i++)
        dp[i][0]=1;

    for(int i=1;i<4;i++){
        for(int j=1;j<=n;j++){
            if(j<=coins[i])
                dp[i][j]=(dp[i-1][j]+dp[i][j-coins[i]])%1000000007;
            else
                dp[i][j]=dp[i-1][j];
        }
    }
    return dp[3][n];
}

```

6.12.7 IP 可能的个数

- 代码

```

@Test
public void test() {
    System.out.println(dp("19216801"));
}

```

```

private int dp(String str) {
    int length = str.length();
    int[][] dp = new int[5][length + 1];

    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < length + 1; j++) {
            if (i == 0 && j==0) {
                dp[i][j] = 1;
                continue;
            }
            if (i == 0) {
                dp[i][j] = 0;
                continue;
            }
            dp[i][j] = 0;
            for (int k = 1; k <= 3; k++) {
                if (j-k>=0 && validate(str.substring(j-k, j))) {
                    dp[i][j] += dp[i-1][j-k];
                }
            }
        }
    }

    return dp[4][length];
}

private boolean validate(String s) {
    if (s.equals("0")) {
        return true;
    }
    if (s.charAt(0) == '0') {
        return false;
    }
    return Integer.parseInt(s) < 256;
}

```

6.13 矩阵



6.13.1 螺旋矩阵

给定一个包含 $m \times n$ 个要素的矩阵，（ m 行， n 列），按照螺旋顺序，返回该矩阵中的所有要素。

示例：

```
输入：
[
  [1, 2, 3, 4],
  [5, 6, 7, 8],
  [9,10,11,12]
]
输出： [1,2,3,4,8,12,11,10,9,5,6,7]
```

6.13.1 .1解题思路

我们定义矩阵的第 k 层是到最近边界距离为 k 的所有顶点。例如，下图矩阵最外层元素都是第 1 层，次外层元素都是第 2 层，然后是第 3 层的。

```
[[1, 1, 1, 1, 1, 1, 1],
 [1, 2, 2, 2, 2, 2, 1],
 [1, 2, 3, 3, 3, 2, 1],
 [1, 2, 2, 2, 2, 2, 1],
 [1, 1, 1, 1, 1, 1, 1]]
```

对于每层，我们从左上方开始以顺时针的顺序遍历所有元素，假设当前层左上角坐标是 (r_1, c_1) ，右下角坐标是 (r_2, c_2) 。

首先，遍历上方的所有元素 (r_1, c) ，按照 $c = c_1, \dots, c_2$ 的顺序。然后遍历右侧的所有元素 (r, c_2) ，按照 $r = r_1+1, \dots, r_2$ 的顺序。如果这一层有四条边（也就是 $r_1 < r_2$ 并且 $c_1 < c_2$ ），我们以下图所示的方式遍历下方的元素和左侧的元素。

[[1, 1, 1, 1, 1, 1],	top: c from $c_1 \dots c_2$
[1, 2, 2, 2, 2, 1],	right: r from $r_1+1 \dots r_2$
[1, 2, 3, 3, 2, 1],	bottom: c from $c_2+1 \dots c_1+1$
[1, 2, 2, 2, 2, 1],	left: r from $r_2+1 \dots r_1+1$
[1, 1, 1, 1, 1, 1]]	

```
public List<Integer> spiralOrder(int[][] matrix) {    ArrayList<Integer> rst =
new ArrayList<Integer>();    if(matrix == null || matrix.length == 0) {
return rst;    }    int rows = matrix.length;    int cols =
matrix[0].length;    int count = 0;    while(count * 2 < rows && count * 2 <
cols){        for (int i = count; i < cols - count; i++) {
rst.add(matrix[count][i]);        }        for (int i = count + 1; i <
rows - count; i++) {            rst.add(matrix[i][cols - count - 1]);        }
        if (rows - 2 * count == 1 || cols - 2 * count == 1) { // 如果只剩1行
或1列            break;        }        for (int i = cols - count -
2; i >= count; i--) {            rst.add(matrix[rows - count - 1][i]);        }
        for (int i = rows - count - 2; i >= count + 1; i--) {
            rst.add(matrix[i][count]);        }        count++;    }    return
rst;}
```

6.13.2 判断数独是否合法

请判定一个数独是否有效。该数独可能只填充了部分数字，其中缺少的数字用 . 表示。

维护一个 HashSet 用来记同一行、同一列、同一九宫格是否存在相同数字

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

示例：

```
输入:[ ["8","3",".",".","7",".",".",".","."],
["6",".",".","1","9","5",".",".","."], [".","9","8",".",".",".",".","6","."],
["8",".",".",".","6",".",".",".","3"], ["4",".",".","8",".","3",".",".","1"],
["7",".",".",".","2",".",".",".","6"], [".","6",".",".",".",".","2","8","."],
[".",".",".","4","1","9",".",".","5"], [".",".",".",".","8",".",".","7","9"]]输出: false解释: 除了第一行的第一个数字从 5 改为 8 以外，空格内其他数字均与 示例1 相同。但由于位于左上角的 3x3 宫内有两个 8 存在，因此这个数独是无效的。
```

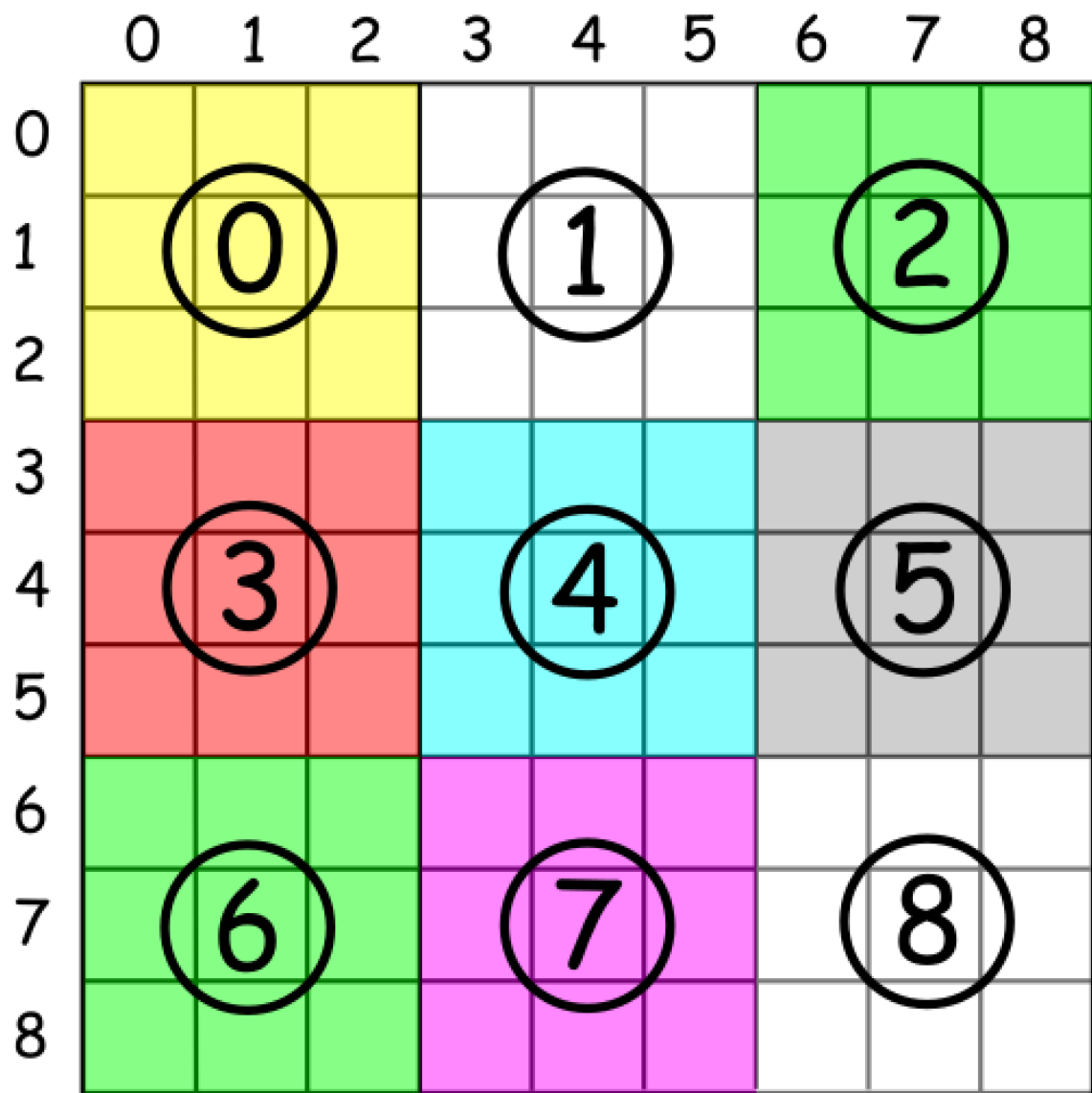
说明:

一个有效的数独（部分已被填充）不一定是可解的。只需要根据以上规则，验证已经填入的数字是否有效即可。给定数独序列只包含数字 1-9 和字符 '.'。给定数独永远是 9x9 形式的。

6.13.2 解题思路

一次迭代 首先，让我们来讨论下面两个问题：

如何枚举子数独？ 可以使用 $\text{box_index} = (\text{row} / 3) * 3 + \text{columns} / 3$ ，其中 / 是整数除法。



如何确保行 / 列 / 子数独中没有重复项？可以利用 `value -> count` 哈希映射来跟踪所有已经遇到的值。

现在，我们完成了这个算法的所有准备工作：

遍历数独。检查每个单元格值是否已经在当前的行 / 列 / 子数独中出现过：如果出现重复，返回 `false`。如果没有，则保留此值以进行进一步跟踪。返回 `true`。

	0	1	2	3	4	5	6	7	8
0	5	3		7					
1	6			1	8	7			
2		8	7					6	
3	8				6				3
4	4			8		3			1
5	7				2				6
6		6					2	8	
7				4	1	7			5
8					8				8

$i = 0, j = 0$

$rows[0] = \{5:1, \}$

$columns[0] = \{5:1, \}$

$boxes[0] = \{5:1, \}$

```
public boolean isValidSudoku(char[][] board) {
    Set seen = new HashSet();
    for (int i=0; i<9; ++i) {
        for (int j=0; j<9; ++j) {
            char
            number = board[i][j];
            if (number != '.')
                if
                (!seen.add(number + " in row " + i) ||
                 !seen.add(number + "
in column " + j) ||
                 !seen.add(number + " in block " + i / 3 +
                 "-" + j / 3))
                    return false;
            }
        }
        return true;
    }
}
```

6.13.3 旋转图像

给定一个 $N \times N$ 的二维矩阵表示图像，90度 顺时针旋转图像。

示例：

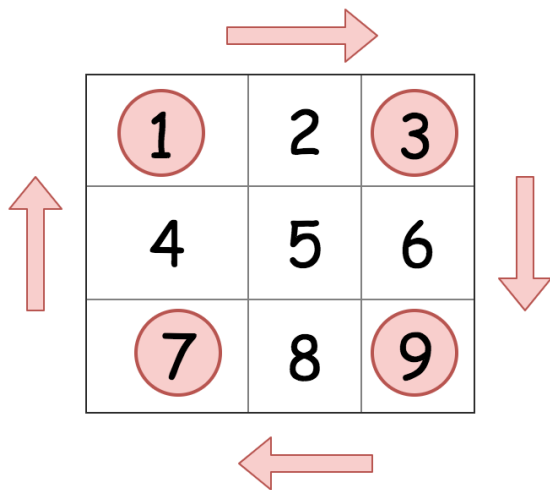
输入：[[1,1,0,0],[1,0,0,1],[0,1,1,1],[1,0,1,0]] 输出：[[1,1,0,0],[0,1,1,0],[0,0,0,1],[1,0,1,0]] 解释：首先翻转每一行：[[0,0,1,1],[1,0,0,1],[1,1,1,0],[0,1,0,1]]；然后反转图片：[[1,1,0,0],[0,1,1,0],[0,0,0,1],[1,0,1,0]]

说明：

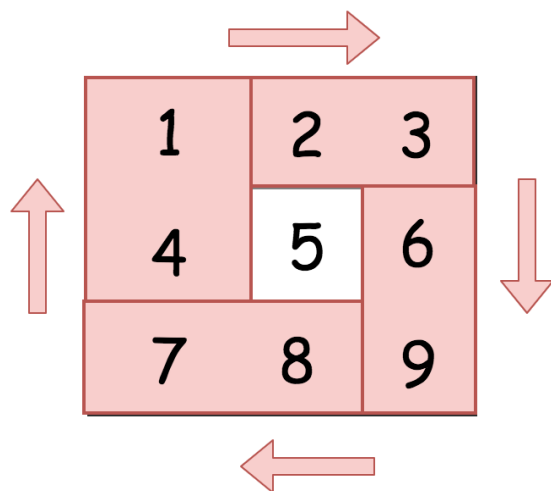
$1 \leq A.length = A[0].length \leq 200 \leq A[i][j] \leq 1$

6.13.3.1 解题思路

我们先来看看每个元素在旋转的过程中是如何移动的：



这提供给我们了一个思路，将给定的矩阵分成四个矩形并且将原问题划归为 **旋转这些矩形** 的问题。



现在的解法很直接 -- 可以在第一个矩形中移动元素并且在 长度为 4 个元素的临时列表中 **移动** 它们。

Temp list

1	2	3
4	5	6
7	8	9

```
public void rotate(int[][] matrix) {
    if (matrix == null || matrix.length == 0 || matrix[0].length == 0) {
        return;
    }
}
```

```

int length = matrix.length;

for (int i = 0; i < length / 2; i++) {
    for (int j = 0; j < (length + 1) / 2; j++){
        int tmp = matrix[i][j];
        matrix[i][j] = matrix[length - j - 1][i];
        matrix[length - j - 1][i] = matrix[length - i - 1][length - j - 1];
        matrix[length - i - 1][length - j - 1] = matrix[j][length - i - 1];
        matrix[j][length - i - 1] = tmp;
    }
}
}

```

6.14 二进制 / 位运算



优点：

特定情况下，计算方便，速度快，被支持面广

如果用算数方法，速度慢，逻辑复杂

位运算不限于一种语言，它是计算机的基本运算方法

6.14.1 知识点预热

(一) 按位与&

两位 全为1，结果才为1

$0 \& 0 = 0$; $0 \& 1 = 0$; $1 \& 0 = 0$; $1 \& 1 = 1$

例如：51&5 即 $0011\ 0011 \& 0000\ 0101 = 0000\ 0001$ 因此51&5=1.

特殊用法

(1) 清零。如果想将一个单元清零，即使其全部二进制位为0，只要与一个各位都是零的数值相与，结果为零。

(2) 取一个数中 指定位。

例如：设 $X=10101110$ ，取X的 低四位，用 $x \& 0000\ 1111 = 0000\ 1110$ 即可得到。

方法：找一个数，对应x要取的位，该数的对应位为1，其余位为零，此数与x进行“与运算”可以得到x中的指定位。

(二) 按位或 |

只要有一个为1，结果就为1。

$0|0=0$; $0|1=1$; $1|0=1$; $1|1=1$;

例如: $51|5$ 即 $0011\ 0011 | 0000\ 0101 = 0011\ 0111$ 因此 $51|5=55$

特殊用法

常用来对一个数据的某些位置1。

方法: 找到一个数, 对应x要置1的位, 该数的对应位为1, 其余位为零。此数与x相或可使x中的某些位置1。

(三) 异或 ^

两个相应位为“异”(值不同), 则该位结果为1, 否则为0

$0^0=0$; $0^1=1$; $1^0=1$; $1^1=0$;

例如: 51^5 即 $0011\ 0011 ^ 0000\ 0101 = 0011\ 0110$ 因此 $51^5=54$

特殊用法

(1) 与1相异或, 使特定位 翻转

方法: 找一个数, 对应X要翻转的位, 该数的对应为1, 其余位为零, 此数与X对应位异或即可。

例如: $X=1010\ 1110$, 使X低四位翻转, 用 $X^0000\ 1111=1010\ 0001$ 即可得到。

(2) 与0相异或, 保留 原值

例如: $X^0000\ 0000 = 1010\ 1110$

(3) 两个变量 交换值

1.借助第三个变量来实现

$C=A; A=B; B=C;$

2.利用加减法实现两个变量的交换

$A=A+B; B=A-B; A=A-B;$

3.用位异或运算来实现, 也是效率最高的

原理: 一个数异或本身等于0; 异或运算符合交换律

```
A=A^B; `B=A^B; `A=A^B
```

(四) 取反与运算~

对一个二进制数按位取反, 即将0变为1, 1变0

```
~1=0 ; ~0=1
```

(五) 左移<<

将一个运算对象的各二进制位全部 左移若干位 (左边的二进制位丢弃, 右边补0)

例如: $2<<1 = 4$ $10<<1=100$

若左移时舍弃的高位 不包含1, 则每 左移一位, 相当于该数 乘以2。

例如:

11(1011)<<2= 0010 1100=22

11(00000000 00000000 00000000 1011)整形32bit

(六) 右移>>

将一个数的各二进制位全部 右移 若干位，正数 左补0，负数 左补1，右边丢弃。若右移时舍 高位不是1（即不是负数），操作数每 右移 一位，相当于该数 除以2。

左补0还是补1得看被移数是正还是负。

例如：4>>2=4/2/2=1

-14（即1111 0010）>>2 =1111 1100=-4

(七) 无符号右移运算>>>

各个位向 右移指定的位数，右移后左边 空出的位用零 来填充，移除 右边的位被丢弃。

例如：-14>>>2

（即 11111111 11111111 11111111 11110010）>>>2

=(00111111 11111111 11111111 11111100)=1073741820

运算符	运算	示例
&	与运算	6 & 3 = 2
	或运算	6 3 = 7
^	异或运算	6 ^ 3 = 5
~	反码	~6 = -7
<<	左移	3 <<2 = 12 3*2*2=12
>>	右移	3 >>1 = 1 3/2=1
>>>	无符号右移	3 >>>1 = 1 3/2=1

6.14.2 只出现一次的数字

给出 $2 * n + 1$ 个数字，除其中一个数字之外其他每个数字均出现两次，找到这个数字。

异或运算具有很好的性质，相同数字异或运算后为0，并且具有交换律和结合律，故将所有数字异或运算后即可得到只出现一次的数字。

示例：

输入：[4,1,2,1,2]

输出：4

6.14.2.1 解题思路

如果我们对 0 和二进制位做 XOR 运算，得到的仍然是这个二进制位

$a \oplus 0 = a$ $a \oplus 0 = a$

如果我们对相同的二进制位做 XOR 运算，返回的结果是 0

$a \oplus a = 0$

XOR 满足交换律和结合律

$a \oplus b \oplus a = (a \oplus a) \oplus b = 0 \oplus b = b$

所以我们只需要将所有的数进行 XOR 操作，得到那个唯一的数字。

```
public int singleNumber(int[] A) {
    if(A == null || A.length == 0) {
        return -1;
    }
    int rst = 0;
    for (int i = 0; i < A.length; i++) {
        rst ^= A[i];
    }
    return rst;
}
```

6.14.2.2 复杂度分析

时间复杂度： $O(n)$ 。我们只需要将 nums 中的元素遍历一遍，所以时间复杂度就是 nums 中的元素个数。空间复杂度： $O(1)$ 。

6.14.3 格雷编码

格雷编码是一个二进制数字系统，在该系统中，两个连续的数值仅有一个二进制的差异。给定一个非负整数 n ，表示该代码中所有二进制的总数，请找出其格雷编码顺序。一个格雷编码顺序必须以 0 开始，并覆盖所有的 2^n 个整数。例子——输入：2；输出：[0, 1, 3, 2]；解释：0 - 00，1 - 01，3 - 11，2 - 10

6.14.3.1 解题思路

格雷码生成公式： $G(i) = i \oplus (i \gg 1)$

```
public ArrayList<Integer> grayCode(int n) {
    ArrayList<Integer> result = new ArrayList<Integer>();
    for (int i = 0; i < (1 << n); i++) {
        result.add(i ^ (i >> 1));
    }
    return result;
}
```

6.15 其他



6.15.1 两个大文件中找出共同记录

6.15.1.1 解题思路

- step1: 遍历文件a, 对每个url求取 $\text{hash}(\text{url})\%1000$, 然后根据所取得的值将url分别存储到1000个小文件(记为a0,a1,...,a999, 每个小文件约300M), 为什么是1000? 主要根据内存大小和要分治的文件大小来计算, 我们就大致可以把320G大小分为1000份, 每份大约300M (当然, 到底能不能分布尽量均匀, 得看hash函数的设计)
- step2: 遍历文件b, 采取和a相同的方式将url分别存储到1000个小文件(记为b0,b1,...,b999) (为什么要这样做? 文件a的hash映射和文件b的hash映射函数要保持一致, 这样的话相同的url就会保存在对应的小文件中, 比如, 如果a中有一个url记录data1被hash到了a99文件中, 那么如果b中也有相同url, 则一定被hash到了b99中) 所以现在问题转换成了: 找出1000对小文件中每一对相同的url (不对应的小文件不可能有相同的url)
- step3: 因为每个hash大约300M, 所以我们再可以采用 (1) 中的想法

6.15.2 ip地址与int类型的转换

6.15.2.1 解题思路

- ip地址转int类型, 例如ip为“192.168.1.116”, 相当于“.”将ip地址分为了4部分, 各部分对应的权值为 $256^3, 256^2, 256, 1$, 相成即可
- int类型转ip地址, 思路类似, 除以权值即可, 但是有部分字符串的操作

6.15.3 整数反转

将一个整数中的数字进行 **颠倒**, 当颠倒后的整数 **溢出**时, 返回 0 (标记为 32 位整数)。

示例:

输入: -123
输出: -321

6.15.3.1 解题思路

利用 **除 10 取余** 的方法, 将最低位和最高 **倒序**输出 即可

```
public int reverseInteger(int n) {
    int reversed_n = 0;

    while (n != 0) {
        int temp = reversed_n * 10 + n % 10;
        n = n / 10;
        if (temp / 10 != reversed_n) {
            reversed_n = 0;
            break;
        }
        reversed_n = temp;
    }
    return reversed_n;
}
```

6.15.4 LRU缓存策略

运用你所掌握的数据结构，设计和实现一个 LRU (最近最少使用) 缓存机制。它应该支持以下操作：获取数据 `get` 和 写入数据 `put`。

获取数据 `get(key)` - 如果密钥 (`key`) 存在于缓存中，则获取密钥的值（总是正数），否则返回 `-1`。
写入数据 `put(key, value)` - 如果密钥不存在，则写入其数据值。当缓存容量达到上限时，它应该在写入新数据之前删除最近最少使用的数据值，从而为新的数据值留出空间。

示例:

```
LRUCache cache = new LRUCache( 2 /* 缓存容量 */ );

cache.put(1, 1);
cache.put(2, 2);
cache.get(1);      // 返回 1
cache.put(3, 3);    // 该操作会使得密钥 2 作废
cache.get(2);      // 返回 -1 (未找到)
cache.put(4, 4);    // 该操作会使得密钥 1 作废
cache.get(1);      // 返回 -1 (未找到)
cache.get(3);      // 返回 3
cache.get(4);      // 返回 4
```

6.15.4.1 解题思路

解法一：

自定义数据结构：

- 实现一个链表用于记录缓存，并处理调用使用频率
- 定义一个 `HashMap` 用于记录缓存内容

```
public class LRUCache {
    private class Node{
        Node prev;
        Node next;
        int key;
        int value;

        public Node(int key, int value) {
            this.key = key;
            this.value = value;
            this.prev = null;
            this.next = null;
        }
    }

    private int capacity;
    private HashMap<Integer, Node> hs = new HashMap<Integer, Node>();
    private Node head = new Node(-1, -1); // 头
    private Node tail = new Node(-1, -1); // 尾

    public LRUCache(int capacity) {
        this.capacity = capacity;
        tail.prev = head;
        head.next = tail;
    }
}
```



```

    }

    public int get(int key) {
        if( !hs.containsKey(key)) {           //key找不到
            return -1;
        }

        // remove current
        Node current = hs.get(key);
        current.prev.next = current.next;
        current.next.prev = current.prev;

        // move current to tail
        move_to_tail(current);                //每次get，使用次数+1，最近使用，放于尾部

        return hs.get(key).value;
    }

    public void set(int key, int value) {      //数据放入缓存
        // get 这个方法会把key挪到最末端，因此，不需要再调用 move_to_tail
        if (get(key) != -1) {
            hs.get(key).value = value;
            return;
        }

        if (hs.size() == capacity) {          //超出缓存上限
            hs.remove(head.next.key);          //删除头部数据
            head.next = head.next.next;
            head.next.prev = head;
        }

        Node insert = new Node(key, value);    //新建节点
        hs.put(key, insert);
        move_to_tail(insert);                  //放于尾部
    }

    private void move_to_tail(Node current) {  //移动数据至尾部
        current.prev = tail.prev;
        tail.prev = current;
        current.prev.next = current;
        current.next = tail;
    }
}

```

解法二：

题目要求实现 LRU 缓存机制，需要在 $O(1)$ 时间内完成如下操作：

- 获取键 / 检查键是否存在
- 设置键
- 删除最先插入的键
- 前两个操作可以用标准的哈希表在 $O(1)$ 时间内完成。

有一种叫做 有序字典 的数据结构，综合了 哈希表 和 链表，在 Java 中为 LinkedHashMap。

下面用这个数据结构来实现。

```

class LRUCache extends LinkedHashMap<Integer, Integer>{

```

```

private int capacity;

public LRUCache(int capacity) {
    super(capacity, 0.75F, true);
    this.capacity = capacity;
}

public int get(int key) {
    return super.getOrDefault(key, -1);
}

public void put(int key, int value) {
    super.put(key, value);
}

@Override
protected boolean removeEldestEntry(Map.Entry<Integer, Integer> eldest) {
    return size() > capacity;
}
}

```

6.15.4.2 复杂度分析

- 时间复杂度：对于 put 和 get 操作复杂度是 $O(1)$ ，因为有序字典中的所有操作：
- `get/in/set/move_to_end/popitem` (`get/containskey/put/remove`) 都可以在常数时间内完成。空间复杂度： $O(\text{capacity})$ ，因为空间只用于有序字典存储最多 `capacity + 1` 个元素。

第七章 Android面试之Flutter相关面试题全解析

Dart部分

*1. Dart 语言的特性? *

- Productive (生产力高, Dart的语法清晰明了, 工具简单但功能强大)
- Fast (执行速度快, Dart提供提前优化编译, 以在移动设备和Web上获得可预测的高性能和快速启动。)
- Portable (易于移植, Dart可编译成ARM和X86代码, 这样Dart移动应用程序可以在iOS、Android和其他地方运行)
- Approachable (容易上手, 充分吸收了高级语言特性, 如果你已经知道C++, C语言, 或者Java, 你可以在短短几天内用Dart来开发)
- Reactive (响应式编程)

*2.Dart的一些重要概念? *

- 在Dart中, 一切都是对象, 所有的对象都是继承自Object
- Dart是强类型语言, 但可以用var或 dynamic来声明一个变量, Dart会自动推断其数据类型,dynamic类似c#
- 没有赋初值的变量都会有默认值null
- Dart支持顶层方法, 如main方法, 可以在方法内部创建方法
- Dart支持顶层变量, 也支持类变量或对象变量

- Dart没有public protected private等关键字，如果某个变量以下划线（_）开头，代表这个变量在库中是私有的

3.Dart 当中的「..」表示什么意思？

- Dart 当中的「..」意思是「级联操作符」，为了方便配置而使用。
- 「..」和「.」不同的是 调用「..」后返回的相当于是 this，而「.」返回的则是该方法返回的值。

· 2. Dart 的作用域

- Dart 没有「public」「private」等关键字，默认就是公开的，私有变量使用下划线 _ 开头。

· 3. Dart 是不是单线程模型？是如何运行的？

- Dart 是单线程模型，如何运行的看这张图：

.

- 引用《Flutter中文网》里的话：

- Dart 在单线程中是以消息循环机制来运行的，其中包含两个任务队列，一个是“微任务队列” **microtask queue**，另一个叫做“事件队列” **event queue**。

- 入口函数 main() 执行完后，消息循环机制便启动了。首先会按照先进先出的顺序逐个执行微任务队列中的任务，当所有微任务队列执行完后便开始执行事件队列中的任务，事件任务执行完毕后再去执行微任务，如此循环往复，生生不息。

4. Dart 多任务如何并行的？

- 刚才也说了，既然 Dart 不存在多线程，那如何进行多任务并行？
- Dart 当中提供了一个 **类似于新线程，但是不共享内存的独立运行的 worker - isolate**。
- 那他们是如何交互的？
- 这里引用 [flutter入门之dart中的并发编程、异步和事件驱动详解](#) 中的一部分答案：

.

- 在dart中，一个Isolate对象其实就是一个isolate执行环境的引用，一般来说我们都是通过当前的isolate去控制其他的isolate完成彼此之间的交互，而当我们想要创建一个新的Isolate可以使用 **Isolate.spawn**方法获取返回的一个新的isolate对象，两个isolate之间使用SendPort相互发送消息，而isolate中也存在了一个与之对应的ReceivePort接受消息用来处理，但是我们需要注意的是，ReceivePort和SendPort在每个isolate都有一对，只有同一个isolate中的ReceivePort才能接受到当前类的SendPort发送的消息并且处理。

5.dart是值传递还是引用传递？

- dart是值传递。我们每次调用函数，传递过去的都是对象的内存地址，而不是这个对象的复制。先来看段代码

```
main(){  
  
  Test a = new Test(5);  
  
  print("a的初始值为: ${a.value}");  
  
  setValue(a);  
  
  print("修改后a的值为: ${a.value}");
```

```
}
```

```
class Test{  
    int value = 1;  
    Test(int newValue){  
        this.value = newValue;  
    }  
}
```

```
setValue(Test s){  
    print("修改value为100");  
    s.value = 100;  
}
```

输出结果为：

a的初始值为：5

修改value为100

修改后a的值为:100

从这里可以看出是值传递，如果只是复制了一个对象，然后把这个新建的对象地址传递到函数里面的话，setValue()函数中的修改是不会影响到main函数中的a的，因为二者所引用的内存地址是不一样的。有些人可能会以以下代码反驳我：

```
main(){  
    int s = 6;  
    setValue(s);  
    print(s); //输出6，而不是7  
}
```

```
class Test{  
    int value = 1;  
    Test(int newValue){  
        this.value = newValue;  
    }  
}
```

```
setValue(int s){  
    s += 1;
```

```
}
```

你看，这输出的不是6吗，在dart中一切皆为对象，如果是引用传递，那为什么是6啊。

答案是这样的，在setValue()方法中，参数s实际上和我们初始化int s = 6的s不是一个对象，只是他们现在指的是同一块内存区域，然后在setValue()中调用s += 1的时候，这块内存区域的对象执行+1操作，然后在堆(类比java)中产生了一个新的对象，s再指向这个对象。所以s参数只是把main函数中的s的内存地址复制过去了，就比如java中的：

```
public class Test {  
  
    public static void main(String[] args) {  
  
        Test a = new Test();  
  
        Test b = a;  
  
        b = new Test();  
  
    }  
}
```

我们只要记住一点，参数是把内存地址传过去了，如果对这个内存地址上的对象修改，那么其他位置的引用该内存地址的变量值也会修改。千万要记住dart中一切都是对象。

6. Dart 属于是强类型语言，但可以用var 来声明变量，Dart 会自推导出数据类型，var 实际上是编译期的‘语法糖’。dynamic 表示动态类型，被编译后，实际是一个 object 类型，在编译期间不进行任何的类型检查，而是在运行期进行类型检查。

7. Dart 中if 等语句只支持bool 类型，switch 支持String 类型。

8.Dart 中数组和 List 是一样的。

9.Dart 中，Runes 代表符号文字，是 UTF-32 编码的字符串，用于如 Runes input = new Runes('\u{1f596} \u{1f44d}');

10.Dart 支持闭包。

11.Dart 中 number 类型分为 int 和 double，没有 float 类型。

13.Dart 中级联操作符 可以方便配置逻辑，如下代码：

```
event  
  
..id = 1  
  
..type = ""  
  
..actor = "";
```

14. 说一下 Future?

· Future，字面意思「未来」，是用来处理异步的工具。

· 刚才也说过：

· Dart 在单线程中是以消息循环机制来运行的，其中包含两个任务队列，一个是“微任务队列” microtask queue，另一个叫做“事件队列” event queue。

- Future 默认情况下其实就是往「事件队列」里插入一个事件，当有空余时间的时候就去执行，当执行完毕后会回调 Future.then(v) 方法。
- 而我們也可以通过使用 Future.microtask 方法来向「微任务队列」中插入一个任务，这样就会提高他执行的效率。
- 因为在 Dart 每一个 isolate 当中，执行优先级为：Main > MicroTask > EventQueue

15. 说一下 Stream?

- Stream 和 Future 一样，都是用来处理异步的工具。
- 但是 Stream 和 Future 不同的地方是 Stream 可以接收多个异步结果，而 Future 只有一个。
- Stream 的创建可以使用 Stream.fromFuture，也可以使用 StreamController 来创建和控制。
- 还有一个注意点是：普通的 Stream 只可以有一个订阅者，如果想要多订阅的话，要使用 asBroadcastStream()。

16. 说一下 mixin?

- 关于什么是 mixin，引用 [张风捷特烈 文章](#)中的：
- 首先mixin是一个定义类的关键字。直译出来是混入，混合的意思 Dart为了支持多重继承，引入了mixin关键字，它最大的特殊处在于：mixin定义的类不能有构造方法，这样可以避免继承多个类而产生的父类构造方法冲突

16.Widget和element和RenderObject之间的关系

首先我详细说下当时的情景，面试官问我Widget和Element之间是不是一对多的关系，如果是增加一个Widget之后，这个关系又是什么。这部分还是没有很好地答案，现在只是一个猜想，如果添加了一个widget，Element树遍历后面所有的Element看类型是否发生改变，有的话再重建RenderObject。Element和Widget之间应该还是一对一的关系，因为每个Widget的context都是独一无二的。等想好了再写上去吧。

17.. widget树的root节点

还是没能理解面试官的意思。。有能够理解的同学请评论告知我一下。现在理解了，面试官的意思应该指的是runApp()方法中的那个的Widget。我当时也想说的，不过忘记这个方法名是啥了。。。

18mixin extends implement之间的关系

- 继承（关键字 extends）

Flutter中的继承是单继承，构造函数不能继承，子类重写超类的方法，要用@[override](#)
 子类调用超类的方法，要用super
 Flutter中的子类可以访问父类中的所有变量和方法，因为Flutter中没有公有、私有的区别

- 混入 mixins（关键字 with）

mixins Dart赋予的多继承特性，类似C#的多继承，而Java是不支持类的多继承
 混合的对象是类，可以混合多个

接口实现（关键字 implements）

Flutter是没有interface的，但是Flutter中的每个类都是一个隐式的接口，这个接口包含类里的所有成员变量，以及定义的方法。

当class被当做interface用时，class中的成员变量也需要在子类里重新实现。在成员变量前加 [@override](#)

这三种关系可以同时存在，但是有前后顺序：

extends -> mixins -> implements

19.Future和microtask执行顺序

MicroTask

Dart 中事件机制的实现：Main isolate 中有一个Looper，但存在两个Queue:Event Queue 和 Microtask Queue。

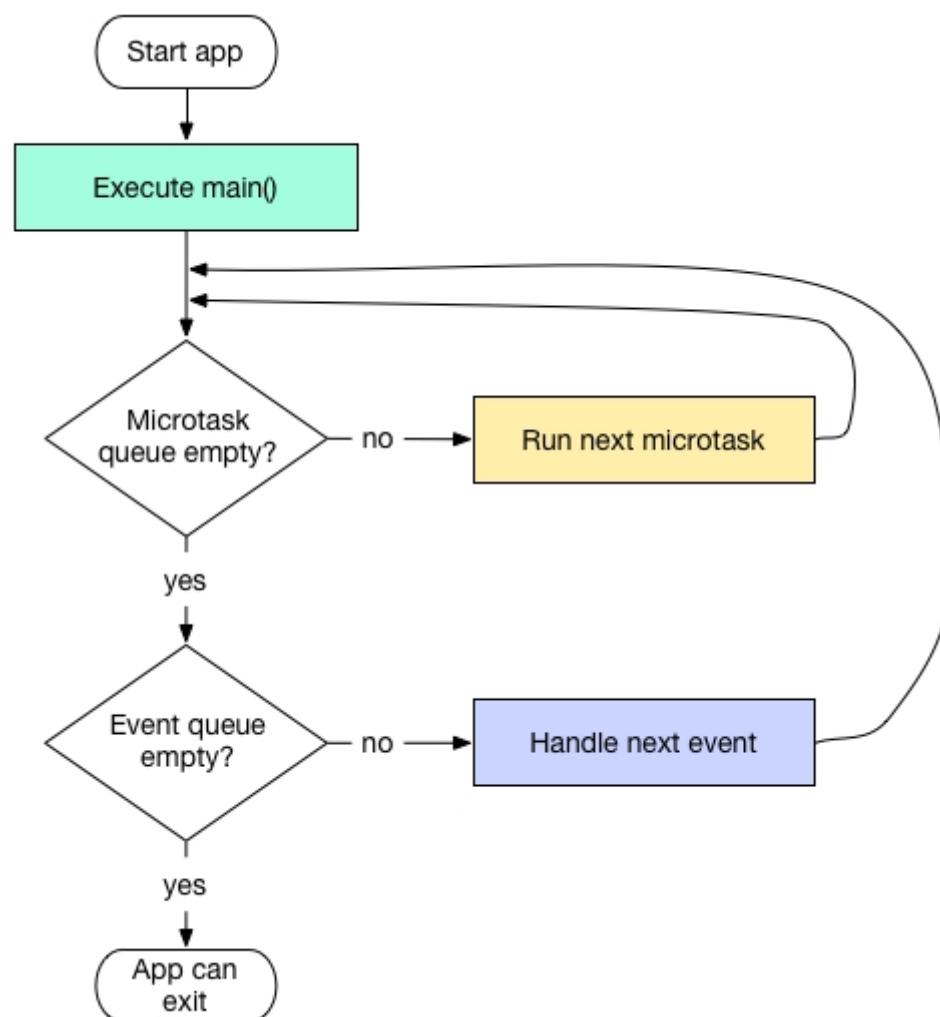
Dart 中事件的执行顺序：Main > MicroTask > EventQueue。

在 Main 中写代码将最先执行；

执行完 Main 中的代码，然后会检查并执行 Microtask Queue 中的任务，

通常使用 scheduleMicrotask 将事件添加到 MicroTask Queue 中；

最后执行 EventQueue 队列中的代码，通常使用 Future 向 EventQueue加入时间，也可以使用 async 和 await 向 EventQueue 加入事件。



Future 先进先出

Future 提供链式调用

```
new Future (() => print('拆分任务_1'))  
    .then((i) => print('拆分任务_2'))  
    .then((i) => print('拆分任务_3'))  
    .whenComplete(()=>print('任务完成'));
```

多Future 和多micTask 的执行顺序

```
void testScheduleMicrotask() {  
    scheduleMicrotask(() => print('Mission_1'));
```

//注释1

```
    new Future.delayed(new Duration(seconds: 1), () => print('Mission_2'));
```

//注释2

```
    new Future(() => print('Mission_3')).then((_) {  
        print('Mission_4');  
        scheduleMicrotask(() => print('Mission_5'));  
    }).then((_) => print('Mission_6'));
```

//注释3

```
    new Future(() => print('Mission_7'))  
        .then((_) => new Future(() => print('Mission_8')))  
        .then((_) => print('Mission_9'));
```

//注释4

```
    new Future(() => print('Mission_10'));
```

```
    scheduleMicrotask(() => print('Mission_11'));
```

```
    print('Mission_12');
```

```
}
```

输出结果

I/flutter (19025): Mission_12

I/flutter (19025): Mission_1

I/flutter (19025): Mission_11

I/flutter (19025): Mission_3

I/flutter (19025): Mission_4

I/flutter (19025): Mission_6

I/flutter (19025): Mission_5

I/flutter (19025): Mission_7

I/flutter (19025): Mission_10

I/flutter (19025): Mission_8

I/flutter (19025): Mission_9

Syncing files to device MIX 3...

I/flutter (19025): Mission_2

****19.await for的使用方式****

代码如下

```
String data;
```

```
getData() async {
```

```
    data = await http.get(Uri.encodeFull(url), headers: {"Accept": "application/json"}); //延迟执行后赋值给data
```

```
}
```

· await关键字必须在async函数内部使用

· 调用async函数必须使用await关键字

· 返回默认是一个future对象

//定义了返回结果值为String类型

```
Future getDatas(String category) async {
```

```
    var request = await _httpClient.getUrl(Uri.parse(url));
```

```
    var response = await request.close();
```

```
    return await response.transform(utf8.decoder).join();
```

```
}
```

```
run() async{
```

```
    int data = await getDatas('keji'); //因为类型不匹配，IDE会报错
```

```
}
```

Future的作用是为了链式调用

//案例3

```
funA(){
```

```
...set an important variable... //设置变量
```

```
}
```

```
funB(){
```

```
...use the important variable... //使用变量
```

```
}
```

```
main(){
```

```
new Future.then(funA()).then(funB()); // 明确表现出了后者依赖前者设置的变量值
```

```
new Future.then(funA()).then(_ {new Future(funB())}); //还可以这样用
```

//链式调用，捕获异常

```
new Future.then(funA(),onError: (e) { handleError(e); }).then(funB(),onError: (e) { handleError(e); })
```

```
}
```

20.赋值操作符

比较有意思的赋值操作符有：

AA ?? "999" ///表示如果 AA 为空，返回999

AA ??= "999" ///表示如果 AA 为空，给 AA 设置成 999

AA ~/999 ///AA 对于 999 整除

21.可选方法参数

Dart 方法可以设置 **参数默认值** 和 **指定名称**。

比如：getDetail(String userName, reposName, {branch = "master"}){} 方法，这里 branch 不设置的话，默认是“master”。参数类型 可以指定或者不指定。调用效果：getRepositoryDetailDao("aaa", "bbbb", branch: "dev");。

22.作用域

Dart **没有**关键词 public 、private 等修饰符，_ 下横向直接代表 private ，**但是有** @protected 注解。

23.构造方法

Dart 中的多构造方法，可以通过命名方法实现。

默认构造方法只能有一个，而通过 Model.empty() 方法可以创建一个空参数的类，其实方法名称随你喜欢，而变量初始化值时，只需要通过 this.name 在构造方法中指定即可：

```
class ModelA {  
  
    String name;  
  
    String tag;  
  
    //默认构造方法，赋值给name和tag  
    ModelA(this.name, this.tag);  
  
    //返回一个空的ModelA  
    ModelA.empty();  
  
    //返回一个设置了name的ModelA  
    ModelA.forName(this.name);  
}
```

24.*getter setter 重写*

Dart 中所有的基础类型、类等都继承 Object，默认值是 NULL，自带 getter 和 setter，而如果是 final 或者 const 的话，那么它只有一个 getter 方法，Object 都支持 getter、setter 重写：

```
@override  
Size get preferredSize {  
    return Size.fromHeight(kTabHeight + indicatorWeight);  
}
```

25.Assert(断言)

assert 只在检查模式有效，在开发过程中，assert(unicorn == null); 只有条件为真才正常，否则直接抛出异常，一般用在开发过程中，某些地方不应该出现什么状态的判断。

26.重写运算符，如下所示重载 operator 后对类进行 +/- 操作。

```
class Vector {  
  
    final int x, y;  
  
    Vector(this.x, this.y);  
  
    Vector operator +(Vector v) => Vector(x + v.x, y + v.y);  
    Vector operator -(Vector v) => Vector(x - v.x, y - v.y);  
  
    ...  
}
```

```
void main() {  
    final v = Vector(2, 3);  
    final w = Vector(2, 2);  
  
    assert(v + w == Vector(4, 5));  
    assert(v - w == Vector(0, 1));  
}
```

<	+		[]
>	/	^	[]=
<=	~/	&	~
>=	*	<<	==
-	%	>>	

Dart 中支持 mixins，按照出现顺序应该为 extends、mixins、implements。

Dart 中可通过 Zone 表示指定代码执行的环境，类似一个沙盒概念，在 Flutter 中 **C++** 运行 Dart 也是在 `_runMainZoned` 内执行 `runZoned` 方法启动，而我们也可以通过 Zone，在运行环境内捕获全局异常等信息：

同时你可以给 runZoned 注册方法，在需要时执行回调，如下代码所示，这样的在一个 Zone 内任何地方，只要能获取 onData 这个 ZoneUnaryCallback，就都可以调用到 handleData

[illegible]

```
}
```

```
///返回得到一个 ZoneUnaryCallback
```

```
var onData = Zone.current.registerUnaryCallback<dynamic, int>(handleData);
```

```
///执行 ZoneUnaryCallback 返回数据
```

```
Zone.current.runUnary(onData, 2);
```

异步逻辑可以通过 `scheduleMicrotask` 可以插入异步执行方法：

```
Zone.current.scheduleMicrotask((){
```

```
  //todo something
```

```
});
```

更多可参看：[《Flutter完整开发实战详解\(十一、全面深入理解Stream\)》](#)

Future

Future 简单了说就是对 Zone 的封装使用。

比如 `Future.microtask` 中主要是执行了 Zone 的 `scheduleMicrotask`，而 `result._complete` 最后调用的是 `_zone.runUnary` 等等。

```
factory Future.microtask(FutureOr computation()) {
```

```
  _Future result = new _Future();
```

```
  scheduleMicrotask(() {
```

```
    try {
```

```
      result._complete(computation());
```

```
    } catch (e, s) {
```

```
      _completeWithErrorCallback(result, e, s);
```

```
    }
```

```
  });
```

```
  return result;
```

```
}
```

Dart 中可通过 `async/await` 或者 `Future` 定义异步操作，而事实上 `async/await` 也只是语法糖，最终还是通过编译器转为 `Future`。

有兴趣看这里：

[generators](#)

[code_generator.dart](#)

[Flutter完整开发实战详解\(十一、全面深入理解Stream\)](#)

Stream

Stream 也是有对Zone 的另外一种封装使用。

Dart 中另外一种异步操作， async* / yield 或者 Stream 可定义 Stream 异步， async* / yield 也只是语法糖，最终还是通过编译器转为 Stream。Stream 还支持同步操作。

1)、Stream 中主要有 Stream、StreamController、StreamSink 和 StreamSubscription 四个关键对象，大致可以总结为：

StreamController：如类名描述，用于整个 Stream 过程的控制，提供各类接口用于创建各种事件流。

StreamSink：一般作为事件的入口，提供如 add， addStream 等。

Stream：事件源本身，一般可用于监听事件或者对事件进行转换，如 listen、where。

StreamSubscription：事件订阅后的对象，表面上用于管理订阅过等各类操作，如 cancel、pause，同时在内部也是事件的中转关键。

2)、**一般通过 StreamController 创建 Stream；通过 StreamSink 添加事件；通过 Stream 监听事件；通过 StreamSubscription 管理订阅。**

3)、Stream 中支持各种变化，比如map、expand、where、take 等操作，同时支持转换为 Future。

更多可参看：[《Flutter完整开发实战详解\(十一、全面深入理解Stream\)》](#)

Flutter 部分

***1. Flutter 是什么？ ***

Flutter是谷歌的移动UI框架，可以快速在iOS和Android上构建高质量的原生用户界面。Flutter可以与现有的代码一起工作。在全世界，Flutter正在被越来越多的开发者和组织使用，并且Flutter是完全免费、开源的。

***2. Flutter 特性有哪些？ ***

快速开发（毫秒级热重载）

· 绚丽UI（内建漂亮的质感设计Material Design和Cupertino Widget和丰富平滑的动画效果和平台感知）

· 响应式(Reactive，用强大而灵活的API解决2D、动画、手势、效果等难题)

- 原生访问功能
- 堪比原生性能

3.基础知识

Flutter 和 React Native 不同主要在于 **Flutter UI是直接通过 skia 渲染的，而 React Native 是将 js 中的控件转化为原生控件，通过原生去渲染的**，相关更多可查看：[《移动端跨平台开发的深度解析》](#)。

Flutter 中存在 Widget、Element、RenderObject、Layer 四棵树，其中 Widget 与 Element 是一对多的关系，

Element 中持有 Widget 和 RenderObject，而 Element 与 RenderObject 是一一对应的关系（除去 Element 不存在 RenderObject 的情况，如 ComponentElement 是不具备 RenderObject），

当 RenderObject 的 isRepaintBoundary 为 true 时，那么个区域形成一个 Layer，所以不是每个 RenderObject 都具有 Layer 的，因为这受 isRepaintBoundary 的影响。

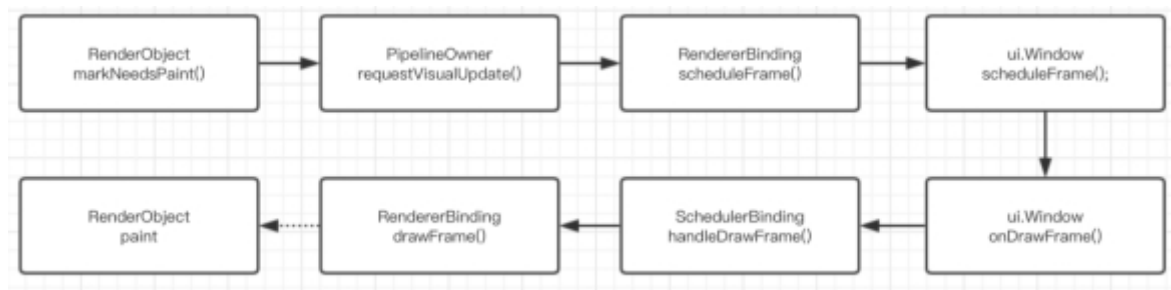
更多相关可查阅 [《Flutter完整开发实战详解\(九、深入绘制原理\)》](#)

Flutter 中 Widget 不可变，每次保持在一帧，如果发生改变是通过 State 实现跨帧状态保存，而真实完成布局和绘制数组的是 RenderObject，Element 充当两者的桥梁，State 就是保存在 Element 中。

Flutter 中的 BuildContext 只是接口，而 Element 实现了它。

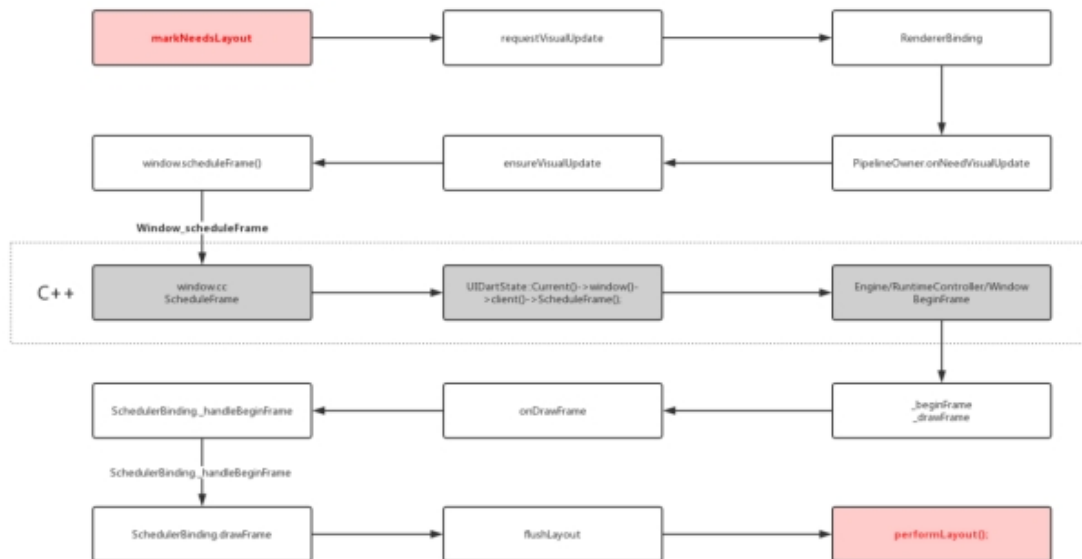
Flutter 中 setState 其实是调用了 markNeedsBuild，该方法内部标记此 Element 为 Dirty，然后在下一帧 WidgetsBinding.drawFrame 才会被绘制，这可以看出 setState 并不是立即生效的。

Flutter 中 RenderObject 在 attach/layout 之后会通过 markNeedsPaint(); 使得页面重绘，流程大概如下：



通过isRepaintBoundary 往上确定了更新区域，通过 requestVisualUpdate 方法触发更新往下绘制。

正常情况 RenderObject 的布局相关方法调用顺序是： layout ->performResize ->performLayout ->markNeedsPaint，但是用户一般不会直接调用 layout，而是**通过** markNeedsLayout，具体流程如下：



Flutter 中一般 **json** 数据从 String 转为 Object 的过程中都需要先经过 Map 类型。

Flutter 中 InheritedWidget 一般用于**状态共享**，如Theme、Localizations、MediaQuery等，都是通过它实现共享状态，这样我们可以通过 context 去获取共享的状态，比如 ThemeData theme = Theme.of(context);

在 Element 的 inheritFromWidgetOfExactType 方法实现里，有一个 Map<Type, InheritedElement> _inheritedWidgets 的对象。

_inheritedWidgets 一般情况下是空的，只有当父控件是 InheritedWidget 或者本身是 InheritedWidgets 时才会有被初始化，而当父控件是 InheritedWidget 时，这个 Map 会被一级一级往下传递与合并。

所以当我们通过 context 调用 inheritFromWidgetOfExactType 时，就可以往上查找到父控件的 Widget。

Flutter 中默认主要通过 runtimeType 和 key 判断更新：

```

static bool canUpdate(Widget oldWidget, Widget newWidget) {
return oldWidget.runtimeType == newWidget.runtimeType
&& oldWidget.key == newWidget.key;
}
}
  
```

4.*Flutter 中的生命周期*

initState() 表示当前 State 将和一个 BuildContext 产生关联，但是此时BuildContext 没有完全装载完成，如果你需要在该方法中获取 BuildContext，可以 new Future.delayed(const Duration(seconds: 0, {}//context)); 一下。

didChangeDependencies() 在 initState() 之后调用，当 State 对象的依赖关系发生变化时，该方法被调用，初始化时也会调用。

deactivate() 当 State 被暂时从视图树中移除时，会调用这个方法，同时页面切换时，也会调用。

dispose() Widget 销毁了，在调用这个方法之前，总会先调用 deactivate()。

didUpdateWidget 当 widget 状态发生变化时，会调用。

通过 StreamBuilder 和 FutureBuilder 我们可以快速使用 Stream 和 Future 快速构建我们的异步控件：
[《Flutter完整开发实战详解\(十一、全面深入理解Stream\)》](#)

Flutter 中 runApp 启动入口其实是一个 WidgetsFlutterBinding，它主要是通过 BindingBase 的子类 GestureBinding、ServicesBinding、SchedulerBinding、PaintingBinding、SemanticsBinding、RendererBinding、WidgetsBinding 等，通过 mixins 的组合而成的。

Flutter 中的 Dart 的线程是以**事件循环和消息队列**的形式存在，包含两个任务队列，一个是 **microtask 内部队列**，一个是 **event 外部队列**，而 **microtask 的优先级又高于 event**。

因为 microtask 的优先级又高于 event，同时会阻塞 event 队列，所以如果 microtask 太多就可能会对触摸、绘制等外部事件造成阻塞卡顿哦。

Flutter 中存在**四大线程**，分别为 UI Runner、GPU Runner、IO Runner，Platform Runner（**原生主线程**），同时在 Flutter 中可以通过 isolate 或者 compute 执行真正的跨线程异步操作。

5.*PlatformView*

Flutter 中通过 PlatformView 可以嵌套原生 View 到 Flutter UI 中，这里面其实是使用了 Presentation + VirtualDisplay + Surface 等实现的，大致原理就是：

使用了类似副屏显示的技术，VirtualDisplay 类代表一个虚拟显示器，调用 DisplayManager 的 createVirtualDisplay() 方法，将虚拟显示器的内容渲染在一个 Surface 控件上，然后将 Surface 的 id 通知给 Dart，让 engine 绘制时，在内存中找到对应的 Surface 画面内存数据，然后绘制出来。em...
实时控件截图渲染显示技术。

Flutter 的 Debug 下是 JIT 模式，release 下是 AOT 模式。

Flutter 中可以通过 mixins AutomaticKeepAliveClientMixin，然后重写 wantKeepAlive 保持住页面，记得在被保持住的页面 build 中调用 super.build。（因为 mixins 特性）。

Flutter 手势事件主要是通过竞技判断的：

主要有 hitTest 把所有需要处理的控件对应的 RenderObject，从 child 到 parent 全部组合成列表，从最里面一直添加到最外层。

然后从队列头的 child 开始 for 循环执行 handleEvent 方法，执行 handleEvent 的过程不会被拦截打断。

一般情况下 Down 事件不会决出胜利者，大部分时候是在 MOVE 或者 UP 的时候才会决出胜利者。

竞技场关闭时只有一个的就直接胜出响应，没有胜利者就拿排在队列第一个强制胜利响应。

同时还有 didExceedDeadline 处理按住时的 Down 事件额外处理，同时手势处理一般在 GestureRecognizer 的子类进行。

更多详细请查看：[《Flutter完整开发实战详解\(十三、全面深入触摸和滑动原理\)》](#)

Flutter 中 ListView 滑动其实都是通过改变 ViewPort 中的 child 布局来实现显示的。

常用状态管理的：目前有 scope_model、flutter_redux、fish_redux、bloc + Stream 等几种模式，具体可见：[《Flutter完整开发实战详解\(十二、全面深入理解状态管理设计\)》](#)

6.*Platform Channel*

Flutter 中可以通过 Platform Channel 让 Dart 代码和原生代码通信的：

- BasicMessageChannel：用于传递字符串和半结构化的信息。
- MethodChannel：用于传递方法调用（method invocation）。
- EventChannel：用于数据流（event streams）的通信。

同时 Platform Channel **并非线程安全的**，更多详细可查阅闲鱼技术的[《深入理解Flutter Platform Channel》](#)

其中基础数据类型映射如下：

7.*Android 启动页

Android 中 Flutter 默认启动时会在 FlutterActivityDelegate.java 中读取 AndroidManifest.xml 内 meta-data 标签，其中 io.flutter.app.android.SplashScreenUntilFirstFrame 标志位如果为 true，就会启动 Splash 画面效果（类似iOS的启动页面）。

启动时原生代码会读取 android.R.attr.windowBackground 得到指定的 Drawable，用于显示启动闪屏效果，之后并且通过 flutterView.addFirstFrameListener，在onFirstFrame 中移除闪屏。

8. Flutter 和 Dart的关系是什么？

Flutter是一个使用Dart语言开发的跨平台移动UI框架，通过自建绘制引擎，能高性能、高保真地进行移动开发。Dart囊括了多数编程语言的优点，它更符合Flutter构建界面的方式。

9. Widget 和 element 和 RenderObject 之间的关系？

- Widget是用户界面的一部分,并且是不可变的。
- Element是在树中特定位置Widget的实例。
- RenderObject是渲染树中的一个对象，它的层次结构是渲染库的核心。

Widget会被inflate（填充）到Element，并由Element管理底层渲染树。Widget并不会直接管理状态及渲染,而是通过State这个对象来管理状态。Flutter创建Element的可见树，相对于Widget来说，是可变的，通常界面开发中，我们不用直接操作Element,而是由框架层实现内部逻辑。就如一个UI视图树中，可能包含有多个TextWidget(Widget被使用多次)，但是放在内部视图树的视角，这些TextWidget都是填充到一个个独立的Element中。Element会持有renderObject和widget的实例。记住，Widget 只是一个配置，RenderObject 负责管理布局、绘制等操作。

在第一次创建 Widget 的时候，会对应创建一个 Element，然后将该元素插入树中。如果之后 Widget 发生了变化，则将其与旧的 Widget 进行比较，并且相应地更新 Element。重要的是，Element 不会被重建，只是更新而已。

10. mixin extends implement 之间的关系?

继承（关键字 extends）、混入 mixins（关键字 with）、接口实现（关键字 implements）。这三者可以同时存在，前后顺序是 extends -> mixins -> implements。

Flutter中的继承是单继承，子类重写超类的方法要用@Override，子类调用超类的方法要用super。

在Flutter中，Mixins是一种在多个类层次结构中复用类代码的方法。mixins的对象是类，mixins绝不是继承，也不是接口，而是一种全新的特性，可以mixins多个类，mixins的使用需要满足一定条件。

***11. 使用mixins的条件是什么? ***

因为mixins使用的条件，随着Dart版本一直在变，这里讲的是Dart2.1中使用mixins的条件：

mixins类只能继承自object

mixins类不能有构造函数

一个类可以mixins多个mixins类

可以mixins多个类，不破坏Flutter的单继承

***12. mixin 怎么指定异常类型? ***

on关键字可用于指定异常类型。on只能用于被mixins标记的类，例如mixins X on A，意思是要mixins X的话，得先接口实现或者继承A。这里A可以是类，也可以是接口，但是在mixins的时候用法有区别。

on 一个类：



```
class A {  
  void a(){  
    print("a");  
  }  
}
```

```
mixin X on A{  
  void x(){  
    print("x");  
  }  
}
```

```
class mixinsX extends A with X{  
}
```



普通代码都是同步执行的，结束后会开始检查microtask中是否有任务，若有则执行，执行完继续检查microtask，直到microtask排队为空。最后会去执行event队列（future）。

future是异步编程，调用本身立即返回，并在稍后的某个时候执行完成时再获得返回结果。在普通代码中可以使用await 等待一个异步调用结束。

*15. Stream 与Future是什么关系? *

Stream 和 Future 是 Dart 异步处理的核心 API。Future 表示稍后获得的一个数据，所有异步的操作的返回值都用 Future 来表示。但是 Future 只能表示一次异步获得的数据。而 Stream 表示多次异步获得的数据。比如界面上的按钮可能会被用户点击多次，所以按钮上的点击事件 (onClick) 就是一个 Stream。简单地说，Future 将返回一个值，而 Stream 将返回多次值。Dart 中统一使用 Stream 处理异步事件流。Stream 和一般的集合类似，都是一组数据，只不过一个是异步推送，一个是同步拉取。

*16. Stream 两种订阅模式? *

Stream有两种订阅模式：**单订阅(single)**和**多订阅 (broadcast)**。单订阅就是只能有一个订阅者，而广播是可以有多个订阅者。这就有点类似于消息服务（Message Service）的处理模式。单订阅类似于点对点，在订阅者出现之前会持有数据，在订阅者出现之后就才转交给它。而广播类似于发布订阅模式，可以同时有多个订阅者，当有数据时就会传递给所有的订阅者，而不管当前是否已有订阅者存在。Stream 默认处于单订阅模式，所以同一个 stream 上的 listen 和其它大多数方法只能调用一次，调用第二次就会报错。但 Stream 可以通过 transform() 方法（返回另一个 Stream）进行连续调用。通过 Stream.asBroadcastStream() 可以将一个单订阅模式的 Stream 转换成一个多订阅模式的 Stream，isBroadcast 属性可以判断当前 Stream 所处的模式。

17. await for 如何使用?

await for是不断获取stream流中的数据，然后执行循环体中的操作。它一般用在直到stream什么时候完成，并且必须等待传递完成之后才能使用，不然就会一直阻塞。



```
Stream stream = new Stream.fromIterable(['不开心', '面试', '没', '过']);
```

```
main() async{  
  
  print('上午被开水烫了脚');  
  
  await for(String s in stream){  
  
    print(s);  
  
  }  
  
  print('晚上还没吃饭');  
  
}
```



*18. Flutter中的Widget、State、Context 的核心概念? 是为了解决什么问题? *

Widget: 在Flutter中，几乎所有东西都是Widget。将一个Widget想象为一个可视化的组件（或与应用可视化方面交互的组件），当你需要构建与布局直接或间接相关的任何内容时，你正在使用Widget。

Widget树: Widget以树结构进行组织。包含其他Widget的widget被称为父Widget(或widget容器)。包含在父widget中的widget被称为子Widget。

Context: 仅仅是已创建的所有Widget树结构中的某个Widget的位置引用。简而言之，将context作为widget树的一部分，其中context所对应的widget被添加到此树中。一个context只从属于一个widget，它和widget一样是链接在一起的，并且会形成一个context树。

State: 定义了StatefulWidget实例的行为，它包含了用于“交互/干预”Widget信息的行为和布局。应用于State的任何更改都会强制重建Widget。

这些状态的引入，主要是为了解决多个部件之间的交互和部件自身状态的维护。

*19. Widget的两种类型是什么? *

StatelessWidget: 一旦创建就不关心任何变化，在下次构建之前都不会改变。它们除了依赖于自身的配置信息（在父节点构建时提供）外不再依赖于任何其他信息。比如典型的Text、Row、Column、Container等，都是StatelessWidget。它的生命周期相当简单：初始化、通过build()渲染。

StatefulWidget: 在生命周期内，该类Widget所持有的数据可能会发生变化，这样的数据被称为**State**，这些拥有动态内部数据的Widget被称为StatefulWidget。比如复选框、Button等。State会与Context相关联，并且此关联是永久性的，State对象将永远不会改变其Context，即使可以在树结构周围移动，也仍将与该context相关联。当**state与context关联时**，state被视为**已挂载**。StatefulWidget由两部分组成，在初始化时必须要在createState()时初始化一个与之相关的State对象。

***20. State 对象的初始化流程? ***

initState(): 一旦State对象被创建，initState方法是第一个（构造函数之后）被调用的方法。可通过重写来执行额外的初始化，如初始化动画、控制器等。重写该方法时，应该首先调用super.initState()。在initState中，无法真正使用context，因为框架还没有完全将其与state关联。initState在该State对象的生命周期内将不会再次调用。

didChangeDependencies(): 这是第二个被调用的方法。在这一阶段，context已经可用。如果你的Widget链接到了一个InheritedWidget并且/或者你需要初始化一些listeners（基于context），通常会重写该方法。

build(BuildContext context): 此方法在didChangeDependencies()、didUpdateWidget()之后被调用。每次State对象更新（或当InheritedWidget有新的通知时）都会调用该方法！我们一般都在build中来编写真正的功能代码。为了强制重建，可以在需要的时候调用setState((){...})方法。

dispose(): 此方法在Widget被废弃时调用。可重写该方法来执行一些清理操作（如解除listeners），并在此之后立即调用super.dispose()。

***21. Widget 唯一标识Key有那几种? ***

在flutter中，每个widget都是被唯一标识的。这个唯一标识在build或rendering阶段由框架定义。该标识对应于可选的Key参数，如果省略，Flutter将会自动生成一个。

在flutter中，主要有4种类型的Key：GlobalKey（确保生成的Key在整个应用中唯一，是很昂贵的，允许element在树周围移动或变更父节点而不会丢失状态）、LocalKey、UniqueKey、ObjectKey。

***22. 什么是Navigator? MaterialApp做了什么? ***

Navigator是在Flutter中负责管理维护页面堆栈的导航器。MaterialApp在需要的时候，会自动为我们创建Navigator。Navigator.of(context)，会使用context来向上遍历Element树，找到MaterialApp提供的_NavigatorState再调用其push/pop方法完成导航操作。

***23. **flutter与React Native有什么不同? ***

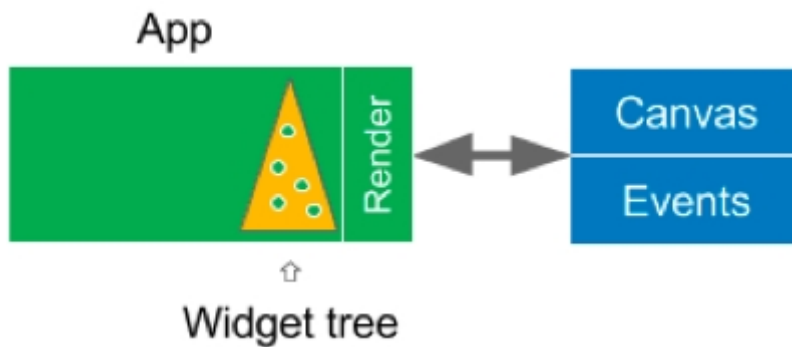
React Native利用JavaScript桥将其小部件转换为OEM小部件。而且由于它不断地进行这种转换（比较和更新周期），因此会产生瓶颈并导致性能下降。

虽然仍然使用反应式视图的优势，但Flutter并没有使用这种桥将其自己的小部件转换为OEM小部件。除了快速和流畅的UI性能和可预测性之外，作为此项的另一个优势，您在Android KitKat设备上看到的内容与您在Android Pie上获得的内容相同。这种兼容性是显而易见的，因为Flutter不使用OEM小部件，并且不受不同Android版本之间的UI / UX更改的影响。

24. **为什么说flutter是原生的

[Flutter应用程序与平台的交互](#)

Flutter使用名为Skia的图形引擎在应用程序端执行所有UI呈现。这意味着它不依赖于平台提供的OEM小部件。它只需要平台的画布来绘制自己的渲染。这确保了可预测性和开发人员对小部件和布局的完全控制。



Flutter内部小部件树

除此之外，Flutter将其结构保持为小部件树。顺便说一下，Flutter中的几乎所有东西都是一个小部件，它使您能够在小部件内部的小部件结构中构建您的应用程序。此内部树结构允许Skia仅呈现需要更新的小部件，并从缓存中检索未更改的甚至移动的小部件。

25. 讲一下flutter的几个特点/优缺点

Dart是用于开发Flutter应用程序的面向对象，垃圾收集的编程语言。它也是由谷歌创建的，但它是开源的，因此它在Google内外都有社区。

除了Google的起源之外，Dart还被选为Flutter的语言，原因如下：它是极少数可以同时编译AOT（提前）和JIT（即时）的语言之一。

在应用程序开发过程中使用JIT编译，因为它可以通过动态编译代码来实现热重新加载（我将在下一个问题中详细讨论）和快速开发周期。

完成开发并准备发布后，将使用AOT编译。然后将代码AOT编译为本机代码，从而实现应用程序的快速启动和高性能执行。

就个人而言，我对Dart的经验是，如果您是熟悉Java或类似语言的开发人员，只需要几天的时间就可以习惯它。因此，如果您是Android开发人员，那么这种语言的学习曲线应该非常低。

凭借其干净但灵活的语法，Dart可以被识别为仅包含任何高级编程语言中最需要的功能的语言。

26. 什么是ScopedModel / BLoC模式？

ScopedModel和BLoC（业务逻辑组件）是常见的Flutter应用程序架构模式，可帮助将业务逻辑与UI代码分离，并使用更少的状态窗口小部件

*27. **什么是stateWidget和statelessWidget？*

内边距margin 和外边距边距padding

```
body: Center(
  child: Container(
    child: new Text("hello zzl ",
    style: TextStyle(
      fontSize: 40.0,
    ),
    textAlign: TextAlign.center,
  ),
  alignment: Alignment.topCenter,
```

```
width: 500.0,  
height: 400.0,  
color: Colors.blue,  
padding: const EdgeInsets.all(50.0),//外边距  
// margin: const EdgeInsets.all(100.0),//内边距  
,  
,
```

填充控件Padding

Padding的布局分为两种情况：

当child为空的时候，会产生一个宽为left+right，高为top+bottom的区域；

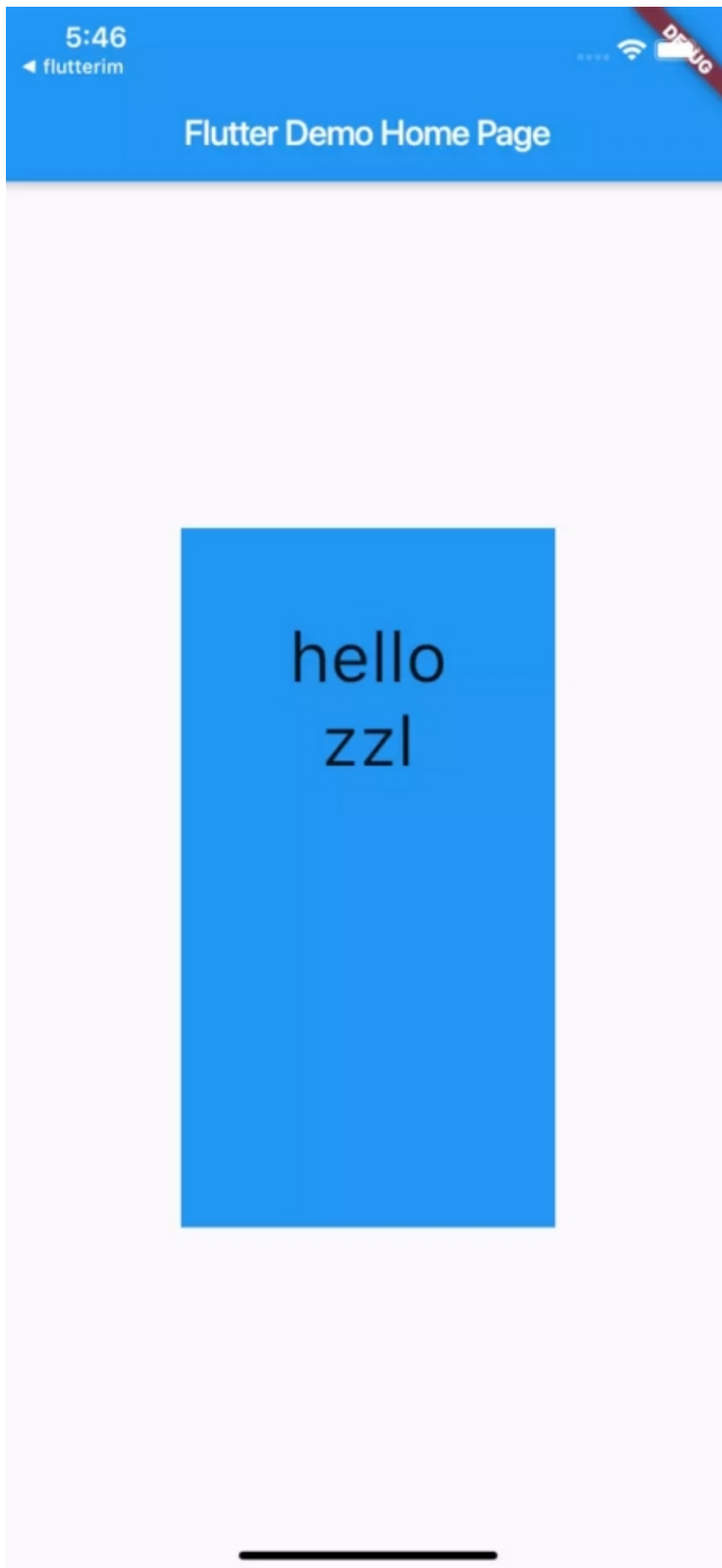
当child不为空的时候，Padding会将布局约束传递给child，根据设置的padding属性，缩小child的布局尺寸。然后Padding将自己调整到child设置了padding属性的尺寸，在child周围创建空白区域。

```
import 'package:flutter/material.dart';  
  
class PaddingDemo extends StatelessWidget{  
  @override  
  Widget build(BuildContext context) {  
    return new Scaffold(  
      appBar: new AppBar(  
        title:new Text("padding填充控件"),  
      ),  
      body: new Padding(  
        padding: const EdgeInsets.all(8.0),  
        child: new Image.asset("images/hua3.png"),  
      ),  
    );  
  }  
}  
  
void main(){  
  runApp(new MaterialApp(  
    title: "padding填充控件",  
    home: new PaddingDemo(),  
  ));
```


***28.如何在Flutter中定义边距和填充? ***

内边距margin 和外边距边距padding

```
body: Center(  
  child: Container(  
    child: new Text("hello zzl ",  
    style: TextStyle(  
      fontSize: 40.0,  
    ),  
    textAlign: TextAlign.center,  
  ),  
  alignment: Alignment.topCenter,  
  width: 500.0,  
  height: 400.0,  
  color: Colors.blue,  
  padding: const EdgeInsets.all(50.0),//外边距  
  // margin: const EdgeInsets.all(100.0),//内边距  
,  
,
```



填充控件Padding

Padding的布局分为两种情况：

当child为空的时候，会产生一个宽为left+right，高为top+bottom的区域；

当child不为空的时候，Padding会将布局约束传递给child，根据设置的padding属性，缩小child的布局尺寸。然后Padding将自己调整到child设置了padding属性的尺寸，在child周围创建空白区域。

```
import 'package:flutter/material.dart';

class PaddingDemo extends StatelessWidget{

  @override

  Widget build(BuildContext context) {

    return new Scaffold(

      appBar: new AppBar(

        title:new Text("padding填充控件"),

      ),

      body: new Padding(

        padding: const EdgeInsets.all(8.0),

        child: new Image.asset("images/hua3.png"),

      ),

    );

  }

}

void main(){

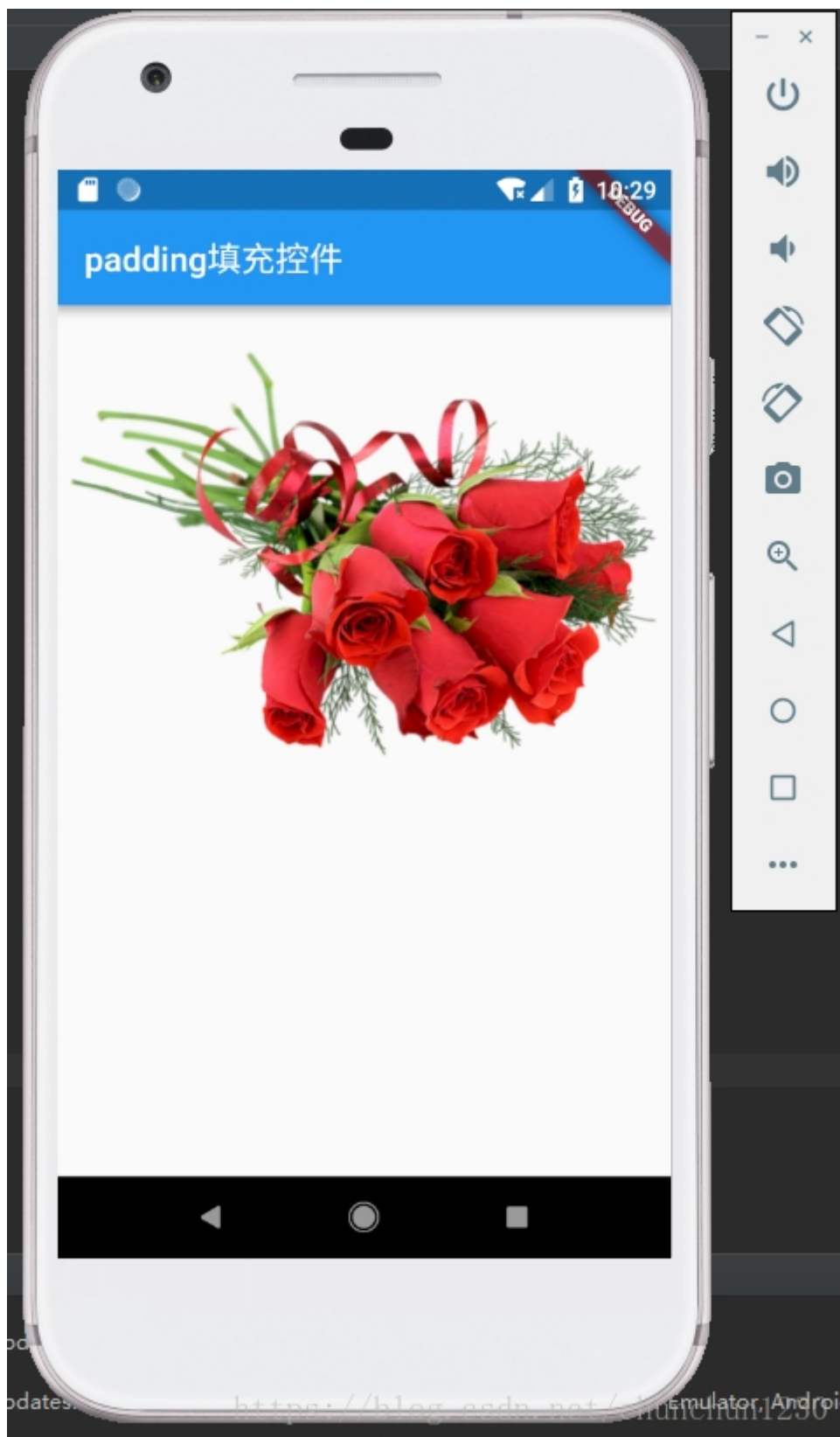
  runApp(new MaterialApp(

    title: "padding填充控件",

    home: new PaddingDemo(),

  ));

}
```



29. 谈一下flutter state的生命周期

State 的生命周期

从上面的例子中可以看到，会要求提供一个含有视图树的。

既然 能够控制一个视图的状态，那它肯定会有一系列的生命周期。

上图就是 State 的生命周期图。

\1. StatefulWidget.createState()

Framework 调用会通过调用 StatefulWidget.createState() 来创建一个 State。

\1. initState()

新创建的 State 会和一个产生关联，此时认为 State 已经被安装好了，initState() 函数将会被调用。

通常，我们可以重写这个函数，进行初始化操作。

\1. didChangeDependencies()

在 initState() 调用结束后，这个函数会被调用。

事实上，当 State 对象的依赖关系发生变化时，这个函数总会被 Framework 调用。

\1. build()

经过以上步骤，系统认为一个 State 已经准备好了，就会调用 build() 来构建视图。

我们需要在这个函数中，返回一个 Widget。

\1. deactivate()

当 State 被暂时从视图树中移除时，会调用这个函数。

页面切换时，也会调用它，因为此时 State 在视图树中的位置发生了变化，需要先暂时移除后添加。

⚠ 注意，重写的时候必须要调用 super.deactivate()。

\1. dispose()

当 State 被永久的从视图树中移除，Framework 会调用该函数。

在销毁前触发，我们可以在这里进行最终的资源释放。

在调用这个函数之前，总会先调用 deactivate()。

⚠ 注意，重写的时候必须要调用 super.dispose()。

\1. didUpdateWidget(covariant T oldWidget)

当 widget 的配置发生变化时，会调用这个函数。

比如，的时候就会调用这个函数。

这个函数调用后，会调用 build()。

\1. setState()

当我需要更新 State 的视图时，需要手动调用这个函数，它会触发 build()。

31.**Flutter和RN的对比。

32.**说一下Hot Reload，Hot Restart，热更新三者的区别和原理。

33.**Flutter是如何做到一套Dart代码可以编译运行在Android和iOS平台的？所以说具体的原理。

34.**Flutter不具备反射，如果要使用反射，你应该如何使用？说一下大概的思路。

35.**Flutter在不使用WebView和S方案的情况下。如何做到热更新？说一下大概思路。

36.**如何让Flutter 编译出来的APP的包大小尽可能的变小？

37.**我们这个项目是一个综合系统的老项目，里面有Android，iOS，还有Web代码，是一个混合开发的项目，现在需要迁移到Flutter，加入你加入团队做这个项目的迁移工作，你觉得这个项目如何工程化、容器化以及架构演变应该从哪些维度思考？

38.**APP启动速度以及页面加载速度一直是我们比较关心的一个问题，特别是混合开发项目，谈谈你对Flutter渲染优化有哪些见解？

39.**谈谈Flutter的内存回收管理机制，以及你平时是怎么处理内存的？内存泄漏和内存溢出你是怎么解决的？

40.**再问一个简单一点的，你是如何把控混合项目开发时的生命周期（比如类似安卓的onCreate、onResume这种）和路由管理的？

41.**Flutter for web 和Flutter1.9推出的Flutter Web有何本质上的区别？

42.**谈谈你认为是Flutter Web应该如何改进？哪些内容可以改造之后可以用于平时的Web开发。谈谈你的改造方案。

43.**谈谈如何打造低延迟的视频直播？为什么这样用？

44. StatefulWidget 的生命周期

· initState(): Widget 初始化当前 State，在当前方法中是不能获取到 Context 的，如想获取，可以试试 Future.delayed()

· didChangeDependencies(): 在 initState() 后调用，State对象依赖关系发生变化的时候也会调用。

· deactivate(): 当 State 被暂时从视图树中移除时会调用这个方法，页面切换时也会调用该方法，和 Android里的 onPause 差不多。

· dispose(): Widget 销毁时调用。

· didUpdateWidget: Widget 状态发生变化的时候调用。

借用 [CoorChice 文章](#) 里的一张图：

45. Flutter 如何与 Android iOS 通信？

Flutter 通过 PlatformChannel 与原生进行交互，其中 PlatformChannel 分为三种：

\1. BasicMessageChannel：用于传递字符串和半结构化的信息。

\2. MethodChannel：用于传递方法调用。Flutter主动调用Native的方法，并获取相应的返回值。

\3. EventChannel：用于数据流（event streams）的通信。

具体可以查看 [闲鱼技术：深入理解 Flutter Platform Channel](#)。

46. 什么是 Widgets、RenderObjects 和 Elements？

· Widget 仅用于存储渲染所需要的信息。

- `RenderObject` 负责管理布局、绘制等操作。

- `Element` 才是这颗巨大的控件树上的实体。

具体可以查看 [\[译\] Flutter, 什么是 Widgets、RenderObjects 和 Elements?](#)

47. 说一下什么是状态管理，为什么需要它？

首先状态其实是一个概念上的东西，区分全局状态和局部状态。

局部状态比如说一个控件中输入的信息，全局状态比如是登录后从后台请求回来的 `userId`。

当全局状态越来越多，多个页面共享一个状态时，我们就需要管理它。

常用的状态管理有：

- `ScopedModel`

- `BLoC`

- `Redux / FishRedux`

- `Provider`

48. 说一下 BLoC 模式？

具体可以查看：[Vadaski - Flutter | 状态管理探索篇——BLoC\(三\)](#)

这里引用一部分：

BLoC是一种利用reactive programming方式构建应用的方法，这是一个由流构成的完全异步的世界。

49. 如何统一管理错误页面？

我们都知道，如果在 Flutter 当中出错的话，那就是一片红。

可以使用 `ErrorWidget.builder` 来自定义一个 `Widget` 就 ok 了。

具体可以看一下 [小德 - 教你自定义Flutter错误页面](#)

第八章 Android面试之必问设计模式

1.请列举出在 JDK中几个常用的设计模式？

单例模式（Singleton

pattern）用于 Runtime，Calendar 和其他的一些类中。工厂模式

（Factory pattern）被用于各种不可变的类如 `Boolean`，像 `Boolean.valueOf`，观察者模式

（Observer pattern）被用于 Swing 和很多的事件监听中。装饰器设计模式（Decorator

design pattern）被用于多个 Java IO 类中。

2.什么是设计模式？你是否在你的代码里面使用过任何设计模式？

设计模式是世界上各种各样程序员用来解决特定设计问题的尝试和测试的方法。设计模式

是代码可用性的延伸

3. Java 中什么叫单例设计模式？请用 Java 写出线程安全的单例模式

单例模式重点在于在整个系统上共享一些创建时较耗资源的对象。整个应用中只维护一个特定类实例，它被所有组件共同使用。Java.lang.Runtime 是单例模式的经典例子。从 Java

4. 在 Java 中，什么叫观察者设计模式（observer design pattern）？

观察者模式是基于对象的状态变化和观察者的通讯，以便他们作出相应的操作。简单的例子就是一个天气系统，当天气变化时必须在展示给公众的视图中进行反映。这个视图对象是一个主体，而不同的视图是观察者。

5. 使用工厂模式最主要的好处是什么？在哪里使用？

工厂模式的最大好处是增加了创建对象时的封装层次。如果你使用工厂来创建对象，之后你可以使用更高级和更高性能的实现来替换原始的产品实现或类，这不需要在调用层做任何修改。

6. 举一个用 Java 实现的装饰模式(decorator design pattern)? 它是作用于对象层次还是类层次？

装饰模式增加了单个对象的能力。Java IO 到处都使用了装饰模式，典型例子就是 Buffered 系列类如 BufferedReader 和 BufferedWriter，它们增强了 Reader 和 Writer 对象，以实现提升性能的 Buffer 层次的读取和写入。

7. 在 Java 中，为什么不允许从静态方法中访问非静态变量？

Java 中不能从静态上下文访问非静态数据只是因为非静态变量是跟具体的对象实例关联的，而静态的却没有和任何实例关联。

8. 设计一个 ATM 机，请说出你的设计思路？比如设计金融系统来说，必须知道它们应该在任何情况下都能够正常工作。

不管是断电还是其他情况，ATM 应该保持正确的状态（事务），想想 加锁（locking）、事务（transaction）、错误条件（error condition）、边界条件（boundary condition）等等。尽管你不能想到具体的设计，但如果你可以指出非功能性需求，提出一些问题，想到关于边界条件，这些都会是很好的。

9. 在 Java 中，什么时候用重载，什么时候用重写？

如果你看到一个类的不同实现有着不同的方式来做同一件事，那么就应该用重写（overriding），而重载（overloading）是用不同的输入做同一件事。在 Java 中，重载的方法签名不同，而重写并不是。

10. 举例说明什么情况下会更倾向于使用抽象类而不是接口？

接口和抽象类都遵循“面向接口而不是实现编码”设计原则，它可以增加代码的灵活性，

可以适应不断变化的需求。下面有几个点可以帮助你回答这个问题：

在 Java 中，你只能继承一个类，但可以实现多个接口。所以一旦你继承了一个类，你就失去了继承其他类的机会了。

接口通常被用来表示附属描述或行为如：Runnable、Cloneable、Serializable 等等，因此当你使用抽象类来表示行为时，你的类就不能同时是 Runnable 和 Cloneable(注：这里的意思是指如果把 Runnable 等实现为抽象类的情况)，因为在 Java 中你不能继承两个类，但当你使用接口时，你的类就可以同时拥有多个不同的行为。

在一些对时间要求比较高的应用中，倾向于使用抽象类，它会比接口稍快一点。

如果希望把一系列行为都规范在类继承层次内，并且可以更好地在同一个地方进行编码，那么抽象类是一个更好的选择。有时，接口和抽象类可以一起使用，接口中定义函数，而在抽象类中定义默认的实现。

11. 设计模式六大原则

a.单一职责原则：就一个类来说，应该只有一个引起它变化的原因

一个类做一件事情，避免职责过多。比如这种情况是不太好的，在一个Activity中既有bean文件，又有http请求，还有adapter等等，这就导致我们需要修改任何一个东西的时候都会导致Activity的改变，这样一来就有多个引起它变化的原因，不符合单一职责原则

b.开放封闭原则：类，模块，函数应该是可以扩展的，但是不可以修改

对于扩展是开放的，对于修改是封闭的。尽量做到面对需求的改变时，我们的代码能保持相对稳定，通过扩展的方式应对变化，而不是修改原有代码实现

c.里氏替换原则：所有引用基类的地方，必须可以透明的时候其子类的对象

里氏替换原则是实现开放封闭原则的重要方式之一，我们知道，使用基类的地方都可以使用子类去实现，因为子类拥有基类的所有方法，所以在程序设计中尽量使用基类类型对对象进行定义，在运行时确定子类类型。

d.依赖倒置原则：高层模块不应该依赖于底层模块，两者都应该依赖于抽象，抽象不应该依赖于细节，细节应该依赖于抽象

依赖倒置原则针对的是模块之间的依赖关系，高层模块指调用端，底层模块指具体的实现类，抽象指接口或抽象类，细节就是实现类。该原则的具体表现就是模块间的依赖通过抽象发生，直线类之间不发生直接依赖关系，依赖通过接口或抽象类产生，降低耦合，比如MVP模式下，View层和P层通过接口产生依赖关系

e.迪米特原则（最少知识原则）：一个软件实体应该尽可能少的与其他实体发生相互作用

迪米特原则要求我们在设计系统时，尽量减少对象之间的交互

f.接口隔离原则：一个类对另一个类的依赖应该建立在最小的接口上

接口隔离原则的关键是接口以及这个接口要小，如何小呢，也就是我们要为专门的类创建专门的接口，这个接口只对它有效，不要试图让一个接口包罗万象，要建立最小的依赖关系

12. 设计模式的分类

设计模式分为三类

创建型设计模式

与对象创建有关包括单例模式，工厂方法模式，抽象工厂模式，建造者模式，原型模式

结构型设计模式

结构性设计模式是从程序的结构上解决模块之间的耦合问题，包括适配器模式，代理模式，装饰模式，外观模式，桥接模式，组合模式和享元模式

行为型设计模式

主要处理类或对象如何交互及如何分配职责，包括策略模式，模板方法模式，观察者模式，迭代器模式，责任链模式，命令模式，备忘录模式，状态模式，访问者模式，中介模式，解析器模式

致谢

书稿终成，掩卷思量，饮水思源，在此谨表达自身的殷切期许与拳拳谢意。非常感谢各位读者耐心的看到本身的最后部分。

其实这六个章节几乎已经包含高级Android 面试中会涉及到的全部内容，从不同的角度引领读者亲身实战，真正地掌握高级编程的核心开发技巧。但是，实例的数量终究有限，希望读者更多地关注于实战中的开发思想，而不是具体的代码逻辑，代码总会不断地更迭，解决问题的思维却历久弥新。本书中的实例更多的是以点带面，读者可以一边阅读和思考，一边编写代码，相信读完本书，一定受益匪浅；同时，通过本书的实例可以解决一些常见的开发需求。

最后，也得感谢两位论坛作者对我们的大力支持，感谢本书得以付梓的幕后英雄腾讯码牛技术团队，包括关键技术资料和项目实战案例搜集整理，谢谢你们！对所引用的书籍表示感谢。

参考资料：

GitHub: <https://github.com/hornhuang> 作者：圆号本昊

思否: <https://segmentfault.com/u/xiangzhihong> 作者：xiangzhihong